

Cuestionario de estudio

Java 17 & 21 — Certificación Oracle OCP

Basado en los temas del curso "Curso Certificación Java 17 & 21"

Canal Hello World Java — Miguel Rugerio

508 preguntas de opción múltiple organizadas en 24 categorías, alineadas con los objetivos del examen Oracle Certified Professional Java SE 17/21 Developer (1Z0-829).

Cómo usar esta guía:

- Responde primero todas las preguntas de una categoría sin mirar las respuestas.
- Anota tus respuestas (A, B, C o D) en una hoja aparte.
- Al final del documento encontrarás la sección de Respuestas y Explicaciones, organizada igual que las preguntas.
- Repite las categorías donde falles más de una pregunta hasta dominar el 90% o más.

Índice de categorías

1. 01. Fundamentos de Java (JVM, JDK, entorno) — 16 preguntas
2. 02. Tipos primitivos, casting y var — 33 preguntas
3. 03. Operadores, Control de Flujo y Switch — 34 preguntas
4. 04. Arrays y Varargs — 11 preguntas
5. 05. Strings, StringBuilder y Bloques de Texto — 22 preguntas
6. 06. POO: Clases, Constructores y Encapsulamiento — 49 preguntas
7. 07. Herencia, Polimorfismo y Casting — 33 preguntas
8. 08. Interfaces y Clases Abstractas — 12 preguntas
9. 10. Enums — 18 preguntas
10. 11. Records — 5 preguntas
11. 12. Sealed Classes y Pattern Matching Avanzado — 6 preguntas
12. 13. Generics, Wildcards y Type Erasure — 42 preguntas
13. 14. Comparable y Comparator — 21 preguntas
14. 15. Lambdas e Interfaces Funcionales — 35 preguntas
15. 16. Referencias a Métodos (Method References) — 14 preguntas
16. 17. Streams API — 38 preguntas
17. 18. Optional — 6 preguntas
18. 19. Collections Framework (List, Set, Map, Deque) — 28 preguntas
19. 20. Concurrencia y Virtual Threads — 8 preguntas
20. 21. Manejo de Excepciones — 12 preguntas
21. 22. BigDecimal y Precisión Numérica — 5 preguntas
22. 24. Internacionalización (Locale, NumberFormat, ResourceBundle) — 19 preguntas
23. 27. Patrones de Diseño (Strategy) — 4 preguntas
24. 09. Clases Anidadas, Internas, Locales y Anónimas — 8 preguntas
25. 23. API de Fecha y Hora (java.time) — 5 preguntas
26. 26. Módulos Java (JPMS) — 5 preguntas
27. 00. Información del examen de certificación — 5 preguntas
28. 25. Entrada/Salida (I/O, File, Path, Scanner) — 14 preguntas

01. Fundamentos de Java (JVM, JDK, entorno)

1. ¿Cuál es la diferencia principal entre JDK, JRE y JVM?

- A) JDK incluye herramientas de desarrollo (javac, etc.) y el JRE; el JRE contiene la JVM y las librerías necesarias para ejecutar; la JVM ejecuta el bytecode
- B) JRE es para desarrollar, JDK es solo para ejecutar programas ya compilados
- C) La JVM compila el código fuente .java directamente a código máquina nativo sin bytecode intermedio
- D) JDK y JRE son sinónimos y ambos requieren instalar la JVM por separado

2. ¿Qué extensión tiene el archivo generado tras compilar un .java con javac?

- A) .class
- B) .jar
- C) .exe
- D) .bytecode

3. ¿Qué instrucción usarías para ejecutar directamente un archivo fuente Java de un solo archivo sin compilar manualmente (característica desde Java 11)?

- A) java MiArchivo.java
- B) javac -run MiArchivo.java
- C) jshell MiArchivo.java --exec
- D) java --compile MiArchivo.java

4. ¿Qué hace el Garbage Collector (recolector de basura) en Java?

- A) Libera automáticamente la memoria de objetos que ya no tienen referencias alcanzables, sin que el programador deba liberarla manualmente
- B) Compila el código fuente a bytecode
- C) Elimina variables locales no usadas antes de compilar
- D) Cierra automáticamente todos los sockets y archivos abiertos

5. ¿Qué firma debe tener obligatoriamente el método main para que la JVM pueda ejecutar un programa?

- A) public static void main(String[] args)
- B) public void main(String[] args)
- C) static void Main(String args)
- D) public static int main(String... args)

6. ¿Qué es JShell, introducido en Java 9?

- A) Una herramienta REPL (Read-Eval-Print Loop) que permite ejecutar fragmentos de código Java de forma interactiva sin crear un proyecto completo
- B) Un nuevo compilador que reemplaza a javac
- C) Un shell del sistema operativo escrito en Java
- D) Una herramienta exclusiva para depurar hilos concurrentes

7. ¿Cuál de las siguientes NO es una firma válida del método main como punto de entrada tradicional (antes de las simplificaciones de Java 21)?

- A) public void static main(String[] args)
- B) public static void main(String[] args)
- C) static public void main(String[] args)
- D) public static final void main(String... args)

8. ¿Qué simplificación introduce Java 21 (a través de un preview feature) respecto al método main tradicional?

- A) Permite declarar una versión simplificada del punto de entrada sin necesidad de 'public static', ni siquiera de parámetros, facilitando el aprendizaje inicial (unnamed classes and instance main methods)
- B) Elimina por completo la necesidad de un método main en cualquier programa
- C) Obliga a que main siempre reciba un array de enteros en lugar de Strings
- D) Permite que main devuelva un valor int como código de salida obligatorio

9. ¿Cuál es el orden correcto y obligatorio de las secciones en un archivo fuente Java?

- A) Declaración de package (si existe), luego declaraciones de import, y finalmente la declaración de la clase (u otros tipos de nivel superior)
- B) import, luego package, luego class
- C) class, luego package, luego import
- D) El orden no importa, el compilador los reordena automáticamente

10. ¿Cuál de los siguientes NO es un identificador válido para una variable en Java?

- A) _ (un único guion bajo solo)
- B) _contador
- C) contador1
- D) \$total

11. ¿Puede un identificador de variable en Java comenzar con un número, por ejemplo '1variable'?

- A) No, un identificador no puede comenzar con un dígito; puede contener dígitos pero no como primer carácter
- B) Sí, sin ninguna restricción
- C) Solo si la variable es final
- D) Solo en variables locales dentro de un método

12. En un programa Java con bloques de inicialización static, bloques de inicialización de instancia y un constructor, ¿cuál es el orden de ejecución al crear la primera instancia de la clase?

- A) Primero los bloques static (una única vez, al cargar la clase), luego los bloques de inicialización de instancia, y finalmente el constructor
- B) Primero el constructor, luego los bloques de instancia, y al final los bloques static
- C) Los bloques static se ejecutan después de cada instancia creada, no solo la primera
- D) El orden es aleatorio y depende de la JVM

13. Si se crean dos instancias sucesivas de la misma clase, ¿cuántas veces se ejecutan los bloques de inicialización static durante ese programa?

- A) Una sola vez en total, sin importar cuántas instancias se creen después (solo se ejecutan al cargar la clase)
- B) Una vez por cada instancia creada
- C) Dos veces, una por cada instancia
- D) Nunca se ejecutan si ya existe al menos una instancia previa

14. ¿Qué regla rige la indentación (espacios en blanco al inicio de cada línea) dentro de un text block?

- A) El compilador elimina automáticamente los espacios en blanco incidentales comunes a todas las líneas, basándose en la línea con menor indentación (incluida la línea de cierre """)
- B) Todos los espacios se conservan exactamente tal como se escribieron, sin ningún procesamiento
- C) Los espacios deben eliminarse manualmente con trim() después de crear el text block
- D) La indentación siempre debe ser de exactamente 4 espacios por línea

15. En un text block, ¿qué efecto tiene colocar la comilla de cierre "" en su propia línea, alineada más a la izquierda que el resto del contenido, en comparación con alinearla junto al contenido?

- A) Alinear la comilla de cierre más a la izquierda reduce el nivel de 'indentación incidental' considerado común, conservando más espacios de sangría en cada línea del texto resultante
- B) No tiene ningún efecto en el resultado final del String
- C) Provoca un error de compilación siempre
- D) Convierte el text block en un String normal de una sola línea

16. En el contexto del examen de certificación, ¿qué significa típicamente la instrucción 'asuma que la clase compila' en el enunciado de una pregunta?

- A) Que no se debe evaluar si hay errores de sintaxis o de compilación general del archivo; el foco de la pregunta está en la lógica y comportamiento en tiempo de ejecución del fragmento mostrado
- B) Que el código definitivamente no compila y hay que encontrar el error
- C) Que la pregunta trata sobre el proceso de compilación de la JVM
- D) Que se debe ignorar completamente el fragmento de código mostrado

02. Tipos primitivos, casting y var

1. ¿Cuáles son los 8 tipos primitivos de Java?

- A) byte, short, int, long, float, double, char, boolean
- B) byte, short, int, long, float, double, String, boolean
- C) int, long, float, double, char, boolean, Integer, Double
- D) byte, int, long, double, char, boolean, var, Object

2. ¿Qué imprime este código?

int x = 130;

byte b = (byte) x;

System.out.println(b);

```
int x = 130;
byte b = (byte) x;
System.out.println(b);
```

- A) -126
- B) 130
- C) Error de compilación
- D) 0

3. ¿Cuál de las siguientes declaraciones con 'var' NO compila?

- A) var x; (sin inicializador)
- B) var lista = new ArrayList<String>();
- C) var texto = "hola";
- D) var numero = 10;

4. ¿Se puede usar 'var' para declarar un atributo (campo) de una clase?

- A) No, 'var' solo es válido para variables locales (dentro de métodos, bloques, for, try-with-resources)
- B) Sí, siempre que el campo sea private
- C) Sí, pero solo si el campo es static final
- D) Sí, en cualquier contexto incluyendo parámetros de métodos

5. ¿Qué es el 'autoboxing' en Java?

- A) La conversión automática que realiza el compilador de un tipo primitivo a su clase wrapper correspondiente (por ejemplo, int a Integer)
- B) Un proceso manual que el programador debe invocar explícitamente con cast
- C) Una técnica para envolver excepciones dentro de otras excepciones
- D) La conversión de un array a una lista

6. ¿Qué imprime este código?

double d = 9.99;

int i = (int) d;

System.out.println(i);

```
double d = 9.99;  
int i = (int) d;  
System.out.println(i);
```

- A) 9
- B) 10
- C) 9.99
- D) Error de compilación

7. ¿Cuál de las siguientes asignaciones requiere un cast explícito para compilar?

- A) `int x = (int) 3.5;` (double a int)
- B) `long l = 100;` (int a long)
- C) `double d = 100;` (int a double)
- D) `float f = 100L;` (long literal a float con widening)

8. ¿Qué es el 'unboxing' en Java?

- A) La conversión automática de un objeto wrapper (como Integer) a su tipo primitivo correspondiente (como int)
- B) El proceso de crear un array de wrappers
- C) Una técnica para deshabilitar el autoboxing
- D) Un método para serializar objetos

9. ¿Qué ocurre si se intenta hacer unboxing de un valor Integer que es null, por ejemplo al asignarlo a un int?

- A) Se lanza `NullPointerException` en tiempo de ejecución
- B) Se asigna automáticamente el valor 0
- C) El compilador lo rechaza en compilación
- D) Se lanza `ClassCastException`

10. ¿Qué imprime este código?

```
Integer a = 100;  
Integer b = 100;  
System.out.println(a == b);  
Integer c = 200;  
Integer d = 200;  
System.out.println(c == d);
```

```
Integer a = 100;  
Integer b = 100;  
System.out.println(a == b);  
Integer c = 200;  
Integer d = 200;  
System.out.println(c == d);
```

- A) true y false
- B) true y true
- C) false y false
- D) false y true

11. ¿Cuál es el método recomendado para convertir un String a un int de forma directa, sin pasar por un objeto Integer?

- A) Integer.parseInt(texto)
- B) Integer.valueOf(texto).getPrimitive()
- C) new Integer(texto).toInt()
- D) int.parse(texto)

12. ¿Por qué 'var resultado = null;' no compila en Java?

- A) Porque el compilador no puede inferir un tipo concreto a partir de null, ya que null es compatible con cualquier tipo referencia y no da información suficiente para la inferencia
- B) Porque var nunca puede usarse junto a una asignación
- C) Porque null solo puede asignarse a variables de tipo Object
- D) Porque var solo funciona con tipos primitivos, no con referencias

13. ¿Se puede usar 'var' para el tipo de una expresión lambda o de una referencia a método asignada directamente, como en 'var f = () -> System.out.println("hola");'?

- A) No, el compilador no puede inferir un tipo funcional concreto solo a partir de una lambda o method reference sin contexto de una interfaz funcional explícita
- B) Sí, siempre funciona sin restricciones
- C) Solo si la lambda no recibe parámetros
- D) Solo si se usa dentro de un Stream

14. ¿Cuáles son los valores por defecto de un campo de instancia boolean y de un campo de instancia char no inicializados explícitamente?

- A) false para boolean, y el carácter nulo '\u0000' para char
- B) null para ambos
- C) false para boolean, y 0 (como número) para char
- D) true para boolean, y un espacio en blanco ' ' para char

15. ¿Tienen las variables locales (declaradas dentro de un método) un valor por defecto si no se inicializan explícitamente?

- A) No, las variables locales no tienen valor por defecto; el compilador exige que se inicialicen antes de usarlas, o produce un error de compilación
- B) Sí, tienen los mismos valores por defecto que los campos de instancia
- C) Sí, siempre valen null sin importar el tipo
- D) Depende de si el método es static o no

16. ¿Cuál de los siguientes literales numéricos con guion bajo NO es válido en Java?

- A) `_1000` (como si fuera el literal numérico completo con guion bajo al inicio)
- B) `1_000_000`
- C) `0x1F_3A` (hexadecimal con guion bajo)
- D) `3.14_159`

17. ¿Qué prefijo se usa en Java para escribir un literal entero en base octal, por ejemplo el número 8 en octal?

- A) 0 (un cero al inicio, por ejemplo `010` representa el número 8 en octal)
- B) `0o` (como en algunos otros lenguajes)
- C) `Oct()` como función envolvente
- D) `#` antes del número

18. Si existen dos clases con el mismo nombre en distintos paquetes (por ejemplo `java.util.Date` y `java.sql.Date`) y se importa una explícitamente con `'import java.util.Date;'`, ¿qué ocurre si además se usa `'import java.sql.*;'` (import con asterisco) en el mismo archivo?

- A) El import explícito (`java.util.Date`) tiene prioridad sobre el import con asterisco; para usar `java.sql.Date` en ese archivo habría que calificarlo completamente como `java.sql.Date`
- B) El compilador siempre falla con error de ambigüedad sin importar el contexto
- C) El import con asterisco siempre gana sobre el explícito
- D) Java elige aleatoriamente cuál `Date` usar en cada línea

19. Si se importan explícitamente dos clases con el mismo nombre simple desde paquetes distintos, por ejemplo `'import java.util.Date;'` y `'import java.sql.Date;'` en el mismo archivo, ¿qué ocurre?

- A) Error de compilación por ambigüedad: no se puede tener dos imports explícitos en conflicto para el mismo nombre simple en un mismo archivo
- B) Java usa automáticamente la última importada
- C) Java usa automáticamente la primera importada
- D) Se combinan ambas clases en una sola en tiempo de compilación

20. ¿Es recomendable, según buenas prácticas, usar imports con asterisco (`import paquete.*;`) en lugar de imports explícitos uno por uno?

- A) No es la práctica recomendada generalmente: los imports explícitos son más claros sobre qué se usa exactamente y reducen el riesgo de conflictos de nombres al añadir nuevas clases a un paquete importado
- B) Sí, siempre es mejor porque reduce el tamaño del archivo fuente
- C) Es exactamente equivalente en todos los aspectos y no importa cuál se use
- D) El import con asterisco es obligatorio a partir de Java 17

21. Según la estrategia de estudio mencionada para el examen de certificación, ¿qué suele ser más efectivo que memorizar teoría de forma aislada?

- A) Resolver constantemente simuladores de examen con preguntas tipo test, ya que exponen los patrones y trampas reales que se repiten en el examen oficial
- B) Memorizar la especificación completa del lenguaje Java (JLS) palabra por palabra
- C) Evitar por completo la teoría y aprender solo mediante prueba y error en un IDE
- D) Estudiar exclusivamente el código fuente de la JVM

22. En un text block, ¿para qué sirve el carácter de escape '\s'?

- A) Para insertar explícitamente un espacio en blanco que se preserve al final de una línea, evitando que el compilador lo elimine como espacio incidental
- B) Para insertar un salto de línea adicional
- C) Para escapar las triples comillas dobles de cierre
- D) Para indicar que la línea debe convertirse a mayúsculas

23. ¿Qué garantiza realmente la llamada a System.gc() en Java?

- A) No garantiza nada: es solo una 'sugerencia' a la JVM para que considere ejecutar el Garbage Collector, pero la JVM puede ignorarla completamente
- B) Fuerza inmediatamente la recolección de todos los objetos elegibles, sin excepción
- C) Detiene el programa hasta que termine la recolección completa
- D) Elimina también los objetos que todavía tienen referencias activas

24. Dado un método sobrecargado con las firmas metodo(int x) y metodo(Integer x), si se invoca metodo(5) con un literal int, ¿cuál versión se prefiere?

- A) La versión con el parámetro primitivo metodo(int x), ya que Java prioriza la coincidencia exacta sin necesidad de autoboxing antes de considerar la versión que requiere convertir el primitivo a su wrapper
- B) La versión con Integer, porque siempre se prefiere el autoboxing
- C) Ambas se ejecutan y el resultado depende del orden de declaración
- D) Produce un error de compilación por ambigüedad

25. ¿Cuál es una trampa común en el examen relacionada con unboxing y valores null?

- A) Asignar un Integer que vale null a una variable int primitiva compila correctamente pero lanza NullPointerException en tiempo de ejecución al intentar hacer unboxing
- B) El compilador siempre detecta y rechaza en compilación cualquier posible unboxing de un valor null
- C) Un Integer nunca puede valer null
- D) El unboxing de null siempre produce el valor 0 sin lanzar excepción

26. En los diagramas de memoria usados para analizar referencias, ¿qué representa una flecha que sale de una variable de tipo objeto?

- A) La referencia (dirección) que esa variable mantiene hacia un objeto ubicado en el heap, no el objeto en sí
- B) El valor primitivo contenido directamente en la variable
- C) Una copia completa del objeto almacenada en la pila (stack)
- D) El nombre de la clase del objeto únicamente, sin relación con memoria

27. ¿Qué diferencia existe entre un bloque de inicialización de instancia y el propio constructor de una clase, en cuanto a cuándo se ejecutan?

- A) El bloque de inicialización de instancia se ejecuta antes del cuerpo del constructor, cada vez que se crea una nueva instancia, independientemente de cuál constructor sobrecargado se invoque
- B) El constructor siempre se ejecuta antes que los bloques de inicialización de instancia
- C) Los bloques de inicialización de instancia solo se ejecutan si el constructor no tiene parámetros
- D) No existe ninguna diferencia, son sintácticamente intercambiables

28. ¿Cuál es la forma correcta de escribir el número 26 en binario como literal en Java?

- A) 0b11010
- B) b11010
- C) 0B_11010 sin dígitos después
- D) binary(11010)

29. ¿Qué caracteres son válidos como dígitos en un literal hexadecimal en Java, además de los prefijos 0x/0X?

- A) Los dígitos del 0 al 9 y las letras de la A a la F (mayúsculas o minúsculas)
- B) Solo los dígitos del 0 al 9
- C) Solo las letras de la A a la F
- D) Los dígitos del 0 al 7 únicamente

30. En el patrón de diseño Strategy explicado con el ejemplo de aves (Aguila, Pato, Pinguino), ¿qué relación representa que la clase Ave tenga una referencia a una interfaz ComportamientoVolar, en lugar de heredar el comportamiento de volar directamente?

- A) Una relación 'Has-A' (composición/delegación), en contraste con una relación 'Is-A' (herencia), permitiendo cambiar el comportamiento de vuelo en tiempo de ejecución
- B) Una relación de herencia múltiple entre Ave y ComportamientoVolar
- C) Una relación de implementación de interfaz directa sin composición
- D) No representa ninguna relación de diseño particular, es solo un atributo cualquiera

31. ¿Por qué Pinguino no debería heredar un método volar() con implementación fija desde una superclase Ave si se sigue estrictamente el enfoque del patrón Strategy?

- A) Porque forzaría a Pinguino a 'volar' de alguna manera (o a sobrescribir el método para no hacerlo), rompiendo el principio de que el comportamiento debe asignarse mediante composición según las capacidades reales de cada subtipo, no heredarse ciegamente
- B) Porque Pinguino no puede ser subclase de Ave bajo ninguna circunstancia
- C) Porque los pingüinos no existen como concepto válido en POO
- D) Porque el patrón Strategy prohíbe el uso de clases abstractas

32. En la evolución de versiones del ejemplo Strategy explicada en el curso (de la versión 0 con herencia tradicional hasta la versión final), ¿qué mejora clave introduce el paso de usar herencia directa a usar una interfaz de estrategia con delegación?

- A) Permite cambiar el comportamiento de un objeto en tiempo de ejecución (por ejemplo, reasignando la estrategia de vuelo de un ave) sin necesidad de crear una nueva subclase para cada combinación de comportamientos
- B) Elimina por completo la necesidad de usar interfaces en el diseño
- C) Hace que el código se ejecute más rápido en todos los casos
- D) Convierte automáticamente las clases en records

33. ¿Qué ventaja aporta dar un 'comportamiento por defecto' en el constructor de una clase que usa el patrón Strategy (por ejemplo, asignar NoVolar por defecto si no se especifica otra estrategia)?

- A) Evita que el objeto quede en un estado inválido o con una referencia nula a la estrategia si el cliente no especifica explícitamente un comportamiento al crear el objeto
- B) Hace que la clase ya no necesite implementar la interfaz de estrategia
- C) Impide que el comportamiento pueda cambiarse después de la creación del objeto
- D) Es un requisito obligatorio del lenguaje Java para todas las clases

03. Operadores, Control de Flujo y Switch

1. ¿Qué imprime el siguiente switch expression?

```
int dia = 3;  
String r = switch(dia){  
case 1,7 -> "Fin de semana";  
default -> "Entre semana";  
};  
System.out.println(r);
```

```
int dia = 3;  
String r = switch(dia){  
case 1, 7 -> "Fin de semana";  
default -> "Entre semana";  
};  
System.out.println(r);
```

- A) Entre semana
- B) Fin de semana
- C) Error de compilación
- D) 3

2. ¿Qué palabra clave se usa en un switch expression para devolver un valor calculado dentro de un bloque {}?

- A) yield
- B) return
- C) break
- D) value

3. ¿Cuál es el resultado de: `System.out.println(5 / 2);` y luego `System.out.println(5.0 / 2);`?

- A) 2 y 2.5
- B) 2.5 y 2.5
- C) 2 y 2
- D) 2.5 y 2

4. ¿Qué diferencia hay entre los operadores '&&' y '&' cuando se usan con dos expresiones booleanas?

- A) '&&' usa cortocircuito (no evalúa el segundo operando si el primero ya determina el resultado); '&' siempre evalúa ambos operandos
- B) '&' es solo para booleanos y '&&' solo para enteros
- C) No hay ninguna diferencia entre ambos
- D) '&&' siempre evalúa ambos operandos y '&' usa cortocircuito

5. ¿Qué imprime este bucle?

```
for(int i = 0; i < 3; i++){  
    if(i == 1) continue;  
    System.out.print(i);  
}
```

```
for(int i = 0; i < 3; i++){  
    if(i == 1) continue;  
    System.out.print(i);  
}
```

- A) 02
- B) 012
- C) 01
- D) 2

6. ¿Es obligatorio el 'break' al final de cada rama en un switch statement clásico (con dos puntos ':')?

- A) No es obligatorio sintácticamente, pero sin él el flujo 'cae' (fall-through) a la siguiente rama, lo cual suele ser un error común
- B) Sí, el compilador rechaza un switch clásico sin break en cada case
- C) El break solo es válido dentro de bucles, nunca en un switch
- D) Solo es necesario en el último case del switch

7. ¿Cuál es el resultado de la expresión '2 + 3 * 4' según la precedencia de operadores en Java?

- A) 14, porque la multiplicación tiene mayor precedencia que la suma y se evalúa primero
- B) 20, porque se evalúa estrictamente de izquierda a derecha ignorando precedencia
- C) 9, resultado de sumar y multiplicar en algún otro orden
- D) Error de compilación por ambigüedad

8. Al operar entre un valor byte y un valor int en una expresión aritmética, por ejemplo 'byte b = 5; int resultado = b + 10;', ¿qué ocurre con el tipo de b durante la operación?

- A) Se promociona automáticamente (widening) a int antes de realizar la suma, ya que las operaciones aritméticas binarias promocionan operandos byte, short y char al menos a int
- B) Se mantiene como byte durante toda la operación
- C) Provoca un error de compilación siempre
- D) Se convierte automáticamente a long, no a int

9. ¿Qué hace el operador lógico XOR (^) aplicado a dos valores boolean, por ejemplo 'true ^ false'?

- A) Devuelve true si exactamente uno de los dos operandos es true (y false si ambos son iguales, ya sea ambos true o ambos false)
- B) Devuelve true solo si ambos operandos son true, igual que &&
- C) Devuelve false siempre que al menos un operando sea true
- D) Es equivalente exactamente al operador ||

10. ¿Por qué se dice que '&&' y '||' realizan 'evaluación de circuito corto' (short-circuit), a diferencia de '&' y '|'?

- A) Porque '&&' y '||' pueden evitar evaluar el segundo operando si el resultado ya queda determinado por el primero, mientras que '&' y '|' siempre evalúan ambos operandos sin importar el resultado parcial
- B) Porque '&&' y '||' son más rápidos en todos los casos sin excepción
- C) Porque '&' y '|' solo funcionan con tipos numéricos, nunca con boolean
- D) Porque '&&' y '||' no pueden usarse dentro de condicionales if

11. ¿Qué imprime el siguiente código?

int x = 5;

int y = x++ + ++x;

System.out.println(y);

```
int x = 5;
int y = x++ + ++x;
System.out.println(y);
```

- A) 12
- B) 11
- C) 13
- D) 10

12. ¿Qué valor tiene 'resultado' tras ejecutar: int a = 10; int resultado = (a > 5) ? 100 : 200;

- A) 100
- B) 200
- C) 10
- D) Error de compilación

13. ¿Qué devuelve el operador módulo (%) al aplicarse a dos enteros, por ejemplo '17 % 5'?

- A) El resto de la división entera entre los dos operandos (2, ya que $17 = 5 \cdot 3 + 2$)
- B) El cociente de la división entera (3)
- C) Siempre 0 si ambos números son positivos
- D) Un valor decimal equivalente a la división real

14. Dado 'int x = 5; int y = x-- - --x;'. ¿qué valor final tienen y, y x?

```
int x = 5;
int y = x-- - --x;
```

- A) y = 2, x = 3
- B) y = 0, x = 4
- C) y = 1, x = 3
- D) y = 2, x = 4

15. ¿Qué produce el operador de complemento a nivel de bits (~) aplicado a un entero, por ejemplo '~5'?

- A) Invierte todos los bits del número y, como regla práctica, el resultado equivale a $-(n+1)$, por lo que ~ 5 es igual a -6
- B) Cambia el signo del número sin modificar su valor absoluto (equivalente a la negación aritmética -n)
- C) Siempre devuelve 0 para cualquier entero
- D) Solo funciona con tipos boolean, nunca con enteros

16. ¿Qué ocurre si se suma 1 al valor máximo representable por un int (Integer.MAX_VALUE), por ejemplo 'int x = Integer.MAX_VALUE + 1;'?

- A) Se produce overflow silencioso: el valor 'da la vuelta' y se convierte en Integer.MIN_VALUE (el número negativo más pequeño representable), sin lanzar ninguna excepción
- B) Se lanza automáticamente una ArithmeticException
- C) El compilador rechaza la operación por desbordamiento detectado en compilación
- D) El resultado se ajusta automáticamente a un tipo long para evitar el problema

17. ¿Qué hace el operador de asignación compuesta '+=' que lo diferencia de escribir 'x = x + valor' de forma explícita, en el caso de tipos más estrechos como byte o short?

- A) El operador compuesto '+=' realiza un cast implícito (narrowing) automático de vuelta al tipo original de la variable, algo que 'x = x + valor' no hace y por eso podría requerir un cast explícito
- B) No existe ninguna diferencia entre ambas formas en ningún caso
- C) '+=' siempre lanza un error de compilación con tipos byte o short
- D) 'x = x + valor' es más eficiente en todos los casos que usar '+='

18. ¿En qué orden se evalúan los operandos de una expresión aritmética con el mismo nivel de precedencia, como en 'a - b + c'?

- A) De izquierda a derecha: primero (a - b), y a ese resultado se le suma c
- B) De derecha a izquierda: primero (b + c), y luego a se le resta ese resultado
- C) Depende del tipo de dato de cada operando
- D) Java elige el orden que optimice el rendimiento, sin garantía fija

19. ¿Cuáles de los siguientes tipos NO son válidos como tipo de la variable de control (selector) en un switch (statement o expression)?

- A) long, double, float y boolean
- B) int, short y byte
- C) char y String
- D) Un tipo enum

20. ¿Es obligatorio incluir una rama 'default' en un switch expression (con arrow ->) que se usa para asignar un valor a una variable?

- A) Sí, salvo que el compilador pueda determinar que todos los casos posibles del tipo están cubiertos exhaustivamente (por ejemplo, todas las constantes de un enum), de lo contrario es obligatorio para garantizar que la expresión siempre produzca un valor
- B) No, nunca es obligatorio en un switch expression
- C) Solo es obligatorio si el switch tiene más de 3 casos
- D) Es obligatorio únicamente en switch statements clásicos, nunca en expressions

21. ¿Se puede usar 'var' como tipo de la variable de control en un switch, por ejemplo 'var dia = 3; switch(dia){...}'?

- A) Sí, siempre que el tipo inferido para 'var' sea uno de los tipos válidos permitidos como selector de switch (como int en este caso)
- B) No, 'var' nunca puede usarse en la variable de control de un switch
- C) Solo si 'var' infiere el tipo String exclusivamente
- D) Solo dentro de switch expressions, nunca en switch statements clásicos

22. En un for-each como 'for(String nombre : listaDeStrings){...}', ¿qué tipo de objeto debe implementar 'listaDeStrings' como mínimo para que el bucle compile?

- A) Debe implementar la interfaz Iterable<T> (como lo hacen todas las Collections del framework, entre otras clases)
- B) Debe ser obligatoriamente un array, nunca una Collection
- C) Debe implementar Comparable<T>
- D) Debe ser necesariamente una List, no puede ser un Set ni un Map directamente

23. ¿Para qué sirve etiquetar un bucle con una 'label' (por ejemplo 'externo: for(...) { for(...) { break externo; } })?

- A) Permite que un break o continue afecte específicamente al bucle etiquetado (por ejemplo, el externo), en lugar de solo al bucle más interno donde se encuentra la instrucción
- B) Sirve únicamente para dar nombre descriptivo sin ningún efecto funcional en el control de flujo
- C) Convierte el bucle en un bucle infinito garantizado
- D) Solo puede usarse con bucles while, nunca con for

24. ¿Qué elimina el pattern matching for instanceof (Java 16+) que antes era necesario escribir manualmente?

- A) El casting explícito y redundante del objeto después de comprobar su tipo con instanceof, ya que la variable del patrón queda automáticamente tipada y lista para usarse dentro del bloque
- B) La propia instrucción instanceof, que deja de ser necesaria por completo
- C) La necesidad de usar bloques if en el código
- D) La posibilidad de comparar contra null

25. ¿Qué es el 'flow scoping' (alcance de flujo) en el contexto del pattern matching con instanceof?

- A) El alcance de la variable de patrón (por ejemplo 's' en 'obj instanceof String s') depende del flujo lógico del código: solo está definitivamente asignada y disponible en las ramas donde el compilador puede garantizar que la comprobación fue verdadera
- B) Es un mecanismo para declarar variables globales accesibles desde cualquier clase
- C) Significa que la variable de patrón siempre está disponible en todo el método sin restricciones
- D) Es un sinónimo de 'scope estático' aplicado a métodos

26. ¿Por qué 'if (obj instanceof String s && s.length() > 5)' compila correctamente, pero un equivalente con '||' en lugar de '&&' (como 'if (obj instanceof String s || s.length() > 5)') NO compila?

- A) Con '&&', si la primera condición es false, la segunda no se evalúa (por eso 's' está garantizada como asignada al evaluar la segunda parte); con '||', si la primera condición es false, Sí se evaluaría la segunda, pero 's' podría no estar definitivamente asignada en ese caso, generando un error de compilación
- B) Porque el operador '||' está prohibido en cualquier expresión que use instanceof
- C) Porque 's' solo puede usarse con el operador '&&', nunca en ninguna otra expresión booleana
- D) Porque ambas expresiones en realidad sí compilan sin ningún problema

27. ¿Cuál es la diferencia entre 'break;' (sin label) y 'break etiqueta;' (con label) dentro de un bucle for anidado dentro de otro bucle for etiquetado como 'etiqueta'?

- A) 'break;' solo termina el bucle interno donde se ejecuta, continuando con el bucle externo; 'break etiqueta;' termina completamente el bucle externo etiquetado (y por tanto también el interno)
- B) Ambas instrucciones tienen exactamente el mismo efecto en cualquier contexto
- C) 'break etiqueta;' solo funciona si el bucle etiquetado es un while, nunca un for
- D) 'break;' sin label siempre termina todos los bucles anidados, no solo el más interno

28. En un switch statement clásico con 'case' basados en valores int, ¿por qué una variable declarada como 'final int x = 5;' puede usarse como valor de un case, pero una variable 'int y = 5;' (no final, aunque nunca se reasigne) generalmente no puede?

- A) Porque los valores de 'case' en un switch sobre int/char/byte/short deben ser constantes conocidas en tiempo de compilación, y solo una variable 'final' inicializada con un valor constante cumple ese requisito; una variable no-final no se considera una constante de compilación aunque su valor no cambie en la práctica
- B) Porque 'final' convierte automáticamente cualquier variable en static
- C) Porque los case nunca pueden usar variables, solo literales escritos directamente
- D) No hay ninguna diferencia real entre ambos casos

29. ¿Puede repetirse el mismo valor de constante en dos 'case' distintos dentro del mismo switch statement?

- A) No, produce un error de compilación por 'valor de case duplicado', ya que cada valor debe ser único dentro del mismo switch
- B) Sí, y en ese caso se ejecutan ambos bloques en secuencia
- C) Sí, pero solo si el switch es un switch expression, no un switch statement
- D) Solo se permite duplicar el valor si ambos case están vacíos

30. Al analizar errores de compilación relacionados con el alcance (scope) de variables, ¿cuál es una causa típica de estos errores según lo revisado en los ejercicios?

- A) Intentar acceder a una variable fuera del bloque {} en el que fue declarada, ya que su visibilidad termina al cerrarse ese bloque
- B) Usar nombres de variable con más de 10 caracteres
- C) Declarar la variable como final dentro de un bucle
- D) Usar la palabra clave var en vez de un tipo explícito

31. En un ejercicio con bucles anidados y un 'break' etiquetado combinado con el operador módulo dentro de la condición, ¿qué habilidad es clave para resolver correctamente este tipo de preguntas de examen?

- A) Rastrear meticulosamente, iteración por iteración, el valor de cada variable de control y evaluar con precisión hacia qué bucle etiquetado apunta cada break o continue
- B) Memorizar de antemano la respuesta sin ejecutar mentalmente el código
- C) Asumir que todo bucle anidado con break etiquetado siempre termina en la primera iteración
- D) Ignorar el operador módulo, ya que rara vez afecta el resultado final

32. ¿Cuál de las siguientes es una trampa común de examen al analizar un switch expression con errores de compilación?

- A) Mezclar tipos de datos incompatibles entre los distintos 'case' y el valor del selector, o cometer errores de punto y coma que generan código inalcanzable o sintaxis inválida
- B) Usar más de un case en una misma línea separado por comas, lo cual siempre es un error
- C) Usar la palabra 'yield' dentro de un switch expression, lo cual está prohibido
- D) Que el switch expression tenga más de 5 casos en total

33. Al revisar preguntas de examen sobre pattern matching con instanceof, ¿qué tipo de trampa se menciona como frecuente relacionada con el formato del código?

- A) Un formato de indentación o salto de línea engañoso que hace parecer que una variable de patrón está disponible en cierto punto del código, cuando en realidad el flujo (flow scoping) no lo garantiza en esa posición
- B) El uso de tabulaciones en vez de espacios, que provoca errores de compilación reales
- C) Que el pattern matching solo funcione si el código está escrito en una sola línea
- D) Que las preguntas de examen nunca incluyan pattern matching con instanceof

34. ¿Por qué se insiste en que dominar los conceptos fundamentales del lenguaje (scope, loops, switch, pattern matching básico) es clave para el examen de certificación, más allá de los temas avanzados?

- A) Porque estos fundamentos son la base sobre la que se construyen los temas más avanzados, y el examen los evalúa con frecuencia, muchas veces combinados de forma sutil en una misma pregunta
- B) Porque el examen no incluye ningún tema avanzado en absoluto
- C) Porque los fundamentos representan menos del 5% del contenido del examen
- D) Porque los temas avanzados nunca aparecen combinados con los fundamentos en una misma pregunta

04. Arrays y Varargs

1. ¿Cuál declaración de un método varargs es válida?

- A) void metodo(String... nombres)
- B) void metodo(String nombres...)
- C) void metodo(...String nombres)
- D) void metodo(String[]... nombres, int x)

2. ¿Qué ocurre si un método tiene un parámetro varargs y otro sobrecargado con array explícito del mismo tipo?

- A) Compila; Java prefiere la coincidencia exacta (el método con array explícito) antes que expandir varargs
- B) Error de compilación por ambigüedad siempre
- C) Siempre se ejecuta la versión varargs
- D) Solo compila si varargs se declara primero en el archivo

3. `int[] a = new int[3];` ¿cuál es el valor por defecto de cada elemento?

- A) 0
- B) null
- C) Error de compilación, hay que inicializar
- D) -1

4. ¿Qué imprime este código?

```
int[][] matriz = {{1,2},{3,4,5}};
```

```
System.out.println(matriz[1].length);
```

```
int[][] matriz = {{1,2},{3,4,5}};  
System.out.println(matriz[1].length);
```

- A) 3
- B) 2
- C) 5
- D) Error de compilación

5. ¿Qué método de la clase Arrays permite convertir un array en una representación de texto legible, como "[1, 2, 3]"?

- A) Arrays.toString(array)
- B) array.print()
- C) Arrays.display(array)
- D) String.valueOf(array) directamente muestra el contenido

6. ¿Qué son realmente los arrays bidimensionales en Java, como `int[][] matriz`?

- A) Son arrays de arrays: matriz es un array cuyos elementos son, a su vez, referencias a otros arrays unidimensionales (que pueden incluso tener distinto tamaño entre sí)
- B) Son una estructura de datos nativa completamente distinta a los arrays unidimensionales, sin relación entre sí
- C) Siempre deben tener el mismo número de filas y columnas, como una matriz matemática estricta
- D) Solo pueden contener tipos primitivos, nunca objetos

7. Si se declara `int[][] matriz = new int[3][];` (sin especificar el tamaño de las filas) y luego se intenta acceder a `matriz[0][0]` sin haber inicializado esa fila, ¿qué ocurre?

- A) Se lanza `NullPointerException`, ya que `matriz[0]` es null (no se ha creado el array interno de esa fila todavía)
- B) Se devuelve automáticamente el valor 0
- C) Se lanza `ArrayIndexOutOfBoundsException`
- D) El código no compila

8. ¿Qué excepción se lanza al intentar acceder a un índice igual o mayor al tamaño (length) de un array, por ejemplo `array[5]` en un array de tamaño 5 (índices válidos 0 a 4)?

- A) `ArrayIndexOutOfBoundsException`
- B) `NullPointerException`
- C) `IllegalArgumentException`
- D) `ArithmeticException`

9. ¿Qué información imprime típicamente `Arrays.stream()` al procesar directamente los elementos de un array bidimensional sin aplanar (por ejemplo, imprimiendo cada elemento del array externo tal cual)?

- A) La representación de la ubicación en memoria (hashcode) de cada sub-array interno, ya que cada elemento del array externo es en sí mismo una referencia a otro array, no un valor simple
- B) Directamente todos los números contenidos en la matriz completa, aplanados en una sola línea
- C) Siempre imprime null para cada fila
- D) Genera un error de compilación porque `Arrays.stream()` no admite arrays de dos dimensiones

10. ¿Cuál es una forma habitual y clara de recorrer todos los elementos de un array bidimensional para procesarlos individualmente?

- A) Usar bucles for-each anidados: uno externo que recorre cada sub-array (fila), y uno interno que recorre los elementos de esa fila
- B) Usar un único bucle for-each simple, ya que Java aplanar automáticamente cualquier array multidimensional
- C) Es obligatorio usar únicamente índices numéricos con for tradicional, nunca for-each, para arrays de más de una dimensión
- D) No es posible iterar arrays bidimensionales, solo se puede acceder por índice fijo

11. ¿Es válido en Java crear un array de tamaño cero, como `int[] vacio = new int[0];`?

- A) Sí, es completamente válido crear un array de tamaño 0; simplemente no tiene ninguna posición disponible para almacenar elementos (no se le pueden 'insertar' valores después, ya que su tamaño es fijo)
- B) No, Java lanza una excepción en tiempo de ejecución al intentar crear un array de tamaño 0
- C) No compila, el tamaño de un array debe ser mayor a 0 obligatoriamente
- D) Sí, pero automáticamente Java lo convierte a tamaño 1

05. Strings, StringBuilder y Bloques de Texto

1. ¿Por qué se dice que String es inmutable en Java?

- A) Porque una vez creado un objeto String, su contenido no puede modificarse; operaciones como concat() devuelven un nuevo objeto
- B) Porque String no permite ser referenciado por más de una variable
- C) Porque el compilador prohíbe declarar más de un String por clase
- D) Porque String usa arrays mutables internamente sincronizados

2. ¿Qué imprime?

String a = "hola";

String b = "hola";

System.out.println(a == b);

```
String a = "hola";  
String b = "hola";  
System.out.println(a == b);
```

- A) true
- B) false
- C) Error de compilación
- D) Depende del JIT

3. ¿Cuál es la sintaxis correcta para iniciar un bloque de texto (text block, Java 15+)?

- A) Triple comillas dobles """" seguidas de salto de línea
- B) Comillas simples triples '''
- C) Corchetes angulares <"">
- D) Prefijo tb" seguido de comillas dobles

4. ¿Qué método se usa para comparar el contenido de dos Strings (no la referencia)?

- A) equals()
- B) ==
- C) compareRef()
- D) same()

5. ¿Por qué se recomienda StringBuilder en lugar de concatenar Strings con '+' dentro de un bucle con muchas iteraciones?

- A) Porque cada concatenación con '+' crea un nuevo objeto String (dado que String es inmutable), lo que es ineficiente; StringBuilder modifica un buffer mutable interno
- B) Porque el operador '+' no está permitido con Strings en bucles
- C) Porque StringBuilder es la única clase que soporta texto en Java
- D) Porque '+' solo funciona fuera de bucles

6. ¿Qué imprime?

String s = " Hola Mundo ";

Svstem.out.println(s.strip().length());

```
String s = " Hola Mundo ";  
System.out.println(s.strip().length());
```

- A) 10
- B) 14
- C) 9
- D) 11

7. ¿Qué método de String permite dividir el texto en un array usando un delimitador, por ejemplo una coma?

- A) split(regex)
- B) divide(regex)
- C) tokenize(regex)
- D) partition(regex)

8. ¿Qué ocurre al llamar al método replace() (u otros métodos de transformación) sobre un StringBuilder dentro de una cadena de construcción (Stream Builder de texto), en cuanto a si modifica el objeto original?

- A) Los métodos de StringBuilder, a diferencia de los de String, sí modifican el objeto original in situ (mutándolo), en lugar de crear un nuevo objeto independiente
- B) Siempre se crea un nuevo objeto inmutable, exactamente igual que con String
- C) replace() en StringBuilder no existe, solo está disponible en String
- D) El comportamiento depende de si el StringBuilder es final o no

9. ¿Qué método de String se usa para obtener un carácter específico según su posición (índice), por ejemplo el tercer carácter de un texto?

- A) charAt(indice)
- B) getChar(indice)
- C) characterAt(indice)
- D) indexChar(indice)

10. ¿Qué comportamiento tiene equals() al comparar dos arrays con el mismo contenido, por ejemplo new int[]{1,2,3}.equals(new int[]{1,2,3})?

- A) Devuelve false, porque los arrays no sobrescriben equals() con comparación de contenido; heredan el comportamiento de Object, que compara referencias (equivalente a ==)
- B) Devuelve true, porque compara automáticamente el contenido elemento por elemento
- C) Lanza una excepción en tiempo de ejecución al comparar dos arrays con equals()
- D) No compila, arrays no tienen el método equals()

11. ¿Qué devuelve Math.round(2.5) en Java?

- A) 3 (Math.round redondea hacia el entero más cercano, y en el caso de .5 exacto redondea hacia arriba)
- B) 2
- C) 2.5 sin cambios, porque Math.round no afecta a doubles
- D) Un error de compilación por tipo incompatible

12. ¿Qué diferencia hay entre Math.floor() y Math.round() al aplicarse a un valor decimal como 4.7?

- A) Math.floor(4.7) devuelve 4.0 (siempre redondea hacia abajo, al entero menor o igual), mientras que Math.round(4.7) devuelve 5 (redondea al entero más cercano)
- B) Ambos métodos devuelven exactamente el mismo resultado en todos los casos
- C) Math.floor() siempre redondea hacia arriba y Math.round() siempre hacia abajo
- D) Math.floor() solo funciona con enteros, nunca con decimales

13. ¿Por qué es importante recordar que LocalDate es immutable al usar métodos como plusDays() o minusMonths()?

- A) Porque estos métodos no modifican el objeto LocalDate original, sino que devuelven una nueva instancia con la fecha modificada; si no se captura ese valor de retorno (por ejemplo, reasignando la variable), el cambio se pierde
- B) Porque plusDays() y minusMonths() lanzan una excepción si se llaman más de una vez sobre el mismo objeto
- C) Porque LocalDate no tiene esos métodos, solo Calendar los tiene
- D) Porque estos métodos solo funcionan si LocalDate se declara como var

14. ¿Qué requisito debe cumplir un array para que Arrays.binarySearch() funcione correctamente y produzca resultados confiables?

- A) El array debe estar previamente ordenado (en orden ascendente según su orden natural o el Comparator usado), ya que la búsqueda binaria asume que los elementos están ordenados
- B) El array debe tener un tamaño par de elementos
- C) El array debe contener únicamente valores positivos
- D) El array debe ser de tipo String exclusivamente

15. ¿Qué devuelve Arrays.binarySearch() cuando el elemento buscado NO se encuentra en el array?

- A) Un valor negativo que codifica el punto de inserción donde debería colocarse el elemento para mantener el orden, calculado como -(punto_insercion) - 1
- B) Siempre devuelve -1 sin más información
- C) Lanza NoSuchElementException
- D) Devuelve 0

16. ¿Qué hace el método indent(n) de String cuando n es un número positivo?

- A) Añade n espacios en blanco al inicio de cada línea del String, y además normaliza los saltos de línea, garantizando que el texto termine con salto de línea
- B) Elimina n espacios del inicio de cada línea
- C) Cuenta cuántos espacios de indentación tiene el texto sin modificarlo
- D) Convierte el texto a un text block automáticamente

17. ¿Qué hace el método translateEscapes() de String?

- A) Convierte secuencias de escape literales escritas como texto (por ejemplo la secuencia de caracteres backslash+n) en su carácter especial real correspondiente (como un salto de línea real)
- B) Elimina todas las secuencias de escape del String sin reemplazarlas por nada
- C) Traduce el texto completo a otro idioma humano
- D) Convierte el String a mayúsculas escapando los acentos

18. ¿Qué produce `String.format("%05d", 42)`?

- A) "00042" (el número se rellena con ceros a la izquierda hasta completar 5 dígitos en total)
- B) "42000" (rellena con ceros a la derecha)
- C) "42" sin ningún cambio
- D) Lanza una excepción por especificador de formato inválido

19. ¿Qué devuelve `Arrays.compare(array1, array2)` al comparar dos arrays de enteros?

- A) Un valor negativo, cero o positivo indicando si array1 es 'menor', 'igual' o 'mayor' que array2, comparando elemento por elemento en orden (similar a `compareTo`), incluyendo el caso de que uno sea un prefijo del otro
- B) Siempre true o false, indicando solo si son exactamente iguales
- C) El número de elementos que difieren entre ambos arrays
- D) Siempre lanza una excepción si los arrays tienen distinta longitud

20. ¿Qué devuelve `Arrays.mismatch(array1, array2)` si ambos arrays son completamente iguales en contenido y longitud?

- A) -1, indicando que no existe ningún índice de discrepancia entre ambos arrays
- B) 0, indicando que el primer elemento coincide
- C) true, indicando que son iguales
- D) Lanza una excepción porque no hay discrepancia que reportar

21. ¿Por qué trabajar con `ZonedDateTime` y cálculos que cruzan un cambio de horario de verano (DST) puede producir resultados aparentemente inesperados, como sumar horas y no obtener el resultado 'esperado' en horas absolutas?

- A) Porque `ZonedDateTime` ajusta automáticamente la hora local según las reglas de la zona horaria, incluyendo saltos o repeticiones de horas durante las transiciones de horario de verano, por lo que sumar un número fijo de horas puede no corresponder exactamente con lo esperado en tiempo 'de reloj' si se cruza esa transición
- B) Porque `ZonedDateTime` no soporta el horario de verano en absoluto y siempre lo ignora
- C) Porque sumar horas a un `ZonedDateTime` siempre lanza una excepción
- D) Porque `ZonedDateTime` convierte automáticamente todo a UTC sin posibilidad de configurarlo

22. Si se ejecuta `'fecha.plusDays(3);'` sobre un objeto `LocalDate` sin reasignar el resultado a ninguna variable, ¿qué ocurre con el valor de `'fecha'` después de esa línea?

- A) `'fecha'` permanece sin cambios, ya que `plusDays()` devuelve un nuevo objeto `LocalDate` y no modifica el original; al no capturar ese valor de retorno, el cambio se pierde
- B) `'fecha'` se actualiza automáticamente con los 3 días sumados
- C) Se lanza una excepción en tiempo de ejecución por no capturar el retorno
- D) El programa no compila si no se asigna el resultado

06. POO: Clases, Constructores y Encapsulamiento

1. ¿Qué ocurre si una clase no define ningún constructor explícito?

- A) El compilador genera automáticamente un constructor por defecto sin argumentos
- B) La clase no puede ser instanciada nunca
- C) Java lanza una excepción en tiempo de ejecución al usar new
- D) Se genera un constructor con todos los atributos como parámetros

2. ¿Qué significa que un atributo sea 'static'?

- A) Pertenece a la clase y es compartido por todas las instancias, en lugar de pertenecer a cada objeto individual
- B) No puede modificarse nunca después de asignarse
- C) Solo puede accederse desde el método main
- D) Se destruye automáticamente al finalizar cada método

3. ¿Cuál es el propósito principal del modificador 'final' aplicado a una variable?

- A) Impedir que la variable sea reasignada después de su primera asignación
- B) Hacer que la variable sea accesible desde cualquier paquete
- C) Convertir la variable en static automáticamente
- D) Permitir que la variable se sobrescriba en subclases

4. ¿Puede una clase declarada como 'final' ser extendida (heredada)?

- A) No, una clase final no puede tener subclases
- B) Sí, siempre
- C) Solo si está en el mismo paquete
- D) Solo si implementa una interfaz

5. ¿Qué es un bloque de inicialización estático (static initializer block) y cuándo se ejecuta?

- A) Un bloque static { ... } que se ejecuta una única vez, cuando la clase se carga por primera vez en la JVM, antes de crear cualquier instancia
- B) Un bloque que se ejecuta cada vez que se crea una nueva instancia de la clase
- C) Un bloque que solo puede contener llamadas a métodos privados
- D) Un bloque que reemplaza al constructor por completo

6. ¿Cuál es el propósito principal del encapsulamiento en POO?

- A) Ocultar el estado interno de un objeto y exponer solo una interfaz pública controlada (getters/setters), protegiendo la integridad de los datos
- B) Permitir que todas las clases accedan directamente a los atributos de cualquier otra clase
- C) Eliminar la necesidad de constructores
- D) Convertir automáticamente todos los métodos en static

7. ¿Qué hace la palabra clave 'this' dentro de un constructor cuando se usa como this(...)?

- A) Invoca a otro constructor sobrecargado de la misma clase (constructor chaining), y debe ser la primera línea del constructor
- B) Invoca al constructor de la superclase
- C) Crea una nueva instancia adicional de la clase actual
- D) Es equivalente a static en ese contexto

8. ¿Qué diferencia hay entre 'String s = "hola";' y 'String s = new String("hola");' en cuanto al Pool de Strings?

- A) La asignación literal reutiliza (o crea) el objeto en el Pool de Strings; new String(...) fuerza la creación de un nuevo objeto en el heap, fuera del pool, aunque su contenido sea idéntico
- B) Ambas formas crean siempre el mismo objeto físico en memoria
- C) new String(...) es más eficiente porque evita el uso del pool
- D) El Pool de Strings solo aplica a partir de Java 17

9. Dado:

String a = new String("hola");

String b = "hola";

System.out.println(a == b);

System.out.println(a.equals(b));

¿Qué imprime?

```
String a = new String("hola");
String b = "hola";
System.out.println(a == b);
System.out.println(a.equals(b));
```

- A) false y true
- B) true y true
- C) false y false
- D) true y false

10. ¿Por qué se dice que el Pool de Strings ayuda a optimizar memoria en aplicaciones Java?

- A) Porque permite que múltiples referencias con el mismo valor literal compartan un único objeto en memoria, en lugar de crear un objeto duplicado por cada literal idéntico
- B) Porque comprime automáticamente el texto almacenado
- C) Porque libera memoria automáticamente cada vez que se usa un String
- D) Porque convierte todos los Strings en tipos primitivos

11. ¿Por qué StringBuilder es mutable mientras que String es inmutable?

- A) StringBuilder mantiene internamente un array de caracteres modificable que puede crecer y cambiar sin crear un nuevo objeto en cada operación, mientras que String nunca modifica su contenido interno una vez creado
- B) Porque StringBuilder no usa memoria heap
- C) Porque String y StringBuilder son en realidad la misma clase internamente
- D) Porque StringBuilder solo puede usarse dentro de bucles

12. En Java, cuando se pasa un objeto como argumento a un método, ¿qué es exactamente lo que se copia?

- A) Se copia el valor de la referencia (la dirección que apunta al objeto), no el objeto en sí; por eso Java se describe como 'siempre paso por valor', incluso con objetos
- B) Se copia el objeto completo, creando una nueva instancia independiente en el heap
- C) No se copia nada, se pasa el objeto original directamente sin ninguna referencia
- D) Depende de si el objeto implementa Cloneable

13. Dado este método:

```
void cambiar(String s){ s = s + " mundo"; }
```

...

```
String texto = "hola";
```

```
cambiar(texto);
```

```
System.out.println(texto);
```

¿Qué imprime?

```
void cambiar(String s){ s = s + " mundo"; }
```

```
String texto = "hola";  
cambiar(texto);  
System.out.println(texto);
```

- A) hola
- B) hola mundo
- C) mundo
- D) Error de compilación

14. ¿Por qué es importante sobrescribir correctamente el método equals() en una clase propia, en lugar de dejar la implementación heredada de Object?

- A) Porque la implementación de Object.equals() compara referencias (equivalente a ==), lo que casi nunca es lo que se quiere al comparar el contenido lógico de dos objetos de una clase propia
- B) Porque Java no permite comparar objetos de clases propias de ninguna otra manera
- C) Porque equals() en Object siempre lanza una excepción
- D) Porque solo las clases final pueden tener equals() sobrescrito

15. ¿Por qué la clase String es 'final' en Java?

- A) Para evitar que se cree una subclase que rompa las garantías de inmutabilidad y el comportamiento esperado del Pool de Strings, ya que una subclase podría sobrescribir métodos y alterar ese comportamiento
- B) Porque String no tiene ningún método que pueda sobrescribirse
- C) Porque 'final' es obligatorio para todas las clases que están en java.lang
- D) Porque de lo contrario ocuparía más memoria

16. ¿Cuándo se considera que un objeto es 'elegible' para el Garbage Collector?

- A) Cuando ya no existe ninguna referencia alcanzable desde el programa que apunte a ese objeto
- B) Cuando el objeto lleva más de 1 segundo sin usarse
- C) Cuando el objeto implementa la interfaz Cloneable
- D) Cuando el programador lo marca manualmente con la palabra clave delete

17. Dado:

Persona p1 = new Persona();

Persona p2 = p1;

p1 = null;

¿Es elegible para Garbage Collection el objeto Persona original?

```
Persona p1 = new Persona();  
Persona p2 = p1;  
p1 = null;
```

- A) No, porque p2 todavía mantiene una referencia alcanzable al mismo objeto
- B) Sí, porque p1 ya es null
- C) Sí, porque p2 es una copia distinta del objeto
- D) No se puede determinar sin ejecutar el programa

18. ¿Qué ocurre si dos objetos se referencian mutuamente entre sí (por ejemplo, objeto A referencia a B y B referencia a A), pero ninguna otra variable del programa referencia a ninguno de los dos?

- A) Ambos son elegibles para el Garbage Collector, ya que un recolector de basura moderno detecta ciclos de referencias sin alcanzabilidad externa (no basta con tener referencias entre sí)
- B) Nunca serán recolectados porque siempre habrá al menos una referencia entre ellos
- C) Java lanza una excepción en tiempo de ejecución al detectar el ciclo
- D) Solo uno de los dos objetos será elegible, nunca ambos

19. ¿Cuál es la diferencia entre aplicar 'final' a una variable de tipo primitivo y aplicarlo a una variable de tipo referencia (objeto)?

- A) En ambos casos impide reasignar la variable a otro valor u objeto; pero si es una referencia a un objeto mutable, el contenido interno del objeto referenciado sí puede seguir modificándose
- B) final en primitivos no tiene ningún efecto
- C) final en referencias impide modificar cualquier atributo del objeto apuntado
- D) final solo puede aplicarse a tipos primitivos, nunca a referencias

20. ¿Qué efecto tiene declarar un método como 'final' en una clase?

- A) Impide que las subclases sobrescriban (override) ese método específico, aunque sí pueden heredarlo y usarlo tal cual
- B) Impide que el método sea invocado más de una vez
- C) Convierte el método en static automáticamente
- D) Impide que el método reciba parámetros

21. ¿Qué hace el método intern() de la clase String?

- A) Busca en el Pool de Strings un objeto con el mismo contenido; si existe, devuelve la referencia a ese objeto del pool, y si no existe, lo añade al pool y devuelve esa referencia
- B) Convierte el String en un StringBuilder mutable
- C) Elimina el objeto String de la memoria inmediatamente
- D) Compara el String con otro usando distinción de mayúsculas y minúsculas

22. Dado:

String a = new String("java").intern();

String b = "java";

System.out.println(a == b);

¿Qué imprime?

```
String a = new String("java").intern();
String b = "java";
System.out.println(a == b);
```

- A) true
- B) false
- C) Error de compilación
- D) Depende de la JVM utilizada

23. ¿Cómo se convierte un StringBuilder a un objeto String una vez terminada la construcción del texto?

- A) Llamando al método toString() del StringBuilder
- B) Haciendo un cast directo (String) sb
- C) Con el método convert() de StringBuilder
- D) No es posible convertir StringBuilder a String

24. ¿Por qué usar StringBuilder en lugar de concatenar con '+' puede considerarse una optimización de memoria relevante para el examen de certificación?

- A) Porque evita la creación de múltiples objetos String intermedios desechables en el heap durante concatenaciones repetidas, reduciendo la presión sobre el Garbage Collector
- B) Porque StringBuilder no usa el heap en absoluto
- C) Porque StringBuilder convierte automáticamente el texto a minúsculas para ahorrar espacio
- D) Porque el operador '+' está prohibido en el examen de certificación

25. De los 6 usos de 'final' repasados (primitivos, objetos mutables, objetos inmutables, clases, métodos de instancia, métodos estáticos), ¿qué distingue a 'final' aplicado a una referencia de un objeto MUTABLE?

- A) Impide reasignar la referencia a otro objeto, pero permite modificar el contenido interno (estado) del objeto mutable al que apunta, a través de sus propios métodos
- B) Impide cualquier modificación, tanto de la referencia como del contenido interno del objeto
- C) Convierte automáticamente el objeto en inmutable
- D) No tiene ningún efecto si el objeto ya es mutable

26. ¿Qué significa que un método estático sea declarado 'final' respecto a las subclases?

- A) Impide que una subclase pueda 'ocultar' (hide) ese método estático definiendo uno propio con la misma firma
- B) Impide que el método estático pueda ser invocado más de una vez en el programa
- C) Convierte el método en un método de instancia automáticamente
- D) No tiene ningún efecto especial en métodos estáticos, solo aplica a métodos de instancia

27. ¿Cuál de las siguientes declaraciones de un método con varargs es sintácticamente inválida?

- A) void metodo(String... nombres, int extra) — varargs como primer parámetro seguido de otro parámetro
- B) void metodo(int extra, String... nombres) — varargs como último parámetro
- C) void metodo(String... nombres) — un único parámetro varargs
- D) void metodo() { } y luego sobrecargarlo con void metodo(String... nombres) — varargs en una sobrecarga distinta

28. ¿Cuántos parámetros varargs puede tener un mismo método como máximo?

- A) Uno solo
- B) Dos, si son de tipos diferentes
- C) Ilimitados, siempre que estén al final
- D) Depende de si el método es static

29. ¿Cuál es el orden de los modificadores de acceso en Java, desde el más restrictivo hasta el más permisivo?

- A) private → default (package) → protected → public
- B) public → protected → default → private
- C) protected → private → public → default
- D) default → private → protected → public

30. ¿A qué elementos del lenguaje se les puede aplicar un modificador de acceso como public, protected, default o private?

- A) A clases de nivel superior (con restricciones, ya que solo public o default aplican ahí), a métodos, a variables (campos) y a constructores
- B) Únicamente a variables locales dentro de métodos
- C) Únicamente a clases, nunca a métodos o campos
- D) Solo a métodos static, nunca a métodos de instancia

31. Si una variable se declara con acceso 'default' (sin modificador explícito) en una clase, ¿desde dónde puede accederse a ella?

- A) Únicamente desde clases ubicadas en el mismo paquete que la clase que la declara
- B) Desde cualquier clase de cualquier paquete, igual que public
- C) Únicamente desde la misma clase, igual que private
- D) Únicamente desde subclases, sin importar el paquete, igual que protected

32. Si la clase Classroom (en el paquete 'escuela') tiene un constructor 'private', ¿puede la clase School (en el mismo paquete 'escuela') crear instancias de Classroom con 'new Classroom()'?

- A) Sí, porque un constructor private es accesible desde cualquier código dentro de la misma clase donde fue declarado, y en este caso ambas clases están en el mismo archivo o se evalúa dentro del contexto de la propia clase Classroom; si School es una clase externa distinta, en realidad NO podría acceder a ese constructor
- B) Sí, siempre, sin ninguna restricción, porque están en el mismo paquete
- C) No, un constructor private nunca es accesible ni siquiera desde dentro de la misma clase
- D) Depende de si School extiende Classroom, sin importar el modificador del constructor

33. ¿Qué permite hacer un 'import estático' (static import) en Java, por ejemplo 'import static java.lang.Math.PI;'?

- A) Acceder directamente al miembro estático (PI en este caso) sin necesidad de calificarlo con el nombre de la clase (Math.PI), simplemente escribiendo PI
- B) Importar todas las clases del paquete java.lang automáticamente
- C) Convertir cualquier método de instancia en estático automáticamente
- D) Es idéntico a un import normal, sin ninguna diferencia funcional

34. ¿Cuál es la diferencia entre 'import static Clase.miembro;' (import estático explícito) e 'import static Clase.*;' (import estático con asterisco)?

- A) El explícito importa solo el miembro estático indicado; el de asterisco importa todos los miembros estáticos (campos y métodos) de esa clase
- B) El de asterisco solo importa métodos, nunca campos
- C) Ambos tienen exactamente el mismo efecto en cualquier caso
- D) El explícito solo funciona con métodos static void

35. ¿Cuándo se ejecutan los bloques de inicialización estática de una clase respecto a la creación de sus instancias?

- A) Se ejecutan una única vez, cuando la clase se carga por primera vez en la JVM, antes de que se cree la primera instancia (si es que se crea alguna)
- B) Se ejecutan cada vez que se crea una nueva instancia de la clase
- C) Se ejecutan solo si se invoca explícitamente un método estático adicional para activarlos
- D) Nunca se ejecutan automáticamente, deben llamarse manualmente

36. ¿Se lanza NullPointerException al invocar un método estático a través de una referencia de objeto que vale null, por ejemplo 'Chimp c = null; c.metodoEstatico();'?

- A) No, no se lanza NullPointerException, porque los métodos estáticos están asociados a la clase, no a la instancia; Java resuelve la llamada usando el tipo declarado de la referencia (Chimp), sin necesidad de acceder al objeto real
- B) Sí, siempre se lanza NullPointerException al usar una referencia null para llamar cualquier método, estático o no
- C) Depende de si el método estático tiene parámetros
- D) El código no compila si la referencia es null

37. ¿En qué contexto se menciona que los imports estáticos son particularmente comunes y útiles en la práctica?

- A) En frameworks de pruebas unitarias (testing), donde se usan frecuentemente métodos estáticos como assertEquals() sin tener que calificarlos con el nombre de la clase
- B) Únicamente en la declaración del método main de cualquier programa
- C) Solo en clases que implementan Serializable
- D) Únicamente en programación de bajo nivel con arrays de bytes

38. Si se pasa un `StringBuilder` como argumento a un método y dentro del método se invoca `sb.append("texto")`, ¿el cambio se refleja en el objeto original fuera del método?

- A) Sí, porque se copia el valor de la referencia (que apunta al mismo objeto en el heap), y `append()` modifica ese objeto mutable directamente a través de esa referencia compartida
- B) No, porque Java siempre crea una copia independiente del objeto al pasarlo como parámetro
- C) Solo si el `StringBuilder` se declara como final en el método que lo recibe
- D) Depende de si el método es `static` o de instancia

39. ¿Cuál es la regla obligatoria sobre la posición de `'this(...)'` cuando se usa para delegar a otro constructor de la misma clase?

- A) Debe ser la primera línea ejecutable dentro del constructor; no puede aparecer después de ninguna otra instrucción
- B) Puede colocarse en cualquier línea del constructor, sin restricción de posición
- C) Debe ser siempre la última línea del constructor
- D) Solo puede usarse dentro de métodos `static`, nunca dentro de constructores

40. ¿Se puede invocar un constructor como si fuera un método normal en algún punto del código, por ejemplo escribiendo `'this.MiClase();'` en medio de un método?

- A) No, los constructores no pueden invocarse como métodos regulares; solo pueden llamarse mediante `'new'` (para crear una instancia) o mediante `this(...)/super(...)` desde otro constructor
- B) Sí, siempre que el constructor sea `public`
- C) Sí, pero solo si el constructor no tiene parámetros
- D) Sí, siempre que se use dentro de un bloque `static`

41. En la resolución de sobrecarga de métodos (`overloading`), ¿cuál es el orden de prioridad que Java sigue al elegir qué versión invocar, de mayor a menor preferencia?

- A) 1) Coincidencia exacta con el tipo primitivo, 2) conversión a través del wrapper correspondiente (autoboxing), 3) coincidencia mediante una superclase como `Object`, 4) por último, la versión con `varargs`
- B) 1) `Varargs` primero, 2) luego wrapper, 3) luego primitivo exacto, 4) por último `Object`
- C) Java elige siempre la primera sobrecarga declarada en el código fuente, en orden de aparición
- D) El orden es aleatorio y no está definido por la especificación del lenguaje

42. ¿Cuál es la diferencia entre `'hiding'` (ocultamiento) y `'overriding'` (sobrescritura) al redefinir en una subclase un método con la misma firma que uno de la superclase?

- A) Si el método es `static`, la subclase lo 'oculta' (`hiding`) y la resolución de qué versión se ejecuta se determina en tiempo de compilación según el tipo de la referencia; si el método es de instancia, la subclase lo 'sobrescribe' (`overriding`) y la versión ejecutada se determina en tiempo de ejecución según el tipo real del objeto (polimorfismo)
- B) `Hiding` y `overriding` son sinónimos exactos sin ninguna diferencia práctica
- C) `Hiding` solo aplica a atributos, nunca a métodos
- D) `Overriding` solo puede hacerse con métodos `static`, nunca con métodos de instancia

43. ¿Qué es la covarianza en tipos de retorno al sobrescribir un método en una subclase?

- A) Permite que el método sobrescrito en la subclase declare un tipo de retorno que sea un subtipo del tipo de retorno declarado en el método original de la superclase, en lugar de exigir exactamente el mismo tipo
- B) Permite que el método sobrescrito reciba parámetros de cualquier tipo sin restricción
- C) Obliga a que el tipo de retorno sea siempre exactamente idéntico, sin ninguna flexibilidad
- D) Solo aplica a métodos que devuelven tipos primitivos, nunca a objetos

44. Si una subclase no invoca explícitamente a super(...) en su constructor, ¿qué hace el compilador automáticamente?

- A) Inserta implícitamente una llamada a super() (sin argumentos) como primera línea del constructor, invocando el constructor sin parámetros de la superclase
- B) No hace nada, y el objeto de la superclase simplemente no se inicializa
- C) Genera un error de compilación siempre que no se invoque super() explícitamente
- D) Invoca automáticamente el constructor de la superclase que tenga más parámetros

45. ¿Qué error de compilación ocurre si la superclase NO tiene un constructor sin argumentos (solo tiene uno con parámetros), y la subclase no invoca explícitamente a super(argumentos)?

- A) Error de compilación, porque el compilador intenta insertar automáticamente 'super();' (sin argumentos), que no existe en la superclase en este caso
- B) No hay ningún error, Java crea automáticamente un constructor sin argumentos para la superclase
- C) Solo genera una advertencia (warning), pero el código compila igual
- D) El error solo ocurre en tiempo de ejecución, no en compilación

46. Al sobrescribir un método en una subclase, ¿se puede reducir su nivel de acceso respecto al método original de la superclase (por ejemplo, de protected a private)?

- A) No, el método sobrescrito solo puede mantener el mismo nivel de acceso o aumentarlo (hacerlo más visible), nunca reducirlo
- B) Sí, se puede reducir libremente el nivel de acceso sin ninguna restricción
- C) Solo se puede reducir si el método original era public
- D) Depende de si la subclase está en el mismo paquete, sin ninguna otra regla

47. ¿Se heredan los métodos privados de una superclase hacia sus subclases?

- A) No, los métodos private no se heredan; por lo tanto, un método con la misma firma en la subclase no lo 'sobrescribe' (override), sino que simplemente es un método nuevo e independiente
- B) Sí, se heredan igual que cualquier método protected o public
- C) Se heredan solo si la subclase está en el mismo paquete
- D) Se heredan solo si el método private es también static

48. ¿Cuáles son características típicas que debe cumplir una clase para considerarse inmutable en Java?

- A) Declarar sus campos como private y final, no proveer métodos setter, inicializar todos los campos en el constructor, y si tiene campos de referencia a objetos mutables, evitar exponer referencias directas a ellos (por ejemplo, devolviendo copias defensivas)
- B) Simplemente marcar la clase como 'final', sin ninguna otra consideración sobre sus campos
- C) Que todos sus métodos sean static
- D) Que la clase no tenga ningún constructor definido

49. En un ejemplo que combina varargs, constructores sobrecargados y métodos estáticos, ¿por qué es importante identificar primero cuál constructor específico se invoca antes de analizar el resto del comportamiento del objeto?

- A) Porque el constructor invocado determina qué inicializaciones específicas ocurren (valores asignados a los campos, posibles delegaciones con `this()`, etc.), lo cual afecta directamente el estado inicial del objeto y por tanto el resultado de cualquier análisis posterior de su comportamiento
- B) Porque en Java el orden de los constructores no afecta en nada al estado final del objeto
- C) Porque los métodos estáticos siempre se ejecutan antes que cualquier constructor, sin excepción
- D) Porque varargs impide que un objeto tenga más de un constructor

07. Herencia, Polimorfismo y Casting

1. ¿Qué es 'overriding' (sobrescritura) de métodos?

- A) Redefinir en una subclase un método con la misma firma que el de la superclase, cambiando su implementación
- B) Definir varios métodos con el mismo nombre pero distintos parámetros en la misma clase
- C) Declarar un método como private en una subclase
- D) Llamar a un método estático desde una instancia

2. ¿Qué palabra clave permite invocar el constructor o método de la superclase desde una subclase?

- A) super
- B) this
- C) parent
- D) extends

3. Dado:

```
class Animal { void sonido(){ System.out.println("..."); } }
```

```
class Perro extends Animal { void sonido(){ System.out.println("Guau"); } }
```

```
Animal a = new Perro();
```

```
a.sonido();
```

¿Qué se imprime?

```
class Animal { void sonido(){ System.out.println("..."); } }  
class Perro extends Animal { void sonido(){ System.out.println("Guau"); } }  
Animal a = new Perro();  
a.sonido();
```

- A) Guau
- B) ...
- C) Error de compilación
- D) Guau seguido de ...

4. Java permite herencia múltiple de clases pero sí de interfaces. ¿Verdadero o falso?

- A) Falso: Java NO permite herencia múltiple de clases (solo una superclase), pero SÍ permite implementar múltiples interfaces
- B) Verdadero, ambas cosas son posibles sin restricción
- C) Falso, tampoco permite implementar múltiples interfaces
- D) Verdadero, pero solo desde Java 21

5. ¿Qué modificador de acceso permite que un miembro sea visible solo dentro del mismo paquete y en subclases de otros paquetes?

- A) protected
- B) private
- C) default (sin modificador)
- D) public

6. ¿Puede un método sobrescrito (override) en una subclase declarar una excepción checked más amplia que la del método original de la superclase?

- A) No, el método sobrescrito solo puede declarar la misma excepción checked, una subclase de ella, o ninguna; nunca una más amplia
- B) Sí, siempre puede declarar cualquier excepción checked adicional
- C) Solo si el método es static
- D) Solo si la superclase es una interfaz

7. ¿Qué es el 'binding estático' (static binding) frente al 'binding dinámico' (dynamic binding)?

- A) El binding estático resuelve en compilación qué método se llama (métodos static, private, final, y sobrecarga); el binding dinámico resuelve en tiempo de ejecución (métodos sobrescritos vía polimorfismo)
- B) Ambos se resuelven siempre en tiempo de compilación
- C) El binding dinámico solo aplica a atributos, nunca a métodos
- D) El binding estático solo existe en interfaces

8. ¿Puede una clase en Java extender (heredar de) más de una clase directamente?

- A) No, Java solo permite herencia simple de clases (una única superclase directa mediante extends); la herencia múltiple de comportamiento se logra mediante la implementación de múltiples interfaces
- B) Sí, una clase puede extender tantas clases como se necesite separándolas con comas
- C) Sí, pero solo hasta un máximo de dos superclases
- D) No, y tampoco se permite implementar más de una interfaz

9. ¿De qué clase derivan, directa o indirectamente, todas las clases en Java, incluso si no se especifica explícitamente ningún 'extends'?

- A) java.lang.Object
- B) java.lang.Class
- C) java.lang.Super
- D) No derivan de ninguna clase común si no se especifica extends

10. ¿Cuál es una diferencia clave entre una clase abstracta y una interfaz en cuanto a los campos que pueden declarar?

- A) Una clase abstracta puede declarar campos de instancia normales (con cualquier modificador y mutables); una interfaz solo puede declarar constantes, que son implícitamente public static final
- B) Una interfaz puede declarar campos mutables de instancia libremente, igual que una clase abstracta
- C) Ninguna de las dos puede declarar ningún tipo de campo
- D) Una clase abstracta no puede declarar ningún campo bajo ninguna circunstancia

11. ¿Qué modificadores son implícitos en un método abstracto declarado dentro de una interfaz (sin cuerpo, sin la palabra 'default' ni 'static')?

- A) public y abstract, aunque no se escriban explícitamente en el código
- B) private y final
- C) protected y static
- D) package-private (default) y final

12. En un problema que combina miembros estáticos, bloques de inicialización y herencia entre una superclase y una subclase, ¿en qué orden se ejecutan los bloques static cuando se crea por primera vez una instancia de la subclase?

- A) Primero los bloques static de la superclase (de arriba hacia abajo si hay varios), y después los bloques static de la subclase, todo esto antes de cualquier bloque de instancia o constructor
- B) Primero los bloques static de la subclase, y después los de la superclase
- C) Los bloques static de la superclase nunca se ejecutan si la subclase también tiene bloques static propios
- D) El orden depende únicamente de en qué paquete estén ubicadas ambas clases

13. ¿Qué papel cumple 'var' al declarar una variable local que luego se usará para hacer casting a otro tipo, considerando que el compilador infiere el tipo real de var en tiempo de compilación?

- A) El tipo inferido para 'var' es el tipo real y concreto de la expresión usada para inicializarla; cualquier casting posterior se analiza contra ese tipo inferido exactamente igual que si se hubiera escrito el tipo explícitamente
- B) 'var' siempre se infiere como Object, por lo que cualquier casting posterior sería válido sin restricciones
- C) 'var' impide que la variable sea casteada a cualquier otro tipo en el futuro
- D) El tipo de 'var' se determina en tiempo de ejecución, no en compilación

14. ¿Qué es el 'upcasting' en Java y por qué generalmente es seguro y no requiere cast explícito?

- A) Es convertir la referencia de un objeto hacia un tipo de su superclase (o interfaz que implementa); es seguro porque toda subclase 'es un' tipo de su superclase, por lo que Java lo permite implícitamente sin necesidad de escribir el cast
- B) Es convertir un objeto hacia un tipo completamente no relacionado, sin ninguna conexión de herencia
- C) Es lo mismo que el downcasting, solo cambia el nombre según el contexto
- D) Requiere siempre un cast explícito porque puede fallar en tiempo de ejecución

15. ¿Por qué el 'downcasting' (convertir una referencia de un tipo general a un tipo más específico) requiere un cast explícito y puede fallar en tiempo de ejecución?

- A) Porque no todo objeto de un tipo general es necesariamente una instancia del subtipo específico al que se quiere convertir; si el objeto real no es de ese subtipo (o uno compatible), se lanza ClassCastException en tiempo de ejecución
- B) Porque el downcasting siempre falla, sin excepción, y nunca debería usarse
- C) Porque Java no permite downcasting bajo ninguna circunstancia
- D) Porque requiere que la clase de destino sea abstracta

16. Dado:

Animal a = new Perro();

Gato g = (Gato) a;

Suponiendo que Gato y Perro son ambas subclases de Animal pero no están relacionadas entre sí, ¿qué ocurre al ejecutar la segunda línea?

```
Animal a = new Perro();  
Gato g = (Gato) a;
```

- A) Se lanza ClassCastException en tiempo de ejecución, porque el objeto real referenciado por 'a' es un Perro, no un Gato, y no existe relación de herencia entre Perro y Gato
- B) Compila y se ejecuta sin ningún problema, convirtiendo el Perro en un Gato
- C) No compila, porque el compilador detecta el error antes de ejecutar
- D) Se asigna null a la variable g sin lanzar ninguna excepción

17. ¿Puede declararse una variable de referencia usando el tipo de una interfaz, en lugar de una clase concreta, por ejemplo 'Volador v = new Aguila();' donde Volador es una interfaz implementada por Aguila?

- A) Sí, es completamente válido y común: la variable puede tener como tipo declarado cualquier interfaz que la clase concreta implemente, permitiendo un código más flexible y desacoplado de implementaciones concretas
- B) No, las variables de referencia solo pueden declararse con el tipo exacto de la clase concreta
- C) Solo es válido si la interfaz no tiene ningún método
- D) Solo es válido si la clase Aguila es abstracta

18. ¿Qué relación existe entre el casting y el polimorfismo al invocar un método sobrescrito a través de una referencia con upcast, por ejemplo 'Animal a = new Perro(); a.hacerSonido();' donde Perro sobrescribe hacerSonido()?

- A) Se ejecuta la versión sobrescrita en Perro (el tipo real del objeto), independientemente de que la referencia 'a' esté declarada como Animal, gracias al polimorfismo en tiempo de ejecución (dynamic dispatch)
- B) Se ejecuta siempre la versión de Animal, ya que la referencia está declarada con ese tipo
- C) Se produce un error de compilación porque Animal no puede invocar métodos de Perro
- D) Depende de si el método hacerSonido() es static o no en Animal

19. ¿Cómo puede usarse el operador instanceof para evitar una ClassCastException antes de realizar un downcasting?

- A) Comprobando primero con 'if (objeto instanceof TipoDestino)' que el objeto realmente es (o hereda de) el tipo al que se quiere castear, y solo entonces realizar el cast dentro de ese bloque, garantizando que sea seguro
- B) instanceof no tiene ninguna relación con evitar errores de casting
- C) Usando instanceof únicamente después de realizar el cast, nunca antes
- D) instanceof solo puede usarse con tipos primitivos, no con objetos

20. En el ejemplo con Bird, Parrot y Macaw (donde Parrot y Macaw extienden Bird y sobrescriben fly()), si se tiene 'Bird b = new Macaw();' y se invoca 'b.fly()'; ¿qué versión de fly() se ejecuta?

- A) La versión sobrescrita en Macaw, ya que el polimorfismo determina la implementación a ejecutar según el tipo real del objeto en tiempo de ejecución, sin importar que la referencia sea de tipo Bird
- B) La versión de Bird, porque la referencia está declarada como Bird
- C) La versión de Parrot, porque Macaw hereda indirectamente de Parrot en este ejemplo
- D) Se ejecutan las tres versiones (Bird, Parrot y Macaw) en cadena

21. ¿Por qué es importante distinguir entre 'la variable de referencia' y 'el objeto' al analizar el efecto del casting sobre el comportamiento de un programa?

- A) Porque el casting cambia el tipo con el que se accede al objeto a través de esa referencia (afectando qué miembros son visibles en tiempo de compilación), pero nunca cambia el objeto real en el heap ni su comportamiento polimórfico real en métodos sobrescritos
- B) Porque el casting siempre crea una copia física distinta del objeto original en memoria
- C) Porque la variable de referencia y el objeto son términos completamente intercambiables sin ninguna diferencia práctica
- D) Porque el casting modifica permanentemente la clase real del objeto en el heap

22. Si se hace upcast de una referencia (por ejemplo, de Macaw a Bird) y luego se intenta acceder a un miembro estático a través de esa referencia casteada, ¿qué determina qué versión del miembro estático se usa?

- A) El tipo declarado de la referencia en el punto de acceso (Bird en este caso), ya que los miembros estáticos se resuelven en tiempo de compilación según el tipo de la referencia, no según el tipo real del objeto
- B) El tipo real del objeto en tiempo de ejecución (Macaw), igual que con métodos de instancia sobrescritos
- C) Siempre se usa la versión de la clase más específica, sin importar el casting
- D) Los miembros estáticos no pueden accederse a través de una referencia de otro tipo

23. ¿Qué relación se menciona entre los records (Java 14+) y las clases inmutables?

- A) Los records ofrecen una forma concisa de crear clases inmutables por defecto (campos final, sin setters, con equals/hashCode/toString generados), ahorrando el código repetitivo (boilerplate) que normalmente se escribe a mano para lograr inmutabilidad
- B) Los records son mutables por defecto, a diferencia de las clases inmutables tradicionales
- C) Los records reemplazan por completo el concepto de clases inmutables, haciendo obsoleto ese patrón
- D) No existe ninguna relación entre records y clases inmutables

24. ¿En qué se diferencia crear una clase anónima que implementa una interfaz, respecto a crear una clase con nombre que usa 'implements' de forma tradicional?

- A) La clase anónima combina la declaración y la instanciación en un único paso, sin nombre propio ni posibilidad de reutilizarse en otro lugar del código; la clase con nombre (implements) se declara por separado y puede instanciarse y reutilizarse cuantas veces se necesite
- B) No existe ninguna diferencia funcional entre ambos enfoques
- C) Una clase anónima nunca puede implementar más de un método de la interfaz
- D) Una clase con nombre tradicional no puede implementar interfaces, solo clases anónimas pueden

25. ¿Puede crearse una clase anónima que extienda una clase declarada como 'final'?

- A) No, una clase final no puede ser extendida por ninguna subclase, incluida una clase anónima, ya que 'final' prohíbe la herencia por completo
- B) Sí, las clases anónimas están exentas de la restricción de 'final'
- C) Solo si la clase final no tiene métodos
- D) Solo si la clase anónima no sobrescribe ningún método

26. ¿Puede una clase anidada estática (static nested class) acceder directamente a los miembros de instancia (no static) de la clase externa que la contiene?

- A) No, al no estar asociada a ninguna instancia particular de la clase externa, una clase anidada estática solo puede acceder a los miembros static de la clase externa, no a los de instancia
- B) Sí, tiene acceso completo a todos los miembros de instancia de la clase externa igual que una clase interna no estática
- C) Solo puede acceder si se declara explícitamente como 'inner' además de static
- D) Solo puede acceder si la clase externa no tiene constructor

27. En el contexto del polimorfismo, ¿qué determina la JVM en tiempo de ejecución al invocar un método de instancia sobrescrito a través de una referencia?

- A) Determina cuál implementación del método ejecutar basándose en el tipo real del objeto al que apunta la referencia, no en el tipo declarado de la variable que la contiene
- B) Determina la implementación basándose únicamente en el tipo declarado de la variable de referencia, ignorando el objeto real
- C) Determina la implementación de forma aleatoria entre todas las disponibles en la jerarquía
- D) Siempre ejecuta la implementación de la superclase más alta en la jerarquía, sin importar las sobrescrituras

28. ¿Pueden los métodos definidos dentro de un enum comportarse de forma distinta para cada constante, similar al polimorfismo entre subclases?

- A) Sí, mediante un método abstracto declarado en el enum, cada constante puede proporcionar su propio cuerpo de implementación (constant-specific class body), logrando un comportamiento polimórfico por constante
- B) No, todos los métodos de un enum deben comportarse exactamente igual para todas sus constantes, sin excepción
- C) Solo es posible si el enum tiene una única constante declarada
- D) Los enums no pueden declarar métodos bajo ninguna circunstancia

29. En un record `record Platypus(String name, double length) {}`, ¿qué genera automáticamente el compilador respecto al constructor y los métodos de acceso?

- A) Genera un constructor canónico que recibe 'name' y 'length' en ese orden y los asigna a los campos correspondientes; y genera métodos accessor llamados exactamente 'name()' y 'length()' (sin prefijo 'get'), que simplemente retornan el valor de cada campo
- B) Genera getters con prefijo tradicional 'getName()' y 'getLength()', como en un JavaBean clásico
- C) No genera ningún constructor, debe escribirse manualmente incluso para records simples
- D) Genera setters 'setName()' y 'setLength()' además de los getters

30. Para ordenar una `List<Platypus>` por el atributo 'length' usando `Collections.sort()`, ¿cuál de las siguientes formas usando `Comparator` es válida y moderna?

- A) `Collections.sort(lista, Comparator.comparingDouble(Platypus::length));`
- B) `Collections.sort(lista);` sin especificar ningún `Comparator`, ya que `Platypus` es un record
- C) `Collections.sort(lista, Platypus.length);`
- D) `Collections.sortByField(lista, "length");`

31. ¿Por qué, al sobrescribir un método genérico heredado en una subclase concreta que fija el tipo parametrizado (por ejemplo, una subclase que extiende `Contenedor<String>`), el método sobrescrito debe usar exactamente 'String' en lugar de repetir el nombre genérico original de la superclase?

- A) Porque una vez que la subclase fija el argumento de tipo (`Contenedor<String>`), el compilador sustituye el parámetro de tipo genérico por 'String' en todos los métodos heredados de esa jerarquía; el método sobrescrito debe coincidir con esa firma ya resuelta, no con el parámetro de tipo genérico original sin resolver
- B) Porque las subclases nunca pueden sobrescribir métodos genéricos de sus superclases
- C) Porque 'String' y el parámetro de tipo genérico original son completamente intercambiables sin ninguna relación con la firma esperada
- D) Porque los métodos genéricos no pueden heredarse en ninguna circunstancia

32. ¿Cuál es la diferencia fundamental entre sobrecarga (overloading) y sobrescritura (override) de métodos en cuanto a la firma del método?

- A) La sobrecarga implica métodos con el MISMO nombre pero DISTINTA lista de parámetros (en la misma clase o jerarquía); la sobrescritura implica un método en una subclase con EXACTAMENTE la misma firma (nombre y parámetros) que uno heredado de la superclase, cambiando su implementación
- B) Ambos términos son exactamente sinónimos, sin ninguna diferencia real
- C) La sobrecarga solo puede ocurrir entre una clase y su superclase, nunca dentro de la misma clase
- D) La sobrescritura permite cambiar libremente la cantidad de parámetros del método original

33. ¿Cuál es una forma común y concisa de iterar sobre un Map usando entrySet() combinado con una referencia a método o lambda?

- A) `mapa.entrySet().forEach(entrada -> System.out.println(entrada.getKey() + "=" + entrada.getValue()));`
- B) `mapa.forEach(System.out::entrySet);`
- C) `mapa.entrySet().stream().collect(System.out::println);`
- D) `mapa.keySet().entrySet().forEach(System.out::println);`

08. Interfaces y Clases Abstractas

1. ¿Qué permiten los métodos 'default' en una interfaz (desde Java 8)?

- A) Proveer una implementación concreta dentro de la interfaz que las clases implementadoras pueden usar o sobrescribir
- B) Hacer que el método sea abstracto obligatoriamente
- C) Convertir la interfaz en una clase abstracta automáticamente
- D) Ejecutarse solo si la clase implementadora es static

2. ¿Puede una clase abstracta tener constructores?

- A) Sí, aunque no se pueda instanciar directamente, sus constructores se ejecutan al instanciar subclases concretas
- B) No, nunca
- C) Solo si es final
- D) Solo si no tiene métodos abstractos

3. ¿Qué es una clase anónima en Java?

- A) Una clase sin nombre que se define e instancia en un único paso, útil para implementar una interfaz o extender una clase de forma puntual
- B) Una clase que no puede tener métodos
- C) Un tipo de enum sin constantes
- D) Una clase que solo puede usarse dentro de records

4. ¿Cuántos métodos abstractos puede tener como máximo una interfaz funcional?

- A) Exactamente uno (además de métodos default/static)
- B) Cero
- C) Máximo dos
- D) Ilimitados

5. ¿Puede una interfaz declarar campos (variables)?

- A) Sí, pero son implícitamente public static final (constantes), no se pueden tener campos de instancia mutables
- B) No, las interfaces nunca pueden declarar variables
- C) Sí, y son mutables como en una clase normal
- D) Solo si la interfaz es sealed

6. ¿Qué son los métodos 'static' dentro de una interfaz (desde Java 8)?

- A) Métodos con implementación que pertenecen a la interfaz misma y se invocan como NombreInterfaz.metodo(), no se heredan a las clases implementadoras
- B) Métodos que deben ser sobrescritos obligatoriamente por cada clase implementadora
- C) Métodos abstractos adicionales
- D) Un sinónimo de los métodos default

7. Si una clase implementa dos interfaces que definen el mismo método default con distinta implementación, ¿qué ocurre?

- A) La clase debe sobrescribir explícitamente ese método para resolver la ambigüedad, o el código no compila
- B) Java elige automáticamente la implementación de la primera interfaz declarada
- C) Se produce un error solo en tiempo de ejecución, no en compilación
- D) Ambas implementaciones se ejecutan en secuencia automáticamente

8. ¿Cuál es el propósito de un método 'private' dentro de una interfaz (introducido en Java 9)?

- A) Permitir compartir código común entre varios métodos default (o static) de la misma interfaz, sin exponer ese método como parte del contrato público de la interfaz
- B) Convertir automáticamente la interfaz en una clase abstracta
- C) Obligar a que las clases implementadoras definan su propia versión de ese método
- D) Reemplazar completamente a los métodos default

9. ¿Puede una clase implementadora de una interfaz sobrescribir un método static declarado en esa interfaz?

- A) No, los métodos static de una interfaz no se heredan ni pueden sobrescribirse por las clases implementadoras; se invocan directamente sobre el nombre de la interfaz
- B) Sí, cualquier clase implementadora puede sobrescribir libremente los métodos static de la interfaz
- C) Solo puede sobrescribirse si la clase implementadora es abstracta
- D) Solo puede sobrescribirse si el método static no tiene cuerpo

10. ¿Cuál es una ventaja clave de usar un record en lugar de una clase tradicional para representar datos inmutables simples?

- A) Se reduce drásticamente el código repetitivo (boilerplate): el compilador genera automáticamente el constructor canónico, accessors, equals(), hashCode() y toString(), en lugar de tener que escribirlos manualmente
- B) Los records permiten mutar sus campos después de creados, a diferencia de las clases tradicionales
- C) Los records no pueden implementar interfaces, a diferencia de las clases tradicionales
- D) Los records son siempre más rápidos en tiempo de ejecución que cualquier clase tradicional equivalente

11. ¿En qué se diferencia un campo static de un campo de instancia en cuanto al número de copias que existen en memoria?

- A) Un campo static tiene una única copia compartida por todas las instancias de la clase; un campo de instancia tiene una copia independiente para cada objeto creado
- B) Un campo static tiene una copia independiente por cada instancia, igual que un campo de instancia
- C) Un campo de instancia solo existe si la clase tiene al menos un método static
- D) No hay ninguna diferencia entre ambos tipos de campos en cuanto a memoria

12. ¿Cuándo es apropiado usar una clase abstracta en lugar de una interfaz para modelar una jerarquía?

- A) Cuando se necesita compartir estado (campos de instancia) y/o código común concreto entre subclases relacionadas, además de posiblemente declarar métodos abstractos que cada subclase deba implementar de forma distinta
- B) Únicamente cuando la clase no necesita tener ningún método
- C) Siempre que se quiera evitar cualquier tipo de herencia
- D) Solo cuando el lenguaje no permite usar interfaces en ese contexto

10. Enums

1. ¿Pueden los enums en Java tener constructores, campos y métodos?

- A) Sí, un enum es una clase especial que puede tener constructores (siempre implícitamente privados), campos y métodos, incluso implementar interfaces
- B) No, un enum solo puede contener constantes simples
- C) Solo puede tener métodos static
- D) Solo puede implementar una interfaz si no tiene constructor

2. ¿Qué método permite obtener la posición ordinal (basada en 0) de una constante enum?

- A) ordinal()
- B) position()
- C) index()
- D) value()

3. ¿Se puede usar un enum en un switch statement/expression sin prefijo del nombre de la clase en los 'case'?

- A) Sí, solo se usa el nombre de la constante (ej. case LUNES ->)
- B) No, siempre hay que escribir DiaSemana.LUNES
- C) Solo si el enum es public
- D) Solo en switch clásico, no en switch expression

4. ¿Qué método permite obtener todas las constantes de un enum como un array?

- A) NombreEnum.values()
- B) NombreEnum.getAll()
- C) NombreEnum.list()
- D) NombreEnum.entries()

5. ¿Puede un enum implementar un cuerpo de clase distinto para cada constante (constant-specific class body)?

- A) Sí, cada constante puede sobrescribir un método abstracto declarado en el enum con su propia implementación
- B) No, todas las constantes deben compartir exactamente el mismo comportamiento
- C) Solo si el enum no tiene constructor
- D) Solo es posible con dos constantes como máximo

6. ¿Qué excepción se lanza si se llama a valueOf() de un enum con un nombre que no corresponde a ninguna constante?

- A) IllegalArgumentException
- B) NullPointerException
- C) ClassCastException
- D) NoSuchFieldException

7. En un ejemplo donde una clase sellada 'SilClass' se declara con 'permits Armadillo', ¿qué ocurre si se intenta crear otra clase 'Perro' que también intente extender SilClass?

- A) Error de compilación, porque solo la clase o clases explícitamente listadas en la cláusula 'permits' (en este caso, únicamente Armadillo) están autorizadas a extender la clase sellada
- B) Compila sin problema, cualquier clase puede extender una clase sellada libremente
- C) Solo genera una advertencia, pero el programa compila y funciona igual
- D) Se permite solo si Perro está en el mismo paquete que SilClass, sin importar la cláusula permits

8. ¿Es correcto decir que un enum en Java 'funciona como una clase' con instancias fijas predefinidas?

- A) Sí, un enum es efectivamente un tipo especial de clase donde las instancias (constantes) están predefinidas y son fijas, pudiendo tener constructores, atributos y métodos igual que una clase normal
- B) No, un enum es fundamentalmente distinto a una clase y no puede tener constructores ni métodos bajo ninguna circunstancia
- C) Un enum solo puede tener constantes numéricas simples, sin ningún comportamiento adicional
- D) Un enum es solo un conjunto de constantes String, sin relación con el concepto de clase

9. ¿Cuándo se vuelve obligatorio el punto y coma (;) después de la lista de constantes en la declaración de un enum?

- A) Cuando el enum incluye elementos adicionales después de las constantes, como constructores, atributos o métodos; en ese caso el punto y coma separa la lista de constantes del resto del cuerpo de la clase
- B) Siempre es obligatorio en cualquier enum, sin excepción
- C) Nunca es obligatorio, es simplemente una preferencia de estilo sin efecto en la compilación
- D) Solo es obligatorio si el enum implementa una interfaz

10. ¿Qué papel juega la encapsulación (campos private con accesores controlados) al diseñar un enum con atributos, como por ejemplo un enum Planeta con un atributo 'masa'?

- A) Permite proteger el estado interno de cada constante del enum (como la masa de cada planeta) exponiendo solo los métodos necesarios para leerlo, evitando modificaciones no controladas del atributo una vez inicializado
- B) La encapsulación no aplica a los enums porque sus atributos son siempre públicos y mutables por defecto
- C) Impide que el enum tenga más de una constante
- D) Obliga a que el atributo sea static en lugar de un campo de instancia por constante

11. ¿Puede el constructor de un enum declararse con acceso 'public' o 'protected'?

- A) No, el constructor de un enum solo puede ser private (o package-private/default, que en la práctica tiene un efecto equivalente al estar restringido al propio enum); nunca puede ser public ni protected
- B) Sí, puede ser public sin ninguna restricción especial
- C) Sí, puede ser protected si el enum implementa una interfaz
- D) Los enums nunca pueden tener constructores de ningún tipo

12. ¿Es correcto decir que un enum siempre tiene, como mínimo, un constructor privado implícito si no se declara ninguno explícitamente?

- A) Sí, si no se declara ningún constructor explícito, el compilador genera uno implícito sin argumentos y con acceso private, igual que ocurre conceptualmente en una clase normal, pero respetando la restricción de que nunca puede ser public
- B) No, un enum nunca tiene ningún constructor si no se declara uno explícitamente
- C) El constructor implícito de un enum siempre es public por defecto
- D) Un enum no puede tener ningún constructor, ni implícito ni explícito

13. ¿Cuándo es estrictamente necesario colocar un punto y coma (;) después de la lista de constantes en un enum?

- A) Cuando después de las constantes se añade cualquier otro miembro (constructor, campos, métodos); en ese caso el punto y coma es obligatorio para delimitar dónde termina la lista de constantes
- B) Siempre es obligatorio en cualquier enum, incluso si solo tiene constantes simples
- C) Nunca es obligatorio bajo ninguna circunstancia
- D) Solo es obligatorio si el enum tiene exactamente una constante

14. ¿Qué elementos genera automáticamente el compilador para un record, evitando que el programador los escriba manualmente?

- A) El constructor canónico, los métodos de acceso (accessors) para cada componente, y las sobrescrituras de equals(), hashCode() y toString()
- B) Solo el método toString(), el resto debe escribirse manualmente
- C) Únicamente setters para cada componente, sin ningún otro método
- D) Solamente el constructor, sin generar accessors ni equals/hashCode/toString

15. ¿Puede un record declarar campos static adicionales, distintos de sus componentes?

- A) Sí, un record puede declarar campos static (compartidos por la clase, no por instancia), aunque no puede declarar campos de instancia adicionales fuera de sus componentes
- B) No, un record no puede declarar ningún campo adicional de ningún tipo
- C) Sí, puede declarar tanto campos static como campos de instancia adicionales libremente
- D) Solo puede declarar campos static si el record implementa Serializable

16. ¿Cuál es un error de compilación común al intentar instanciar directamente una clase abstracta con 'new', por ejemplo 'new Animal();' donde Animal es abstract?

- A) El compilador rechaza la instanciación directa, ya que una clase abstracta no puede instanciarse por sí misma; solo se pueden crear instancias de sus subclases concretas (no abstractas)
- B) Compila sin problema y crea una instancia genérica de la clase abstracta
- C) Solo falla si la clase abstracta tiene métodos abstractos, pero compila si no los tiene
- D) Falla únicamente en tiempo de ejecución, no en compilación

17. ¿Qué error de compilación ocurre si una clase concreta (no abstracta) extiende una clase abstracta pero no implementa todos los métodos abstractos heredados?

- A) Error de compilación: una clase concreta (no declarada como abstract) está obligada a proveer implementación para todos los métodos abstractos heredados; si no lo hace, debe declararse ella misma como abstract
- B) No hay ningún error, los métodos abstractos no implementados simplemente no estarán disponibles en esa clase
- C) Solo genera una advertencia, pero el programa compila y se ejecuta igual
- D) El error solo ocurre en tiempo de ejecución al invocar el método no implementado

18. ¿Qué error de compilación surge si una clase intenta implementar una interfaz sellada (sealed) sin estar listada en su cláusula 'permits'?

- A) Error de compilación, ya que solo las clases explícitamente autorizadas en la cláusula permits de la interfaz sellada pueden implementarla
- B) No hay ningún error, cualquier clase puede implementar una interfaz sellada libremente
- C) Solo se genera una advertencia sin impedir la compilación
- D) El error solo se detecta en tiempo de ejecución al crear una instancia

11. Records

1. ¿Son mutables los componentes de un record?

- A) No, todos los campos de un record son implícitamente final e inmutables
- B) Sí, se pueden reasignar con setters generados
- C) Solo si se declaran como var
- D) Depende de si el record implementa Serializable

2. ¿Qué es un 'constructor compacto' en un record?

- A) Una forma de escribir el constructor canónico sin repetir la lista de parámetros, útil para validar o normalizar valores antes de la asignación implícita
- B) Un constructor que no acepta parámetros
- C) Un constructor privado obligatorio en todo record
- D) Un método estático que reemplaza al constructor

3. ¿Puede un record implementar interfaces?

- A) Sí, un record puede implementar una o varias interfaces, igual que una clase normal
- B) No, los records no pueden implementar ninguna interfaz
- C) Solo pueden implementar Comparable
- D) Solo pueden implementar Serializable

4. ¿Puede un record declarar campos de instancia adicionales fuera de los componentes definidos en su cabecera?

- A) No, un record solo puede tener los campos correspondientes a sus componentes declarados; sí puede tener campos static adicionales
- B) Sí, puede declarar cualquier campo de instancia adicional libremente
- C) Sí, pero solo si son públicos
- D) Sí, pero solo si el record es sealed

5. ¿Qué genera el compilador si defines explícitamente el accessor de un componente de un record, por ejemplo `public int x() { return x * -1; }`?

- A) Usa la implementación personalizada en lugar del accessor generado automáticamente, permitiendo modificar qué se devuelve
- B) Produce un error de compilación porque no se puede sobrescribir un accessor
- C) Ignora la implementación personalizada y usa siempre la generada
- D) Solo funciona si el método se declara private

12. Sealed Classes y Pattern Matching Avanzado

1. ¿Para qué sirve la palabra clave 'sealed' en una clase o interfaz (Java 17)?

- A) Para restringir explícitamente qué clases pueden extenderla o implementarla, usando la cláusula 'permits'
- B) Para hacerla immutable automáticamente como un record
- C) Para prohibir que tenga métodos abstractos
- D) Para convertirla automáticamente en final

2. Una subclase de una clase sealed, ¿qué modificador debe declarar obligatoriamente?

- A) final, sealed o non-sealed
- B) static
- C) abstract siempre
- D) private

3. En Java 21, ¿qué aporta el pattern matching para switch sobre records (record patterns)?

- A) Permite desestructurar los componentes de un record directamente en el 'case' (ej. case Punto(int x, int y) -> ...)
- B) Permite modificar los valores del record dentro del switch
- C) Obliga a que el record implemente Comparable
- D) Solo funciona con enums, no con records

4. ¿Pueden las clases permitidas ('permits') de una clase sealed estar en un paquete diferente?

- A) Sí, pero solo si pertenecen al mismo módulo, o si están en el mismo paquete cuando no se usan módulos
- B) No, siempre deben estar exactamente en el mismo archivo .java
- C) Sí, sin ninguna restricción de paquete o módulo
- D) Solo si son records

5. ¿Qué ventaja aporta un switch exhaustivo sobre una jerarquía sealed junto con pattern matching?

- A) El compilador puede verificar que se cubren todos los subtipos posibles, sin necesitar una rama 'default', mejorando la seguridad en compilación
- B) Permite que el switch se ejecute en paralelo automáticamente
- C) Elimina la necesidad de declarar 'permits'
- D) Convierte automáticamente el sealed en un enum

6. ¿Qué patrón usarías en un case para descomponer un record Punto(int x, int y) y aplicar una guarda adicional 'cuando x sea positivo' (Java 21, guarded patterns)?

- A) case Punto(int x, int y) when x > 0 -> ...
- B) case Punto(int x, int y) if x > 0 -> ...
- C) case Punto(int x, int y) && x > 0 -> ...
- D) No es posible añadir condiciones adicionales dentro de un case

13. Generics, Wildcards y Type Erasure

1. ¿Qué problema resuelven los Generics en Java?

- A) Proveen seguridad de tipos en tiempo de compilación para colecciones y clases, evitando ClassCastException en tiempo de ejecución y castings manuales
- B) Permiten que una variable cambie de tipo dinámicamente en tiempo de ejecución
- C) Eliminan la necesidad de interfaces
- D) Sirven solo para arrays primitivos

2. ¿Qué significa el comodín '?' extends Number' en una declaración de generics?

- A) Acepta el tipo Number o cualquiera de sus subtipos (útil para lectura/producción de datos, 'Producer Extends')
- B) Acepta solo el tipo Number exacto
- C) Acepta cualquier tipo excepto Number
- D) Permite modificar la colección añadiendo elementos de cualquier subtipo libremente

3. ¿Por qué List<String> NO es un subtipo de List<Object> en Java (a diferencia de los arrays)?

- A) Porque los generics son invariantes: aunque String sea subtipo de Object, List<String> y List<Object> son tipos distintos sin relación de subtipo entre sí
- B) Porque List<String> no puede contener objetos de tipo Object
- C) Porque los generics no soportan herencia en absoluto
- D) Porque solo los arrays son válidos en Java, List<> es obsoleto

4. ¿Qué es 'type erasure' (borrado de tipos) en Java?

- A) El proceso por el cual la información de tipo genérico se elimina en tiempo de compilación, quedando solo el tipo raw (u Object) en el bytecode
- B) Un proceso que elimina variables no usadas del código fuente
- C) Una técnica para borrar clases genéricas del classpath
- D) Un mecanismo del recolector de basura para objetos genéricos

5. ¿Qué significa el comodín '?' super Integer' en una declaración de generics?

- A) Acepta Integer o cualquiera de sus supertipos (útil para escribir/consumir datos, 'Consumer Super')
- B) Acepta solo subtipos de Integer
- C) Acepta cualquier tipo excepto Integer
- D) Es sintácticamente inválido en Java

6. ¿Se puede crear un array de un tipo genérico parametrizado, como new List<String>[10]?

- A) No, Java no permite crear directamente arrays de tipos genéricos parametrizados debido al conflicto entre la covarianza de arrays y el borrado de tipos
- B) Sí, sin ninguna restricción
- C) Solo si el tipo genérico es Object
- D) Solo dentro de métodos static

7. ¿Qué es un método genérico y cómo se declara su parámetro de tipo?

- A) Un método que declara su propio parámetro de tipo antes del tipo de retorno, por ejemplo: `public <T> T primero(List<T> lista)`
- B) Un método que solo puede usarse dentro de clases genéricas
- C) Un método que acepta cualquier objeto sin necesidad de declarar tipo alguno
- D) Un método que siempre debe devolver `Object`

8. ¿Los métodos heredados de la clase `Object` (como `equals()`, `hashCode()` o `toString()`) cuentan para el límite de 'un único método abstracto' que debe tener una interfaz funcional?

- A) No, si una interfaz declara explícitamente un método abstracto con la misma firma que uno de `Object` (por ejemplo, un `boolean equals(Object o)`), ese método NO cuenta para el límite de método abstracto único, ya que toda clase ya provee una implementación heredada de `Object`
- B) Sí, cualquier método abstracto adicional, sin importar su origen, siempre rompe la condición de interfaz funcional
- C) Los métodos de `Object` nunca pueden declararse dentro de una interfaz
- D) Solo cuenta si el método se declara como `default`, nunca si se declara como abstracto

9. ¿Qué ocurre si una interfaz funcional (con un único método abstracto propio) es extendida por otra interfaz que declara un segundo método abstracto adicional?

- A) La nueva interfaz (la que extiende) ya NO es una interfaz funcional, porque ahora tiene dos métodos abstractos en total (el heredado más el nuevo); dejaría de poder implementarse mediante una única lambda
- B) La interfaz extendida sigue siendo funcional automáticamente sin importar cuántos métodos abstractos adicionales declare
- C) Es un error de compilación extender una interfaz funcional bajo cualquier circunstancia
- D) Los métodos abstractos heredados nunca cuentan, solo los declarados directamente en la propia interfaz

10. ¿Qué función cumple la anotación `@FunctionalInterface` colocada sobre una interfaz?

- A) Indica la intención de que esa interfaz sea funcional, y hace que el compilador verifique y reporte un error si la interfaz NO cumple con la condición de tener exactamente un método abstracto; sin embargo, la anotación en sí no es obligatoria para que una interfaz sea tratada como funcional
- B) Convierte automáticamente cualquier interfaz en funcional, sin importar cuántos métodos abstractos tenga
- C) Es obligatoria para poder usar lambdas con esa interfaz
- D) Elimina la necesidad de declarar ningún método dentro de la interfaz

11. ¿Qué diferencia de contrato existe entre `BiConsumer<T,U>` y un `UnaryOperator<T>` en cuanto a la cantidad de argumentos y el retorno?

- A) `BiConsumer<T,U>` recibe dos argumentos (de tipos `T` y `U`) y no devuelve nada (`accept`); `UnaryOperator<T>` recibe un único argumento de tipo `T` y devuelve un valor del mismo tipo `T` (`apply`)
- B) Ambas interfaces reciben la misma cantidad de argumentos y tienen el mismo tipo de retorno
- C) `BiConsumer` no puede usarse con expresiones lambda, solo con clases anónimas
- D) `UnaryOperator` siempre recibe dos argumentos, igual que `BiConsumer`

12. ¿Qué ventaja aporta migrar código que usaba clases tradicionales con herencia (por ejemplo, para representar distintos comportamientos) hacia el uso de interfaces funcionales con lambdas?

- A) Reduce significativamente el código repetitivo (boilerplate) al eliminar la necesidad de declarar clases completas por cada comportamiento distinto, permitiendo expresar la lógica de forma concisa directamente donde se necesita
- B) Hace que el programa deje de ser orientado a objetos por completo
- C) Elimina la posibilidad de usar polimorfismo en el programa
- D) Obliga a reescribir toda la jerarquía de clases como enums

13. ¿Qué problema de diseño resolvían los generics (introducidos en Java 5) respecto a tener que crear una clase contenedora específica para cada tipo de dato (por ejemplo, una clase ContenedorDeString y otra ContenedorDeInteger por separado)?

- A) Los generics permiten escribir una única clase contenedora parametrizada por un tipo T, reutilizable para cualquier tipo de dato, eliminando la duplicación de código que antes requería una clase distinta por cada tipo específico
- B) Los generics eliminan por completo la necesidad de usar clases contenedoras
- C) Los generics solo funcionan con tipos primitivos, no con clases contenedoras de objetos
- D) Los generics obligan a usar Object en lugar de cualquier tipo específico, sin ninguna ventaja adicional

14. ¿Qué significa declarar un método genérico a nivel de método, independientemente de si la clase que lo contiene es o no genérica, usando por ejemplo '<E> void metodo(E elemento)'?

- A) El método declara su propio parámetro de tipo E, que se determina de forma independiente cada vez que se invoca el método, sin necesidad de que la clase contenedora sea genérica ni de que E coincida con algún parámetro de tipo de la clase
- B) Solo las clases genéricas pueden declarar métodos con su propio parámetro de tipo adicional
- C) El parámetro de tipo E de un método genérico debe ser obligatoriamente el mismo que el de la clase contenedora
- D) Los métodos genéricos no pueden combinarse con clases que también sean genéricas

15. ¿Cuál es la convención de nomenclatura habitual para los nombres de parámetros de tipo genérico de una sola letra en Java, como T, E, K, V?

- A) T para 'Type' (tipo genérico general), E para 'Element' (elemento de una colección), K para 'Key' (clave) y V para 'Value' (valor), como en Map<K,V>
- B) No existe ninguna convención, los nombres se eligen completamente al azar
- C) T siempre significa 'Test', y E siempre significa 'Error'
- D) K y V solo pueden usarse dentro de clases abstractas, nunca en interfaces genéricas

16. ¿Qué representa un wildcard acotado superiormente, como 'List<? extends Number>', en cuanto a las operaciones permitidas sobre esa lista?

- A) Permite leer elementos de la lista tratándolos como Number (o un supertipo), pero no permite añadir elementos (salvo null), ya que el compilador no puede garantizar el tipo exacto de los elementos que realmente contiene la lista
- B) Permite tanto leer como añadir libremente cualquier subtipo de Number sin ninguna restricción
- C) Prohíbe cualquier operación de lectura sobre la lista, solo permite escritura
- D) Solo puede usarse con listas de tipos primitivos, nunca con listas de objetos

17. ¿Qué representa un wildcard acotado inferiormente, como 'List<? super Integer>', en cuanto a las operaciones permitidas sobre esa lista?

- A) Permite añadir elementos de tipo Integer (o subtipos) de forma segura, ya que la lista garantiza ser de tipo Integer o algún supertipo de Integer; pero al leer, solo se puede garantizar que los elementos son al menos de tipo Object
- B) Permite leer los elementos directamente como Integer sin ninguna restricción adicional
- C) Prohíbe cualquier operación de escritura, solo permite lectura como Integer
- D) Es sintácticamente idéntico a 'List<? extends Integer>', sin ninguna diferencia de comportamiento

18. Antes de la introducción de generics en Java 5, ¿qué era necesario hacer al recuperar un elemento de una colección que internamente almacenaba objetos de tipo Object (como las collections 'raw' de aquella época)?

- A) Era necesario hacer un casting explícito manual del Object recuperado hacia el tipo real esperado (por ejemplo, '(String) lista.get(0)'), con el riesgo de un ClassCastException en tiempo de ejecución si el tipo real no coincidía
- B) No era necesario ningún casting, el compilador infería automáticamente el tipo real sin generics
- C) Las colecciones antes de Java 5 no podían almacenar objetos, solo tipos primitivos
- D) Se debía usar obligatoriamente reflection para acceder a cualquier elemento de la colección

19. En una clase genérica como 'class Box<T> { private List<T> elementos; ... }', ¿qué gana el código en seguridad de tipos respecto a la versión sin generics equivalente (usando Object)?

- A) El compilador verifica en tiempo de compilación que solo se añadan y recuperen elementos del tipo T declarado, evitando errores de tipo que antes solo se detectaban en tiempo de ejecución mediante ClassCastException
- B) No hay ninguna diferencia de seguridad de tipos entre ambas versiones
- C) La versión con generics es más lenta en tiempo de ejecución que la versión con Object
- D) Los generics eliminan la necesidad de declarar ningún método en la clase Box

20. ¿Es obligatorio, a nivel de compilador, usar los nombres de convención (T, E, K, V) para los parámetros de tipo genérico, o se trata solo de una recomendación de estilo?

- A) Es solo una convención de estilo ampliamente adoptada; el compilador acepta cualquier identificador válido como nombre de parámetro de tipo genérico, aunque usar nombres no convencionales puede dificultar la lectura del código para otros desarrolladores
- B) El compilador rechaza cualquier nombre de parámetro de tipo que no sea T, E, K o V
- C) Solo se permite usar nombres de una sola letra, sin ninguna excepción
- D) Las convenciones de nomenclatura son parte de la sintaxis obligatoria del lenguaje, verificada por el compilador

21. ¿Puede un método static de una clase genérica Box<T> usar directamente el parámetro de tipo T de la clase (por ejemplo, 'static T obtenerDefault() { ... })'?

- A) No, un método static no puede referirse al parámetro de tipo T de la clase, ya que T está asociado a cada instancia particular y los métodos static no dependen de ninguna instancia; si el método static necesita ser genérico, debe declarar su propio parámetro de tipo independiente
- B) Sí, sin ninguna restricción, un método static puede usar libremente el parámetro de tipo de la clase contenedora
- C) Solo puede usarlo si la clase Box tiene un constructor sin parámetros
- D) Solo puede usarlo si T se declara también como static dentro de la clase

22. ¿Cómo se puede usar Stream de forma segura sobre una colección genérica List<T> dentro de un método de una clase genérica, para procesar sus elementos?

- A) Invocando lista.stream() normalmente; como el compilador ya conoce el tipo T asociado a esa instancia particular, el Stream resultante es un Stream<T> con toda la seguridad de tipos habitual, permitiendo aplicar operaciones como filter/map/forEach sobre elementos de tipo T
- B) No es posible usar streams sobre colecciones de tipo genérico, solo sobre tipos concretos como String o Integer
- C) Se requiere primero castear la lista a List<Object> antes de poder usar stream()
- D) Los streams sobre tipos genéricos siempre pierden la información de tipo, comportándose como Stream<Object>

23. ¿Cuál es la sintaxis correcta para declarar un parámetro de tipo genérico T que debe ser tanto un subtipo de Number como implementar Comparable<T>, combinando ambas restricciones (bounded type con múltiples límites)?

- A) <T extends Number & Comparable<T>>
- B) <T extends Number, Comparable<T>>
- C) <T implements Number & Comparable<T>>
- D) <T extends Number | Comparable<T>>

24. Al declarar múltiples límites (bounds) para un parámetro de tipo genérico, como en '<T extends Number & Comparable<T>>', ¿qué regla de orden debe respetarse si uno de los límites es una clase?

- A) La clase debe declararse primero, antes de cualquier interfaz, en la lista de límites separados por '&'; solo se permite una clase como límite, pero pueden añadirse varias interfaces después de ella
- B) El orden nunca importa, se pueden escribir los límites en cualquier secuencia
- C) Las interfaces siempre deben ir antes que la clase en la lista de límites
- D) Solo se permite declarar una interfaz como límite, nunca una clase

25. ¿Cómo se relaciona la covarianza de tipos de retorno (al sobrescribir un método) con los generics, por ejemplo si una clase Contenedor<T> declara un método 'T obtener()' y una subclase concreta ContenedorDeTexto extiende Contenedor<String>?

- A) En ContenedorDeTexto, el método sobrescrito puede (y de hecho, efectivamente) devuelve String en lugar de T genérico, porque el compilador sustituye T por String en esa instancia concreta de la jerarquía, siendo coherente con las reglas de covarianza de tipos de retorno
- B) El método sobrescrito debe declarar obligatoriamente el tipo T literal en la subclase, sin poder especializarse a String
- C) La covarianza de tipos de retorno no aplica en absoluto a clases que usan generics
- D) El método sobrescrito en ContenedorDeTexto debe devolver Object en lugar de String

26. Si se declara una clase genérica como 'class Contenedor<T extends CharSequence>', ¿por qué se puede usar directamente 'String' como argumento de tipo, por ejemplo 'Contenedor<String>'?

- A) Porque String implementa la interfaz CharSequence, por lo que cumple con la restricción del bounded type declarado (T extends CharSequence), permitiendo que String sea un argumento de tipo válido para esa clase genérica
- B) Porque CharSequence y String son exactamente la misma interfaz con distinto nombre
- C) Porque cualquier clase puede usarse como argumento de tipo sin importar el bounded type declarado
- D) Porque String extiende directamente la clase CharSequence, en lugar de implementarla como interfaz

27. ¿Por qué una lista declarada con un wildcard no acotado, como 'List<?> lista', solo permite operaciones de lectura muy limitadas (tratando los elementos como Object) y no permite añadir elementos (salvo null)?

- A) Porque el wildcard '?' representa 'algún tipo desconocido', y el compilador no puede garantizar qué tipo específico es realmente, por lo que solo permite operaciones seguras para cualquier tipo posible (como leer como Object), pero no añadir elementos de un tipo concreto que podría no coincidir con el tipo real desconocido
- B) Porque List<?> siempre está vacía por definición y nunca puede contener elementos
- C) Porque List<?> es sinónimo exacto de List<Object>, con las mismas capacidades de escritura
- D) Porque el wildcard sin acotar solo puede usarse con colecciones de tipos primitivos

28. ¿Qué es el 'diamond operator' (<>) introducido para simplificar la sintaxis de generics, y desde qué versión de Java está disponible?

- A) Es la sintaxis '<>' que permite omitir repetir el tipo genérico en el lado derecho de una instanciación, por ejemplo 'List<String> lista = new ArrayList<>();' en lugar de repetir '<String>' en ambos lados; está disponible desde Java 7
- B) Es un operador matemático nuevo introducido específicamente para comparar tipos genéricos entre sí
- C) Es una anotación que reemplaza completamente a los wildcards
- D) Está disponible únicamente desde Java 17 en adelante, no en versiones anteriores

29. ¿Pueden los records usar parámetros de tipo genérico, por ejemplo 'record Par<T, U>(T primero, U segundo) {'?

- A) Sí, los records pueden ser genéricos igual que las clases normales, declarando sus propios parámetros de tipo que se usan en los tipos de sus componentes
- B) No, los records nunca pueden ser genéricos bajo ninguna circunstancia
- C) Solo pueden ser genéricos si tienen exactamente un componente
- D) Los records genéricos requieren obligatoriamente implementar Comparable

30. En un ejemplo con una jerarquía Helado (superclase) y HeladoDeChocolate (subclase), ¿por qué NO se puede asignar directamente un 'Contenedor<HeladoDeChocolate>' a una variable declarada como 'Contenedor<Helado>', aunque HeladoDeChocolate sea subtipo de Helado?

- A) Porque los generics son invariantes: a pesar de que HeladoDeChocolate es subtipo de Helado, Contenedor<HeladoDeChocolate> y Contenedor<Helado> son tipos completamente distintos sin relación de subtipo entre sí, por lo que la asignación directa no es válida sin wildcards
- B) Porque Contenedor no puede ser una clase genérica en absoluto
- C) Porque HeladoDeChocolate debe declararse como 'final' para poder usarse con generics
- D) Porque los generics solo funcionan con tipos que implementan Comparable

31. ¿Cómo permite un wildcard acotado, como 'Contenedor<? extends Helado>', aceptar de forma flexible tanto un Contenedor<Helado> como un Contenedor<HeladoDeChocolate>?

- A) El wildcard '? extends Helado' representa 'algún subtipo desconocido de Helado', por lo que una variable de ese tipo puede referenciar contenedores parametrizados con Helado o con cualquiera de sus subtipos, resolviendo la limitación de invariancia de los generics para casos de solo lectura
- B) No es posible lograr esa flexibilidad de ninguna manera en Java
- C) Solo funciona si Contenedor implementa Iterable
- D) El wildcard convierte automáticamente todos los subtipos de Helado en el mismo tipo exacto

32. ¿Qué relación tiene 'Comparator.naturalOrder()' con la interfaz Comparable, al usarse para ordenar una colección?

- A) Comparator.naturalOrder() devuelve un Comparator que delega en el propio compareTo() de los elementos (que deben implementar Comparable), aplicando el 'orden natural' definido por esa clase, en lugar de definir una lógica de comparación externa distinta
- B) Comparator.naturalOrder() ignora completamente cualquier implementación de Comparable de los elementos
- C) Comparator.naturalOrder() solo puede usarse con tipos primitivos, nunca con objetos
- D) Comparator.naturalOrder() invierte automáticamente el orden natural definido por compareTo()

33. ¿Por qué NO es posible crear directamente un array de un tipo genérico dentro de una clase genérica, como 'T[] array = new T[10];' dentro de una clase Contenedor<T>?

- A) Debido al type erasure, la JVM no conserva información del tipo T en tiempo de ejecución; permitir 'new T[10]' sin saber el tipo real produciría problemas de seguridad de tipos en tiempo de ejecución, por lo que el compilador lo prohíbe directamente
- B) Porque los arrays nunca pueden usarse dentro de ninguna clase genérica bajo ninguna circunstancia
- C) Porque T siempre se interpreta como int, y no se pueden crear arrays de tipos primitivos genéricos
- D) Porque esta operación sí es válida sin ninguna restricción en Java 17

34. ¿Puede un parámetro de tipo genérico T usarse directamente como un tipo primitivo, por ejemplo declarando una clase 'class Caja<T>' e intentando instanciarla como 'Caja<int>'?

- A) No, los generics en Java solo funcionan con tipos referencia (clases, no primitivos); para usar un tipo similar a int se debe usar su wrapper correspondiente, como Caja<Integer>
- B) Sí, Java permite usar tipos primitivos directamente como argumentos de tipo genérico sin ninguna restricción
- C) Solo se permite con 'long' y 'double', pero no con 'int'
- D) Los tipos primitivos y los generics son completamente incompatibles, ni siquiera con wrappers

35. ¿Por qué una variable static de una clase genérica no puede declararse usando el parámetro de tipo T de esa clase, por ejemplo 'static T valorCompartido;' dentro de 'class Caja<T>'?

- A) Porque T está asociado a cada instancia particular de la clase genérica (Caja<String>, Caja<Integer>, etc.), mientras que una variable static es compartida por todas las instancias sin distinción de su tipo parametrizado específico, generando una contradicción de diseño
- B) Porque las variables static nunca pueden inicializarse en Java
- C) Porque T solo puede usarse dentro de métodos, nunca como tipo de un campo
- D) Porque esta declaración sí es completamente válida sin ninguna restricción

36. Debido al type erasure, ¿por qué no se pueden declarar en la misma clase dos métodos sobrecargados cuyas firmas solo difieran en sus parámetros de tipo genérico, por ejemplo 'void metodo(List<String> lista)' y 'void metodo(List<Integer> lista)'?

- A) Porque tras el borrado de tipos, ambas firmas se convierten en 'void metodo(List lista)' (usando el tipo raw List), siendo exactamente la misma firma de método, lo que el compilador rechaza por firma duplicada
- B) Porque List<String> y List<Integer> son tipos completamente diferentes en tiempo de ejecución, sin ningún conflicto real
- C) Porque no se permite usar List como parámetro de ningún método sobrecargado
- D) Porque esta sobrecarga sí es perfectamente válida y compila sin ningún problema

37. ¿Qué comportamiento de solo lectura tiene una lista declarada con wildcard no acotado, como 'List<?> lista = obtenerListaDesconocida();', respecto a añadir elementos?

- A) No se puede invocar add() con ningún tipo de elemento (excepto null), ya que el compilador no puede garantizar qué tipo específico acepta realmente la lista detrás del wildcard '?'; sin embargo, sí se pueden leer elementos tratándolos genéricamente como Object
- B) Se puede añadir cualquier tipo de objeto libremente, ya que '?' representa 'cualquier tipo permitido'
- C) List<?> siempre está vacía y nunca puede contener elementos, ni siquiera al leerlos
- D) Solo se pueden añadir elementos de tipo String, sin importar el tipo real de la lista

38. ¿Por qué un método como 'static double sumar(List<? extends Number> lista)' en una clase utilitaria CollectionUtils puede aceptar tanto List<Integer> como List<Double>, mientras que un parámetro 'List<Number> lista' sin wildcard NO aceptaría directamente List<Integer>?

- A) El wildcard '? extends Number' acepta una lista de Number o de cualquier subtipo suyo (Integer, Double, etc.), aprovechando la flexibilidad de lectura que dan los wildcards acotados; en cambio, 'List<Number>' exige exactamente ese tipo exacto, ya que los generics son invariantes y List<Integer> no es subtipo de List<Number>
- B) 'List<Number>' y 'List<? extends Number>' son exactamente equivalentes en cualquier contexto
- C) El wildcard solo funciona con listas vacías, sin elementos
- D) Los wildcards solo pueden usarse con tipos String, nunca con tipos numéricos

39. En un método utilitario que solo necesita LEER y sumar los valores numéricos de una colección (sin añadir elementos), ¿por qué es preferible usar 'List<? extends Number>' en lugar de un tipo genérico como '<T extends Number> List<T>'?

- A) Ambos enfoques son técnicamente válidos, pero el wildcard '? extends Number' comunica más claramente que el método solo lee los datos sin necesitar un parámetro de tipo nombrado, siendo la opción preferida cuando no se necesita referenciar ese tipo en ningún otro lugar de la firma o cuerpo del método
- B) El wildcard es la única opción posible; un método genérico con <T extends Number> nunca compilaría en este contexto
- C) El tipo genérico con nombre (T) es siempre más eficiente en tiempo de ejecución que el wildcard
- D) No existe ninguna diferencia entre ambos enfoques en ningún aspecto

40. Dada la jerarquía A (superclase), B extends A, y C extends B, ¿qué tipo de wildcard permitiría a un método aceptar una lista de B, o de cualquier subtipo de B (incluyendo C), pero no de A?

- A) List<? extends B>
- B) List<? super B>
- C) List<A>
- D) List<C>

41. ¿Qué advertencia o conflicto puede surgir al declarar un record genérico que además implementa Comparable, como 'record Par<T extends Comparable<T>>(T valor) implements Comparable<Par<T>>'?

- A) Es necesario asegurar consistencia entre el bounded type del parámetro T (que ya exige ser Comparable<T>) y la implementación de Comparable<Par<T>> del propio record, para que compareTo() pueda delegar correctamente en el compareTo() de T sin conflictos de tipos ni advertencias del compilador por uso de tipos raw o incompatibles
- B) Los records nunca pueden implementar Comparable bajo ninguna circunstancia
- C) No puede existir ningún conflicto, ya que los records ignoran completamente los bounded types
- D) Un record genérico nunca puede tener más de un componente

42. ¿Por qué Deque es frecuentemente preferido sobre LinkedList 'pura' cuando se necesita explícitamente un comportamiento de pila (LIFO) o cola (FIFO) en el código?

- A) Porque Deque expone una interfaz con métodos específicamente pensados para operaciones en ambos extremos (push/pop para LIFO, offer/poll para FIFO), comunicando más claramente la intención del código, aunque LinkedList sea frecuentemente la implementación subyacente utilizada
- B) Porque LinkedList no puede usarse nunca como Queue o Stack
- C) Porque Deque es una clase concreta distinta, sin relación con LinkedList
- D) Porque Deque solo funciona con tipos primitivos, a diferencia de LinkedList

14. Comparable y Comparator

1. ¿Cuál es la diferencia principal entre Comparable y Comparator?

- A) Comparable se implementa en la propia clase para definir su orden natural (compareTo); Comparator se define externamente y permite múltiples estrategias de ordenación (compare)
- B) Comparable es para colecciones y Comparator solo para arrays
- C) Comparator solo funciona con tipos primitivos
- D) No hay diferencia, son sinónimos desde Java 17

2. ¿Qué valor devuelve compareTo() si el objeto actual es 'mayor' que el que se compara?

- A) Un valor positivo
- B) Un valor negativo
- C) Siempre 1 exactamente
- D) 0

3. ¿Cómo se encadenan múltiples criterios de comparación usando Comparator (por ejemplo, por apellido y luego por nombre)?

- A) Comparator.comparing(Persona::getApellido).thenComparing(Persona::getNombre)
- B) Comparator.and(Persona::getApellido, Persona::getNombre)
- C) Comparator.merge(apellido, nombre)
- D) No es posible encadenar comparadores en Java

4. ¿Qué ocurre si dos objetos son 'iguales' según equals() pero compareTo() no devuelve 0 para ellos?

- A) Se dice que la clase tiene un orden 'inconsistente con equals', lo cual puede causar comportamientos inesperados en colecciones ordenadas como TreeSet o TreeMap
- B) Java lanza una excepción automáticamente en tiempo de compilación
- C) No tiene ningún efecto práctico en ninguna colección
- D) Se corrige automáticamente al usar Comparator en vez de Comparable

5. ¿Cómo se invierte el orden producido por un Comparator existente, por ejemplo para ordenar de mayor a menor?

- A) comparator.reversed()
- B) comparator.invert()
- C) Comparator.negate(comparator)
- D) comparator.opposite()

6. ¿Qué método estático de Comparator crea un comparador a partir de una función que extrae una clave comparable de cada elemento?

- A) Comparator.comparing(funcionExtractora)
- B) Comparator.by(funcionExtractora)
- C) Comparator.key(funcionExtractora)
- D) Comparator.extract(funcionExtractora)

7. ¿Por qué se dice que Comparable 'se queda corto' cuando se necesita ordenar objetos por múltiples criterios distintos?

- A) Porque Comparable define un único método `compareTo()` por clase, lo que fija un único 'orden natural'; para ordenar por un criterio diferente (o varios) sería necesario modificar esa única implementación cada vez, en lugar de tener múltiples estrategias de orden simultáneas
- B) Porque Comparable no puede implementarse en ninguna clase que tenga más de un atributo
- C) Porque Comparable solo funciona con tipos primitivos, nunca con objetos complejos
- D) Porque Comparable requiere obligatoriamente el uso de expresiones lambda

8. ¿Puede usarse Comparator para ordenar objetos de una clase que NO implementa Comparable?

- A) Sí, Comparator es completamente independiente de si la clase implementa Comparable o no, ya que la lógica de comparación se define externamente en el propio Comparator
- B) No, Comparator solo puede usarse en clases que ya implementan Comparable
- C) Solo si la clase tiene un método `toString()` sobrescrito
- D) Solo si la clase es final

9. De las tres formas de implementar un Comparator mostradas (clase separada, clase anónima, expresión lambda), ¿qué tienen en común en cuanto al método que finalmente se debe proveer?

- A) Las tres formas terminan proveyendo una implementación del método `compare(T o1, T o2)`, ya sea mediante una clase completa, una clase anónima al vuelo, o el cuerpo conciso de una lambda
- B) Cada forma implementa un método completamente distinto, sin relación entre sí
- C) Solo la clase separada implementa `compare()`; las otras dos formas usan un método diferente
- D) Las tres formas requieren sobrescribir `compareTo()`, no `compare()`

10. ¿Por qué la versión con expresión lambda de un Comparator suele considerarse más concisa que la versión con clase anónima equivalente?

- A) Porque la lambda elimina la sintaxis repetitiva de declarar la clase anónima, el nombre del método y los tipos de los parámetros (que el compilador puede inferir), dejando solo la lógica esencial de comparación
- B) Porque una lambda nunca puede lanzar excepciones, a diferencia de una clase anónima
- C) Porque las lambdas se ejecutan más rápido en tiempo de ejecución que cualquier clase anónima
- D) Porque una lambda no puede acceder a variables externas, haciendo el código más simple

11. ¿Qué método estático de Comparator es común usar como punto de partida al construir una lambda o method reference para ordenar por un atributo específico, por ejemplo la edad de una lista de empleados?

- A) `Comparator.comparing(Empleado::getEdad)`
- B) `Comparator.orderBy(Empleado::getEdad)`
- C) `Comparator.sortBy(Empleado::getEdad)`
- D) `Comparator.criteria(Empleado::getEdad)`

12. ¿Por qué es relevante recordar el 'cache de Integer' (valores entre -128 y 127) al analizar el comportamiento de un HashSet<Integer> en ejercicios de examen?

- A) Porque dentro de ese rango, los objetos Integer autoboxeados con el mismo valor son literalmente el mismo objeto compartido (gracias al cache), lo cual puede afectar el análisis de '==' entre elementos, aunque el comportamiento de duplicados de un HashSet siempre depende de equals()/hashCode(), no de identidad de referencia
- B) Porque un HashSet nunca puede almacenar valores Integer fuera de ese rango
- C) Porque el cache de Integer convierte automáticamente cualquier HashSet en un TreeSet
- D) Porque los valores fuera de ese rango no pueden compararse con equals()

13. ¿Qué ocurre en tiempo de ejecución si se intenta añadir elementos a un TreeSet cuyos elementos NO implementan Comparable, sin proporcionar tampoco un Comparator explícito al crear el TreeSet?

- A) Se lanza ClassCastException al intentar insertar el primer par de elementos que deban compararse entre sí, ya que el TreeSet no tiene ninguna forma de determinar su orden
- B) El TreeSet los inserta sin ningún problema, usando el orden de inserción como respaldo automático
- C) El programa no compila si los elementos no implementan Comparable
- D) El TreeSet convierte automáticamente los elementos a String para poder compararlos

14. ¿Qué diferencia hay entre los métodos add()/remove() y offer()/poll() de la interfaz Deque en cuanto al manejo de casos límite (como una estructura llena o vacía)?

- A) add() y remove() lanzan una excepción si la operación no puede completarse (por ejemplo, remove() en una estructura vacía); offer() y poll() manejan esos mismos casos de forma más segura, devolviendo false o null respectivamente en lugar de lanzar una excepción
- B) add() y offer() son exactamente sinónimos sin ninguna diferencia de comportamiento
- C) poll() siempre lanza una excepción si la estructura está vacía, igual que remove()
- D) offer() solo puede usarse con Deques de tamaño fijo

15. ¿Qué devuelven los métodos keySet() y entrySet() de la interfaz Map?

- A) keySet() devuelve una vista de tipo Set con todas las claves del mapa; entrySet() devuelve una vista de tipo Set con todos los pares clave-valor (Map.Entry) del mapa, permitiendo iterar sobre ambos a la vez
- B) keySet() devuelve una List con las claves ordenadas alfabéticamente siempre
- C) entrySet() devuelve únicamente los valores del mapa, sin las claves asociadas
- D) Ambos métodos devuelven exactamente lo mismo, son sinónimos

16. ¿Cómo se relaciona la interfaz BiConsumer<K,V> con el método forEach() de un Map, como en 'mapa.forEach((clave, valor) -> System.out.println(clave + "=" + valor));'?

- A) El método forEach() de Map espera precisamente un BiConsumer<K,V>, ya que necesita procesar dos valores por cada entrada (la clave y el valor) sin producir ningún resultado de retorno, lo cual coincide exactamente con el contrato accept(K,V) de BiConsumer
- B) Map.forEach() solo acepta un Consumer de un único parámetro, ignorando las claves
- C) BiConsumer no puede usarse con estructuras tipo Map, solo con listas
- D) El método forEach() de Map requiere obligatoriamente una clase anónima, no puede usarse con lambdas

17. ¿Qué hace el método `getOrDefault(clave, valorPorDefecto)` de la interfaz `Map`?

- A) Devuelve el valor asociado a la clave si existe en el mapa; si la clave no está presente, devuelve el `valorPorDefecto` proporcionado sin lanzar ninguna excepción ni modificar el mapa
- B) Inserta la clave con el `valorPorDefecto` si no existe, modificando el mapa
- C) Lanza una excepción si la clave no existe en el mapa
- D) Elimina la clave del mapa si su valor coincide con el `valorPorDefecto`

18. ¿Qué hace el método `merge(clave, valor, funcionRemapeo)` de la interfaz `Map` cuando la clave YA existe en el mapa?

- A) Aplica la función de remapeo (`BiFunction`) al valor existente y al nuevo valor proporcionado, y almacena el resultado como el nuevo valor asociado a esa clave (o elimina la entrada si la función devuelve `null`)
- B) Siempre sobrescribe el valor existente con el nuevo valor, ignorando la función de remapeo
- C) Lanza una excepción si la clave ya existe en el mapa
- D) Ignora la operación por completo si la clave ya existe

19. ¿Por qué es importante validar contra `null` dentro de la implementación de `compareTo()` al implementar `Comparable` en una clase propia?

- A) Porque si `compareTo()` recibe un argumento `null` y no lo maneja explícitamente, se lanzaría `NullPointerException` al intentar comparar sus atributos internos contra los de un objeto inexistente; el contrato de `Comparable` espera que se lance `NullPointerException` de forma consistente, no un comportamiento indefinido
- B) Porque `compareTo()` nunca puede recibir `null` como argumento bajo ninguna circunstancia, ya que el compilador lo previene
- C) Porque validar `null` convierte automáticamente la clase en inmutable
- D) Porque sin esa validación, el método no compila

20. ¿Cómo se encadenan varios criterios de ordenamiento con `Comparator` para ordenar records de `Empleado`, primero por departamento y luego por salario, en orden descendente para el salario?

- A) `Comparator.comparing(Empleado::departamento).thenComparing(Comparator.comparing(Empleado::salario).reversed())`
- B) `Comparator.and(Empleado::departamento, Empleado::salario).reversed()`
- C) `Comparator.merge(Empleado::departamento, Empleado::salario)`
- D) No es posible combinar orden ascendente y descendente en el mismo encadenamiento

21. ¿Por qué `Arrays.binarySearch()` (o `Collections.binarySearch()`) requiere que la colección o array esté ordenado en orden ascendente antes de aplicarse?

- A) Porque el algoritmo de búsqueda binaria divide repetidamente el rango de búsqueda a la mitad, descartando la mitad donde el elemento buscado no puede estar, según comparaciones que solo son válidas si los elementos están ordenados de forma ascendente y consistente
- B) Porque el orden ascendente es solo una recomendación estética, sin ningún efecto en el resultado real de la búsqueda
- C) Porque la búsqueda binaria funciona igual de bien con cualquier orden, incluso datos completamente desordenados
- D) Porque de lo contrario el método no compila

15. Lambdas e Interfaces Funcionales

1. ¿Cuál es la sintaxis correcta de una expresión lambda que recibe dos enteros y devuelve su suma?

- A) (a, b) -> a + b
- B) function(a, b) => a + b
- C) lambda(a, b) { return a + b; }
- D) (a, b) => { a + b }

2. ¿Qué interfaz funcional del paquete `java.util.function` representa una operación que recibe un argumento y devuelve un boolean?

- A) `Predicate<T>`
- B) `Function<T,R>`
- C) `Supplier<T>`
- D) `Consumer<T>`

3. ¿Qué interfaz funcional representa una operación que NO recibe argumentos pero produce un resultado?

- A) `Supplier<T>`
- B) `Consumer<T>`
- C) `Function<T,R>`
- D) `BiFunction<T,U,R>`

4. ¿Puede una expresión lambda acceder y modificar una variable local externa capturada?

- A) No, solo puede leer variables locales externas que sean 'effectively final' (efectivamente finales); no puede reasignarlas
- B) Sí, puede modificar libremente cualquier variable local externa
- C) Solo si la variable es static
- D) Solo dentro de un Comparator

5. ¿Qué interfaz funcional recibe un argumento y no devuelve ningún resultado (void)?

- A) `Consumer<T>`
- B) `Supplier<T>`
- C) `Function<T,R>`
- D) `Predicate<T>`

6. ¿Qué interfaz funcional recibe un argumento de tipo T y produce un resultado de tipo R distinto?

- A) `Function<T,R>`
- B) `UnaryOperator<T>`
- C) `Predicate<T>`
- D) `BiConsumer<T,U>`

7. ¿Puede una expresión lambda lanzar una excepción checked si la interfaz funcional que implementa no la declara en su método abstracto?

- A) No, si el método abstracto de la interfaz funcional no declara esa excepción checked, la lambda no puede lanzarla (no compila)
- B) Sí, cualquier lambda puede lanzar cualquier excepción sin restricciones
- C) Solo si la lambda se asigna a una variable Object
- D) Solo si la excepción es unchecked, nunca aplica a checked de todos modos

8. ¿Qué requisito estructural debe cumplir una interfaz para poder ser implementada mediante una expresión lambda?

- A) Debe ser una interfaz funcional: declarar exactamente un único método abstracto (además de posibles métodos default o static)
- B) Debe tener al menos dos métodos abstractos declarados
- C) Debe extender obligatoriamente la interfaz Comparable
- D) Debe declarar únicamente campos, sin ningún método

9. ¿Qué hace la interfaz funcional Predicate<T> y cuál es su método abstracto principal?

- A) Representa una condición evaluada sobre un objeto de tipo T, mediante el método test(T t), que devuelve true o false
- B) Representa una transformación de un tipo T a un tipo R distinto, mediante el método apply()
- C) Representa una operación sin retorno sobre un tipo T, mediante el método accept()
- D) Representa la creación de un nuevo valor de tipo T sin argumentos, mediante el método get()

10. Al filtrar un array de empleados usando distintos Predicate (por edad, por salario, por longitud de nombre), ¿qué ventaja aporta usar Predicate en lugar de escribir un bucle con if distinto para cada criterio?

- A) Permite encapsular cada criterio de filtrado como un objeto independiente y reutilizable, que puede combinarse o intercambiarse fácilmente sin duplicar la lógica del bucle de recorrido
- B) Predicate hace que el filtrado se ejecute en paralelo automáticamente en todos los casos
- C) Predicate elimina completamente la necesidad de cualquier tipo de bucle o iteración en el código
- D) No hay ninguna ventaja real, ambos enfoques son exactamente equivalentes en todos los aspectos

11. ¿Cómo se declara una interfaz funcional personalizada que use generics, por ejemplo una que reciba un tipo T y devuelva un tipo R?

- A) Declarando una interfaz con la anotación opcional @FunctionalInterface, un único método abstracto que use los parámetros de tipo genérico declarados en la propia interfaz (por ejemplo, interface Transformador<T, R> { R transformar(T entrada); })
- B) Las interfaces funcionales no pueden usar generics, solo tipos concretos
- C) Se debe declarar como una clase abstracta genérica, no como interfaz
- D) Los generics solo pueden usarse en interfaces con más de un método abstracto

12. ¿Qué paquete de la biblioteca estándar de Java provee un conjunto de interfaces funcionales predefinidas y listas para usar, como Predicate, Function, Supplier y Consumer?

- A) java.util.function
- B) java.util.functional
- C) java.lang.function
- D) java.util.lambda

13. ¿El hecho de que una interfaz funcional contenga métodos static y default rompe la regla de que debe tener 'un único método abstracto'?

- A) No, la regla de 'un único método abstracto' solo aplica a los métodos SIN cuerpo (abstractos); los métodos static y default sí tienen implementación propia y no cuentan para ese límite, por lo que una interfaz funcional puede tener varios de ellos además de su único método abstracto
- B) Sí, cualquier método adicional (static o default) convierte automáticamente la interfaz en no funcional
- C) No, pero solo se permite un método default como máximo, sin límite para los static
- D) No, pero solo se permite un método static como máximo, sin límite para los default

14. ¿Cómo se invoca un método static declarado dentro de una interfaz funcional, por ejemplo un método 'static Predicate<T> siempreVerdadero()' dentro de Predicate?

- A) Se invoca directamente sobre el nombre de la interfaz, como 'Predicate.siempreVerdadero()', sin necesidad de tener una instancia de la interfaz
- B) Se invoca sobre cualquier instancia lambda que implemente la interfaz, como 'miLambda.siempreVerdadero()'
- C) No puede invocarse directamente, requiere reflexión
- D) Solo puede invocarse desde dentro de otra interfaz

15. ¿Por qué un método default de una interfaz funcional SÍ requiere una instancia (una lambda o implementación concreta) para poder invocarse, a diferencia de un método static?

- A) Porque un método default forma parte del contrato de instancia de la interfaz, y su implementación suele operar sobre 'this' (la instancia actual), por lo que necesita un objeto concreto sobre el cual ejecutarse
- B) Porque los métodos default nunca tienen cuerpo, igual que los abstractos
- C) Porque los métodos default solo pueden invocarse desde dentro de la propia interfaz, nunca desde fuera
- D) No hay ninguna diferencia real entre invocar un método default y uno static

16. ¿Qué hace el método default negate() de la interfaz Predicate<T>?

- A) Devuelve un nuevo Predicate<T> que representa la negación lógica del predicado original: donde el original devolvía true, el nuevo devuelve false, y viceversa
- B) Modifica el predicado original invirtiendo su lógica permanentemente
- C) Elimina el predicado de memoria
- D) Convierte el Predicate en un Supplier

17. ¿Qué hacen los métodos default and() y or() de Predicate<T>, por ejemplo 'predicado1.and(predicado2)'?

- A) Combinan dos predicados en uno nuevo: and() devuelve true solo si ambos predicados originales son true (equivalente lógico a &&); or() devuelve true si al menos uno de los dos es true (equivalente lógico a ||)
- B) Ambos métodos hacen exactamente lo mismo, son sinónimos
- C) and() ejecuta ambos predicados en paralelo en hilos distintos
- D) or() reemplaza el predicado original por el segundo, ignorando el primero

18. ¿Qué se entiende por 'función de orden superior' (higher-order function) en el contexto de un método que devuelve una expresión lambda?

- A) Es un método que trata las funciones (lambdas o referencias a método) como valores de primera clase: puede recibirlas como parámetros y/o devolverlas como resultado, en lugar de solo trabajar con datos simples
- B) Es un método que siempre se ejecuta antes que cualquier otro método en el programa
- C) Es un método que solo puede ser static
- D) Es un sinónimo de método sobrecargado (overloaded)

19. ¿Cuál es el contrato (entrada y salida) de la interfaz funcional Supplier<T>?

- A) No recibe ningún argumento y devuelve un valor de tipo T, mediante el método get()
- B) Recibe un argumento de tipo T y no devuelve nada, mediante el método accept()
- C) Recibe un argumento de tipo T y devuelve un boolean, mediante el método test()
- D) Recibe dos argumentos de tipo T y devuelve uno de tipo T, mediante el método apply()

20. ¿Cuál es la diferencia de contrato entre Consumer<T> y BiConsumer<T,U>?

- A) Consumer<T> recibe un único argumento de tipo T y no devuelve nada (accept(T t)); BiConsumer<T,U> recibe dos argumentos de tipos T y U distintos y tampoco devuelve nada (accept(T t, U u))
- B) Consumer<T> devuelve un valor booleano, mientras BiConsumer<T,U> no devuelve nada
- C) BiConsumer<T,U> solo puede usarse con tipos numéricos
- D) Ambas interfaces son exactamente idénticas, solo cambia el nombre

21. ¿Qué relación de contrato tiene UnaryOperator<T> respecto a Function<T,R>?

- A) UnaryOperator<T> es un caso especial de Function<T,R> donde el tipo de entrada y el tipo de salida son exactamente el mismo (T), es decir, Function<T,T>
- B) UnaryOperator<T> recibe dos argumentos, a diferencia de Function que recibe uno solo
- C) UnaryOperator<T> no devuelve ningún valor, a diferencia de Function
- D) No existe ninguna relación entre ambas interfaces

22. ¿Qué relación de contrato tiene BinaryOperator<T> respecto a BiFunction<T,U,R>?

- A) BinaryOperator<T> es un caso especial de BiFunction donde ambos argumentos de entrada y el resultado son del mismo tipo T (equivalente a BiFunction<T,T,T>)
- B) BinaryOperator<T> recibe un único argumento, a diferencia de BiFunction que recibe dos
- C) BinaryOperator<T> siempre devuelve un boolean, a diferencia de BiFunction
- D) No existe relación alguna entre ambas interfaces

23. ¿Por qué existen versiones 'primitivas' especializadas de las interfaces funcionales, como IntPredicate, IntFunction<R> o ToIntFunction<T>, en lugar de usar siempre las versiones genéricas con Integer?

- A) Para evitar el costo de autoboxing/unboxing entre int e Integer en operaciones frecuentes, mejorando el rendimiento al trabajar directamente con el tipo primitivo en lugar de su wrapper
- B) Porque las versiones genéricas con Integer no existen en Java
- C) Porque las versiones primitivas permiten manejar excepciones checked, a diferencia de las genéricas
- D) Porque solo las versiones primitivas pueden usarse con expresiones lambda

24. ¿Qué método de la interfaz List permite eliminar todos los elementos que cumplan una condición dada, usando directamente un Predicate?

- A) removeIf(Predicate<? super E> filtro)
- B) deleteIf(Predicate filtro)
- C) filterOut(Predicate filtro)
- D) clearMatching(Predicate filtro)

25. ¿Qué método de la interfaz List permite reemplazar cada elemento aplicando una función de transformación (UnaryOperator), modificando la lista en el mismo lugar?

- A) replaceAll(UnaryOperator<E> operador)
- B) transformAll(Function operador)
- C) mapInPlace(Function operador)
- D) updateEach(Consumer operador)

26. ¿Qué contrato define la interfaz funcional BooleanSupplier, y en qué se diferencia de Supplier<Boolean>?

- A) BooleanSupplier define el método getAsBoolean(), que no recibe argumentos y devuelve un boolean primitivo, evitando el autoboxing/unboxing que ocurriría al usar Supplier<Boolean> (que devuelve un objeto Boolean)
- B) BooleanSupplier y Supplier<Boolean> son exactamente idénticos en todos los aspectos
- C) BooleanSupplier recibe un argumento booleano y no devuelve nada
- D) BooleanSupplier solo puede usarse dentro de un switch expression

27. ¿Qué significa que una variable local sea 'effectively final' (efectivamente final) para poder ser capturada dentro de una lambda?

- A) Significa que la variable, aunque no se haya declarado explícitamente con la palabra 'final', nunca es reasignada después de su inicialización a lo largo de todo su alcance; si se reasignara en algún punto, dejaría de ser effectively final y la lambda no podría capturarla
- B) Significa que la variable debe declararse obligatoriamente con la palabra 'final' de forma explícita para poder usarse en una lambda
- C) Significa que la variable es un campo de instancia, no una variable local
- D) Significa que la variable solo puede ser de tipo primitivo

28. ¿Por qué una lambda puede leer libremente un campo de instancia de la clase contenedora, sin la restricción de 'effectively final' que sí aplica a las variables locales?

- A) Porque los campos de instancia no se 'capturan por valor' como las variables locales; la lambda accede a ellos a través de una referencia implícita a 'this' (el objeto contenedor), por lo que ve siempre el valor actual del campo, incluso si cambia después
- B) Porque los campos de instancia siempre son static por definición
- C) Porque los campos de instancia nunca pueden usarse dentro de una lambda
- D) Porque los campos de instancia se convierten automáticamente en variables locales dentro de la lambda

29. ¿Qué conflicto de nombres puede surgir si un parámetro de una lambda tiene el mismo nombre que una variable local ya existente en el método contenedor, por ejemplo '(x) -> x + 1' cuando ya existe una variable local llamada 'x' en ese método?

- A) Error de compilación por conflicto de nombres ('variable already defined'), ya que el parámetro de la lambda no puede reutilizar el nombre de una variable local ya visible en ese mismo ámbito
- B) No hay ningún conflicto, Java permite la redefinición sin ningún problema
- C) Solo genera una advertencia (warning), pero compila y usa la variable local, ignorando el parámetro
- D) El parámetro de la lambda siempre tiene prioridad silenciosamente sobre la variable local externa

30. ¿En qué se diferencia el alcance (scope) de una variable local declarada dentro del cuerpo de una lambda, respecto a una variable de instancia de la clase contenedora?

- A) La variable local dentro de la lambda solo es visible dentro del cuerpo de esa lambda específica; la variable de instancia es visible en toda la clase (según su modificador de acceso) y persiste mientras exista el objeto, no solo durante la ejecución de la lambda
- B) Ambas tienen exactamente el mismo alcance y tiempo de vida
- C) La variable local dentro de la lambda es visible en toda la clase, igual que un campo de instancia
- D) Las variables de instancia no pueden coexistir con lambdas en la misma clase

31. ¿Qué requisito de tipos debe cumplirse para poder combinar dos Predicate con el método default or(), como en 'predicado1.or(predicado2)'?

- A) Ambos predicados deben operar sobre el mismo tipo genérico T (o tipos compatibles), ya que or() combina ambos en un nuevo Predicate<T> que evalúa la condición sobre el mismo tipo de argumento
- B) Los predicados deben ser de tipos genéricos completamente distintos para poder combinarse con or()
- C) or() solo puede usarse si ambos predicados provienen de expresiones lambda, nunca de referencias a método
- D) No existe ninguna restricción de tipos al usar or()

32. ¿Cuál es la diferencia de orden de ejecución entre los métodos default compose() y andThen() de la interfaz Function?

- A) f.compose(g) ejecuta primero g y luego aplica f al resultado (equivalente a f(g(x))); f.andThen(g) ejecuta primero f y luego aplica g al resultado (equivalente a g(f(x)))
- B) compose() y andThen() son exactamente sinónimos, con el mismo orden de ejecución
- C) compose() ejecuta ambas funciones en paralelo, mientras andThen() las ejecuta en secuencia
- D) andThen() solo puede usarse con Predicate, nunca con Function

33. Dadas dos funciones UnaryOperator<Integer> duplicar = x -> x * 2; y sumarUno = x -> x + 1;, ¿qué resultado produce 'duplicar.andThen(sumarUno).apply(3)'?

```
UnaryOperator<Integer> duplicar = x -> x * 2;
UnaryOperator<Integer> sumarUno = x -> x + 1;
duplicar.andThen(sumarUno).apply(3);
```

- A) 7 (primero se duplica 3 obteniendo 6, y luego se le suma 1, obteniendo 7)
- B) 8 (primero se suma 1 a 3 obteniendo 4, y luego se duplica, obteniendo 8)
- C) 6 (solo se aplica duplicar, ignorando sumarUno)
- D) 4 (solo se aplica sumarUno, ignorando duplicar)

34. ¿Puede usarse 'var' como nombre de un parámetro de una lambda, por ejemplo '(var x, var y) -> x + y'?

- A) Sí, se puede usar la palabra 'var' para inferir el tipo de los parámetros de una lambda desde Java 11, pero si se usa 'var' para un parámetro, debe usarse consistentemente para TODOS los parámetros de esa lambda, no se puede mezclar 'var' con tipos explícitos en la misma lista de parámetros
- B) No, 'var' nunca puede usarse en parámetros de lambda bajo ninguna circunstancia
- C) Sí, y se puede mezclar libremente con tipos explícitos en la misma lista de parámetros
- D) 'var' solo puede usarse si la lambda tiene un único parámetro

35. ¿Qué ocurre si una lambda declara un parámetro llamado 'var' usado como identificador simple, por ejemplo intentando escribir '(var) -> var + 1' donde 'var' es literalmente el nombre del parámetro?

- A) No es válido: 'var' no puede usarse como nombre de identificador para un parámetro de lambda, ya que 'var' es una palabra reservada contextual con un significado especial en ese contexto, no un identificador libre
- B) Es completamente válido, 'var' puede usarse libremente como cualquier otro identificador
- C) Solo es válido si el parámetro es de tipo primitivo
- D) Solo es válido dentro de un Predicate, no en otras interfaces funcionales

16. Referencias a Métodos (Method References)

1. ¿Cuál de estas es la sintaxis de una referencia a método estático?

- A) Clase::metodoEstatico
- B) Clase->metodoEstatico
- C) Clase.metodoEstatico()
- D) new Clase::metodo

2. ¿Qué representa la referencia a método `String::toUpperCase` usada como `Function<String,String>`?

- A) Una referencia a un método de instancia sobre un tipo arbitrario, equivalente a `s -> s.toUpperCase()`
- B) Una referencia a un constructor
- C) Una referencia a un método estático de la clase `String`
- D) Un error de compilación porque `toUpperCase` no es estático

3. ¿Cómo se escribe una referencia a constructor para crear instancias de `ArrayList` mediante un `Supplier`?

- A) `ArrayList::new`
- B) `new ArrayList()::create`
- C) `ArrayList.new()`
- D) `Supplier.of(ArrayList)`

4. ¿Cuál es la sintaxis de una referencia a un método de instancia sobre un objeto particular ya existente, por ejemplo `'obj::metodo'`?

- A) `objetoExistente::metodoDeInstancia` — el objeto ya está fijado como receptor de la llamada
- B) `Clase::metodoDeInstancia` sin objeto
- C) `objetoExistente->metodoDeInstancia`
- D) `new objetoExistente::metodo`

5. ¿A qué expresión lambda equivale la referencia de método `Math::max` usada como `BinaryOperator<Integer>`?

- A) `(a, b) -> Math.max(a, b)`
- B) `(a) -> Math.max(a)`
- C) `() -> Math.max()`
- D) `(a, b) -> a.max(b)`

6. ¿Cuáles son los cuatro tipos principales de referencias a métodos en Java?

- A) Referencia a método estático, referencia a método de instancia sobre un objeto particular ya existente, referencia a método de instancia sobre un objeto arbitrario de una clase, y referencia a constructor
- B) Referencia a método público, protegido, privado y de paquete
- C) Referencia a método de clase, de interfaz, de enum y de record
- D) Referencia a método con cero, uno, dos y tres parámetros

7. ¿Por qué se dice que 'el contexto' (la interfaz funcional de destino) es fundamental para interpretar correctamente una referencia a método?

- A) Porque una misma sintaxis de referencia a método (por ejemplo, Clase::metodo) puede interpretarse de formas distintas según los parámetros y el tipo de retorno que exija la interfaz funcional a la que se está asignando, especialmente al distinguir entre una referencia a método estático y una referencia sobre un objeto arbitrario
- B) Porque el contexto determina si la referencia a método puede lanzar excepciones o no
- C) Porque sin contexto ninguna referencia a método compila, sin excepción
- D) Porque el contexto solo afecta a las referencias a constructores, no a las demás

8. Dado 'Function<String, Integer> f = String::length;', ¿a qué tipo de referencia a método corresponde esta expresión?

- A) Referencia a un método de instancia sobre un objeto arbitrario de una clase: el primer parámetro de la interfaz funcional (el String de entrada) se convierte en el objeto sobre el cual se invoca el método length()
- B) Referencia a un método estático de la clase String
- C) Referencia a un constructor de la clase String
- D) Referencia a un método de instancia sobre un objeto particular ya existente

9. ¿Cómo afecta la sobrecarga de métodos (overloading) a la resolución de una referencia a método, por ejemplo 'Clase::metodo' cuando existen varias versiones sobrecargadas de 'metodo'?

- A) El compilador determina cuál versión sobrecargada usar comparando la firma de cada una contra el método abstracto de la interfaz funcional de destino, eligiendo (si existe) la que coincida en cantidad y tipos de parámetros y en el tipo de retorno
- B) La referencia a método siempre falla en compilación si el método referenciado está sobrecargado
- C) Se ejecutan todas las versiones sobrecargadas en secuencia al invocar la referencia
- D) Java elige aleatoriamente una de las versiones sobrecargadas en tiempo de ejecución

10. ¿Qué firma debe tener la interfaz funcional de destino para poder usarse con una referencia a constructor como 'ArrayList::new'?

- A) Debe tener un método abstracto cuyos parámetros (en tipo y cantidad) coincidan con alguno de los constructores disponibles de la clase, y cuyo tipo de retorno sea compatible con el tipo de la clase (o un supertipo de ella)
- B) Cualquier interfaz funcional puede usarse sin importar su firma, ya que 'new' ignora los parámetros esperados
- C) Debe ser exclusivamente la interfaz Supplier, ninguna otra interfaz es compatible con referencias a constructor
- D) Debe tener exactamente cero parámetros en todos los casos, sin excepción

11. ¿Cómo se usaría una referencia a método de instancia sobre un objeto particular ya existente junto con la interfaz Consumer<T>, por ejemplo para imprimir cada elemento de una lista con System.out::println?

- A) 'System.out' es el objeto particular ya existente, y println es su método de instancia; la referencia System.out::println es compatible con Consumer<T> porque recibe un argumento (el objeto a imprimir) y no devuelve nada, igual que el método accept(T t) de Consumer
- B) System.out::println solo puede usarse con Supplier, nunca con Consumer
- C) Esta referencia requiere que T sea obligatoriamente de tipo String
- D) No es posible usar System.out::println como referencia a método en ningún contexto

12. ¿Cómo encajaría una referencia a método estático de dos parámetros, como 'Integer::sum', con la interfaz BiFunction<Integer, Integer, Integer>?

- A) Integer.sum(int a, int b) es un método estático que recibe dos enteros y devuelve su suma; esto coincide exactamente con el contrato de BiFunction<Integer, Integer, Integer>, que recibe dos argumentos de tipo Integer y devuelve un Integer mediante apply(T t, U u)
- B) Integer::sum nunca puede usarse como BiFunction porque solo aplica a Consumer
- C) BiFunction no admite referencias a métodos estáticos, solo lambdas explícitas
- D) Integer::sum solo es compatible con UnaryOperator, no con BiFunction

13. ¿Cómo se usaría una referencia a método de instancia sobre un objeto arbitrario de una clase junto con la interfaz BiConsumer<T,U>, por ejemplo con 'List::add'?

- A) El primer parámetro de BiConsumer (de tipo T, la lista) se convierte en el objeto receptor sobre el cual se invoca add(), y el segundo parámetro (de tipo U) se pasa como argumento del método add(elemento); es decir, List::add equivale a (lista, elemento) -> lista.add(elemento)
- B) List::add solo puede usarse como Consumer de un solo parámetro, nunca como BiConsumer
- C) Esta referencia requiere que la lista sea inmutable para funcionar correctamente
- D) No es posible usar métodos de instancia con BiConsumer bajo ninguna circunstancia

14. ¿Qué ventaja práctica ofrecen las referencias a métodos frente a escribir la lambda equivalente de forma explícita, cuando el cuerpo de la lambda simplemente invoca un método ya existente?

- A) Hacen el código más legible y conciso al eliminar la sintaxis redundante de los parámetros y el cuerpo de la lambda, dejando claro de un vistazo qué método existente se está reutilizando como implementación de la interfaz funcional
- B) Las referencias a métodos siempre se ejecutan más rápido en tiempo de ejecución que cualquier lambda equivalente
- C) Las referencias a métodos permiten evitar por completo el uso de interfaces funcionales
- D) No existe ninguna ventaja real, son estrictamente equivalentes en todos los aspectos, incluida la legibilidad

17. Streams API

1. ¿Qué caracteriza a los streams en Java respecto a las colecciones que los originan?

- A) Un stream no almacena datos propios; procesa datos de una fuente de forma perezosa (lazy) y puede consumirse una sola vez
- B) Un stream modifica siempre la colección original
- C) Un stream se puede recorrer múltiples veces como una lista
- D) Un stream requiere que la colección esté ordenada previamente

2. ¿Cuál de las siguientes es una operación TERMINAL de Stream (no intermedia)?

- A) collect()
- B) filter()
- C) map()
- D) sorted()

3. ¿Qué hace el siguiente código?

List<String> nombres = List.of("Ana","Luis","Eva");

nombres.stream().filter(n -> n.length()>3).forEach(Svstem.out::println);

```
List<String> nombres = List.of("Ana", "Luis", "Eva");  
nombres.stream()  
.filter(n -> n.length() > 3)  
.forEach(System.out::println);
```

- A) Imprime solo "Luis" porque es el único nombre con más de 3 caracteres
- B) Imprime los tres nombres
- C) No compila
- D) Imprime "Ana" y "Eva"

4. ¿Qué clase se usa típicamente para acumular resultados de un Stream en una colección mediante collect(), como Collectors.toList()?

- A) java.util.stream.Collectors
- B) java.util.Collection
- C) java.util.Accumulator
- D) java.util.stream.Reducer

5. ¿Qué hace Collectors.groupingBy()?

- A) Agrupa los elementos de un stream en un Map según una función clasificadora, devolviendo un Map<clave, List<elementos>>
- B) Ordena los elementos alfabéticamente
- C) Elimina duplicados del stream
- D) Convierte el stream en un array

6. ¿Qué devuelve la operación terminal reduce() en un Stream?

- A) Combina los elementos del stream en un único resultado acumulado, aplicando repetidamente una función de combinación binaria
- B) Elimina elementos duplicados
- C) Divide el stream en dos partes
- D) Ordena los elementos de forma descendente

7. ¿Cuál es la diferencia entre un stream secuencial y uno paralelo (parallelStream())?

- A) El stream paralelo divide el procesamiento entre múltiples hilos usando el ForkJoinPool común, mientras que el secuencial procesa los elementos uno a uno en un solo hilo
- B) El stream paralelo siempre garantiza mantener el orden original de los elementos en cualquier operación
- C) No hay ninguna diferencia práctica de rendimiento
- D) El stream paralelo solo puede usarse con listas de tipo primitivo

8. ¿Qué método de Stream permite limitar la cantidad de elementos procesados, por ejemplo a los primeros 5?

- A) limit(5)
- B) take(5)
- C) first(5)
- D) cut(5)

9. ¿Qué método permite obtener un IntStream de números consecutivos, por ejemplo del 1 al 10?

- A) IntStream.rangeClosed(1, 10)
- B) IntStream.of(1, 10)
- C) IntStream.sequence(1, 10)
- D) IntStream.between(1, 10)

10. ¿Por qué se dice que un Stream 'envuelve' una fuente de datos (como una colección) en lugar de ser una estructura de datos en sí misma?

- A) Porque un Stream no almacena elementos propios; describe una secuencia de operaciones (filter, map, etc.) a aplicar sobre los datos de la fuente original, calculando resultados de forma perezosa solo cuando se invoca una operación terminal
- B) Porque un Stream siempre hace una copia física completa de la colección original antes de procesarla
- C) Porque un Stream modifica permanentemente la colección original de la que proviene
- D) Porque un Stream solo puede usarse con arrays, nunca con Collections

11. ¿Por qué se dice que un Stream es de 'uso único', y qué ocurre si se intenta reutilizar un Stream después de haber ejecutado una operación terminal sobre él?

- A) Una vez ejecutada una operación terminal (como collect(), forEach() o count()), el Stream queda 'consumido' y no puede volver a usarse; intentar invocar otra operación sobre el mismo Stream lanza IllegalStateException
- B) Un Stream puede reutilizarse indefinidamente sin ninguna restricción, igual que una List
- C) El Stream se reinicia automáticamente a su estado original después de cada operación terminal
- D) Solo los streams paralelos tienen esta restricción de uso único, los secuenciales no

12. ¿Qué precaución especial se debe tomar al trabajar con un stream infinito, como el generado por `Stream.iterate()` o `Stream.generate()` sin límite explícito?

- A) Es imprescindible aplicar una operación intermedia que acote el stream, como `limit(n)`, antes de una operación terminal; de lo contrario, el programa intentaría procesar infinitos elementos y nunca terminaría (o consumiría toda la memoria/CPU disponible)
- B) Los streams infinitos siempre se limitan automáticamente a 1000 elementos por la JVM
- C) No es necesario tomar ninguna precaución especial, Java maneja automáticamente los streams infinitos sin ningún riesgo
- D) Un stream infinito solo puede usarse con operaciones terminales que devuelvan boolean

13. ¿Qué diferencia hay entre las operaciones terminales `findFirst()` y `findAny()` de un Stream?

- A) `findFirst()` garantiza devolver el primer elemento que cumple la condición según el orden del stream (relevante en streams secuenciales u ordenados); `findAny()` puede devolver cualquier elemento que cumpla la condición, sin garantía de orden, lo cual permite mayor eficiencia en streams paralelos
- B) Ambos métodos son exactamente sinónimos, sin ninguna diferencia de comportamiento
- C) `findAny()` siempre devuelve más elementos que `findFirst()`
- D) `findFirst()` solo puede usarse con streams paralelos, nunca con streams secuenciales

14. ¿Por qué operaciones como `Stream.max(Comparator)` y `Stream.min(Comparator)` devuelven un `Optional` en lugar del valor directamente?

- A) Porque el stream podría estar vacío (sin ningún elemento), en cuyo caso no existiría ningún máximo o mínimo que devolver; `Optional` comunica explícitamente esa posible ausencia de resultado, en lugar de devolver `null` o lanzar una excepción inesperada
- B) Porque `max()` y `min()` siempre requieren que el stream contenga exactamente un elemento
- C) Porque `Optional` mejora el rendimiento de estas operaciones sobre streams grandes
- D) Porque el tipo de retorno `Optional` es obligatorio para cualquier operación terminal de `Stream`, sin excepción

15. ¿Cuáles son las tres variantes (sobrecargas) del método `reduce()` de `Stream`, y qué las diferencia principalmente?

- A) `reduce(BinaryOperator)` que devuelve `Optional<T>` (sin valor identidad); `reduce(T identidad, BinaryOperator)` que devuelve `T` directamente (con un valor inicial/identidad); y `reduce(T identidad, BiFunction acumulador, BinaryOperator combinador)` para streams paralelos, donde el combinador une resultados parciales de distintos hilos
- B) Las tres versiones de `reduce()` son exactamente idénticas en firma y comportamiento
- C) `reduce()` solo tiene una única forma posible, sin sobrecargas adicionales
- D) Las tres versiones difieren únicamente en el tipo de dato que procesan (`int`, `double`, `String`), no en su estructura

16. ¿Por qué el 'valor de identidad' (`identity`) usado en `reduce()` debe ser un elemento neutro respecto a la operación de combinación, por ejemplo 0 para una suma o 1 para una multiplicación?

- A) Porque combinar el valor de identidad con cualquier otro elemento mediante la operación no debe alterar el resultado (por ejemplo, sumar 0 no cambia nada, multiplicar por 1 tampoco), garantizando que el resultado final sea correcto independientemente de cómo se distribuya el cálculo (por ejemplo, en un stream paralelo)
- B) El valor de identidad puede ser cualquier número arbitrario sin ninguna restricción especial
- C) El valor de identidad siempre debe ser el mayor valor posible del tipo de dato usado
- D) El valor de identidad solo es relevante en streams secuenciales, no en paralelos

17. ¿Cuál es la diferencia esencial entre las operaciones intermedias filter() y map() de un Stream?

- A) filter() selecciona un subconjunto de los elementos originales según una condición (Predicate), sin transformarlos; map() transforma cada elemento en otro valor (posiblemente de un tipo distinto), manteniendo la misma cantidad de elementos en el stream resultante
- B) filter() y map() son operaciones exactamente equivalentes, solo cambia el nombre
- C) map() reduce la cantidad de elementos del stream, igual que filter()
- D) filter() siempre convierte los elementos a otro tipo de dato, igual que map()

18. Al procesar un Stream<String> de números representados como texto (por ejemplo "3", "7", "2") para sumarlos como enteros, ¿qué combinación de operaciones sería típica usando map() y reduce()?

- A) stream.map(Integer::parseInt).reduce(0, Integer::sum); primero se transforma cada String a int con map() usando la referencia a método Integer::parseInt, y luego se suman todos los valores con reduce(), usando 0 como identidad e Integer::sum como operador binario
- B) stream.filter(Integer::parseInt).collect(Collectors.toList()); usando filter() en lugar de map() para la conversión de tipos
- C) stream.reduce("", String::concat).map(Integer::parseInt); aplicando reduce() antes que map()
- D) No es posible sumar Strings numéricos usando Stream, se requiere un bucle for tradicional obligatoriamente

19. ¿Qué hace Collectors.toCollection(TreeSet::new) al usarse en collect(), en comparación con Collectors.toList()?

- A) Permite recolectar los elementos del stream en un tipo de colección específico (en este caso, un TreeSet, que elimina duplicados y mantiene orden), en lugar de la List genérica que produce toList()
- B) Collectors.toCollection() solo puede usarse con ArrayList, nunca con otras implementaciones
- C) Ambos collectors producen exactamente el mismo tipo de resultado, sin ninguna diferencia
- D) Collectors.toCollection(TreeSet::new) requiere que los elementos sean todos de tipo String

20. ¿Qué hace flatMap() en un Stream cuando cada elemento original produce, a su vez, otro Stream (por ejemplo, un Stream<List<Integer>>)?

- A) Aplica la función dada a cada elemento (que produce un Stream individual) y luego 'aplana' todos esos streams internos en un único Stream continuo, en lugar de dejar un Stream de streams anidados
- B) Simplemente cuenta cuántos streams internos existen, sin combinarlos
- C) Elimina todos los elementos duplicados entre los distintos streams internos
- D) Convierte automáticamente cada stream interno en una lista separada, sin combinarlos

21. ¿Cuál es la diferencia principal entre map() y flatMap() en cuanto a la relación entre elementos de entrada y de salida?

- A) map() transforma cada elemento en exactamente OTRO elemento (relación uno a uno); flatMap() transforma cada elemento en un Stream de posiblemente varios elementos, que luego se aplanan (relación uno a muchos)
- B) map() y flatMap() son exactamente equivalentes en cualquier situación
- C) flatMap() siempre reduce el stream a un único elemento final
- D) map() solo puede usarse con tipos primitivos, flatMap() solo con objetos

22. ¿Cuál es la diferencia clave entre peek() y forEach() en el contexto de un pipeline de Stream?

- A) peek() es una operación INTERMEDIA (perezosa, pensada principalmente para depuración, que no dispara la ejecución por sí sola); forEach() es una operación TERMINAL que sí dispara el procesamiento completo del stream
- B) peek() y forEach() son sinónimos exactos, con el mismo comportamiento
- C) forEach() es intermedia y peek() es terminal, al contrario de lo habitual
- D) peek() solo puede usarse en streams paralelos, nunca en secuenciales

23. ¿Por qué modificar estructuralmente (añadir o eliminar elementos de) la colección fuente mientras se procesa un Stream derivado de ella puede causar problemas o comportamiento indefinido?

- A) Porque el Stream no crea una copia independiente de los datos; opera sobre la fuente original de forma perezosa, por lo que modificar esa fuente durante el procesamiento puede causar ConcurrentModificationException o resultados inconsistentes/indefinidos
- B) Porque los streams siempre bloquean automáticamente la colección fuente contra cualquier modificación, sin excepción
- C) Porque modificar la colección fuente durante el procesamiento del stream no tiene ningún efecto, ya que el stream siempre usa una copia interna
- D) Porque esto solo es un problema en streams paralelos, nunca en streams secuenciales

24. ¿Qué ventaja de rendimiento ofrecen los streams primitivos especializados (IntStream, LongStream, DoubleStream) frente a un Stream<Integer>, Stream<Long> o Stream<Double> genérico?

- A) Evitan el costo de autoboxing/unboxing al operar directamente sobre valores primitivos internamente, en lugar de envolver cada valor en su clase wrapper correspondiente (Integer, Long, Double), mejorando el rendimiento en operaciones numéricas intensivas
- B) No existe ninguna diferencia de rendimiento real entre ambos enfoques
- C) Los streams primitivos solo pueden procesar un elemento a la vez, a diferencia de los genéricos
- D) Los streams primitivos no pueden usarse con operaciones terminales como sum() o average()

25. ¿Cómo se puede convertir una List<Integer> en un DoubleStream, aplanando y transformando en un solo paso mediante flatMap y mapToDouble?

- A) listaDeListas.stream().flatMap(List::stream).mapToDouble(Integer::doubleValue); primero se aplanan la estructura anidada con flatMap(List::stream), y luego se convierte cada Integer a double primitivo con mapToDouble(Integer::doubleValue)
- B) listaDeListas.stream().mapToDouble(List::stream); intentando usar mapToDouble directamente sobre listas anidadas sin aplanar primero
- C) No es posible realizar esta conversión combinando flatMap y mapToDouble en el mismo pipeline
- D) listaDeListas.parallelStream().toDouble(); usando un método toDouble() inexistente

26. ¿Qué método se usa para extraer el valor primitivo double contenido en un OptionalDouble, análogo a get() en un Optional<T> genérico?

- A) getAsDouble()
- B) get()
- C) doubleValue()
- D) orElseDouble()

27. ¿Qué operaciones estadísticas convenientes ofrecen directamente los streams primitivos (IntStream, DoubleStream, etc.) que no están disponibles de la misma forma en un Stream<T> genérico?

- A) Métodos como sum(), average() (que devuelve OptionalDouble), max(), min() y summaryStatistics(), todos calculando directamente estadísticas numéricas sobre los valores primitivos del stream
- B) Los streams primitivos no ofrecen ninguna ventaja estadística especial sobre los genéricos
- C) Solo ofrecen el método count(), igual que cualquier Stream genérico
- D) Solo pueden calcular la suma, ningún otro tipo de estadística

28. ¿En qué se diferencia el comportamiento de map() y filter() cuando se aplican sobre un Optional<T>, en comparación con su comportamiento sobre un Stream<T>?

- A) Sobre un Optional, map()/filter() operan sobre 'como máximo un valor' (aplicando la transformación o condición solo si el Optional tiene valor presente, devolviendo Optional vacío en caso contrario); sobre un Stream, operan sobre potencialmente muchos elementos de la secuencia
- B) map() y filter() se comportan exactamente igual en Optional y en Stream, sin ninguna diferencia conceptual
- C) Optional no tiene métodos map() ni filter() disponibles, solo Stream los tiene
- D) filter() sobre un Optional siempre lanza una excepción si el Optional está vacío

29. ¿Por qué es recomendable evitar tener un 'Optional dentro de otro Optional' (por ejemplo, Optional<Optional<String>>), y cómo ayuda flatMap() en este contexto?

- A) Un Optional anidado es confuso e innecesariamente complejo de manejar (hay que 'desempaquetar' dos veces); usar flatMap() en lugar de map() cuando la función de transformación ya devuelve un Optional evita ese anidamiento, aplanando el resultado en un único Optional en lugar de uno doblemente envuelto
- B) Los Optional anidados son en realidad más eficientes en memoria que un Optional simple
- C) No existe ninguna forma de evitar Optional anidados en Java
- D) map() y flatMap() producen siempre el mismo resultado sobre un Optional, sin ninguna diferencia

30. ¿Por qué una expresión lambda usada dentro de una operación de Stream (como map()) NO puede lanzar directamente una excepción checked, si la interfaz funcional correspondiente (como Function) no la declara?

- A) Porque el cuerpo de la lambda debe respetar la firma del método abstracto de la interfaz funcional que implementa; si ese método (como apply() de Function) no declara throws para esa excepción checked, el compilador rechaza que la lambda la lance directamente
- B) Porque las excepciones checked están completamente prohibidas en cualquier parte de un programa Java que use streams
- C) Porque los streams capturan automáticamente cualquier excepción sin necesidad de manejo explícito
- D) Porque solo las excepciones unchecked pueden existir en Java, las checked fueron eliminadas en Java

17

31. ¿Cuál es una estrategia común para 'envolver' una excepción checked lanzada dentro de una lambda usada en un Stream, de forma que el código compile?

- A) Capturar la excepción checked dentro de un bloque try-catch en el cuerpo de la lambda, y relanzarla envuelta en una excepción unchecked (como una RuntimeException personalizada), que sí es compatible con la firma de la interfaz funcional sin necesidad de declarar throws
- B) Es imposible manejar excepciones checked de ninguna forma dentro de una lambda usada en streams
- C) Basta con añadir la palabra 'checked' antes de la lambda para que compile automáticamente
- D) Se debe convertir el Stream completo en una lista antes de poder manejar cualquier excepción

32. ¿Qué propósito cumple la interfaz `Splitter` en el procesamiento de streams?

- A) Permite dividir ('split') una fuente de datos en partes más pequeñas que pueden procesarse de forma independiente (potencialmente en paralelo), además de recorrer secuencialmente los elementos, siendo la base sobre la que se implementa el procesamiento paralelo eficiente de streams
- B) Sirve únicamente para ordenar alfabéticamente los elementos de un stream
- C) Es una interfaz obsoleta que fue reemplazada completamente por `Iterator` desde Java 8
- D) Solo puede usarse con streams de tipo `String`, nunca con otros tipos de datos

33. ¿Qué hace el `Collector Collectors.joining(delimitador)` al aplicarse sobre un `Stream<String>`?

- A) Concatena todos los elementos `String` del stream en un único `String` resultante, insertando el delimitador especificado entre cada par de elementos consecutivos
- B) Divide cada `String` del stream en partes separadas según el delimitador dado
- C) Cuenta cuántas veces aparece el delimitador dentro de cada `String` del stream
- D) Ordena los `Strings` del stream según el delimitador proporcionado como criterio

34. ¿Qué problema resuelve la función de fusión (merge function) que se puede proporcionar como tercer argumento de `Collectors.toMap(keyMapper, valueMapper, mergeFunction)`?

- A) Determina cómo combinar los valores cuando dos elementos del stream producen la MISMA clave al aplicar `keyMapper`, evitando la excepción `IllegalStateException` que ocurriría por defecto ante claves duplicadas
- B) Determina el tipo de mapa (`HashMap`, `TreeMap`) que se usará como resultado final
- C) Ordena las claves del mapa resultante alfabéticamente
- D) Convierte automáticamente todas las claves a mayúsculas antes de insertarlas

35. ¿Qué garantiza siempre el resultado de `Collectors.partitioningBy(predicado)`, a diferencia de `groupingBy()`?

- A) `partitioningBy()` siempre produce un `Map<Boolean, List<T>>` con exactamente dos claves (`true` y `false`), incluso si alguna de las dos particiones queda vacía; `groupingBy()` puede producir un número variable de claves según los valores distintos generados por la función clasificadora
- B) `partitioningBy()` y `groupingBy()` son exactamente idénticos en su tipo de resultado
- C) `partitioningBy()` puede producir cualquier cantidad de particiones, igual que `groupingBy()`
- D) `partitioningBy()` nunca puede usarse con un `Predicate`, solo con una `Function`

36. ¿Qué hace la combinación '`Collectors.groupingBy(funcionClasificadora, Collectors.counting())`'?

- A) Agrupa los elementos según la función clasificadora, y para cada grupo, en lugar de una lista de elementos, almacena el CONTEO (`Long`) de cuántos elementos cayeron en ese grupo
- B) Agrupa los elementos y calcula la suma total de todos los grupos combinados
- C) Elimina los grupos que tengan más de un elemento
- D) Produce el mismo resultado que `groupingBy()` sin el segundo argumento, ignorándolo

37. ¿Qué recomendación práctica se sugiere al construir un pipeline de Stream complejo con múltiples operaciones intermedias encadenadas?

- A) Comenzar construyendo y verificando el pipeline con operaciones simples primero, dividiendo la lógica en partes más pequeñas y manejables para facilitar la depuración, en lugar de escribir de una sola vez una cadena larga y compleja de operaciones
- B) Escribir siempre el pipeline completo de una sola vez, sin dividirlo en partes, para maximizar el rendimiento
- C) Evitar por completo el uso de operaciones intermedias encadenadas, usando solo una operación terminal directa
- D) Los pipelines de Stream nunca deben superar una única operación intermedia, por regla general

38. ¿Qué ventaja ofrece calcular múltiples estadísticas (como suma, promedio, mínimo y máximo) en una sola pasada usando un Collector como summarizingInt/summarizingDouble, en lugar de procesar el stream varias veces (una por cada estadística)?

- A) Permite obtener todas esas métricas procesando la fuente de datos una única vez, evitando el costo de recorrer (o consumir) el stream múltiples veces, lo cual además sería imposible directamente ya que un stream es de uso único
- B) No existe ninguna ventaja real de rendimiento entre ambos enfoques
- C) Un Collector de resumen (summarizing) solo puede calcular una única estadística a la vez, igual que procesar el stream varias veces
- D) Esta técnica solo funciona con streams paralelos, nunca con streams secuenciales

18. Optional

1. ¿Cuál es el propósito principal de la clase `Optional<T>`?

- A) Representar explícitamente la posible ausencia de un valor, evitando `NullPointerException` y forzando un manejo explícito del caso 'vacío'
- B) Reemplazar por completo el uso de `null` en cualquier contexto, incluidos campos de clase
- C) Mejorar el rendimiento de las colecciones
- D) Servir como una nueva colección ordenada

2. ¿Qué hace `Optional.ofNullable(valor)` si 'valor' es `null`?

- A) Devuelve un `Optional` vacío (`Optional.empty()`) sin lanzar excepción
- B) Lanza `NullPointerException` inmediatamente
- C) Devuelve `null`
- D) No compila

3. ¿Cuál es una forma recomendada de obtener un valor por defecto de un `Optional` si está vacío?

- A) `optional.orElse(valorPorDefecto)`
- B) `optional.getDefault()`
- C) `optional.value()`
- D) `optional.orNull()`

4. ¿Qué hace `Optional.map(funcion)` si el `Optional` está vacío?

- A) Devuelve directamente un `Optional` vacío, sin aplicar la función
- B) Lanza `NoSuchElementException`
- C) Ejecuta la función igualmente con un valor `null`
- D) No compila

5. ¿Qué excepción lanza `optional.get()` si el `Optional` está vacío?

- A) `NoSuchElementException`
- B) `NullPointerException`
- C) `IllegalStateException`
- D) `OptionalNotPresentException`

6. ¿Cuál es una buena práctica respecto al uso de `Optional` como tipo de un campo de clase o parámetro de método?

- A) Se recomienda evitarlo: `Optional` está pensado principalmente como tipo de retorno de métodos, no para campos ni parámetros
- B) Se recomienda usar `Optional` en todos los campos de todas las clases
- C) `Optional` debe usarse siempre en lugar de tipos primitivos
- D) No existe ninguna recomendación al respecto

19. Collections Framework (List, Set, Map, Deque)

1. ¿Cuál es la diferencia principal entre List, Set y Map en el Collections Framework?

- A) List permite duplicados y mantiene orden de inserción; Set no permite duplicados; Map almacena pares clave-valor
- B) List no permite duplicados; Set sí los permite; Map es solo para arrays
- C) Los tres son idénticos, solo cambia el nombre
- D) Set mantiene siempre orden alfabético automáticamente y List nunca permite acceso por índice

2. ¿Qué implementación de List garantiza acceso rápido $O(1)$ por índice, pero inserciones/eliminaciones costosas en el medio?

- A) ArrayList
- B) LinkedList
- C) Vector solamente
- D) TreeSet

3. ¿Qué característica distingue a un TreeMap de un HashMap?

- A) TreeMap mantiene sus claves ordenadas (orden natural o Comparator); HashMap no garantiza ningún orden
- B) TreeMap permite claves duplicadas y HashMap no
- C) HashMap es más lento que TreeMap en todos los casos
- D) TreeMap no permite valores null nunca y HashMap sí siempre

4. ¿Qué devuelve List.of(1,2,3) en cuanto a mutabilidad?

- A) Una lista inmutable; intentar añadir o eliminar elementos lanza UnsupportedOperationException
- B) Una lista completamente mutable equivalente a ArrayList
- C) Un array primitivo int[]
- D) Un Set en lugar de una List

5. ¿Qué debe sobrescribir correctamente una clase para usarse como clave en un HashMap de forma consistente?

- A) equals() y hashCode() (contrato de Object)
- B) Comparable obligatoriamente
- C) Serializable obligatoriamente
- D) Cloneable obligatoriamente

6. ¿Qué estructura de datos implementa típicamente una Queue de tipo FIFO (primero en entrar, primero en salir)?

- A) LinkedList (implementando Queue)
- B) TreeSet
- C) Stack únicamente
- D) HashMap

7. ¿Qué diferencia hay entre Iterator y ListIterator?

- A) ListIterator extiende Iterator añadiendo la capacidad de recorrer en ambas direcciones y modificar elementos durante la iteración (add, set)
- B) Iterator solo funciona con Map, ListIterator solo con List
- C) No hay ninguna diferencia real entre ambos
- D) ListIterator no permite eliminar elementos mientras que Iterator sí

8. ¿Qué excepción se lanza al modificar una colección (por ejemplo, con remove()) directamente mientras se itera sobre ella con un for-each, sin usar el propio Iterator?

- A) ConcurrentModificationException
- B) UnsupportedOperationException
- C) IllegalStateException
- D) NoSuchElementException

9. ¿Cuáles son las tres interfaces principales del Collections Framework de Java que representan las estructuras de datos fundamentales?

- A) List, Set y Map
- B) Array, Stream y Optional
- C) Iterator, Iterable y Comparator
- D) Queue, Stack y Vector únicamente

10. ¿Qué estructura permite usar una misma implementación tanto como Queue (FIFO, primero en entrar-primero en salir) como Stack (LIFO, último en entrar-primero en salir)?

- A) Deque (double-ended queue), que permite inserciones y eliminaciones eficientes en ambos extremos, sirviendo tanto para comportamiento FIFO como LIFO según los métodos que se usen
- B) Solo ArrayList puede comportarse como Queue y Stack simultáneamente
- C) TreeSet, gracias a su orden natural interno
- D) HashMap, usando las claves como orden de inserción

11. ¿Qué criterio de ordenamiento usa un TreeSet por defecto para mantener sus elementos ordenados, si no se le proporciona un Comparator explícito?

- A) El orden natural definido por el método compareTo() de la interfaz Comparable, que los elementos deben implementar en ese caso
- B) Un orden completamente aleatorio en cada ejecución del programa
- C) El orden de inserción de los elementos, igual que una LinkedHashSet
- D) El orden alfabético del nombre de la clase de cada elemento, sin relación con su contenido

12. En un ejemplo con una clase Cliente añadida a un HashSet<Cliente>, si Cliente no sobrescribe equals() ni hashCode(), ¿qué ocurre al intentar añadir un 'Cliente' con los mismos datos (pero como un objeto diferente) para verificar si ya existe?

- A) El HashSet lo trata como un elemento distinto y lo añade igualmente, ya que sin sobrescribir equals()/hashCode() se usa la implementación heredada de Object (comparación por identidad de referencia), y dos objetos distintos con el mismo contenido no se consideran iguales
- B) El HashSet detecta automáticamente que el contenido es igual y rechaza la inserción duplicada, sin importar si equals()/hashCode() están sobrescritos
- C) Se lanza una excepción en tiempo de ejecución al intentar añadir un Cliente 'duplicado'
- D) El HashSet reemplaza automáticamente el Cliente anterior con el nuevo, fusionando ambos objetos

13. ¿Por qué es importante que hashCode() sea consistente con equals() (es decir, que objetos considerados iguales por equals() produzcan el mismo hashCode())?

- A) Porque las colecciones basadas en tablas hash (como HashSet o HashMap) usan hashCode() para determinar en qué 'bucket' buscar un elemento antes de usar equals() para confirmar la igualdad exacta; si dos objetos iguales según equals() tuvieran distinto hashCode(), la colección podría no encontrarlos correctamente como duplicados
- B) Porque hashCode() y equals() son métodos completamente independientes sin ninguna relación funcional entre sí
- C) Porque hashCode() determina el orden de iteración en un TreeSet
- D) Porque sin esta consistencia, el programa no compila

14. ¿Qué tipo de lista devuelve Arrays.asList(array), y qué restricción particular tiene respecto a su tamaño?

- A) Devuelve una lista de tamaño fijo respaldada directamente por el array original; se pueden modificar los elementos existentes (set), pero no se pueden añadir ni eliminar elementos (add/remove lanzan UnsupportedOperationException)
- B) Devuelve una lista completamente inmutable donde ni siquiera se pueden modificar los elementos existentes
- C) Devuelve una copia independiente del array, sin ninguna relación con el array original
- D) Devuelve una lista de tamaño dinámico idéntica a una ArrayList normal

15. ¿Qué ocurre si se modifica un elemento del array original después de haber creado una lista con Arrays.asList(array)?

- A) El cambio se refleja también en la lista, ya que ambos (array y lista) comparten el mismo almacenamiento subyacente; no son estructuras independientes
- B) La lista permanece completamente aislada del array, sin verse afectada por cambios posteriores en el array
- C) Se lanza una excepción automáticamente al modificar el array después de crear la lista
- D) El cambio solo se refleja si la lista se declaró como var

16. ¿Qué diferencia hay entre List.copyOf(coleccionOriginal) y simplemente reutilizar la colección original para crear una lista 'immutable'?

- A) List.copyOf() crea una nueva colección independiente, completamente inmutable, con una copia de los elementos actuales; los cambios posteriores en la colección original ya NO afectan a esta copia inmutable, a diferencia de Arrays.asList()
- B) List.copyOf() y Arrays.asList() se comportan exactamente igual en todos los aspectos
- C) List.copyOf() modifica la colección original para hacerla inmutable in situ
- D) List.copyOf() solo funciona con arrays, nunca con otras colecciones existentes

17. ¿Por qué insertar un elemento en una posición intermedia de una ArrayList es generalmente más costoso que hacerlo en una LinkedList?

- A) Porque ArrayList se respalda en un array interno, y al insertar en el medio es necesario desplazar todos los elementos posteriores una posición, lo cual es una operación $O(n)$; LinkedList solo necesita reajustar las referencias de los nodos vecinos, una operación $O(1)$ una vez ubicada la posición
- B) Porque LinkedList no permite insertar elementos en posiciones intermedias bajo ninguna circunstancia
- C) Porque ArrayList siempre tiene un tamaño fijo que no puede modificarse tras su creación
- D) Porque LinkedList siempre almacena los elementos en un array oculto internamente, igual que ArrayList

18. ¿Qué hace el método `removeIf(Predicate)` al aplicarse sobre una colección mutable, por ejemplo `'lista.removeIf(n -> n < 0);'` para eliminar todos los números negativos?

- A) Recorre la colección y elimina todos los elementos para los cuales el Predicate dado devuelve true, modificando la colección original directamente sin necesidad de un bucle explícito ni de gestionar manualmente un Iterator
- B) Crea una nueva colección con los elementos filtrados, dejando la colección original sin cambios
- C) Solo marca los elementos como eliminados, pero no reduce el tamaño real de la colección
- D) Requiere que la colección esté ordenada previamente para funcionar correctamente

19. En una `LinkedList<Integer>`, ¿qué diferencia hay entre invocar `remove(1)` y `remove(Integer.valueOf(1))`?

- A) `remove(1)` interpreta el argumento como un índice (int) y elimina el elemento en la posición 1; `remove(Integer.valueOf(1))` interpreta el argumento como un objeto Integer y elimina la PRIMERA OCURRENCIA del valor 1 en la lista, sin importar su posición
- B) Ambas invocaciones son exactamente equivalentes en cualquier caso
- C) `remove(1)` siempre lanza una excepción de compilación por ambigüedad
- D) `remove(Integer.valueOf(1))` elimina todos los elementos con valor 1 en la lista, no solo el primero

20. ¿Por qué esta ambigüedad entre `remove(int)` y `remove(Object)` es especialmente relevante en listas de tipo Integer, pero no representa un problema en listas de tipo String, por ejemplo?

- A) Porque Integer es un tipo que también puede interpretarse como un índice numérico (int), generando la ambigüedad de cuál sobrecarga de `remove()` aplica; String no tiene esa ambigüedad porque nunca podría confundirse con un índice int
- B) Porque String no tiene ningún método `remove()` disponible en absoluto
- C) Porque Integer nunca puede eliminarse de una lista bajo ninguna circunstancia
- D) Porque las listas de tipo String no permiten sobrecarga de métodos

21. ¿Qué combinación de pasos permite convertir el conjunto de pares clave-valor de un Map en una `List<Map.Entry<K,V>>` para, por ejemplo, poder ordenarlos con `Collections.sort()`?

- A) Invocar `mapa.entrySet()` para obtener un `Set<Map.Entry<K,V>>`, y luego pasar ese Set al constructor de una `ArrayList` (o usar `new ArrayList<>(mapa.entrySet())`) para obtener una lista mutable y ordenable
- B) Los pares clave-valor de un Map nunca pueden convertirse a una List bajo ninguna circunstancia
- C) Basta con llamar directamente a `mapa.sort()` sin ningún paso intermedio
- D) Solo es posible convertir las claves (`keySet()`) a lista, nunca los pares completos

22. ¿Por qué `Collections.binarySearch()` sobre una `LinkedList` suele ser menos eficiente que sobre una `ArrayList`, aunque ambas puedan usarse con el método?

- A) Porque `LinkedList` no permite acceso directo (aleatorio) $O(1)$ a una posición intermedia como sí lo hace `ArrayList`; para acceder al 'elemento del medio' en cada paso del algoritmo de búsqueda binaria, `LinkedList` debe recorrer secuencialmente los nodos desde un extremo, degradando el rendimiento del algoritmo
- B) Porque `LinkedList` nunca puede usarse con `Collections.binarySearch()`, el método lanza una excepción
- C) Porque `ArrayList` siempre está desordenada, a diferencia de `LinkedList`
- D) Porque `LinkedList` solo permite almacenar tipos primitivos, no objetos comparables

23. ¿Qué papel juegan Comparable y Comparator al momento de decidir cómo se ordena un TreeSet de objetos personalizados antes de aplicar cualquier búsqueda o iteración ordenada?

- A) Si no se proporciona un Comparator al crear el TreeSet, este usará el orden natural definido por compareTo() (Comparable) de los elementos; si se proporciona un Comparator explícito al constructor del TreeSet, este tiene prioridad sobre el orden natural, sin importar si los elementos también implementan Comparable
- B) El TreeSet siempre ignora tanto Comparable como Comparator, usando el orden de inserción
- C) Comparator solo puede usarse si los elementos no implementan Comparable en absoluto
- D) Comparable tiene siempre prioridad sobre cualquier Comparator proporcionado explícitamente

24. ¿Qué hace el método setValue(nuevoValor) de la interfaz Map.Entry, y qué efecto tiene sobre el Map original del cual proviene esa entrada?

- A) Modifica el valor asociado a esa entrada directamente en el Map subyacente del cual se obtuvo (siempre que ese Map sea mutable), reflejando el cambio inmediatamente en el mapa original, y devuelve el valor anterior que tenía esa entrada
- B) Solo modifica una copia local de la entrada, sin afectar al Map original en ningún caso
- C) Lanza una excepción siempre que se invoque, independientemente del tipo de Map
- D) Elimina la entrada completa del Map, sustituyéndola por una nueva

25. ¿Qué ocurre si se invoca setValue() sobre una entrada obtenida del entrySet() de un mapa inmutable, como uno creado con Map.of()?

- A) Se lanza UnsupportedOperationException, ya que las estructuras (y sus entradas) provenientes de colecciones inmutables como las creadas con Map.of() no permiten ningún tipo de modificación, incluido cambiar el valor de una entrada individual
- B) El cambio se aplica sin ningún problema, ya que setValue() siempre está permitido independientemente de la mutabilidad del mapa de origen
- C) Se crea automáticamente un nuevo mapa mutable con el cambio aplicado
- D) El valor cambia solo temporalmente, revirtiéndose al finalizar la iteración

26. Si se necesita modificar los valores de una lista o mapa obtenido de una fuente inmutable (por ejemplo, Map.of() o List.of()), ¿cuál es una estrategia común para lograrlo?

- A) Crear una copia mutable independiente a partir de la estructura inmutable (por ejemplo, con 'new ArrayList<>(listaInmutable)' o 'new HashMap<>(mapaInmutable)'), y aplicar las modificaciones sobre esa nueva copia mutable, dejando la estructura original inmutable intacta
- B) Modificar directamente la estructura inmutable original usando reflection para evitar la restricción
- C) No existe ninguna forma de trabajar con los datos de una estructura inmutable; hay que descartarlos por completo
- D) Usar únicamente el método replaceAll() directamente sobre la estructura inmutable original, ya que sí está permitido en estas colecciones

27. ¿En qué posición exacta de la firma de un método static debe declararse su propio parámetro de tipo genérico, por ejemplo en 'public static <T extends Comparable<T>> T maximo(List<T> lista)'?

- A) Inmediatamente antes del tipo de retorno del método (en este caso, antes de la segunda 'T'), no antes de la palabra 'static' ni en ningún otro lugar de la firma
- B) Después del nombre del método, entre paréntesis junto a los parámetros normales
- C) Antes de la palabra 'static', al inicio de toda la declaración del método
- D) No es necesario declarar ninguna posición especial, el compilador lo infiere automáticamente sin sintaxis explícita

28. ¿Por qué un método genérico static que declara su propio parámetro de tipo T (como en '<T> void metodo(T valor)') Sí es válido, mientras que intentar usar directamente el T de la clase contenedora en un método static NO lo es?

- A) Porque en el primer caso, el método declara y resuelve su propio parámetro de tipo T de forma independiente en cada invocación, sin depender de ninguna instancia; en el segundo caso, el T de la clase está asociado a una instancia específica (Clase<String>, Clase<Integer>, etc.), lo cual es incompatible con un contexto static que no tiene instancia asociada
- B) Ambos casos son igual de inválidos y ninguno compila
- C) Ambos casos son igual de válidos sin ninguna diferencia real
- D) La diferencia depende únicamente de si el método tiene parámetros adicionales o no

20. Concurrency y Virtual Threads

1. ¿Qué interfaz representa una tarea que puede devolver un resultado y lanzar excepciones checked, a diferencia de Runnable?

- A) Callable<V>
- B) Runnable
- C) Thread
- D) Executor

2. ¿Qué provee un ExecutorService respecto a crear Thread manualmente?

- A) Un framework de alto nivel para administrar un pool de hilos reutilizables, enviar tareas (submit/execute) y gestionar su ciclo de vida
- B) Solo permite ejecutar un hilo a la vez
- C) Reemplaza completamente la necesidad de sincronización
- D) Es exclusivo para aplicaciones de red

3. ¿Qué son los 'virtual threads' introducidos en Java 21 (JEP 444)?

- A) Hilos ligeros gestionados por la JVM (no por el sistema operativo) que permiten crear millones de hilos concurrentes con bajo costo de memoria, ideales para aplicaciones con mucha I/O
- B) Hilos que solo pueden ejecutar código funcional (lambdas)
- C) Un reemplazo para las clases enum
- D) Hilos que se ejecutan exclusivamente en la GPU

4. ¿Qué palabra clave permite garantizar que solo un hilo a la vez ejecute un bloque de código crítico?

- A) synchronized
- B) volatile
- C) static
- D) transient

5. ¿Cómo se crea rápidamente un ExecutorService basado en virtual threads en Java 21?

- A) Executors.newVirtualThreadPerTaskExecutor()
- B) Executors.newFixedThreadPool(1)
- C) Thread.ofVirtual().createPool()
- D) new VirtualExecutorService()

6. ¿Qué método de la clase Thread permite pausar la ejecución del hilo actual durante un tiempo determinado?

- A) Thread.sleep(milisegundos)
- B) Thread.pause(milisegundos)
- C) Thread.wait(milisegundos) sin sincronización
- D) Thread.delay(milisegundos)

7. ¿Qué es una 'race condition' (condición de carrera) en programación concurrente?

- A) Una situación en la que el resultado del programa depende del orden impredecible de ejecución de múltiples hilos accediendo a datos compartidos sin la sincronización adecuada
- B) Un mecanismo que garantiza que los hilos se ejecuten siempre en el mismo orden
- C) Un tipo de excepción lanzada exclusivamente por `ExecutorService`
- D) Una técnica de optimización de rendimiento en la JVM

8. ¿Qué aporta la clase `java.util.concurrent.atomic.AtomicInteger` frente a un `int` normal en un entorno multihilo?

- A) Provee operaciones atómicas (como `incrementAndGet()`) que se ejecutan de forma indivisible sin necesidad de bloques `synchronized`, evitando condiciones de carrera
- B) Convierte automáticamente el entero en un tipo `BigInteger`
- C) Solo puede usarse fuera de hilos concurrentes
- D) Reemplaza completamente a `ExecutorService`

21. Manejo de Excepciones

1. ¿Cuál es la diferencia entre una excepción 'checked' y una 'unchecked'?

- A) Las checked deben declararse (throws) o manejarse obligatoriamente en compilación (extienden Exception); las unchecked no obligan a manejo explícito (extienden RuntimeException)
- B) Las unchecked solo pueden lanzarse dentro de un try-catch
- C) Las checked no pueden ser capturadas nunca con catch
- D) No hay diferencia real, es solo una convención de nombres

2. ¿Qué garantiza el bloque 'finally' en un try-catch-finally?

- A) Se ejecuta siempre, haya o no excepción, incluso si hay un return dentro del try o catch (salvo System.exit() o error grave de JVM)
- B) Solo se ejecuta si no ocurrió ninguna excepción
- C) Solo se ejecuta si hubo una excepción no capturada
- D) Se ejecuta antes que el bloque try

3. ¿Qué ventaja ofrece 'try-with-resources' introducido en Java 7?

- A) Cierra automáticamente los recursos que implementan AutoCloseable al finalizar el bloque try, sin necesidad de un finally explícito
- B) Permite capturar cualquier tipo de excepción sin declarar catch
- C) Elimina la necesidad de manejar excepciones por completo
- D) Solo funciona con archivos, no con conexiones de red o BD

4. ¿Qué clase es la superclase común de Exception y Error?

- A) Throwable
- B) RuntimeException
- C) Object
- D) IOException

5. ¿Se puede tener múltiples bloques catch para un mismo try, capturando distintos tipos de excepción?

- A) Sí, se pueden encadenar varios catch; el orden importa: las excepciones más específicas deben ir antes que las más generales
- B) No, solo se permite un único catch por try
- C) Sí, pero el orden de los catch nunca importa
- D) Solo se permite si todas las excepciones son unchecked

6. ¿Qué permite el 'multi-catch' introducido en Java 7 (catch (IOException | SQLException e))?

- A) Capturar varios tipos de excepción no relacionados jerárquicamente en un único bloque catch, evitando duplicar código de manejo
- B) Capturar todas las excepciones posibles automáticamente sin especificarlas
- C) Ejecutar el bloque catch varias veces en secuencia
- D) Es solo válido para excepciones unchecked

7. ¿Qué es una excepción personalizada (custom exception) y cómo se crea típicamente?

- A) Una clase que extiende Exception (checked) o RuntimeException (unchecked) para representar un error específico del dominio de la aplicación
- B) Una excepción que solo puede lanzarse desde el método main
- C) Una excepción que no puede tener mensaje ni causa asociada
- D) Un tipo de anotación que reemplaza al try-catch

8. ¿Qué es una 'suppressed exception' (excepción suprimida) en el contexto de try-with-resources?

- A) Es una excepción adicional lanzada durante el cierre automático de un recurso (en el método close()), que ocurre DESPUÉS de que ya se había lanzado una excepción principal en el bloque try; en lugar de reemplazar la excepción principal, se adjunta como 'suprimida' y puede recuperarse con getSuppressed()
- B) Es una excepción que Java ignora completamente sin dejar ningún rastro
- C) Es un tipo de excepción que solo puede ocurrir en bloques catch, nunca en try
- D) Es sinónimo exacto de una excepción checked

9. ¿Es obligatorio incluir un bloque catch o finally al usar try-with-resources, como en 'try (Recurso r = new Recurso()) { ... }'?

- A) No, un try-with-resources puede existir por sí solo, sin bloque catch ni finally, ya que el propio mecanismo de cierre automático de recursos es la característica principal; catch y finally siguen siendo opcionales adicionales si se necesitan
- B) Sí, siempre es obligatorio tener al menos un catch después de un try-with-resources
- C) Sí, siempre es obligatorio tener un finally, aunque el catch sea opcional
- D) No es válida la sintaxis try-with-resources sin al menos una de las dos cláusulas adicionales

10. Si un try-with-resources declara múltiples recursos, como 'try (Recurso1 r1 = ...; Recurso2 r2 = ...) {...}', ¿en qué orden se cierran esos recursos al finalizar el bloque?

- A) En orden INVERSO al que fueron declarados: se cierra primero r2 y después r1, ya que el cierre sigue una disciplina similar a una pila (el último recurso abierto es el primero en cerrarse)
- B) En el mismo orden en que fueron declarados: primero r1 y después r2
- C) El orden de cierre es indefinido y puede variar en cada ejecución
- D) Solo se cierra el último recurso declarado; los demás permanecen abiertos

11. ¿Qué ocurre si un bloque 'finally' contiene una sentencia 'return' (o lanza una nueva excepción), y el bloque try (o catch) también había lanzado una excepción previamente?

- A) El return (o la nueva excepción) del bloque finally tiene prioridad y efectivamente 'descarta' o reemplaza la excepción original que se había lanzado en try/catch, la cual se pierde silenciosamente sin propagarse
- B) La excepción original del try/catch siempre tiene prioridad sobre cualquier cosa que ocurra en finally
- C) Ambas, la excepción original y la acción de finally, se combinan y se propagan juntas como una excepción compuesta
- D) El código no compila si finally contiene un return junto con una excepción pendiente

12. ¿En qué orden deben cerrarse los recursos si una clase personalizada como Turkey implementa AutoCloseable y se anida dentro de otro recurso Duck que también lo implementa, ambos en el mismo try-with-resources?

- A) El recurso declarado en último lugar en la lista del try-with-resources se cierra primero (orden inverso de declaración), independientemente de si están conceptualmente 'anidados' o no en la lógica del programa
- B) Siempre se cierra primero el recurso mencionado primero en la lista, sin excepción
- C) El orden de cierre depende únicamente del orden alfabético de los nombres de las clases
- D) Solo se cierra el recurso que lanzó una excepción, el otro permanece abierto

22. BigDecimal y Precisión Numérica

1. ¿Por qué se recomienda usar BigDecimal en lugar de double para cálculos monetarios?

- A) Porque double usa representación de punto flotante binario que introduce errores de redondeo; BigDecimal representa números decimales exactos con precisión arbitraria
- B) Porque double no puede almacenar números negativos
- C) Porque BigDecimal es más rápido en todas las operaciones aritméticas
- D) Porque double no permite más de 2 decimales

2. ¿Cuál es la forma recomendada de crear un BigDecimal a partir de un valor decimal exacto como 19.99?

- A) new BigDecimal("19.99") usando un String, para evitar imprecisión del double
- B) new BigDecimal(19.99) usando el double directamente, siempre es exacto
- C) BigDecimal.valueOf(19.99f) usando float
- D) No es posible crear BigDecimal con decimales

3. ¿Por qué comparar dos BigDecimal con equals() puede dar resultados inesperados, por ejemplo entre new BigDecimal("2.0") y new BigDecimal("2.00")?

- A) Porque equals() considera también la escala (número de decimales); para comparar solo el valor numérico se debe usar compareTo()
- B) Porque BigDecimal no sobrescribe equals()
- C) Porque BigDecimal siempre lanza excepción al comparar
- D) Porque equals() en BigDecimal compara referencias, no valores

4. ¿Qué método de BigDecimal se usa para redondear un valor a una cantidad específica de decimales con un modo de redondeo determinado?

- A) setScale(decimales, RoundingMode.HALF_UP) (u otro RoundingMode)
- B) round(decimales)
- C) truncate(decimales)
- D) toFixed(decimales)

5. ¿Por qué BigDecimal es inmutable, igual que String?

- A) Porque cada operación aritmética (add, subtract, multiply...) devuelve un nuevo objeto BigDecimal en lugar de modificar el original, garantizando consistencia y seguridad en entornos concurrentes
- B) Porque solo puede contener un dígito a la vez
- C) Porque no permite operaciones aritméticas en absoluto
- D) Porque BigDecimal está deprecado desde Java 17

24. Internacionalización (Locale, NumberFormat, ResourceBundle)

1. ¿Qué clase representa una región geográfica, política o cultural específica para adaptar formatos y textos (i18n)?

- A) java.util.Locale
- B) java.util.Region
- C) java.util.Country
- D) java.text.Zone

2. ¿Cómo se cargan mensajes traducidos según el idioma usando ResourceBundle?

- A) ResourceBundle.getBundle("nombreBase", locale) busca automáticamente el archivo .properties más específico que coincida con el Locale dado, con fallback al bundle por defecto
- B) Se debe escribir manualmente un if-else para cada idioma soportado
- C) ResourceBundle solo soporta inglés y español
- D) Los archivos .properties deben estar compilados como clases .class

3. ¿Qué clase se usa para formatear números o monedas de acuerdo a un Locale específico?

- A) java.text.NumberFormat
- B) java.lang.NumberParser
- C) java.util.Formatter.Locale
- D) java.text.LocaleFormat

4. ¿Cómo se crea un Locale específico para español de México?

- A) Locale.of("es", "MX") (o el constructor equivalente new Locale("es","MX") en versiones previas)
- B) new Locale("MX-es")
- C) Locale.create("spanish", "mexico")
- D) Locale.MEXICO_SPANISH

5. ¿Qué convención de nombres de archivo sigue un ResourceBundle basado en propiedades para el Locale francés (fr_FR), con nombre base 'mensajes'?

- A) mensajes_fr_FR.properties
- B) mensajes-fr-FR.properties
- C) fr_FR_mensajes.properties
- D) mensajes.fr.FR.properties

6. ¿Qué diferencia hay entre usar el carácter '#' y el carácter '0' en un patrón de DecimalFormat, por ejemplo en "#0.00"?

- A) '#' representa un dígito OPCIONAL (que no se muestra si no hay valor en esa posición, evitando ceros innecesarios), mientras que '0' representa un dígito OBLIGATORIO (que siempre se muestra, rellenando con cero si no hay valor real en esa posición)
- B) Ambos símbolos son exactamente equivalentes en cualquier patrón de DecimalFormat
- C) '#' solo puede usarse para la parte entera, y '0' solo para la parte decimal
- D) '0' representa un dígito opcional y '#' uno obligatorio, al contrario de lo habitual

7. ¿Qué diferencia de resultado esperarías entre 'NumberFormat.getCurrencyInstance(Locale.US)' y 'NumberFormat.getCurrencyInstance(Locale.GERMANY)' al formatear el mismo valor numérico?

- A) Cada uno formatea el número según las convenciones monetarias de su región: Locale.US usaría el símbolo de dólar (\$) y el formato típico estadounidense (con punto decimal), mientras que Locale.GERMANY usaría el símbolo de euro (€) y las convenciones alemanas (como la coma como separador decimal)
- B) Ambos producen exactamente el mismo resultado, ya que NumberFormat ignora el Locale para moneda
- C) Locale.GERMANY siempre lanza una excepción al usarse con getCurrencyInstance()
- D) Solo Locale.US puede usarse con getCurrencyInstance(), las demás regiones no son compatibles

8. ¿Qué representa el 'compact number format' (formato de número compacto) introducido para representar valores grandes de forma abreviada, como "7K" o "7M"?

- A) Es una forma de NumberFormat que abrevia números grandes usando sufijos como K (miles), M (millones), etc., adaptando la abreviatura según el Locale y la magnitud del número, útil para interfaces de usuario donde se prioriza la brevedad sobre la precisión exacta
- B) Es un formato que solo puede usarse con números negativos
- C) Es sinónimo exacto de DecimalFormat sin ninguna diferencia real
- D) Solo funciona con la clase BigDecimal, no con NumberFormat

9. ¿Qué excepción se lanza típicamente al intentar parsear (convertir de texto a número) un String que no tiene un formato numérico válido según el patrón esperado, usando NumberFormat.parse()?

- A) ParseException, una excepción checked que debe manejarse o declararse explícitamente
- B) NumberFormatException, siempre unchecked sin necesidad de manejo explícito
- C) IllegalArgumentException genérica sin relación con el formato de números
- D) ClassCastException, ya que el parsing intenta convertir tipos incompatibles

10. ¿Cuál es el orden exacto del algoritmo de búsqueda (fallback) que sigue ResourceBundle.getBundle() al buscar el archivo de propiedades más adecuado para un Locale específico, por ejemplo español de México (es_MX)?

- A) Primero busca el archivo más específico (nombreBase_es_MX.properties); si no lo encuentra, busca uno solo con el idioma (nombreBase_es.properties); si tampoco existe, recurre al archivo base sin sufijo de idioma/país (nombreBase.properties); si ninguno existe, lanza MissingResourceException
- B) Siempre usa directamente el archivo base sin sufijos, ignorando cualquier archivo específico de idioma o país
- C) Busca aleatoriamente entre todos los archivos .properties disponibles en el classpath
- D) Requiere que el archivo específico exista obligatoriamente, sin ningún mecanismo de fallback

11. ¿Qué excepción se lanza si se solicita una clave específica que no existe en NINGUNO de los archivos de propiedades encontrados por el algoritmo de fallback de ResourceBundle?

- A) MissingResourceException
- B) NoSuchElementException
- C) KeyNotFoundException
- D) IllegalArgumentException

12. ¿Cómo se combina MessageFormat con ResourceBundle para producir mensajes dinámicos y localizados, por ejemplo "Hola {0}, bienvenido a {1}"?

- A) Se obtiene la plantilla del mensaje (con marcadores como {0}, {1}) desde el ResourceBundle mediante getString(), y luego se usa MessageFormat.format(plantilla, parametro1, parametro2) para sustituir esos marcadores con los valores reales proporcionados en tiempo de ejecución
- B) MessageFormat reemplaza completamente a ResourceBundle, no se usan juntos
- C) Los marcadores {0}, {1} solo pueden usarse dentro de código Java, nunca dentro de archivos .properties
- D) ResourceBundle sustituye automáticamente los marcadores sin necesidad de MessageFormat

13. ¿Cuál es una diferencia clave entre usar la clase Properties directamente y usar ResourceBundle para gestionar configuración o mensajes de una aplicación?

- A) Properties se centra en la carga y gestión manual de pares clave-valor desde cualquier fuente (útil para configuración general, incluso modificable en tiempo de ejecución con setProperty()); ResourceBundle está específicamente diseñado para internacionalización, con su propio algoritmo de fallback automático basado en Locale
- B) Properties y ResourceBundle son exactamente lo mismo, con distinto nombre solo por convención histórica
- C) ResourceBundle permite modificar valores en tiempo de ejecución, mientras que Properties es siempre de solo lectura
- D) Properties solo puede usarse con archivos XML, nunca con archivos .properties de texto plano

14. ¿Qué hace el método getProperty(clave, valorPorDefecto) de la clase Properties si la clave solicitada no existe en el objeto Properties?

- A) Devuelve el valorPorDefecto proporcionado como segundo argumento, sin lanzar ninguna excepción
- B) Lanza MissingResourceException, igual que ResourceBundle
- C) Inserta automáticamente la clave con el valorPorDefecto en el objeto Properties
- D) Devuelve null sin ningún valor por defecto, ignorando el segundo argumento

15. ¿Por qué usar un patrón como "HH:mm" con un objeto LocalDate (en lugar de LocalDateTime o LocalTime) provoca un error en tiempo de ejecución al formatear?

- A) Porque LocalDate no contiene ninguna información de hora (HH, mm); el patrón de formato debe ser compatible con los campos que realmente contiene el objeto temporal, y usar patrones de hora sobre un LocalDate lanza una excepción en tiempo de ejecución (UnsupportedTemporalTypeException)
- B) Porque LocalDate no puede usarse nunca con DateTimeFormatter bajo ninguna circunstancia
- C) Porque el patrón "HH:mm" es sintácticamente inválido en cualquier contexto
- D) Porque LocalDate requiere obligatoriamente especificar también la zona horaria en el patrón

16. ¿Es correcto asumir que ResourceBundle, tras encontrar un archivo específico para un Locale (por ejemplo, zoo_es.properties), seguirá buscando también en el archivo default por cada clave individual que falte?

- A) No, es un error común: una vez que ResourceBundle determina cuál archivo de bundle usar según el algoritmo de fallback de Locale, usa exclusivamente ESE archivo (no vuelve a buscar en otros Locales) para las claves; si una clave específica no está en ese archivo bundle elegido, se lanza MissingResourceException directamente
- B) Sí, ResourceBundle siempre revisa todos los archivos disponibles (específico, idioma, default) para cada clave individual que se solicite
- C) ResourceBundle nunca puede fallar una vez que encuentra el primer archivo, todas las claves siempre están disponibles
- D) El comportamiento depende de si se usa getString() o getObject(), siendo distinto para cada método

17. ¿Por qué el patrón de DateTimeFormatter "HH o'clock" (sin escapar la comilla simple) produce un error, y cómo se corrige correctamente?

- A) La comilla simple (') tiene un significado especial en los patrones de DateTimeFormatter (delimita texto literal); para incluir una comilla simple literal dentro del texto, debe escaparse duplicándola, como en "HH 'o'clock"
- B) El patrón es completamente correcto tal como está escrito, sin ningún problema
- C) El error se debe a que "o'clock" contiene una letra minúscula, lo cual no está permitido en ningún patrón
- D) Los patrones de DateTimeFormatter no permiten ningún tipo de texto literal, solo símbolos de fecha/hora

18. ¿Cuál es la sintaxis correcta para construir un objeto Locale para inglés de Estados Unidos, evitando el error común de escribir 'new Locale("en_US")' como un único String?

- A) new Locale("en", "US") (idioma en minúscula y país en mayúscula como argumentos separados), o alternativamente usando Locale.Builder para mayor claridad
- B) new Locale("en_US") es la única forma correcta y válida de construir el Locale
- C) new Locale("US", "en") invirtiendo el orden de idioma y país
- D) new Locale("EN-US") usando mayúsculas y un guion en lugar de guion bajo

19. Según las reglas repasadas en los ejercicios, ¿qué caracteres de un patrón DecimalFormat determinan si un dígito es obligatorio o solo opcional en el resultado formateado?

- A) '0' indica un dígito obligatorio (se muestra siempre, incluso rellenando con cero); '#' indica un dígito opcional (no se muestra si no hay valor real en esa posición)
- B) '#' es obligatorio y '0' es opcional, al contrario de la regla estándar
- C) Ambos símbolos, '0' y '#', son completamente intercambiables sin ninguna diferencia
- D) Ninguno de los dos símbolos tiene efecto alguno en DecimalFormat, se ignoran en el patrón

27. Patrones de Diseño (Strategy)

1. ¿Qué problema resuelve el patrón de diseño Strategy?

- A) Permite definir una familia de algoritmos intercambiables, encapsulando cada uno en una implementación separada, de modo que el algoritmo pueda variar independientemente del código que lo usa
- B) Permite crear una única instancia global de una clase
- C) Convierte una interfaz incompatible en otra que el cliente espera
- D) Permite añadir responsabilidades a un objeto dinámicamente envolviéndolo

2. En Java moderno, ¿qué característica del lenguaje permite implementar el patrón Strategy de forma muy concisa sin crear clases separadas para cada algoritmo?

- A) Expresiones lambda que implementan una interfaz funcional que representa la estrategia
- B) Los records
- C) Los enums simples
- D) Los bloques de texto

3. ¿Cuál es una ventaja de usar Strategy frente a múltiples bloques if-else o switch para seleccionar un algoritmo?

- A) Facilita añadir nuevas estrategias sin modificar el código existente que las usa (principio abierto/cerrado)
- B) Hace que el código se ejecute más rápido siempre
- C) Elimina la necesidad de interfaces
- D) Convierte el código automáticamente en concurrente

4. En el patrón Strategy, ¿qué rol cumple la clase 'Contexto'?

- A) Mantiene una referencia a una estrategia (interfaz común) y delega en ella la ejecución del algoritmo, sin conocer los detalles de la implementación concreta
- B) Es la implementación concreta de un algoritmo específico
- C) Es la interfaz que define el contrato del algoritmo
- D) Es una clase estática que no puede cambiar en tiempo de ejecución

09. Clases Anidadas, Internas, Locales y Anónimas

1. ¿Qué es una clase interna (inner class) no estática en Java?

- A) Una clase definida dentro de otra clase que mantiene una referencia implícita a una instancia de la clase contenedora, permitiendo acceder a sus miembros de instancia
- B) Una clase que solo puede definirse dentro de un método main
- C) Una clase que no puede tener métodos propios
- D) Un sinónimo exacto de clase estática anidada

2. ¿Qué es una clase local (local class) en Java?

- A) Una clase definida dentro del cuerpo de un método, visible solo dentro de ese método
- B) Una clase que solo puede usarse en el paquete java.util
- C) Una clase que no puede implementar interfaces
- D) Un sinónimo de clase anónima

3. ¿Puede una clase interna no estática acceder directamente a variables private de la clase externa que la contiene?

- A) Sí, las clases internas tienen acceso completo a todos los miembros (incluidos private) de la clase externa que las contiene
- B) No, nunca puede acceder a miembros private de la clase externa
- C) Solo si ambas clases están en el mismo archivo pero en paquetes distintos
- D) Solo si la clase interna es también private

4. ¿Puede una interfaz declararse como 'sealed' en Java, restringiendo qué clases o interfaces pueden implementarla o extenderla?

- A) Sí, al igual que las clases, una interfaz puede declararse sealed con una cláusula permits, restringiendo qué clases pueden implementarla y qué interfaces pueden extenderla
- B) No, sealed solo puede aplicarse a clases, nunca a interfaces
- C) Sí, pero una interfaz sealed no puede tener ningún método
- D) Sí, pero solo si la interfaz no tiene métodos default

5. ¿Cuál es la diferencia principal entre una clase interna (inner class) no estática y una clase anidada estática (static nested class)?

- A) La clase interna no estática requiere una instancia de la clase externa para existir y tiene acceso a sus miembros de instancia; la clase anidada estática no requiere esa instancia y no tiene acceso implícito a los miembros de instancia de la clase externa
- B) Ambas requieren siempre una instancia de la clase externa para poder crearse
- C) La clase anidada estática no puede tener métodos propios, a diferencia de la clase interna
- D) No existe ninguna diferencia real entre ambas, solo cambia el nombre

6. ¿Qué es una 'clase local' (local class) y dónde se declara?

- A) Es una clase declarada dentro del cuerpo de un método (o bloque), visible únicamente dentro de ese método/bloque donde fue definida
- B) Es una clase declarada obligatoriamente en el mismo archivo que la clase main
- C) Es sinónimo exacto de una clase anónima, sin ninguna diferencia
- D) Es una clase que solo puede implementar interfaces, nunca extender otras clases

7. ¿Qué error común se debe evitar al trabajar con clases anidadas y el acceso a variables locales del método que las contiene?

- A) Intentar que una clase local o anónima acceda a una variable local externa que no sea 'effectively final' (que se reasigna después de declararse), lo que produce un error de compilación
- B) Declarar más de una clase local por método, lo cual está prohibido por el compilador
- C) Usar clases anidadas dentro de un bucle for, lo cual siempre falla en tiempo de ejecución
- D) Declarar una clase anidada estática dentro de una interfaz, lo cual nunca es válido

8. ¿Cómo se relaciona la encapsulación con el diseño de clases anidadas privadas dentro de una clase externa?

- A) Una clase anidada declarada como private solo es accesible dentro de la clase externa que la contiene, lo cual permite ocultar detalles de implementación interna que no deberían exponerse fuera de esa clase
- B) La encapsulación no puede aplicarse a clases anidadas, solo a campos y métodos
- C) Una clase anidada nunca puede ser private, solo public o default
- D) La encapsulación obliga a que toda clase anidada sea siempre static

23. API de Fecha y Hora (java.time)

1. ¿Qué clase del paquete java.time representa una fecha sin hora, como '2026-07-07'?
 - A) LocalDate
 - B) LocalDateTime
 - C) LocalTime
 - D) Instant
2. ¿Qué clase representa una fecha y hora sin información de zona horaria?
 - A) LocalDateTime
 - B) ZonedDateTime
 - C) Instant
 - D) OffsetDateTime
3. ¿Son mutables los objetos de las clases del paquete java.time como LocalDate y LocalDateTime?
 - A) No, son inmutables; métodos como plusDays() devuelven una nueva instancia en lugar de modificar el objeto original
 - B) Sí, son completamente mutables como Calendar
 - C) Solo LocalDate es inmutable, LocalDateTime no
 - D) Depende de si se usa un builder
4. ¿Qué clase se usa para representar una cantidad de tiempo basada en fechas (años, meses, días), como la diferencia entre dos LocalDate?
 - A) Period
 - B) Duration
 - C) Instant
 - D) Interval
5. ¿Qué método se usa para calcular la diferencia entre dos LocalDate en días?
 - A) ChronoUnit.DAYS.between(fecha1, fecha2)
 - B) fecha1.diff(fecha2)
 - C) fecha1.minus(fecha2)
 - D) Period.diasEntre(fecha1, fecha2)

26. Módulos Java (JPMS)

1. ¿Qué archivo declara un módulo en el sistema de módulos de Java (JPMS, introducido en Java 9)?
 - A) module-info.java
 - B) module.xml
 - C) package-info.java
 - D) moduleconfig.properties
2. ¿Qué directiva dentro de module-info.java indica que un módulo depende de otro módulo?
 - A) requires nombreModulo;
 - B) imports nombreModulo;
 - C) depends nombreModulo;
 - D) uses nombreModulo;
3. ¿Qué directiva permite que otros módulos accedan a las clases públicas de un paquete específico?
 - A) exports nombrePaquete;
 - B) public nombrePaquete;
 - C) open nombrePaquete;
 - D) expose nombrePaquete;
4. ¿Cuál es uno de los principales beneficios del sistema de módulos (JPMS) respecto al classpath tradicional?
 - A) Permite encapsulamiento fuerte a nivel de módulo: un paquete no exportado no es accesible desde fuera, incluso si sus clases son públicas
 - B) Elimina por completo la necesidad de compilar el código
 - C) Hace que todos los paquetes sean automáticamente públicos entre módulos
 - D) Reemplaza completamente a Maven y Gradle
5. ¿Qué directiva se usa en module-info.java para declarar que un módulo abre un paquete a la reflexión (por ejemplo para frameworks como Hibernate)?
 - A) opens nombrePaquete;
 - B) reflect nombrePaquete;
 - C) unlock nombrePaquete;
 - D) grant nombrePaquete;

00. Información del examen de certificación

1. ¿Cuál es la duración del examen de certificación Oracle Java 21 (1Z0-830), en comparación con el examen Java 17 (1Z0-829)?

- A) El examen Java 21 dura 2 horas, más tiempo que las 90 minutos del examen Java 17
- B) Ambos exámenes duran exactamente lo mismo, 90 minutos
- C) El examen Java 21 dura menos tiempo que el de Java 17
- D) El examen Java 21 no tiene límite de tiempo

2. ¿Qué tema importante del examen Java 17 fue eliminado en el examen de certificación Java 21?

- A) JDBC (Java Database Connectivity)
- B) Streams y expresiones lambda
- C) Generics y colecciones
- D) Manejo de excepciones

3. ¿Cuál es el porcentaje mínimo de aciertos necesario para aprobar tanto el examen Java 17 como el Java 21, y cuántas preguntas tiene cada examen?

- A) 68% de aciertos mínimo, con 50 preguntas en ambos exámenes
- B) 50% de aciertos mínimo, con 68 preguntas
- C) 80% de aciertos mínimo, con 100 preguntas
- D) 68% de aciertos mínimo, pero con distinto número de preguntas en cada examen

4. Aproximadamente, ¿qué porcentaje del temario del examen Java 17 sigue siendo aplicable y relevante para el examen Java 21?

- A) Entre el 85% y el 90%
- B) Menos del 30%
- C) Exactamente el 50%
- D) El 100%, son idénticos en todos los temas

5. ¿Qué ventaja tiene el vale (voucher) de examen mencionado en el curso respecto a qué exámenes de certificación Java puede usarse?

- A) Es válido para varios exámenes de certificación Oracle Java, incluyendo Java 8, 11, 17 y 21, no solo para uno específico
- B) Solo puede usarse exclusivamente para el examen Java 21
- C) Solo es válido para exámenes de certificación distintos a Java, como bases de datos
- D) Expira a las 24 horas de haberlo comprado

25. Entrada/Salida (I/O, File, Path, Scanner)

1. ¿Cuál es la diferencia principal entre la clase heredada `java.io.File` y la interfaz `java.nio.file.Path` introducida en NIO.2?

- A) `File` representa de forma más limitada una ruta de archivo o directorio con un conjunto reducido de operaciones; `Path` (parte de la API NIO.2, más moderna) ofrece una representación más rica y flexible del sistema de archivos, integrándose con la clase utilitaria `Files` para operaciones más completas
- B) `File` y `Path` son exactamente equivalentes, sin ninguna diferencia real de diseño o capacidades
- C) `Path` solo puede usarse para archivos, nunca para representar directorios
- D) `File` fue introducida después que `Path`, siendo la API más moderna de las dos

2. ¿Qué método de la clase utilitaria `Files` permite obtener un objeto `Path` a partir de un `String` que representa una ruta, como una alternativa moderna a crear un `File` directamente?

- A) `Path.of("ruta/al/archivo.txt")` (o el método equivalente `Paths.get()`, usado en versiones previas a esta simplificación)
- B) `new File("ruta/al/archivo.txt").toPath()` es la única forma posible de obtenerlo
- C) `Files.createPath("ruta/al/archivo.txt")`
- D) `Path.create("ruta/al/archivo.txt")`

3. ¿Qué método de la clase `Files` permite verificar si un archivo o directorio existe en el sistema de archivos, dado un objeto `Path`?

- A) `Files.exists(path)`
- B) `path.exists()`
- C) `Files.isPresent(path)`
- D) `path.checkExists()`

4. ¿Cuál es la diferencia conceptual entre un 'stream de bytes' (como `InputStream/OutputStream`) y un 'stream de caracteres' (como `Reader/Writer`) en la API de I/O de Java?

- A) Los streams de bytes trabajan con datos binarios sin interpretación de codificación de texto (útiles para imágenes, archivos binarios, etc.); los streams de caracteres están diseñados específicamente para texto, manejando la codificación de caracteres (charset) de forma apropiada
- B) Ambos tipos de streams son completamente intercambiables sin ninguna diferencia real de propósito
- C) Los streams de caracteres solo pueden usarse para archivos, nunca para otras fuentes de datos
- D) Los streams de bytes son exclusivos de NIO.2, mientras que los streams de caracteres son parte de la API original de I/O

5. ¿Por qué se recomienda usar `try-with-resources` al trabajar con streams de I/O como `FileInputStream` o `BufferedReader`?

- A) Porque estos streams implementan `AutoCloseable` (o `Closeable`) y mantienen recursos del sistema operativo (como descriptores de archivo) que deben liberarse explícitamente; `try-with-resources` garantiza su cierre automático incluso si ocurre una excepción durante su uso
- B) Porque los streams de I/O se cierran automáticamente sin necesidad de ningún mecanismo especial, por lo que `try-with-resources` es puramente opcional sin ningún beneficio real
- C) Porque `try-with-resources` es la única sintaxis que permite crear un `FileInputStream`, sin alternativas
- D) Porque sin `try-with-resources`, el programa no compilaría al usar estos streams

6. ¿Qué método de la clase Files permite leer todo el contenido de un archivo de texto pequeño en una única List<String>, con cada línea como un elemento?

- A) Files.readAllLines(path)
- B) Files.readText(path)
- C) Files.getLines(path)
- D) path.readAllLines()

7. ¿Por qué se prefiere usar BufferedReader (envolviendo un FileReader) en lugar de leer un archivo de texto carácter por carácter directamente con FileReader, en términos de rendimiento?

- A) BufferedReader introduce un búfer interno que reduce drásticamente la cantidad de operaciones de lectura físicas al sistema de archivos, leyendo bloques más grandes de una vez y sirviendo los datos desde memoria, en lugar de acceder al disco en cada lectura individual de carácter
- B) BufferedReader no ofrece ninguna ventaja de rendimiento real sobre FileReader directo
- C) BufferedReader solo puede usarse para escribir archivos, nunca para leerlos
- D) FileReader ya incluye internamente un búfer idéntico al de BufferedReader, haciendo la envoltura redundante

8. ¿Qué método de la clase Files permite escribir una colección de líneas de texto directamente a un archivo, como alternativa conveniente a usar un Writer manualmente?

- A) Files.write(path, listaDeLineas)
- B) Files.saveLines(path, listaDeLineas)
- C) Files.print(path, listaDeLineas)
- D) path.writeLines(listaDeLineas)

9. ¿Qué clase se usa comúnmente para leer datos ingresados por el usuario desde la entrada estándar (System.in) de forma sencilla, incluyendo métodos como nextInt() o nextLine()?

- A) Scanner (java.util.Scanner)
- B) Reader (java.io.Reader) directamente sin ninguna clase adicional
- C) System.console() es la única forma posible de leer entrada del usuario
- D) InputStream directamente, sin ninguna clase envolvente

10. ¿Qué diferencia hay entre los métodos next() y nextLine() de Scanner al leer texto ingresado por el usuario?

- A) next() lee solo hasta el siguiente espacio en blanco (un único 'token'), mientras que nextLine() lee toda la línea completa hasta el salto de línea, incluyendo espacios intermedios
- B) Ambos métodos son exactamente equivalentes, sin ninguna diferencia real de comportamiento
- C) nextLine() solo puede leer números, nunca texto
- D) next() lee siempre toda la entrada disponible hasta el final del programa

11. ¿Qué ventaja ofrece la clase System.console() sobre Scanner al interactuar con el usuario en una aplicación de consola, particularmente para leer contraseñas?

- A) Console ofrece el método readPassword(), que lee la entrada del usuario SIN mostrarla en pantalla (oculta los caracteres tecleados), una funcionalidad de seguridad que Scanner no provee directamente
- B) Console es más rápida en todos los casos que Scanner para leer cualquier tipo de entrada
- C) Console solo puede usarse para escribir datos, nunca para leerlos
- D) System.console() siempre está disponible, incluso dentro de un IDE, a diferencia de Scanner

12. ¿Qué puede ocurrir si se llama a un método de Scanner como nextInt() cuando la entrada del usuario no corresponde a un número entero válido?

- A) Se lanza InputMismatchException, una excepción unchecked que indica que el token leído no coincide con el tipo esperado
- B) El programa simplemente ignora la entrada inválida y continúa sin ningún error
- C) Se lanza ParseException, una excepción checked que debe manejarse obligatoriamente
- D) Scanner convierte automáticamente cualquier entrada no numérica a 0

13. ¿Qué método de Scanner permite verificar si existe más entrada disponible para leer como un entero, antes de invocar nextInt() y arriesgarse a una excepción?

- A) hasNextInt()
- B) isNextInt()
- C) checkNextInt()
- D) canReadInt()

14. ¿Por qué es importante cerrar un Scanner que envuelve System.in una vez que ya no se necesita, y qué precaución especial existe al hacerlo en aplicaciones que puedan necesitar leer más entrada después?

- A) Cerrar el Scanner libera recursos asociados, pero cerrar un Scanner que envuelve System.in también cierra el propio flujo de entrada estándar subyacente; si se intenta leer más entrada después de cerrarlo (incluso con un nuevo Scanner), se producirá un error, por lo que conviene cerrar el Scanner de System.in solo al finalizar definitivamente la interacción con el usuario
- B) Cerrar un Scanner nunca tiene ningún efecto sobre System.in, se puede cerrar y reabrir libremente sin ninguna consecuencia
- C) No es necesario cerrar nunca un Scanner, ya que se cierra automáticamente al finalizar el programa sin ninguna excepción
- D) Cerrar un Scanner de System.in reinicia automáticamente el flujo de entrada estándar para que pueda volver a leerse desde el principio

Respuestas y explicaciones

01. Fundamentos de Java (JVM, JDK, entorno)

1. ¿Cuál es la diferencia principal entre JDK, JRE y JVM?

Respuesta correcta: A) JDK incluye herramientas de desarrollo (javac, etc.) y el JRE; el JRE contiene la JVM y las librerías necesarias para ejecutar; la JVM ejecuta el bytecode

El JDK (Java Development Kit) contiene compilador, herramientas y el JRE. El JRE (Java Runtime Environment) trae la JVM y librerías estándar para ejecutar. La JVM interpreta/compila JIT el bytecode .class.

2. ¿Qué extensión tiene el archivo generado tras compilar un .java con javac?

Respuesta correcta: A) .class

javac genera archivos .class que contienen bytecode, ejecutado luego por la JVM.

3. ¿Qué instrucción usarías para ejecutar directamente un archivo fuente Java de un solo archivo sin compilar manualmente (característica desde Java 11)?

Respuesta correcta: A) java MiArchivo.java

Desde JEP 330 (Java 11) se puede ejecutar 'java MiArchivo.java' para lanzar un programa de un solo archivo fuente sin compilación explícita.

4. ¿Qué hace el Garbage Collector (recolector de basura) en Java?

Respuesta correcta: A) Libera automáticamente la memoria de objetos que ya no tienen referencias alcanzables, sin que el programador deba liberarla manualmente

El Garbage Collector es un proceso de la JVM que identifica y libera memoria ocupada por objetos que ya no son alcanzables desde ninguna referencia activa del programa.

5. ¿Qué firma debe tener obligatoriamente el método main para que la JVM pueda ejecutar un programa?

Respuesta correcta: A) public static void main(String[] args)

La JVM busca exactamente 'public static void main(String[] args)' (o su variante con varargs String... args) como punto de entrada del programa.

6. ¿Qué es JShell, introducido en Java 9?

Respuesta correcta: A) Una herramienta REPL (Read-Eval-Print Loop) que permite ejecutar fragmentos de código Java de forma interactiva sin crear un proyecto completo

JShell es una consola interactiva (REPL) para probar expresiones, declaraciones y fragmentos de Java sin necesidad de compilar un proyecto completo.

7. ¿Cuál de las siguientes NO es una firma válida del método main como punto de entrada tradicional (antes de las simplificaciones de Java 21)?

Respuesta correcta: A) public void static main(String[] args)

Los modificadores 'public static' pueden aparecer en cualquier orden entre sí (static public void main también es válido), y se permite añadir 'final', así como usar varargs (String... args) en vez de String[] args. Sin embargo, 'void' debe aparecer inmediatamente antes del nombre del método 'main', nunca antes de 'static' rompiendo ese orden relativo incorrectamente como en 'public void static main'.

8. ¿Qué simplificación introduce Java 21 (a través de un preview feature) respecto al método main tradicional?

Respuesta correcta: A) Permite declarar una versión simplificada del punto de entrada sin necesidad de 'public static', ni siquiera de parámetros, facilitando el aprendizaje inicial (unnamed classes and instance main methods)

Java 21 introduce como preview (JEP 445) la posibilidad de escribir clases sin nombre y métodos main de instancia simplificados (sin public static, sin parámetros obligatorios), pensado para facilitar los primeros pasos de aprendizaje, aunque el método main tradicional sigue siendo válido y es el que se evalúa formalmente en el examen 17.

9. ¿Cuál es el orden correcto y obligatorio de las secciones en un archivo fuente Java?

Respuesta correcta: A) Declaración de package (si existe), luego declaraciones de import, y finalmente la declaración de la clase (u otros tipos de nivel superior)

Un archivo fuente Java debe seguir el orden: package (opcional, como máximo una), imports (opcionales, cero o más), y después las declaraciones de tipos (clases, interfaces, enums, records). Alterar este orden produce un error de compilación.

10. ¿Cuál de los siguientes NO es un identificador válido para una variable en Java?

Respuesta correcta: A) _ (un único guion bajo solo)

Desde Java 9, usar un único carácter '_' (guion bajo solo) como identificador está prohibido y genera error de compilación, ya que se reserva como posible palabra clave futura. Sin embargo, '_contador', 'contador1' y '\$total' sí son identificadores válidos (pueden empezar con letra, \$ o _, pero no ser exactamente '_').

11. ¿Puede un identificador de variable en Java comenzar con un número, por ejemplo '1variable'?

Respuesta correcta: A) No, un identificador no puede comenzar con un dígito; puede contener dígitos pero no como primer carácter

Los identificadores en Java deben comenzar con una letra, guion bajo (_) o signo de dólar (\$); los dígitos solo pueden aparecer después del primer carácter.

12. En un programa Java con bloques de inicialización static, bloques de inicialización de instancia y un constructor, ¿cuál es el orden de ejecución al crear la primera instancia de la clase?

Respuesta correcta: A) Primero los bloques static (una única vez, al cargar la clase), luego los bloques de inicialización de instancia, y finalmente el constructor

Al cargar la clase por primera vez se ejecutan los bloques static (una sola vez para toda la clase). Luego, cada vez que se crea una instancia, se ejecutan en orden los bloques de inicialización de instancia y, finalmente, el cuerpo del constructor.

13. Si se crean dos instancias sucesivas de la misma clase, ¿cuántas veces se ejecutan los bloques de inicialización static durante ese programa?

Respuesta correcta: A) Una sola vez en total, sin importar cuántas instancias se creen después (solo se ejecutan al cargar la clase)

Los bloques static están asociados a la clase, no a las instancias; se ejecutan una única vez cuando la clase se carga en la JVM, independientemente de cuántos objetos se instancien después.

14. ¿Qué regla rige la indentación (espacios en blanco al inicio de cada línea) dentro de un text block?

Respuesta correcta: A) El compilador elimina automáticamente los espacios en blanco incidentales comunes a todas las líneas, basándose en la línea con menor indentación (incluida la línea de cierre """)

El compilador aplica un algoritmo de 'incidental white space stripping': determina el mínimo de espacios en blanco iniciales comunes a todas las líneas no vacías (considerando también la línea con las comillas de cierre) y los elimina de cada línea, preservando la indentación relativa deseada por el programador.

15. En un text block, ¿qué efecto tiene colocar la comilla de cierre "" en su propia línea, alineada más a la izquierda que el resto del contenido, en comparación con alinearla junto al contenido?

Respuesta correcta: A) Alinear la comilla de cierre más a la izquierda reduce el nivel de 'indentación incidental' considerado común, conservando más espacios de sangría en cada línea del texto resultante

La posición de la línea de cierre participa en el cálculo del mínimo de espacios en blanco comunes que se eliminan; si se coloca más a la izquierda que el contenido, se conserva mayor sangría relativa en el texto resultante, y si se alinea con el contenido, se elimina más indentación.

16. En el contexto del examen de certificación, ¿qué significa típicamente la instrucción 'asuma que la clase compila' en el enunciado de una pregunta?

Respuesta correcta: A) Que no se debe evaluar si hay errores de sintaxis o de compilación general del archivo; el foco de la pregunta está en la lógica y comportamiento en tiempo de ejecución del fragmento mostrado

Esa frase orienta al candidato a centrarse en el comportamiento en tiempo de ejecución (valores, flujo, resultados) del código, dando por hecho que compila correctamente, en lugar de buscar errores sintácticos que no son el objetivo de esa pregunta en particular.

02. Tipos primitivos, casting y var

1. ¿Cuáles son los 8 tipos primitivos de Java?

Respuesta correcta: A) byte, short, int, long, float, double, char, boolean

Los 8 tipos primitivos son: byte, short, int, long, float, double, char y boolean. String no es primitivo.

2. ¿Qué imprime este código?

```
int x = 130;  
byte b = (byte) x;  
System.out.println(b);
```

Respuesta correcta: A) -126

byte es de 8 bits (rango -128 a 127). 130 se desborda: $130 - 256 = -126$ debido al overflow al hacer narrowing cast.

3. ¿Cuál de las siguientes declaraciones con 'var' NO compila?

Respuesta correcta: A) var x; (sin inicializador)

'var' requiere inicialización en la misma línea porque el compilador infiere el tipo a partir del valor asignado; no puede usarse sin inicializador ni como tipo de campo/parámetro.

4. ¿Se puede usar 'var' para declarar un atributo (campo) de una clase?

Respuesta correcta: A) No, 'var' solo es válido para variables locales (dentro de métodos, bloques, for, try-with-resources)

'var' (inferencia de tipo local, LVTI) solo aplica a variables locales, no a campos de clase, parámetros de método o tipos de retorno.

5. ¿Qué es el 'autoboxing' en Java?

Respuesta correcta: A) La conversión automática que realiza el compilador de un tipo primitivo a su clase wrapper correspondiente (por ejemplo, int a Integer)

El autoboxing convierte automáticamente tipos primitivos en sus wrappers (int→Integer, double→Double, etc.) cuando el contexto lo requiere, por ejemplo al añadir un int a una List<Integer>.

6. ¿Qué imprime este código?

```
double d = 9.99;  
int i = (int) d;  
System.out.println(i);
```

Respuesta correcta: A) 9

El casting explícito de double a int simplemente trunca la parte decimal (no redondea), por lo que 9.99 se convierte en 9.

7. ¿Cuál de las siguientes asignaciones requiere un cast explícito para compilar?

Respuesta correcta: A) int x = (int) 3.5; (double a int)

Las conversiones de un tipo 'más ancho' a uno 'más estrecho' (narrowing), como double a int, requieren cast explícito porque pueden perder precisión. Las conversiones de ensanchamiento (widening) como int→long son implícitas.

8. ¿Qué es el 'unboxing' en Java?

Respuesta correcta: A) La conversión automática de un objeto wrapper (como Integer) a su tipo primitivo correspondiente (como int)

El unboxing convierte automáticamente un objeto wrapper (Integer, Double, etc.) a su primitivo equivalente cuando el contexto lo requiere, por ejemplo en una operación aritmética.

9. ¿Qué ocurre si se intenta hacer unboxing de un valor Integer que es null, por ejemplo al asignarlo a un int?

Respuesta correcta: A) Se lanza NullPointerException en tiempo de ejecución

Intentar convertir un Integer null a un int primitivo provoca NullPointerException en tiempo de ejecución, ya que no existe un valor primitivo que represente 'ausencia de valor'.

10. ¿Qué imprime este código?

```
Integer a = 100;  
Integer b = 100;  
System.out.println(a == b);  
Integer c = 200;  
Integer d = 200;  
System.out.println(c == d);
```

Respuesta correcta: A) true y false

Java cachea automáticamente los objetos Integer en el rango -128 a 127 (Integer Cache); por eso a==b da true (mismo objeto cacheado), pero 200 está fuera del rango cacheado, por lo que c==d compara referencias distintas y da false.

11. ¿Cuál es el método recomendado para convertir un String a un int de forma directa, sin pasar por un objeto Integer?

Respuesta correcta: A) Integer.parseInt(texto)

Integer.parseInt(String) devuelve directamente un int primitivo; Integer.valueOf(String) en cambio devuelve un objeto Integer (con posible uso de la caché).

12. ¿Por qué 'var resultado = null;' no compila en Java?

Respuesta correcta: A) Porque el compilador no puede inferir un tipo concreto a partir de null, ya que null es compatible con cualquier tipo referencia y no da información suficiente para la inferencia

La inferencia de tipos de var necesita determinar un tipo concreto a partir del valor inicial; como null no aporta ninguna información de tipo (es compatible con cualquier referencia), el compilador no puede inferir nada y rechaza la declaración.

13. ¿Se puede usar 'var' para el tipo de una expresión lambda o de una referencia a método asignada directamente, como en 'var f = () -> System.out.println("hola");'?

Respuesta correcta: A) No, el compilador no puede inferir un tipo funcional concreto solo a partir de una lambda o method reference sin contexto de una interfaz funcional explícita

Una lambda por sí sola no tiene un tipo inherente; su tipo depende de la interfaz funcional de destino (el 'target type'). Como var necesita inferir un tipo concreto sin ese contexto adicional, no es posible asignar directamente una lambda o method reference a una variable var.

14. ¿Cuáles son los valores por defecto de un campo de instancia boolean y de un campo de instancia char no inicializados explícitamente?

Respuesta correcta: A) false para boolean, y el carácter nulo '\u0000' para char

Los campos de instancia numéricos se inicializan a 0 (o 0.0), boolean a false, y char al carácter Unicode nulo '\u0000' (que se imprime como un carácter vacío/invisible), no como el número 0 aunque internamente char es un tipo numérico de 16 bits.

15. ¿Tienen las variables locales (declaradas dentro de un método) un valor por defecto si no se inicializan explícitamente?

Respuesta correcta: A) No, las variables locales no tienen valor por defecto; el compilador exige que se inicialicen antes de usarlas, o produce un error de compilación

A diferencia de los campos de instancia o static, las variables locales no reciben un valor por defecto automático; el compilador verifica que estén 'definitely assigned' (inicializadas) antes de cualquier uso, y de lo contrario marca un error de compilación.

16. ¿Cuál de los siguientes literales numéricos con guion bajo NO es válido en Java?

Respuesta correcta: A) `_1000` (como si fuera el literal numérico completo con guion bajo al inicio)

Los guiones bajos en literales numéricos pueden ir entre dígitos para mejorar la legibilidad, pero nunca al principio ni al final del literal, ni justo antes o después de un punto decimal, prefijo (0x, 0b) o sufijo (L, F, D). '`_1000`' no es un literal numérico válido con guion bajo (de hecho, sería interpretado como un identificador, no un número).

17. ¿Qué prefijo se usa en Java para escribir un literal entero en base octal, por ejemplo el número 8 en octal?

Respuesta correcta: A) 0 (un cero al inicio, por ejemplo `010` representa el número 8 en octal)

Un literal que comienza con '0' (seguido de dígitos del 0 al 7) se interpreta como octal en Java; por ejemplo, `010` en código representa el valor decimal 8, no diez.

18. Si existen dos clases con el mismo nombre en distintos paquetes (por ejemplo `java.util.Date` y `java.sql.Date`) y se importa una explícitamente con `'import java.util.Date;'`, ¿qué ocurre si además se usa `'import java.sql.*;'` (import con asterisco) en el mismo archivo?

Respuesta correcta: A) El import explícito (`java.util.Date`) tiene prioridad sobre el import con asterisco; para usar `java.sql.Date` en ese archivo habría que calificarlo completamente como `java.sql.Date`

Un import explícito (de una clase concreta) tiene mayor prioridad que un import con asterisco (de todo un paquete) al resolver un nombre simple ambiguo; para acceder a la otra clase con el mismo nombre simple, es necesario referenciarla con su nombre completamente calificado (`paquete.Clase`).

19. Si se importan explícitamente dos clases con el mismo nombre simple desde paquetes distintos, por ejemplo `'import java.util.Date;'` y `'import java.sql.Date;'` en el mismo archivo, ¿qué ocurre?

Respuesta correcta: A) Error de compilación por ambigüedad: no se puede tener dos imports explícitos en conflicto para el mismo nombre simple en un mismo archivo

Dos imports explícitos que resuelven al mismo nombre simple (`Date`) generan un conflicto irresoluble en tiempo de compilación; en ese caso hay que eliminar uno de los imports y usar el nombre completamente calificado para esa clase en el código.

20. ¿Es recomendable, según buenas prácticas, usar imports con asterisco (`import paquete.*;`) en lugar de imports explícitos uno por uno?

Respuesta correcta: A) No es la práctica recomendada generalmente: los imports explícitos son más claros sobre qué se usa exactamente y reducen el riesgo de conflictos de nombres al añadir nuevas clases a un paquete importado

Aunque el import con asterisco (*) es sintácticamente válido y cómodo, la práctica recomendada suele preferir imports explícitos porque documentan claramente las dependencias usadas y evitan conflictos de nombres futuros si el paquete importado agrega nuevas clases con nombres coincidentes.

21. Según la estrategia de estudio mencionada para el examen de certificación, ¿qué suele ser más efectivo que memorizar teoría de forma aislada?

Respuesta correcta: A) Resolver constantemente simuladores de examen con preguntas tipo test, ya que exponen los patrones y trampas reales que se repiten en el examen oficial

La práctica intensiva con simuladores de examen (baterías de preguntas tipo test similares a las oficiales) ayuda a reconocer los patrones, trampas y formatos de pregunta reales, complementando el estudio teórico y siendo, en la práctica, más efectiva para aprobar que la teoría aislada.

22. En un text block, ¿para qué sirve el carácter de escape '\s'?

Respuesta correcta: A) Para insertar explícitamente un espacio en blanco que se preserve al final de una línea, evitando que el compilador lo elimine como espacio incidental

Dentro de un text block, el compilador elimina por defecto los espacios en blanco al final de cada línea; '\s' permite forzar un espacio literal que se conserve exactamente en esa posición, incluso al final de una línea.

23. ¿Qué garantiza realmente la llamada a System.gc() en Java?

Respuesta correcta: A) No garantiza nada: es solo una 'sugerencia' a la JVM para que considere ejecutar el Garbage Collector, pero la JVM puede ignorarla completamente

Un mito común es pensar que System.gc() fuerza la recolección de basura; en realidad es solo una sugerencia (hint) a la JVM, que puede decidir libremente ejecutar el GC en ese momento, hacerlo después, o incluso ignorar la petición.

24. Dado un método sobrecargado con las firmas metodo(int x) y metodo(Integer x), si se invoca metodo(5) con un literal int, ¿cuál versión se prefiere?

Respuesta correcta: A) La versión con el parámetro primitivo metodo(int x), ya que Java prioriza la coincidencia exacta sin necesidad de autoboxing antes de considerar la versión que requiere convertir el primitivo a su wrapper

En la resolución de sobrecarga, Java primero busca coincidencias exactas sin conversiones (o solo con widening), y solo si no existe una opción así, recurre al autoboxing/unboxing. Por eso metodo(int) se prefiere sobre metodo(Integer) al pasar un literal int.

25. ¿Cuál es una trampa común en el examen relacionada con unboxing y valores null?

Respuesta correcta: A) Asignar un Integer que vale null a una variable int primitiva compila correctamente pero lanza NullPointerException en tiempo de ejecución al intentar hacer unboxing

El compilador permite la asignación de un Integer (que podría ser null en tiempo de ejecución) a una variable int, confiando en el unboxing automático; sin embargo, si el valor real es null en ese momento, se lanza NullPointerException al intentar convertirlo a primitivo, siendo una trampa clásica de examen.

26. En los diagramas de memoria usados para analizar referencias, ¿qué representa una flecha que sale de una variable de tipo objeto?

Respuesta correcta: A) La referencia (dirección) que esa variable mantiene hacia un objeto ubicado en el heap, no el objeto en sí

En estos diagramas, las variables de tipo referencia se representan en la pila (stack) apuntando mediante una flecha hacia el objeto real almacenado en el heap; distintas variables pueden tener flechas apuntando al mismo objeto.

27. ¿Qué diferencia existe entre un bloque de inicialización de instancia y el propio constructor de una clase, en cuanto a cuándo se ejecutan?

Respuesta correcta: A) El bloque de inicialización de instancia se ejecuta antes del cuerpo del constructor, cada vez que se crea una nueva instancia, independientemente de cuál constructor sobrecargado se invoque

Los bloques de inicialización de instancia se ejecutan en el orden en que aparecen en el código, después de la llamada implícita o explícita a super(), pero antes del resto del cuerpo del constructor, sin importar cuál constructor sobrecargado se esté usando.

28. ¿Cuál es la forma correcta de escribir el número 26 en binario como literal en Java?

Respuesta correcta: A) 0b11010

Los literales binarios se escriben con el prefijo '0b' o '0B' seguido de dígitos 0 y 1; 0b11010 equivale al valor decimal 26.

29. ¿Qué caracteres son válidos como dígitos en un literal hexadecimal en Java, además de los prefijos 0x/0X?

Respuesta correcta: A) Los dígitos del 0 al 9 y las letras de la A a la F (mayúsculas o minúsculas)

Un literal hexadecimal usa dígitos 0-9 junto con letras A-F (o a-f, sin distinción de mayúsculas/minúsculas) para representar los valores del 10 al 15, precedido por el prefijo 0x o 0X.

30. En el patrón de diseño Strategy explicado con el ejemplo de aves (Aguila, Pato, Pinguino), ¿qué relación representa que la clase Ave tenga una referencia a una interfaz ComportamientoVolar, en lugar de heredar el comportamiento de volar directamente?

Respuesta correcta: A) Una relación 'Has-A' (composición/delegación), en contraste con una relación 'Is-A' (herencia), permitiendo cambiar el comportamiento de vuelo en tiempo de ejecución

Al mantener una referencia a una interfaz (ComportamientoVolar) como atributo, Ave 'tiene un' comportamiento de vuelo (Has-A, composición) en lugar de heredarlo fijo de una superclase (Is-A). Esto permite intercambiar la estrategia concreta (SiVolar, NoVolar, AleatorioVolar) en tiempo de ejecución sin modificar la clase Ave.

31. ¿Por qué Pinguino no debería heredar un método volar() con implementación fija desde una superclase Ave si se sigue estrictamente el enfoque del patrón Strategy?

Respuesta correcta: A) Porque forzaría a Pinguino a 'volar' de alguna manera (o a sobrescribir el método para no hacerlo), rompiendo el principio de que el comportamiento debe asignarse mediante composición según las capacidades reales de cada subtipo, no heredarse ciegamente

Uno de los problemas clásicos de usar herencia rígida para comportamientos variables es que fuerza a todas las subclases a heredar (o sobrescribir forzosamente) un comportamiento que no siempre aplica; el patrón Strategy resuelve esto asignando a cada ave la estrategia de vuelo (SiVolar, NoVolar, AleatorioVolar) que realmente le corresponde, mediante composición en lugar de herencia fija.

32. En la evolución de versiones del ejemplo Strategy explicada en el curso (de la versión 0 con herencia tradicional hasta la versión final), ¿qué mejora clave introduce el paso de usar herencia directa a usar una interfaz de estrategia con delegación?

Respuesta correcta: A) Permite cambiar el comportamiento de un objeto en tiempo de ejecución (por ejemplo, reasignando la estrategia de vuelo de un ave) sin necesidad de crear una nueva subclase para cada combinación de comportamientos

Con herencia tradicional, cada combinación de comportamiento requeriría una subclase distinta y fija en tiempo de compilación. Al delegar el comportamiento a una interfaz de estrategia (ComportamientoVolar) asignada como atributo, se puede cambiar dinámicamente esa estrategia en tiempo de ejecución, ganando flexibilidad sin explosión de subclases.

33. ¿Qué ventaja aporta dar un 'comportamiento por defecto' en el constructor de una clase que usa el patrón Strategy (por ejemplo, asignar NoVolar por defecto si no se especifica otra estrategia)?

Respuesta correcta: A) Evita que el objeto quede en un estado inválido o con una referencia nula a la estrategia si el cliente no especifica explícitamente un comportamiento al crear el objeto

Proveer una estrategia por defecto en el constructor (por ejemplo, asumir que un ave no vuela salvo que se indique lo contrario) garantiza que el objeto siempre tenga un comportamiento válido asignado, evitando NullPointerException al invocar el método delegado si el cliente no proporcionó una estrategia explícita.

03. Operadores, Control de Flujo y Switch

1. ¿Qué imprime el siguiente switch expression?

```
int dia = 3;  
String r = switch(dia){  
case 1,7 -> "Fin de semana";  
default -> "Entre semana";  
};  
System.out.println(r);
```

Respuesta correcta: A) Entre semana

Con la nueva sintaxis de switch expression (flechas), múltiples valores se agrupan con comas. Como dia=3 no coincide con 1 ni 7, cae en default.

2. ¿Qué palabra clave se usa en un switch expression para devolver un valor calculado dentro de un bloque {}?

Respuesta correcta: A) yield

'yield' se usa dentro de un bloque {} de un switch expression para producir el valor resultante.

3. ¿Cuál es el resultado de: System.out.println(5 / 2); y luego System.out.println(5.0 / 2);?

Respuesta correcta: A) 2 y 2.5

La división entera (int/int) trunca el decimal, da 2. Si al menos un operando es double, el resultado es double, da 2.5.

4. ¿Qué diferencia hay entre los operadores '&&' y '&' cuando se usan con dos expresiones booleanas?

Respuesta correcta: A) '&&' usa cortocircuito (no evalúa el segundo operando si el primero ya determina el resultado); '&' siempre evalúa ambos operandos

El operador '&&' (AND lógico con cortocircuito) no evalúa el segundo operando si el primero ya es false; '&' (AND bit a bit/lógico) siempre evalúa ambos lados, lo cual es relevante si el segundo operando tiene efectos secundarios.

5. ¿Qué imprime este bucle?

```
for(int i = 0; i < 3; i++){  
    if(i == 1) continue;  
    System.out.print(i);  
}
```

Respuesta correcta: A) 02

continue salta directamente a la siguiente iteración sin ejecutar el resto del cuerpo del bucle; cuando i==1 se omite el print, por lo que se imprime 0 y luego 2.

6. ¿Es obligatorio el 'break' al final de cada rama en un switch statement clásico (con dos puntos ':')?

Respuesta correcta: A) No es obligatorio sintácticamente, pero sin él el flujo 'cae' (fall-through) a la siguiente rama, lo cual suele ser un error común

En un switch clásico, si no se coloca 'break', la ejecución continúa (fall-through) hacia el siguiente case, ejecutando su código también, lo cual suele ser una fuente de errores si no es intencional.

7. ¿Cuál es el resultado de la expresión '2 + 3 * 4' según la precedencia de operadores en Java?

Respuesta correcta: A) 14, porque la multiplicación tiene mayor precedencia que la suma y se evalúa primero

La multiplicación (*) tiene mayor precedencia que la suma (+), por lo que se evalúa primero 3*4=12, y luego 2+12=14.

8. Al operar entre un valor byte y un valor int en una expresión aritmética, por ejemplo 'byte b = 5; int resultado = b + 10;', ¿qué ocurre con el tipo de b durante la operación?

Respuesta correcta: A) Se promociona automáticamente (widening) a int antes de realizar la suma, ya que las operaciones aritméticas binarias promocionan operandos byte, short y char al menos a int

En expresiones aritméticas, Java promociona automáticamente los operandos de tipo byte, short y char a int (binary numeric promotion) antes de aplicar el operador, incluso si el resultado se asignará a otro tipo compatible.

9. ¿Qué hace el operador lógico XOR (^) aplicado a dos valores boolean, por ejemplo 'true ^ false'?

Respuesta correcta: A) Devuelve true si exactamente uno de los dos operandos es true (y false si ambos son iguales, ya sea ambos true o ambos false)

El operador XOR (^) sobre booleanos devuelve true cuando los dos operandos son diferentes entre sí (uno true y otro false), y false cuando son iguales (ambos true o ambos false).

10. ¿Por qué se dice que '&&' y '||' realizan 'evaluación de circuito corto' (short-circuit), a diferencia de '&' y '|'?

Respuesta correcta: A) Porque '&&' y '||' pueden evitar evaluar el segundo operando si el resultado ya queda determinado por el primero, mientras que '&' y '|' siempre evalúan ambos operandos sin importar el resultado parcial

Con '&&', si el primer operando es false, el resultado ya es false y el segundo operando no se evalúa; con '||', si el primero es true, el resultado ya es true. '&' y '|' (también aplicables a boolean) siempre evalúan ambos operandos, lo cual importa si el segundo tiene efectos secundarios.

11. ¿Qué imprime el siguiente código?

```
int x = 5;  
int y = x++ + ++x;  
System.out.println(y);
```

Respuesta correcta: A) 12

x++ (post-incremento) usa el valor actual de x (5) en la expresión y luego incrementa x a 6. Después, ++x (pre-incremento) incrementa x a 7 y usa ese nuevo valor. La suma es 5 + 7 = 12.

12. ¿Qué valor tiene 'resultado' tras ejecutar: `int a = 10; int resultado = (a > 5) ? 100 : 200;`

Respuesta correcta: A) 100

El operador ternario (condición ? valorSiTrue : valorSiFalse) evalúa (a > 5), que es true (10 > 5), por lo que 'resultado' toma el valor 100.

13. ¿Qué devuelve el operador módulo (%) al aplicarse a dos enteros, por ejemplo '17 % 5'?

Respuesta correcta: A) El resto de la división entera entre los dos operandos (2, ya que 17 = 5*3 + 2)

El operador módulo devuelve el resto de la división entera; 17 dividido entre 5 da cociente 3 y resto 2, por lo que 17 % 5 es igual a 2.

14. Dado 'int x = 5; int y = x-- - --x;', ¿qué valor final tienen y, y x?

Respuesta correcta: A) y = 2, x = 3

x-- usa el valor actual (5) en la resta y luego decreuenta x a 4. Después, --x decreuenta x a 3 y usa ese valor (3) en la resta. Entonces y = 5 - 3 = 2, y x queda finalmente en 3.

15. ¿Qué produce el operador de complemento a nivel de bits (~) aplicado a un entero, por ejemplo '~5'?

Respuesta correcta: A) Invierte todos los bits del número y, como regla práctica, el resultado equivale a -(n+1), por lo que ~5 es igual a -6

El operador ~ invierte cada bit de la representación binaria del número (complemento a uno). Una regla práctica útil es que ~n equivale a -(n+1); por eso ~5 da como resultado -6.

16. ¿Qué ocurre si se suma 1 al valor máximo representable por un int (Integer.MAX_VALUE), por ejemplo 'int x = Integer.MAX_VALUE + 1;'?

Respuesta correcta: A) Se produce overflow silencioso: el valor 'da la vuelta' y se convierte en Integer.MIN_VALUE (el número negativo más pequeño representable), sin lanzar ninguna excepción

Java no lanza excepciones por overflow en operaciones aritméticas con tipos primitivos como int; en su lugar, el valor se desborda silenciosamente (wraps around) siguiendo la aritmética de complemento a dos, por lo que Integer.MAX_VALUE + 1 resulta en Integer.MIN_VALUE.

17. ¿Qué hace el operador de asignación compuesta '+=' que lo diferencia de escribir 'x = x + valor' de forma explícita, en el caso de tipos más estrechos como byte o short?

Respuesta correcta: A) El operador compuesto '+=' realiza un cast implícito (narrowing) automático de vuelta al tipo original de la variable, algo que 'x = x + valor' no hace y por eso podría requerir un cast explícito

Por ejemplo, con 'byte b = 10; b += 5;' el compilador inserta un cast implícito de vuelta a byte, mientras que 'b = b + 5;' fallaría en compilación porque la suma se promociona a int y no se puede asignar directamente a byte sin un cast explícito. Esta es una diferencia sutil frecuente en preguntas de examen.

18. ¿En qué orden se evalúan los operandos de una expresión aritmética con el mismo nivel de precedencia, como en 'a - b + c'?

Respuesta correcta: A) De izquierda a derecha: primero (a - b), y a ese resultado se le suma c

Los operadores binarios con la misma precedencia (como + y -) se asocian de izquierda a derecha; en 'a - b + c' primero se calcula (a - b), y a ese resultado se le suma c.

19. ¿Cuáles de los siguientes tipos NO son válidos como tipo de la variable de control (selector) en un switch (statement o expression)?

Respuesta correcta: A) long, double, float y boolean

Un switch en Java admite como selector: byte, short, char, int (y sus wrappers), String y tipos enum (además de var cuando infiere alguno de estos tipos). No admite long, double, float ni boolean como tipo del selector.

20. ¿Es obligatorio incluir una rama 'default' en un switch expression (con arrow ->) que se usa para asignar un valor a una variable?

Respuesta correcta: A) Sí, salvo que el compilador pueda determinar que todos los casos posibles del tipo están cubiertos exhaustivamente (por ejemplo, todas las constantes de un enum), de lo contrario es obligatorio para garantizar que la expresión siempre produzca un valor

Como un switch expression debe producir siempre un valor, el compilador exige exhaustividad: o se cubren todos los valores posibles (por ejemplo todas las constantes del enum), o se debe incluir un 'default' que garantice un valor para cualquier caso no cubierto explícitamente.

21. ¿Se puede usar 'var' como tipo de la variable de control en un switch, por ejemplo 'var dia = 3; switch(dia){...}'?

Respuesta correcta: A) Sí, siempre que el tipo inferido para 'var' sea uno de los tipos válidos permitidos como selector de switch (como int en este caso)

'var' puede usarse para declarar la variable de control de un switch siempre que el tipo que el compilador infiera a partir de su inicialización sea uno de los tipos permitidos como selector (char, byte, short, int, String o enum).

22. En un for-each como 'for(String nombre : listaDeStrings){...}', ¿qué tipo de objeto debe implementar 'listaDeStrings' como mínimo para que el bucle compile?

Respuesta correcta: A) Debe implementar la interfaz Iterable<T> (como lo hacen todas las Collections del framework, entre otras clases)

El for-each funciona con cualquier array o cualquier objeto que implemente Iterable<T> (interfaz que expone un Iterator), lo cual incluye a todas las implementaciones de Collection (List, Set, etc.); Map no implementa Iterable directamente, pero sí lo hacen sus vistas como keySet(), values() o entrySet().

23. ¿Para qué sirve etiquetar un bucle con una 'label' (por ejemplo 'externo: for(...) { for(...) { break externo; } }')?

Respuesta correcta: A) Permite que un break o continue afecte específicamente al bucle etiquetado (por ejemplo, el externo), en lugar de solo al bucle más interno donde se encuentra la instrucción

Sin una label, break/continue solo afectan al bucle inmediatamente contenedor. Al etiquetar un bucle externo y usar 'break etiqueta;' o 'continue etiqueta;' desde un bucle anidado, se puede salir de (o continuar) específicamente ese bucle externo etiquetado, no solo el interno.

24. ¿Qué elimina el pattern matching for instanceof (Java 16+) que antes era necesario escribir manualmente?

Respuesta correcta: A) El casting explícito y redundante del objeto después de comprobar su tipo con instanceof, ya que la variable del patrón queda automáticamente tipada y lista para usarse dentro del bloque

Antes de Java 16, tras comprobar 'if (obj instanceof String)' era necesario castear manualmente 'String s = (String) obj;' para poder usar el valor como String. El pattern matching permite escribir 'if (obj instanceof String s)' y usar directamente 's' ya tipado, sin cast explícito adicional.

25. ¿Qué es el 'flow scoping' (alcance de flujo) en el contexto del pattern matching con instanceof?

Respuesta correcta: A) El alcance de la variable de patrón (por ejemplo 's' en 'obj instanceof String s') depende del flujo lógico del código: solo está definitivamente asignada y disponible en las ramas donde el compilador puede garantizar que la comprobación fue verdadera

El 'flow scoping' determina el alcance de la variable de patrón según el flujo del programa: por ejemplo, en 'if (obj instanceof String s) { usar s aquí }' la variable 's' es visible dentro del bloque if; pero también puede ser visible después del if si el flujo garantiza que, de continuar la ejecución, obj definitivamente era String (por ejemplo, si el if hace return en el caso contrario).

26. ¿Por qué 'if (obj instanceof String s && s.length() > 5)' compila correctamente, pero un equivalente con '||' en lugar de '&&' (como 'if (obj instanceof String s || s.length() > 5)') NO compila?

Respuesta correcta: A) Con '&&', si la primera condición es false, la segunda no se evalúa (por eso 's' está garantizada como asignada al evaluar la segunda parte); con '||', si la primera condición es false, Sí se evaluaría la segunda, pero 's' podría no estar definitivamente asignada en ese caso, generando un error de compilación

El compilador exige que una variable esté 'definitely assigned' antes de usarse. Con '&&' (cortocircuito), si la parte del instanceof es false, la evaluación se detiene y nunca se llega a usar 's'; pero con '||', si el instanceof es false, sí se evaluaría 's.length()', y en ese camino 's' no fue asignada (porque el objeto no era String), lo que el compilador rechaza.

27. ¿Cuál es la diferencia entre 'break;' (sin label) y 'break etiqueta;' (con label) dentro de un bucle for anidado dentro de otro bucle for etiquetado como 'etiqueta'?

Respuesta correcta: A) 'break;' solo termina el bucle interno donde se ejecuta, continuando con el bucle externo; 'break etiqueta;' termina completamente el bucle externo etiquetado (y por tanto también el interno)

Un break sin label solo afecta al bucle (o switch) inmediatamente contenedor. Con una label apuntando al bucle externo, 'break etiqueta;' termina ese bucle externo específico (y, como consecuencia, también sale del bucle interno anidado dentro de él).

28. En un switch statement clásico con 'case' basados en valores int, ¿por qué una variable declarada como 'final int x = 5;' puede usarse como valor de un case, pero una variable 'int y = 5;' (no final, aunque nunca se reasigne) generalmente no puede?

Respuesta correcta: A) Porque los valores de 'case' en un switch sobre int/char/byte/short deben ser constantes conocidas en tiempo de compilación, y solo una variable 'final' inicializada con un valor constante cumple ese requisito; una variable no-final no se considera una constante de compilación aunque su valor no cambie en la práctica

Los valores usados en los 'case' de un switch sobre tipos como int deben ser constant expressions evaluables en tiempo de compilación. Una variable 'final' inicializada con un valor constante (compile-time constant) cumple este requisito, pero una variable no-final, aunque nunca cambie en tiempo de ejecución, no se considera una constante en compilación y produce un error si se usa como valor de case.

29. ¿Puede repetirse el mismo valor de constante en dos 'case' distintos dentro del mismo switch statement?

Respuesta correcta: A) No, produce un error de compilación por 'valor de case duplicado', ya que cada valor debe ser único dentro del mismo switch

Cada valor de 'case' debe ser único dentro de un mismo switch; si dos case tienen el mismo valor constante, el compilador lo detecta como un error, independientemente del tipo del selector (int, String, enum, etc.).

30. Al analizar errores de compilación relacionados con el alcance (scope) de variables, ¿cuál es una causa típica de estos errores según lo revisado en los ejercicios?

Respuesta correcta: A) Intentar acceder a una variable fuera del bloque {} en el que fue declarada, ya que su visibilidad termina al cerrarse ese bloque

Una variable declarada dentro de un bloque (por ejemplo, dentro de un if, un for, o llaves {} explícitas) solo es visible dentro de ese bloque; intentar referenciarla fuera de él, incluso en el mismo método, produce un error de compilación de tipo 'cannot find symbol' o similar.

31. En un ejercicio con bucles anidados y un 'break' etiquetado combinado con el operador módulo dentro de la condición, ¿qué habilidad es clave para resolver correctamente este tipo de preguntas de examen?

Respuesta correcta: A) Rastrear meticulosamente, iteración por iteración, el valor de cada variable de control y evaluar con precisión hacia qué bucle etiquetado apunta cada break o continue

Este tipo de preguntas de examen requiere simular mentalmente (o en papel) la ejecución paso a paso, llevando registro exacto de los valores de las variables de control en cada iteración, y determinando con precisión qué bucle específico interrumpe o continúa cada break/continue etiquetado.

32. ¿Cuál de las siguientes es una trampa común de examen al analizar un switch expression con errores de compilación?

Respuesta correcta: A) Mezclar tipos de datos incompatibles entre los distintos 'case' y el valor del selector, o cometer errores de punto y coma que generan código inalcanzable o sintaxis inválida

Errores comunes en preguntas de examen sobre switch expressions incluyen usar tipos de datos incompatibles entre los case y el selector, colocar punto y coma de más generando bloques de código inalcanzables, o cometer errores de sintaxis sutiles que rompen la compilación, más que problemas relacionados con la cantidad de casos.

33. Al revisar preguntas de examen sobre pattern matching con instanceof, ¿qué tipo de trampa se menciona como frecuente relacionada con el formato del código?

Respuesta correcta: A) Un formato de indentación o salto de línea engañoso que hace parecer que una variable de patrón está disponible en cierto punto del código, cuando en realidad el flujo (flow scoping) no lo garantiza en esa posición

Una trampa señalada es que el formato visual del código (indentación, saltos de línea) puede sugerir erróneamente que una variable de patrón declarada con instanceof está disponible en cierto punto, cuando el análisis real de flow scoping determina que en ese punto del código no está garantizada como asignada, lo que puede llevar a error de compilación si se usa ahí.

34. ¿Por qué se insiste en que dominar los conceptos fundamentales del lenguaje (scope, loops, switch, pattern matching básico) es clave para el examen de certificación, más allá de los temas avanzados?

Respuesta correcta: A) Porque estos fundamentos son la base sobre la que se construyen los temas más avanzados, y el examen los evalúa con frecuencia, muchas veces combinados de forma sutil en una misma pregunta

El curso enfatiza que un dominio sólido de los fundamentos (scope, control de flujo, switch, pattern matching) es indispensable porque el examen frecuentemente combina varios de estos conceptos fundamentales dentro de una misma pregunta, y sin esa base es fácil cometer errores en preguntas aparentemente sencillas.

04. Arrays y Varargs

1. ¿Cuál declaración de un método varargs es válida?

Respuesta correcta: A) void metodo(String... nombres)

La sintaxis correcta de varargs es 'Tipo... nombre'. Solo puede haber un parámetro varargs y debe ser el último de la lista.

2. ¿Qué ocurre si un método tiene un parámetro varargs y otro sobrecargado con array explícito del mismo tipo?

Respuesta correcta: A) Compila; Java prefiere la coincidencia exacta (el método con array explícito) antes que expandir varargs

La resolución de sobrecarga en Java prioriza la coincidencia sin varargs (aplicabilidad estricta) antes de considerar métodos varargs.

3. `int[] a = new int[3];` ¿cuál es el valor por defecto de cada elemento?

Respuesta correcta: A) 0

Los arrays de tipos primitivos numéricos se inicializan por defecto a 0 (boolean a false, char a '\u0000', referencias a null).

4. ¿Qué imprime este código?

```
int[][] matriz = {{1,2},{3,4,5}};  
System.out.println(matriz[1].length);
```

Respuesta correcta: A) 3

Java permite arrays multidimensionales 'irregulares' (jagged arrays), donde cada fila puede tener distinto tamaño. `matriz[1]` es `{3,4,5}`, cuyo `length` es 3.

5. ¿Qué método de la clase Arrays permite convertir un array en una representación de texto legible, como "[1, 2, 3]"?

Respuesta correcta: A) Arrays.toString(array)

Arrays.toString() genera una representación legible de un array unidimensional. Para arrays multidimensionales existe Arrays.deepToString().

6. ¿Qué son realmente los arrays bidimensionales en Java, como int[][] matriz?

Respuesta correcta: A) Son arrays de arrays: matriz es un array cuyos elementos son, a su vez, referencias a otros arrays unidimensionales (que pueden incluso tener distinto tamaño entre sí)

En Java no existen arrays multidimensionales verdaderos a nivel de memoria; un array bidimensional es, en realidad, un array cuyos elementos son referencias a otros arrays (posiblemente de distinto tamaño, formando un 'jagged array'), lo cual es clave para entender la asignación de memoria.

7. Si se declara 'int[][] matriz = new int[3][];' (sin especificar el tamaño de las filas) y luego se intenta acceder a 'matriz[0][0]' sin haber inicializado esa fila, ¿qué ocurre?

Respuesta correcta: A) Se lanza NullPointerException, ya que matriz[0] es null (no se ha creado el array interno de esa fila todavía)

Al declarar 'new int[3][]' se crea únicamente el array externo de tamaño 3, pero cada una de sus posiciones (cada 'fila') queda como null hasta que se le asigne explícitamente un array interno; acceder a un índice de una fila null lanza NullPointerException.

8. ¿Qué excepción se lanza al intentar acceder a un índice igual o mayor al tamaño (length) de un array, por ejemplo array[5] en un array de tamaño 5 (índices válidos 0 a 4)?

Respuesta correcta: A) ArrayIndexOutOfBoundsException

Acceder a un índice fuera del rango válido de un array (0 hasta length-1) lanza ArrayIndexOutOfBoundsException en tiempo de ejecución, no en compilación.

9. ¿Qué información imprime típicamente Arrays.stream() al procesar directamente los elementos de un array bidimensional sin aplanar (por ejemplo, imprimiendo cada elemento del array externo tal cual)?

Respuesta correcta: A) La representación de la ubicación en memoria (hashcode) de cada sub-array interno, ya que cada elemento del array externo es en sí mismo una referencia a otro array, no un valor simple

Como cada elemento de un array bidimensional externo es en realidad una referencia a otro array, al imprimir directamente esos elementos (por ejemplo con Arrays.stream(matriz).forEach(System.out::println)) se obtiene la representación por defecto de cada sub-array (algo como '[I@hashcode]'), en lugar de los números que contiene; para ver los valores reales hay que iterar un nivel más profundo.

10. ¿Cuál es una forma habitual y clara de recorrer todos los elementos de un array bidimensional para procesarlos individualmente?

Respuesta correcta: A) Usar bucles for-each anidados: uno externo que recorre cada sub-array (fila), y uno interno que recorre los elementos de esa fila

Un patrón común y legible para recorrer un array bidimensional es anidar dos for-each: el externo itera sobre cada fila (que es en sí un array), y el interno itera sobre los elementos individuales de esa fila.

11. ¿Es válido en Java crear un array de tamaño cero, como 'int[] vacio = new int[0];'?

Respuesta correcta: A) Sí, es completamente válido crear un array de tamaño 0; simplemente no tiene ninguna posición disponible para almacenar elementos (no se le pueden 'insertar' valores después, ya que su tamaño es fijo)

Un array de tamaño 0 es perfectamente válido en Java (por ejemplo, como valor de retorno para representar 'ninguna coincidencia' en lugar de null); su longitud (length) es fija en 0 y, como con cualquier array, no se pueden agregar elementos después de creado, ya que los arrays no son redimensionables.

05. Strings, StringBuilder y Bloques de Texto

1. ¿Por qué se dice que String es inmutable en Java?

Respuesta correcta: A) Porque una vez creado un objeto String, su contenido no puede modificarse; operaciones como concat() devuelven un nuevo objeto

Los objetos String son inmutables: cualquier método que 'modifica' un String (concat, replace, toUpperCase...) realmente crea y devuelve un nuevo objeto.

2. ¿Qué imprime?

```
String a = "hola";  
String b = "hola";  
System.out.println(a == b);
```

Respuesta correcta: A) true

Los literales String se almacenan en el String pool. Ambas variables referencian el mismo objeto interno, así que '==' (comparación de referencia) da true.

3. ¿Cuál es la sintaxis correcta para iniciar un bloque de texto (text block, Java 15+)?

Respuesta correcta: A) Triple comillas dobles """" seguidas de salto de línea

Un text block comienza con tres comillas dobles y un salto de línea inmediatamente después, terminando también con tres comillas dobles.

4. ¿Qué método se usa para comparar el contenido de dos Strings (no la referencia)?

Respuesta correcta: A) equals()

'equals()' compara el contenido carácter por carácter; '==' compara si ambas referencias apuntan al mismo objeto en memoria.

5. ¿Por qué se recomienda StringBuilder en lugar de concatenar Strings con '+' dentro de un bucle con muchas iteraciones?

Respuesta correcta: A) Porque cada concatenación con '+' crea un nuevo objeto String (dado que String es inmutable), lo que es ineficiente; StringBuilder modifica un buffer mutable interno

Al ser String inmutable, cada '+' genera un nuevo objeto, desperdiciando memoria y tiempo en bucles largos. StringBuilder usa un array de caracteres mutable interno, evitando esas creaciones repetidas.

6. ¿Qué imprime?

```
String s = " Hola Mundo ";  
System.out.println(s.strip().length());
```

Respuesta correcta: A) 10

strip() (Java 11+) elimina espacios en blanco al inicio y al final. "Hola Mundo" sin los espacios circundantes tiene 10 caracteres.

7. ¿Qué método de String permite dividir el texto en un array usando un delimitador, por ejemplo una coma?

Respuesta correcta: A) split(regex)

split(String regex) divide el String en un array de substrings usando la expresión regular indicada como delimitador.

8. ¿Qué ocurre al llamar al método replace() (u otros métodos de transformación) sobre un StringBuilder dentro de una cadena de construcción (Stream Builder de texto), en cuanto a si modifica el objeto original?

Respuesta correcta: A) Los métodos de StringBuilder, a diferencia de los de String, sí modifican el objeto original in situ (mutándolo), en lugar de crear un nuevo objeto independiente

A diferencia de String (inmutable, donde replace() devuelve un nuevo objeto), los métodos de StringBuilder como replace() modifican directamente el buffer interno del mismo objeto, reflejando el cambio sin crear una nueva instancia.

9. ¿Qué método de String se usa para obtener un carácter específico según su posición (índice), por ejemplo el tercer carácter de un texto?

Respuesta correcta: A) charAt(índice)

charAt(int index) devuelve el carácter en la posición indicada (basada en 0) del String; por ejemplo, "hola".charAt(0) devuelve 'h'.

10. ¿Qué comportamiento tiene equals() al comparar dos arrays con el mismo contenido, por ejemplo new int[]{1,2,3}.equals(new int[]{1,2,3})?

Respuesta correcta: A) Devuelve false, porque los arrays no sobrescriben equals() con comparación de contenido; heredan el comportamiento de Object, que compara referencias (equivalente a ==)

Un malentendido común es asumir que equals() en arrays compara el contenido; en realidad, los arrays no sobrescriben equals(), por lo que se usa el de Object (comparación de identidad de referencia). Para comparar contenido de arrays se debe usar Arrays.equals(array1, array2).

11. ¿Qué devuelve Math.round(2.5) en Java?

Respuesta correcta: A) 3 (Math.round redondea hacia el entero más cercano, y en el caso de .5 exacto redondea hacia arriba)

Math.round() redondea al entero más cercano; para valores con fracción exactamente 0.5, redondea hacia arriba (hacia el entero positivo mayor), por lo que Math.round(2.5) da como resultado 3.

12. ¿Qué diferencia hay entre Math.floor() y Math.round() al aplicarse a un valor decimal como 4.7?

Respuesta correcta: A) Math.floor(4.7) devuelve 4.0 (siempre redondea hacia abajo, al entero menor o igual), mientras que Math.round(4.7) devuelve 5 (redondea al entero más cercano)

Math.floor() siempre trunca hacia el entero inferior más cercano (redondeo hacia abajo), devolviendo un double; Math.round() redondea al entero más próximo (hacia arriba si la fracción es ≥ 0.5), devolviendo un long o int según el tipo de entrada.

13. ¿Por qué es importante recordar que LocalDate es inmutable al usar métodos como plusDays() o minusMonths()?

Respuesta correcta: A) Porque estos métodos no modifican el objeto LocalDate original, sino que devuelven una nueva instancia con la fecha modificada; si no se captura ese valor de retorno (por ejemplo, reasignando la variable), el cambio se pierde

Al ser inmutable, cada operación de LocalDate como plusDays() devuelve un nuevo objeto con el resultado; una trampa común de examen es escribir 'fecha.plusDays(5);' sin reasignar el resultado (por ejemplo 'fecha = fecha.plusDays(5);'), lo que hace que el cambio se pierda porque el objeto original permanece sin modificar.

14. ¿Qué requisito debe cumplir un array para que Arrays.binarySearch() funcione correctamente y produzca resultados confiables?

Respuesta correcta: A) El array debe estar previamente ordenado (en orden ascendente según su orden natural o el Comparator usado), ya que la búsqueda binaria asume que los elementos están ordenados

Arrays.binarySearch() implementa el algoritmo de búsqueda binaria, que requiere que el array esté ordenado de antemano; si no lo está, el resultado de la búsqueda es indefinido/incorrecto, aunque el método no lo detecte ni lance una excepción por sí mismo.

15. ¿Qué devuelve Arrays.binarySearch() cuando el elemento buscado NO se encuentra en el array?

Respuesta correcta: A) Un valor negativo que codifica el punto de inserción donde debería colocarse el elemento para mantener el orden, calculado como -(punto_insercion) - 1

Si el elemento no se encuentra, Arrays.binarySearch() devuelve un valor negativo que indica dónde debería insertarse el elemento para mantener el array ordenado, calculado como $-(\text{punto_insercion}) - 1$, en lugar de simplemente devolver -1.

16. ¿Qué hace el método indent(n) de String cuando n es un número positivo?

Respuesta correcta: A) Añade n espacios en blanco al inicio de cada línea del String, y además normaliza los saltos de línea, garantizando que el texto termine con salto de línea

String.indent(n), con n positivo, agrega n espacios al inicio de cada línea del texto; si n es negativo, en cambio, intenta eliminar hasta esa cantidad de espacios en blanco iniciales de cada línea (sin bajar de cero).

17. ¿Qué hace el método `translateEscapes()` de `String`?

Respuesta correcta: A) Convierte secuencias de escape literales escritas como texto (por ejemplo la secuencia de caracteres `backslash+n`) en su carácter especial real correspondiente (como un salto de línea real)

`translateEscapes()` interpreta secuencias de escape que aparecen como texto literal (por ejemplo, un `backslash` seguido de la letra `n`) y las convierte en el carácter especial real que representan (como un salto de línea), útil al procesar cadenas que contienen escapes sin procesar.

18. ¿Qué produce `String.format("%05d", 42)`?

Respuesta correcta: A) "00042" (el número se rellena con ceros a la izquierda hasta completar 5 dígitos en total)

El especificador `%05d` indica un entero (`d`) con ancho mínimo de 5 caracteres, relleno con ceros (`0`) a la izquierda si el número tiene menos dígitos; `42` se convierte en `"00042"`.

19. ¿Qué devuelve `Arrays.compare(array1, array2)` al comparar dos arrays de enteros?

Respuesta correcta: A) Un valor negativo, cero o positivo indicando si `array1` es 'menor', 'igual' o 'mayor' que `array2`, comparando elemento por elemento en orden (similar a `compareTo`), incluyendo el caso de que uno sea un prefijo del otro

`Arrays.compare()` compara lexicográficamente elemento por elemento; si todos los elementos comunes son iguales pero uno de los arrays es más corto, el array más corto se considera 'menor'. Devuelve un entero negativo, cero o positivo, análogo a `compareTo()`.

20. ¿Qué devuelve `Arrays.mismatch(array1, array2)` si ambos arrays son completamente iguales en contenido y longitud?

Respuesta correcta: A) -1, indicando que no existe ningún índice de discrepancia entre ambos arrays

`Arrays.mismatch()` devuelve el índice de la primera posición donde los arrays difieren; si son completamente iguales (mismo contenido y longitud), devuelve `-1` para indicar que no existe ninguna discrepancia.

21. ¿Por qué trabajar con `ZonedDateTime` y cálculos que cruzan un cambio de horario de verano (DST) puede producir resultados aparentemente inesperados, como sumar horas y no obtener el resultado 'esperado' en horas absolutas?

Respuesta correcta: A) Porque `ZonedDateTime` ajusta automáticamente la hora local según las reglas de la zona horaria, incluyendo saltos o repeticiones de horas durante las transiciones de horario de verano, por lo que sumar un número fijo de horas puede no corresponder exactamente con lo esperado en tiempo 'de reloj' si se cruza esa transición

`ZonedDateTime` tiene en cuenta las reglas de la zona horaria asociada, incluyendo las transiciones de horario de verano (donde una hora puede repetirse o saltarse); esto puede hacer que operaciones aritméticas de tiempo (como sumar horas) produzcan un resultado en 'hora de reloj' distinto al que se obtendría con una simple suma aritmética ajena a esas reglas.

22. Si se ejecuta `'fecha.plusDays(3);'` sobre un objeto `LocalDate` sin reasignar el resultado a ninguna variable, ¿qué ocurre con el valor de `'fecha'` después de esa línea?

Respuesta correcta: A) `'fecha'` permanece sin cambios, ya que `plusDays()` devuelve un nuevo objeto `LocalDate` y no modifica el original; al no capturar ese valor de retorno, el cambio se pierde

`LocalDate` es inmutable; `plusDays()` (como todos sus métodos de modificación) devuelve un nuevo objeto sin alterar el original. Si no se reasigna el resultado (por ejemplo `'fecha = fecha.plusDays(3);'`), la variable `'fecha'` sigue apuntando al objeto original sin cambios, siendo una trampa clásica de examen.

06. POO: Clases, Constructores y Encapsulamiento

1. ¿Qué ocurre si una clase no define ningún constructor explícito?

Respuesta correcta: A) El compilador genera automáticamente un constructor por defecto sin argumentos

Si no se declara ningún constructor, el compilador provee uno por defecto (sin parámetros) que llama implícitamente a `super()`.

2. ¿Qué significa que un atributo sea 'static'?

Respuesta correcta: A) Pertenece a la clase y es compartido por todas las instancias, en lugar de pertenecer a cada objeto individual

Un miembro static pertenece a la clase, no a instancias individuales; existe una sola copia compartida por todos los objetos.

3. ¿Cuál es el propósito principal del modificador 'final' aplicado a una variable?

Respuesta correcta: A) Impedir que la variable sea reasignada después de su primera asignación

'final' en una variable impide reasignación tras la inicialización. En una clase impide herencia; en un método impide override.

4. ¿Puede una clase declarada como 'final' ser extendida (heredada)?

Respuesta correcta: A) No, una clase final no puede tener subclases

'final' en una clase prohíbe que otras clases la extiendan, por ejemplo java.lang.String es final.

5. ¿Qué es un bloque de inicialización estático (static initializer block) y cuándo se ejecuta?

Respuesta correcta: A) Un bloque static { ... } que se ejecuta una única vez, cuando la clase se carga por primera vez en la JVM, antes de crear cualquier instancia

Un bloque static{} se ejecuta una sola vez al cargar la clase en memoria, útil para inicializar campos static complejos, antes de cualquier instanciación.

6. ¿Cuál es el propósito principal del encapsulamiento en POO?

Respuesta correcta: A) Ocultar el estado interno de un objeto y exponer solo una interfaz pública controlada (getters/setters), protegiendo la integridad de los datos

El encapsulamiento oculta los detalles internos (atributos generalmente private) y expone un comportamiento controlado a través de métodos públicos, protegiendo la consistencia del objeto.

7. ¿Qué hace la palabra clave 'this' dentro de un constructor cuando se usa como this(...)?

Respuesta correcta: A) Invoca a otro constructor sobrecargado de la misma clase (constructor chaining), y debe ser la primera línea del constructor

this(...) se usa para llamar a otro constructor de la misma clase (encadenamiento de constructores), y debe ser obligatoriamente la primera sentencia dentro del constructor.

8. ¿Qué diferencia hay entre 'String s = "hola";' y 'String s = new String("hola");' en cuanto al Pool de Strings?

Respuesta correcta: A) La asignación literal reutiliza (o crea) el objeto en el Pool de Strings; new String(...) fuerza la creación de un nuevo objeto en el heap, fuera del pool, aunque su contenido sea idéntico

Cuando se usa un literal, la JVM busca o inserta el valor en el String Pool para reutilizar objetos con igual contenido. Con 'new String(...)' se fuerza la creación de un objeto adicional en el heap normal, independiente del pool, aunque su valor sea igual.

9. Dado:

```
String a = new String("hola");
```

```
String b = "hola";
```

```
System.out.println(a == b);
```

```
System.out.println(a.equals(b));
```

¿Qué imprime?

Respuesta correcta: A) false y true

'a' apunta a un objeto en el heap creado explícitamente con new (fuera del pool), mientras 'b' apunta al objeto del pool; por eso '==' (comparación de referencias) da false. Sin embargo, .equals() compara el contenido carácter por carácter, y ambos contienen "hola", por lo que da true.

10. ¿Por qué se dice que el Pool de Strings ayuda a optimizar memoria en aplicaciones Java?

Respuesta correcta: A) Porque permite que múltiples referencias con el mismo valor literal compartan un único objeto en memoria, en lugar de crear un objeto duplicado por cada literal idéntico

El Pool de Strings es un área especial del heap donde se almacenan literales; si dos variables usan el mismo literal, ambas referencian el mismo objeto compartido, ahorrando memoria al evitar duplicados.

11. ¿Por qué StringBuilder es mutable mientras que String es inmutable?

Respuesta correcta: A) StringBuilder mantiene internamente un array de caracteres modificable que puede crecer y cambiar sin crear un nuevo objeto en cada operación, mientras que String nunca modifica su contenido interno una vez creado

StringBuilder está diseñado con un buffer de caracteres mutable y métodos como append() e insert() que modifican ese buffer directamente, evitando la creación repetida de objetos que ocurriría al concatenar Strings inmutables.

12. En Java, cuando se pasa un objeto como argumento a un método, ¿qué es exactamente lo que se copia?

Respuesta correcta: A) Se copia el valor de la referencia (la dirección que apunta al objeto), no el objeto en sí; por eso Java se describe como 'siempre paso por valor', incluso con objetos

Java siempre pasa argumentos por valor. Para tipos referencia, lo que se copia es el valor de la referencia (la 'dirección' al objeto), por lo que el método recibe una copia de esa referencia que apunta al mismo objeto; modificar el objeto a través de ella afecta al original, pero reasignar la referencia dentro del método no afecta a la variable original.

13. Dado este método:

```
void cambiar(String s){ s = s + " mundo"; }
```

...

```
String texto = "hola";
```

```
cambiar(texto);
```

```
System.out.println(texto);
```

¿Qué imprime?

Respuesta correcta: A) hola

Dentro de cambiar(), el parámetro 's' es una copia de la referencia. Al hacer 's = s + " mundo"', se crea un nuevo objeto String y se reasigna la variable local 's' para que apunte a él, pero esto no afecta a la referencia original 'texto' fuera del método, que sigue apuntando a "hola".

14. ¿Por qué es importante sobrescribir correctamente el método equals() en una clase propia, en lugar de dejar la implementación heredada de Object?

Respuesta correcta: A) Porque la implementación de Object.equals() compara referencias (equivalente a ==), lo que casi nunca es lo que se quiere al comparar el contenido lógico de dos objetos de una clase propia

Object.equals() por defecto compara identidad de referencia (mismo objeto en memoria). Si se quiere que dos instancias distintas se consideren 'iguales' por su contenido (por ejemplo, dos objetos Persona con el mismo nombre y edad), es necesario sobrescribir equals() con la lógica de comparación deseada.

15. ¿Por qué la clase String es 'final' en Java?

Respuesta correcta: A) Para evitar que se cree una subclase que rompa las garantías de inmutabilidad y el comportamiento esperado del Pool de Strings, ya que una subclase podría sobrescribir métodos y alterar ese comportamiento

Declarar String como final impide la herencia, protegiendo su inmutabilidad y el correcto funcionamiento del Pool de Strings; si se pudiera heredar, una subclase podría alterar el comportamiento esperado y romper supuestos de seguridad y de rendimiento en toda la plataforma.

16. ¿Cuándo se considera que un objeto es 'elegible' para el Garbage Collector?

Respuesta correcta: A) Cuando ya no existe ninguna referencia alcanzable desde el programa que apunte a ese objeto

Un objeto se vuelve elegible para el Garbage Collector cuando ninguna variable de referencia activa (alcanzable desde el código en ejecución) apunta ya a él, por ejemplo al reasignar la única referencia a null o a otro objeto.

17. Dado:

```
Persona p1 = new Persona();
```

```
Persona p2 = p1;
```

```
p1 = null;
```

¿Es elegible para Garbage Collection el objeto Persona original?

Respuesta correcta: A) No, porque p2 todavía mantiene una referencia alcanzable al mismo objeto

Aunque p1 se reasigna a null, p2 seguía apuntando al mismo objeto Persona antes de esa reasignación, por lo que el objeto sigue siendo alcanzable a través de p2 y no es elegible para el GC.

18. ¿Qué ocurre si dos objetos se referencian mutuamente entre sí (por ejemplo, objeto A referencia a B y B referencia a A), pero ninguna otra variable del programa referencia a ninguno de los dos?

Respuesta correcta: A) Ambos son elegibles para el Garbage Collector, ya que un recolector de basura moderno detecta ciclos de referencias sin alcanzabilidad externa (no basta con tener referencias entre sí)

El Garbage Collector de la JVM usa un algoritmo de alcanzabilidad (reachability) desde raíces del programa (variables locales, static, etc.), no un simple conteo de referencias; por eso puede detectar y recolectar 'islas' de objetos que solo se referencian entre sí pero son inalcanzables desde el resto del programa.

19. ¿Cuál es la diferencia entre aplicar 'final' a una variable de tipo primitivo y aplicarlo a una variable de tipo referencia (objeto)?

Respuesta correcta: A) En ambos casos impide reasignar la variable a otro valor u objeto; pero si es una referencia a un objeto mutable, el contenido interno del objeto referenciado sí puede seguir modificándose

final impide la reasignación de la propia variable (ya sea un valor primitivo o una referencia), pero no hace inmutable al objeto apuntado por una referencia final: se pueden seguir invocando métodos que modifiquen el estado interno del objeto, si la clase lo permite.

20. ¿Qué efecto tiene declarar un método como 'final' en una clase?

Respuesta correcta: A) Impide que las subclases sobrescriban (override) ese método específico, aunque sí pueden heredarlo y usarlo tal cual

Un método final puede ser heredado y utilizado por subclases, pero estas no pueden proporcionar una implementación distinta (no pueden sobrescribirlo).

21. ¿Qué hace el método intern() de la clase String?

Respuesta correcta: A) Busca en el Pool de Strings un objeto con el mismo contenido; si existe, devuelve la referencia a ese objeto del pool, y si no existe, lo añade al pool y devuelve esa referencia

intern() permite mover (o reutilizar) manualmente un String creado fuera del pool (por ejemplo con new String(...)) hacia el Pool de Strings, devolviendo la referencia compartida equivalente.

22. Dado:

```
String a = new String("java").intern();
```

```
String b = "java";
```

```
System.out.println(a == b);
```

¿Qué imprime?

Respuesta correcta: A) true

Aunque 'new String("java")' crea un objeto fuera del pool, al llamar a .intern() se obtiene la referencia al objeto ya existente en el Pool de Strings (el mismo que referencia 'b' mediante el literal), por lo que '==' da true.

23. ¿Cómo se convierte un StringBuilder a un objeto String una vez terminada la construcción del texto?

Respuesta correcta: A) Llamando al método toString() del StringBuilder

StringBuilder.toString() genera un nuevo objeto String inmutable con el contenido actual acumulado en el buffer del StringBuilder.

24. ¿Por qué usar StringBuilder en lugar de concatenar con '+' puede considerarse una optimización de memoria relevante para el examen de certificación?

Respuesta correcta: A) Porque evita la creación de múltiples objetos String intermedios desechables en el heap durante concatenaciones repetidas, reduciendo la presión sobre el Garbage Collector

Cada concatenación con '+' sobre Strings inmutables genera un nuevo objeto descartando el anterior; en bucles o construcciones largas esto genera muchos objetos temporales que luego debe procesar el Garbage Collector. StringBuilder evita esto modificando un único buffer mutable.

25. De los 6 usos de 'final' repasados (primitivos, objetos mutables, objetos inmutables, clases, métodos de instancia, métodos estáticos), ¿qué distingue a 'final' aplicado a una referencia de un objeto MUTABLE?

Respuesta correcta: A) Impide reasignar la referencia a otro objeto, pero permite modificar el contenido interno (estado) del objeto mutable al que apunta, a través de sus propios métodos

final en una variable de referencia solo bloquea la reasignación de esa variable a otro objeto; si el objeto referenciado es mutable (como un ArrayList o un objeto con setters), su contenido interno puede seguir modificándose normalmente a través de sus métodos.

26. ¿Qué significa que un método estático sea declarado 'final' respecto a las subclases?

Respuesta correcta: A) Impide que una subclase pueda 'ocultar' (hide) ese método estático definiendo uno propio con la misma firma

Los métodos estáticos no se 'sobrescriben' (override) como los de instancia, sino que se 'ocultan' (hiding) si una subclase declara un método static con la misma firma; marcar el método estático original como final impide que las subclases realicen esa ocultación.

27. ¿Cuál de las siguientes declaraciones de un método con varargs es sintácticamente inválida?

Respuesta correcta: A) void metodo(String... nombres, int extra) — varargs como primer parámetro seguido de otro parámetro

Un parámetro varargs debe ser siempre el último de la lista de parámetros del método; colocarlo antes de otro parámetro (como en 'String... nombres, int extra') es un error de compilación.

28. ¿Cuántos parámetros varargs puede tener un mismo método como máximo?

Respuesta correcta: A) Uno solo

Un método solo puede declarar un único parámetro varargs; no es posible tener dos o más parámetros varargs en la misma firma de método.

29. ¿Cuál es el orden de los modificadores de acceso en Java, desde el más restrictivo hasta el más permisivo?

Respuesta correcta: A) private → default (package) → protected → public

De más restrictivo a más permisivo: private (solo la misma clase) → default/package (mismo paquete) → protected (mismo paquete + subclases en otros paquetes) → public (accesible desde cualquier lugar).

30. ¿A qué elementos del lenguaje se les puede aplicar un modificador de acceso como public, protected, default o private?

Respuesta correcta: A) A clases de nivel superior (con restricciones, ya que solo public o default aplican ahí), a métodos, a variables (campos) y a constructores

Los modificadores de acceso se aplican a miembros de clase (campos, métodos, constructores) y a clases de nivel superior (aunque estas últimas solo admiten public o default/package, no protected ni private). Las variables locales dentro de un método, en cambio, no llevan modificadores de acceso.

31. Si una variable se declara con acceso 'default' (sin modificador explícito) en una clase, ¿desde dónde puede accederse a ella?

Respuesta correcta: A) Únicamente desde clases ubicadas en el mismo paquete que la clase que la declara

El acceso 'default' (o de paquete) permite el acceso desde cualquier clase que esté en el mismo paquete que la clase declarante, pero no desde clases de otros paquetes, aunque sean subclases.

32. Si la clase Classroom (en el paquete 'escuela') tiene un constructor 'private', ¿puede la clase School (en el mismo paquete 'escuela') crear instancias de Classroom con 'new Classroom()'?

Respuesta correcta: A) Sí, porque un constructor private es accesible desde cualquier código dentro de la misma clase donde fue declarado, y en este caso ambas clases están en el mismo archivo o se evalúa dentro del contexto de la propia clase Classroom; si School es una clase externa distinta, en realidad NO podría acceder a ese constructor

Un constructor (o cualquier miembro) private solo es accesible dentro de la misma clase donde se declara, sin importar si otras clases están en el mismo paquete; el modificador 'default' sí permitiría el acceso entre clases del mismo paquete, pero 'private' es más restrictivo que eso.

33. ¿Qué permite hacer un 'import estático' (static import) en Java, por ejemplo 'import static java.lang.Math.PI;'?

Respuesta correcta: A) Acceder directamente al miembro estático (PI en este caso) sin necesidad de calificarlo con el nombre de la clase (Math.PI), simplemente escribiendo PI

Un import estático permite referenciar un miembro static (campo o método) de otra clase sin tener que escribir el nombre de la clase como prefijo; 'import static java.lang.Math.PI;' permite usar simplemente 'PI' en lugar de 'Math.PI'.

34. ¿Cuál es la diferencia entre 'import static Clase.miembro;' (import estático explícito) e 'import static Clase.*;' (import estático con asterisco)?

Respuesta correcta: A) El explícito importa solo el miembro estático indicado; el de asterisco importa todos los miembros estáticos (campos y métodos) de esa clase

Igual que con los imports normales, el import estático explícito trae un único miembro específico, mientras que la versión con asterisco importa todos los miembros estáticos disponibles de esa clase, con el riesgo de generar más conflictos de nombres.

35. ¿Cuándo se ejecutan los bloques de inicialización estática de una clase respecto a la creación de sus instancias?

Respuesta correcta: A) Se ejecutan una única vez, cuando la clase se carga por primera vez en la JVM, antes de que se cree la primera instancia (si es que se crea alguna)

Los bloques static se ejecutan una única vez al cargar la clase (por ejemplo, la primera vez que se referencia esa clase en el programa, ya sea creando una instancia, accediendo a un miembro static, etc.), independientemente de cuántas instancias se creen después.

36. ¿Se lanza NullPointerException al invocar un método estático a través de una referencia de objeto que vale null, por ejemplo 'Chimp c = null; c.metodoEstatico();'?

Respuesta correcta: A) No, no se lanza NullPointerException, porque los métodos estáticos están asociados a la clase, no a la instancia; Java resuelve la llamada usando el tipo declarado de la referencia (Chimp), sin necesidad de acceder al objeto real

Aunque la sintaxis 'c.metodoEstatico()' parece invocar el método sobre el objeto 'c', en realidad el compilador resuelve la llamada usando el tipo declarado de la variable (Chimp) en tiempo de compilación; como los métodos estáticos no dependen de una instancia real, no se produce NullPointerException aunque 'c' sea null en tiempo de ejecución.

37. ¿En qué contexto se menciona que los imports estáticos son particularmente comunes y útiles en la práctica?

Respuesta correcta: A) En frameworks de pruebas unitarias (testing), donde se usan frecuentemente métodos estáticos como assertEquals() sin tener que calificarlos con el nombre de la clase

Un uso muy común de los imports estáticos es en frameworks de testing (como JUnit), donde métodos estáticos de aserciones (por ejemplo assertEquals, assertTrue) se importan estáticamente para escribir pruebas más limpias y legibles, sin repetir el nombre de la clase en cada línea.

38. Si se pasa un StringBuilder como argumento a un método y dentro del método se invoca sb.append("texto"), ¿el cambio se refleja en el objeto original fuera del método?

Respuesta correcta: A) Sí, porque se copia el valor de la referencia (que apunta al mismo objeto en el heap), y append() modifica ese objeto mutable directamente a través de esa referencia compartida

Como Java pasa por valor la referencia (no el objeto), el parámetro dentro del método apunta al mismo objeto StringBuilder que la variable original. Al ser StringBuilder mutable, invocar append() modifica ese objeto compartido, y el cambio es visible también a través de la referencia original fuera del método.

39. ¿Cuál es la regla obligatoria sobre la posición de 'this(...)' cuando se usa para delegar a otro constructor de la misma clase?

Respuesta correcta: A) Debe ser la primera línea ejecutable dentro del constructor; no puede aparecer después de ninguna otra instrucción

Cuando se usa this(...) para invocar otro constructor sobrecargado de la misma clase, debe ser obligatoriamente la primera sentencia del constructor; el compilador rechaza cualquier código antes de esa llamada.

40. ¿Se puede invocar un constructor como si fuera un método normal en algún punto del código, por ejemplo escribiendo 'this.MiClase();' en medio de un método?

Respuesta correcta: A) No, los constructores no pueden invocarse como métodos regulares; solo pueden llamarse mediante 'new' (para crear una instancia) o mediante this(...)/super(...) desde otro constructor

Un constructor no es un método normal invocable en cualquier punto del código; solo se ejecuta al crear una instancia con 'new', o al delegar explícitamente desde otro constructor mediante this(...) o super(...), y únicamente como la primera línea de ese constructor.

41. En la resolución de sobrecarga de métodos (overloading), ¿cuál es el orden de prioridad que Java sigue al elegir qué versión invocar, de mayor a menor preferencia?

Respuesta correcta: A) 1) Coincidencia exacta con el tipo primitivo, 2) conversión a través del wrapper correspondiente (autoboxing), 3) coincidencia mediante una superclase como Object, 4) por último, la versión con varargs

Java prioriza, en este orden: coincidencia exacta sin conversión (incluyendo widening entre primitivos), luego autoboxing/unboxing al wrapper correspondiente, después coincidencia por una superclase (como Object), y solo como último recurso, si nada más aplica, la versión con parámetro varargs.

42. ¿Cuál es la diferencia entre 'hiding' (ocultamiento) y 'overriding' (sobrescritura) al redefinir en una subclase un método con la misma firma que uno de la superclase?

Respuesta correcta: A) Si el método es static, la subclase lo 'oculta' (hiding) y la resolución de qué versión se ejecuta se determina en tiempo de compilación según el tipo de la referencia; si el método es de instancia, la subclase lo 'sobrescribe' (overriding) y la versión ejecutada se determina en tiempo de ejecución según el tipo real del objeto (polimorfismo)

Los métodos static se resuelven mediante 'static/early binding' según el tipo de la referencia en tiempo de compilación (hiding); los métodos de instancia se resuelven mediante 'dynamic binding' según el tipo real del objeto en tiempo de ejecución (overriding, verdadero polimorfismo).

43. ¿Qué es la covarianza en tipos de retorno al sobrescribir un método en una subclase?

Respuesta correcta: A) Permite que el método sobrescrito en la subclase declare un tipo de retorno que sea un subtipo del tipo de retorno declarado en el método original de la superclase, en lugar de exigir exactamente el mismo tipo

Desde Java 5, al sobrescribir un método, el tipo de retorno en la subclase puede ser un subtipo (covariante) del tipo de retorno declarado en la superclase, en lugar de tener que coincidir exactamente; por ejemplo, un método que originalmente devuelve Animal puede sobrescribirse devolviendo Perro, si Perro extiende Animal.

44. Si una subclase no invoca explícitamente a super(...) en su constructor, ¿qué hace el compilador automáticamente?

Respuesta correcta: A) Inserta implícitamente una llamada a super() (sin argumentos) como primera línea del constructor, invocando el constructor sin parámetros de la superclase

Si el constructor de una subclase no comienza con this(...) ni super(...), el compilador inserta implícitamente 'super();' como primera línea, invocando el constructor sin argumentos de la superclase; si la superclase no tiene un constructor sin argumentos disponible, esto produce un error de compilación.

45. ¿Qué error de compilación ocurre si la superclase NO tiene un constructor sin argumentos (solo tiene uno con parámetros), y la subclase no invoca explícitamente a super(argumentos)?

Respuesta correcta: A) Error de compilación, porque el compilador intenta insertar automáticamente 'super();' (sin argumentos), que no existe en la superclase en este caso

Cuando la superclase carece de un constructor sin argumentos, la subclase está obligada a invocar explícitamente un super(argumentos) válido como primera línea de su propio constructor; de lo contrario, el intento implícito del compilador de insertar 'super();' falla porque ese constructor no existe, generando un error de compilación.

46. Al sobrescribir un método en una subclase, ¿se puede reducir su nivel de acceso respecto al método original de la superclase (por ejemplo, de protected a private)?

Respuesta correcta: A) No, el método sobrescrito solo puede mantener el mismo nivel de acceso o aumentarlo (hacerlo más visible), nunca reducirlo

Una regla fundamental del overriding es que el método en la subclase no puede tener un nivel de acceso más restrictivo que el del método original en la superclase; puede mantenerlo igual o ampliarlo (por ejemplo, de protected a public), pero nunca reducirlo (de protected a private no sería válido).

47. ¿Se heredan los métodos privados de una superclase hacia sus subclases?

Respuesta correcta: A) No, los métodos private no se heredan; por lo tanto, un método con la misma firma en la subclase no lo 'sobrescribe' (override), sino que simplemente es un método nuevo e independiente

Los métodos private pertenecen exclusivamente a la clase donde se declaran y no son visibles ni heredados por las subclases; si una subclase declara un método con la misma firma, es un método completamente nuevo, no una sobreescritura del método privado de la superclase.

48. ¿Cuáles son características típicas que debe cumplir una clase para considerarse inmutable en Java?

Respuesta correcta: A) Declarar sus campos como private y final, no proveer métodos setter, inicializar todos los campos en el constructor, y si tiene campos de referencia a objetos mutables, evitar exponer referencias directas a ellos (por ejemplo, devolviendo copias defensivas)

Una clase inmutable típicamente combina: campos private final inicializados en el constructor, ausencia de setters, la clase en sí puede marcarse final para evitar subclases que rompan la inmutabilidad, y si contiene referencias a objetos mutables, se toman precauciones (como copias defensivas) para que su estado no pueda alterarse después de construido el objeto.

49. En un ejemplo que combina varargs, constructores sobrecargados y métodos estáticos, ¿por qué es importante identificar primero cuál constructor específico se invoca antes de analizar el resto del comportamiento del objeto?

Respuesta correcta: A) Porque el constructor invocado determina qué inicializaciones específicas ocurren (valores asignados a los campos, posibles delegaciones con this(), etc.), lo cual afecta directamente el estado inicial del objeto y por tanto el resultado de cualquier análisis posterior de su comportamiento

Cuando existen varios constructores sobrecargados (incluyendo posibles versiones con varargs) que pueden delegarse entre sí con this(), identificar correctamente cuál se invoca en cada caso es el primer paso indispensable para poder rastrear con precisión qué valores terminan asignados a los campos del objeto.

07. Herencia, Polimorfismo y Casting

1. ¿Qué es 'overriding' (sobrescritura) de métodos?

Respuesta correcta: A) Redefinir en una subclase un método con la misma firma que el de la superclase, cambiando su implementación

El overriding ocurre cuando una subclase redefine un método heredado con la misma firma; el overloading es tener varios métodos con el mismo nombre pero distinta lista de parámetros.

2. ¿Qué palabra clave permite invocar el constructor o método de la superclase desde una subclase?

Respuesta correcta: A) super

'super' referencia la superclase; super() invoca su constructor y super.metodo() invoca su implementación de un método.

3. Dado:

```
class Animal { void sonido(){ System.out.println("..."); } }  
class Perro extends Animal { void sonido(){ System.out.println("Guau"); } }  
Animal a = new Perro();
```

a.sonido();

¿Qué se imprime?

Respuesta correcta: A) Guau

El polimorfismo en tiempo de ejecución (dynamic dispatch) hace que se ejecute la versión sobrescrita del objeto real (Perro), no la del tipo de referencia declarado (Animal).

4. Java permite herencia múltiple de clases pero sí de interfaces. ¿Verdadero o falso?

Respuesta correcta: A) Falso: Java NO permite herencia múltiple de clases (solo una superclase), pero SÍ permite implementar múltiples interfaces

Una clase solo puede extender una única superclase (extends), pero puede implementar múltiples interfaces (implements A, B, C).

5. ¿Qué modificador de acceso permite que un miembro sea visible solo dentro del mismo paquete y en subclases de otros paquetes?

Respuesta correcta: A) protected

'protected' permite el acceso desde el mismo paquete y también desde subclases ubicadas en paquetes distintos. El modificador 'default' (sin palabra clave) solo permite acceso dentro del mismo paquete.

6. ¿Puede un método sobrescrito (override) en una subclase declarar una excepción checked más amplia que la del método original de la superclase?

Respuesta correcta: A) No, el método sobrescrito solo puede declarar la misma excepción checked, una subclase de ella, o ninguna; nunca una más amplia

Una regla de overriding es que el método no puede lanzar excepciones checked nuevas o más amplias que las declaradas en el método de la superclase, para preservar la sustituibilidad de Liskov.

7. ¿Qué es el 'binding estático' (static binding) frente al 'binding dinámico' (dynamic binding)?

Respuesta correcta: A) El binding estático resuelve en compilación qué método se llama (métodos static, private, final, y sobrecarga); el binding dinámico resuelve en tiempo de ejecución (métodos sobrescritos vía polimorfismo)

Los métodos static, private y final (y la resolución de sobrecarga) se enlazan en compilación (static binding); los métodos de instancia sobrescritos se resuelven en runtime según el tipo real del objeto (dynamic binding/dispatch).

8. ¿Puede una clase en Java extender (heredar de) más de una clase directamente?

Respuesta correcta: A) No, Java solo permite herencia simple de clases (una única superclase directa mediante extends); la herencia múltiple de comportamiento se logra mediante la implementación de múltiples interfaces

A diferencia de otros lenguajes, Java restringe la herencia de clases a una única superclase directa (extends), evitando los problemas de ambigüedad de la herencia múltiple; sin embargo, sí permite implementar múltiples interfaces (implements A, B, C).

9. ¿De qué clase derivan, directa o indirectamente, todas las clases en Java, incluso si no se especifica explícitamente ningún 'extends'?

Respuesta correcta: A) java.lang.Object

Toda clase en Java que no declare explícitamente 'extends OtraClase' extiende implícitamente de java.lang.Object, que es la raíz de la jerarquía de clases y provee métodos como equals(), hashCode(), toString(), entre otros.

10. ¿Cuál es una diferencia clave entre una clase abstracta y una interfaz en cuanto a los campos que pueden declarar?

Respuesta correcta: A) Una clase abstracta puede declarar campos de instancia normales (con cualquier modificador y mutables); una interfaz solo puede declarar constantes, que son implícitamente public static final

Una clase abstracta se comporta como una clase normal en cuanto a campos (puede tener campos de instancia mutables con cualquier modificador de acceso); una interfaz, en cambio, solo puede declarar constantes, que son automáticamente public static final, sin estado mutable de instancia.

11. ¿Qué modificadores son implícitos en un método abstracto declarado dentro de una interfaz (sin cuerpo, sin la palabra 'default' ni 'static')?

Respuesta correcta: A) public y abstract, aunque no se escriban explícitamente en el código

Un método declarado sin cuerpo dentro de una interfaz (y sin las palabras default o static) es implícitamente public y abstract, sin necesidad de escribir esos modificadores explícitamente; de hecho, escribir 'public abstract' es redundante pero válido.

12. En un problema que combina miembros estáticos, bloques de inicialización y herencia entre una superclase y una subclase, ¿en qué orden se ejecutan los bloques static cuando se crea por primera vez una instancia de la subclase?

Respuesta correcta: A) Primero los bloques static de la superclase (de arriba hacia abajo si hay varios), y después los bloques static de la subclase, todo esto antes de cualquier bloque de instancia o constructor

Al cargar una jerarquía de clases, la JVM ejecuta primero los bloques static de la superclase (siguiendo la jerarquía desde la clase más alta hacia abajo) y luego los de la subclase, todo esto una única vez, antes de que se ejecuten los bloques de instancia o el constructor de la instancia que se está creando.

13. ¿Qué papel cumple 'var' al declarar una variable local que luego se usará para hacer casting a otro tipo, considerando que el compilador infiere el tipo real de var en tiempo de compilación?

Respuesta correcta: A) El tipo inferido para 'var' es el tipo real y concreto de la expresión usada para inicializarla; cualquier casting posterior se analiza contra ese tipo inferido exactamente igual que si se hubiera escrito el tipo explícitamente

Aunque se use la palabra 'var', el compilador determina y fija un tipo concreto en tiempo de compilación según la expresión de inicialización; a partir de ahí, esa variable se comporta exactamente igual que si se hubiera declarado con el tipo explícito, incluyendo las reglas normales de casting.

14. ¿Qué es el 'upcasting' en Java y por qué generalmente es seguro y no requiere cast explícito?

Respuesta correcta: A) Es convertir la referencia de un objeto hacia un tipo de su superclase (o interfaz que implementa); es seguro porque toda subclase 'es un' tipo de su superclase, por lo que Java lo permite implícitamente sin necesidad de escribir el cast

El upcasting mueve la referencia hacia un tipo 'más general' (superclase o interfaz); como toda instancia de una subclase cumple automáticamente el contrato de su superclase (relación 'is-a'), Java permite este casting de forma implícita, sin necesidad de escribirlo explícitamente ni riesgo de error en tiempo de ejecución.

15. ¿Por qué el 'downcasting' (convertir una referencia de un tipo general a un tipo más específico) requiere un cast explícito y puede fallar en tiempo de ejecución?

Respuesta correcta: A) Porque no todo objeto de un tipo general es necesariamente una instancia del subtipo específico al que se quiere convertir; si el objeto real no es de ese subtipo (o uno compatible), se lanza ClassCastException en tiempo de ejecución

A diferencia del upcasting, el downcasting no está garantizado en compilación: el compilador permite el cast si existe alguna relación de herencia posible, pero solo en tiempo de ejecución se verifica si el objeto real es efectivamente una instancia de ese subtipo; si no lo es, se lanza ClassCastException.

16. Dado:

Animal a = new Perro();

Gato g = (Gato) a;

Suponiendo que Gato y Perro son ambas subclases de Animal pero no están relacionadas entre sí, ¿qué ocurre al ejecutar la segunda línea?

Respuesta correcta: A) Se lanza ClassCastException en tiempo de ejecución, porque el objeto real referenciado por 'a' es un Perro, no un Gato, y no existe relación de herencia entre Perro y Gato

Aunque el compilador permite el cast porque tanto Gato como Perro son subclases de Animal (existe una relación jerárquica compatible en teoría), en tiempo de ejecución la JVM verifica el tipo real del objeto; como el objeto es efectivamente un Perro (no un Gato), se lanza ClassCastException.

17. ¿Puede declararse una variable de referencia usando el tipo de una interfaz, en lugar de una clase concreta, por ejemplo 'Volador v = new Aguila();' donde Volador es una interfaz implementada por Aguila?

Respuesta correcta: A) Sí, es completamente válido y común: la variable puede tener como tipo declarado cualquier interfaz que la clase concreta implemente, permitiendo un código más flexible y desacoplado de implementaciones concretas

Declarar variables usando el tipo de una interfaz (en lugar del tipo concreto) es una práctica común que favorece el polimorfismo y el desacoplamiento, permitiendo cambiar la implementación concreta sin modificar el código que usa la variable, siempre que respete el contrato de la interfaz.

18. ¿Qué relación existe entre el casting y el polimorfismo al invocar un método sobrescrito a través de una referencia con upcast, por ejemplo 'Animal a = new Perro(); a.hacerSonido();' donde Perro sobrescribe hacerSonido()?

Respuesta correcta: A) Se ejecuta la versión sobrescrita en Perro (el tipo real del objeto), independientemente de que la referencia 'a' esté declarada como Animal, gracias al polimorfismo en tiempo de ejecución (dynamic dispatch)

Para métodos de instancia sobrescritos, Java determina en tiempo de ejecución qué implementación ejecutar basándose en el tipo real del objeto (Perro), no en el tipo declarado de la referencia (Animal); esto es precisamente lo que permite el polimorfismo, independientemente del upcasting realizado.

19. ¿Cómo puede usarse el operador instanceof para evitar una ClassCastException antes de realizar un downcasting?

Respuesta correcta: A) Comprobando primero con 'if (objeto instanceof TipoDestino)' que el objeto realmente es (o hereda de) el tipo al que se quiere castear, y solo entonces realizar el cast dentro de ese bloque, garantizando que sea seguro

Verificar 'if (objeto instanceof TipoDestino)' antes de castear permite confirmar en tiempo de ejecución que el objeto es compatible con el tipo destino, evitando así intentar un cast que fallaría con ClassCastException; con el pattern matching moderno, además se obtiene directamente la variable ya casteada.

20. En el ejemplo con Bird, Parrot y Macaw (donde Parrot y Macaw extienden Bird y sobrescriben fly()), si se tiene 'Bird b = new Macaw();' y se invoca 'b.fly();', ¿qué versión de fly() se ejecuta?

Respuesta correcta: A) La versión sobrescrita en Macaw, ya que el polimorfismo determina la implementación a ejecutar según el tipo real del objeto en tiempo de ejecución, sin importar que la referencia sea de tipo Bird

Al ser fly() un método de instancia sobrescrito, la JVM resuelve la llamada usando el tipo real del objeto (Macaw), no el tipo declarado de la referencia (Bird), demostrando el comportamiento polimórfico independientemente del upcasting de la variable.

21. ¿Por qué es importante distinguir entre 'la variable de referencia' y 'el objeto' al analizar el efecto del casting sobre el comportamiento de un programa?

Respuesta correcta: A) Porque el casting cambia el tipo con el que se accede al objeto a través de esa referencia (afectando qué miembros son visibles en tiempo de compilación), pero nunca cambia el objeto real en el heap ni su comportamiento polimórfico real en métodos sobrescritos

El casting solo afecta la 'vista' o tipo con el que se accede al objeto a través de una referencia particular (determinando qué miembros son accesibles en compilación), pero el objeto en sí, su tipo real y su comportamiento en métodos sobrescritos (polimorfismo) permanecen exactamente iguales sin importar cómo se le haga referencia.

22. Si se hace upcast de una referencia (por ejemplo, de Macaw a Bird) y luego se intenta acceder a un miembro estático a través de esa referencia casteada, ¿qué determina qué versión del miembro estático se usa?

Respuesta correcta: A) El tipo declarado de la referencia en el punto de acceso (Bird en este caso), ya que los miembros estáticos se resuelven en tiempo de compilación según el tipo de la referencia, no según el tipo real del objeto

A diferencia de los métodos de instancia (polimórficos, resueltos por el tipo real del objeto), los miembros estáticos se resuelven en tiempo de compilación según el tipo declarado de la referencia utilizada para acceder a ellos; por eso, si se castea a Bird, se usará la versión estática asociada a Bird, independientemente del tipo real del objeto.

23. ¿Qué relación se menciona entre los records (Java 14+) y las clases inmutables?

Respuesta correcta: A) Los records ofrecen una forma concisa de crear clases inmutables por defecto (campos final, sin setters, con equals/hashCode/toString generados), ahorrando el código repetitivo (boilerplate) que normalmente se escribe a mano para lograr inmutabilidad

Los records están diseñados precisamente para modelar datos inmutables de forma concisa: sus componentes son implícitamente private final, generan automáticamente accessors, equals(), hashCode() y toString(), ahorrando gran parte del código repetitivo que antes había que escribir manualmente para lograr una clase inmutable equivalente.

24. ¿En qué se diferencia crear una clase anónima que implementa una interfaz, respecto a crear una clase con nombre que usa 'implements' de forma tradicional?

Respuesta correcta: A) La clase anónima combina la declaración y la instanciación en un único paso, sin nombre propio ni posibilidad de reutilizarse en otro lugar del código; la clase con nombre (implements) se declara por separado y puede instanciarse y reutilizarse cuantas veces se necesite

Las clases anónimas son útiles para implementaciones de uso único y puntual, ya que se declaran e instancian en la misma expresión sin nombre propio; una clase con nombre que usa implements se define de forma independiente y reutilizable en múltiples puntos del programa.

25. ¿Puede crearse una clase anónima que extienda una clase declarada como 'final'?

Respuesta correcta: A) No, una clase final no puede ser extendida por ninguna subclase, incluida una clase anónima, ya que 'final' prohíbe la herencia por completo

El modificador final en una clase prohíbe cualquier tipo de herencia, sin excepciones; esto incluye a las clases anónimas, que técnicamente también son un tipo de subclase (o implementación) creada al vuelo, y por tanto no pueden extender una clase final.

26. ¿Puede una clase anidada estática (static nested class) acceder directamente a los miembros de instancia (no static) de la clase externa que la contiene?

Respuesta correcta: A) No, al no estar asociada a ninguna instancia particular de la clase externa, una clase anidada estática solo puede acceder a los miembros static de la clase externa, no a los de instancia

Como una clase anidada estática no está vinculada a una instancia específica de la clase externa, solo puede acceder directamente a los miembros static de esa clase externa; para acceder a miembros de instancia necesitaría recibir o crear una referencia explícita a un objeto de la clase externa.

27. En el contexto del polimorfismo, ¿qué determina la JVM en tiempo de ejecución al invocar un método de instancia sobrescrito a través de una referencia?

Respuesta correcta: A) Determina cuál implementación del método ejecutar basándose en el tipo real del objeto al que apunta la referencia, no en el tipo declarado de la variable que la contiene

El polimorfismo en tiempo de ejecución (dynamic method dispatch) hace que la JVM seleccione la implementación del método sobrescrito según el tipo real del objeto (el que fue efectivamente creado con 'new'), independientemente del tipo con el que se haya declarado la variable de referencia usada para invocarlo.

28. ¿Pueden los métodos definidos dentro de un enum comportarse de forma distinta para cada constante, similar al polimorfismo entre subclases?

Respuesta correcta: A) Sí, mediante un método abstracto declarado en el enum, cada constante puede proporcionar su propio cuerpo de implementación (constant-specific class body), logrando un comportamiento polimórfico por constante

Un enum puede declarar un método abstracto que cada constante implemente con su propio cuerpo específico (constant-specific class body), logrando un comportamiento similar al polimorfismo de subclases pero dentro de las constantes fijas de un mismo enum.

29. En un record 'record Platypus(String name, double length) {}', ¿qué genera automáticamente el compilador respecto al constructor y los métodos de acceso?

Respuesta correcta: A) Genera un constructor canónico que recibe 'name' y 'length' en ese orden y los asigna a los campos correspondientes; y genera métodos accessor llamados exactamente 'name()' y 'length()' (sin prefijo 'get'), que simplemente retornan el valor de cada campo

Un record genera el constructor canónico según el orden de sus componentes declarados, y métodos accessor con el mismo nombre del componente (sin el prefijo 'get' de los JavaBeans tradicionales); en este caso, 'name()' y 'length()', y no genera ningún setter, ya que los records son inmutables.

30. Para ordenar una List<Platypus> por el atributo 'length' usando Collections.sort(), ¿cuál de las siguientes formas usando Comparator es válida y moderna?

Respuesta correcta: A) Collections.sort(lista, Comparator.comparingDouble(Platypus::length));

Comparator.comparingDouble() (variante especializada para double, evitando autoboxing) combinada con una referencia al accessor length() del record, crea un Comparator adecuado para pasar a Collections.sort(lista, comparator), ordenando la lista según ese atributo.

31. ¿Por qué, al sobrescribir un método genérico heredado en una subclase concreta que fija el tipo parametrizado (por ejemplo, una subclase que extiende Contenedor<String>), el método sobrescrito debe usar exactamente 'String' en lugar de repetir el nombre genérico original de la superclase?

Respuesta correcta: A) Porque una vez que la subclase fija el argumento de tipo (Contenedor<String>), el compilador sustituye el parámetro de tipo genérico por 'String' en todos los métodos heredados de esa jerarquía; el método sobrescrito debe coincidir con esa firma ya resuelta, no con el parámetro de tipo genérico original sin resolver

Cuando una subclase concreta fija el argumento de tipo de su superclase genérica (por ejemplo, extendiendo Contenedor<String>), el compilador sustituye el parámetro de tipo genérico por el tipo concreto (String) en los métodos heredados; para sobrescribir correctamente esos métodos, la subclase debe usar esa firma ya resuelta con 'String', no la variable de tipo genérica original sin resolver.

32. ¿Cuál es la diferencia fundamental entre sobrecarga (overloading) y sobrescritura (override) de métodos en cuanto a la firma del método?

Respuesta correcta: A) La sobrecarga implica métodos con el MISMO nombre pero DISTINTA lista de parámetros (en la misma clase o jerarquía); la sobrescritura implica un método en una subclase con EXACTAMENTE la misma firma (nombre y parámetros) que uno heredado de la superclase, cambiando su implementación

La sobrecarga (overloading) permite tener varios métodos con el mismo nombre pero distinta lista de parámetros (incluso en la misma clase); la sobrescritura (overriding) requiere que la subclase reproduzca exactamente la misma firma (nombre y parámetros) del método heredado de la superclase, proporcionando una implementación distinta para ese mismo contrato.

33. ¿Cuál es una forma común y concisa de iterar sobre un Map usando entrySet() combinado con una referencia a método o lambda?

Respuesta correcta: A) `mapa.entrySet().forEach(entrada -> System.out.println(entrada.getKey() + "=" + entrada.getValue()));`

Obtener `entrySet()` (una vista de los pares clave-valor como objetos `Map.Entry`) y usar `forEach()` con una lambda que accede a `getKey()` y `getValue()` de cada entrada es una forma común y legible de iterar sobre un Map combinando streams/forEach con interfaces funcionales.

08. Interfaces y Clases Abstractas

1. ¿Qué permiten los métodos 'default' en una interfaz (desde Java 8)?

Respuesta correcta: A) Proveer una implementación concreta dentro de la interfaz que las clases implementadoras pueden usar o sobrescribir

Los métodos default tienen cuerpo dentro de la interfaz, permitiendo evolucionar interfaces sin romper implementaciones existentes.

2. ¿Puede una clase abstracta tener constructores?

Respuesta correcta: A) Sí, aunque no se pueda instanciar directamente, sus constructores se ejecutan al instanciar subclases concretas

Las clases abstractas pueden tener constructores, campos y métodos concretos; se invocan mediante `super()` al construir una subclase concreta.

3. ¿Qué es una clase anónima en Java?

Respuesta correcta: A) Una clase sin nombre que se define e instancia en un único paso, útil para implementar una interfaz o extender una clase de forma puntual

Una clase anónima combina la declaración de una subclase (o implementación de interfaz) y su instanciación en una sola expresión, sin nombre propio.

4. ¿Cuántos métodos abstractos puede tener como máximo una interfaz funcional?

Respuesta correcta: A) Exactamente uno (además de métodos default/static)

Una interfaz funcional (anotada opcionalmente con `@FunctionalInterface`) debe declarar exactamente un método abstracto, lo que permite implementarla con una expresión lambda.

5. ¿Puede una interfaz declarar campos (variables)?

Respuesta correcta: A) Sí, pero son implícitamente public static final (constantes), no se pueden tener campos de instancia mutables

Cualquier campo declarado en una interfaz es implícitamente `public static final`, es decir, una constante compartida, no un campo de instancia.

6. ¿Qué son los métodos 'static' dentro de una interfaz (desde Java 8)?

Respuesta correcta: A) Métodos con implementación que pertenecen a la interfaz misma y se invocan como NombreInterfaz.metodo(), no se heredan a las clases implementadoras

Los métodos static en interfaces se invocan sobre el nombre de la interfaz (como utilidades relacionadas), y no se heredan ni pueden sobrescribirse en las clases implementadoras.

7. Si una clase implementa dos interfaces que definen el mismo método default con distinta implementación, ¿qué ocurre?

Respuesta correcta: A) La clase debe sobrescribir explícitamente ese método para resolver la ambigüedad, o el código no compila

Ante un conflicto de métodos default heredados de múltiples interfaces, el compilador exige que la clase implementadora sobrescriba el método explícitamente para eliminar la ambigüedad.

8. ¿Cuál es el propósito de un método 'private' dentro de una interfaz (introducido en Java 9)?

Respuesta correcta: A) Permitir compartir código común entre varios métodos default (o static) de la misma interfaz, sin exponer ese método como parte del contrato público de la interfaz

Los métodos private en interfaces (Java 9+) permiten factorizar lógica común reutilizada por varios métodos default (o static) dentro de la misma interfaz, sin exponer esa lógica interna a las clases implementadoras ni formar parte del contrato público.

9. ¿Puede una clase implementadora de una interfaz sobrescribir un método static declarado en esa interfaz?

Respuesta correcta: A) No, los métodos static de una interfaz no se heredan ni pueden sobrescribirse por las clases implementadoras; se invocan directamente sobre el nombre de la interfaz

Los métodos static de una interfaz pertenecen exclusivamente a la interfaz misma; no se heredan por las clases implementadoras y por tanto no pueden ser sobrescritos por ellas, se acceden como NombreInterfaz.metodo().

10. ¿Cuál es una ventaja clave de usar un record en lugar de una clase tradicional para representar datos inmutables simples?

Respuesta correcta: A) Se reduce drásticamente el código repetitivo (boilerplate): el compilador genera automáticamente el constructor canónico, accessors, equals(), hashCode() y toString(), en lugar de tener que escribirlos manualmente

La ventaja principal de los records es la reducción de código repetitivo: en lugar de escribir manualmente constructor, getters, equals(), hashCode() y toString() para una clase de datos inmutable, el compilador los genera automáticamente a partir de los componentes declarados.

11. ¿En qué se diferencia un campo static de un campo de instancia en cuanto al número de copias que existen en memoria?

Respuesta correcta: A) Un campo static tiene una única copia compartida por todas las instancias de la clase; un campo de instancia tiene una copia independiente para cada objeto creado

Un campo static pertenece a la clase, existiendo una única copia compartida en memoria accesible por todas las instancias; en cambio, cada instancia de la clase mantiene su propia copia independiente de los campos de instancia (no static).

12. ¿Cuándo es apropiado usar una clase abstracta en lugar de una interfaz para modelar una jerarquía?

Respuesta correcta: A) Cuando se necesita compartir estado (campos de instancia) y/o código común concreto entre subclases relacionadas, además de posiblemente declarar métodos abstractos que cada subclase deba implementar de forma distinta

Las clases abstractas son apropiadas cuando las subclases comparten no solo un contrato de métodos, sino también estado (campos) y/o comportamiento concreto común, algo que las interfaces (centradas principalmente en contratos de comportamiento) no modelan tan naturalmente, aunque puedan tener métodos default.

10. Enums

1. ¿Pueden los enums en Java tener constructores, campos y métodos?

Respuesta correcta: A) Sí, un enum es una clase especial que puede tener constructores (siempre implícitamente privados), campos y métodos, incluso implementar interfaces

Los enums son clases especiales; pueden tener campos, constructores (implícitamente private) y métodos, además de implementar interfaces y sobrescribir métodos por constante.

2. ¿Qué método permite obtener la posición ordinal (basada en 0) de una constante enum?

Respuesta correcta: A) ordinal()

ordinal() devuelve la posición de la constante según el orden de declaración, comenzando en 0. name() devuelve su nombre como String.

3. ¿Se puede usar un enum en un switch statement/expression sin prefijo del nombre de la clase en los 'case'?

Respuesta correcta: A) Sí, solo se usa el nombre de la constante (ej. case LUNES ->)

En un switch sobre un enum, los case usan directamente el nombre de la constante sin calificar con el nombre de la clase.

4. ¿Qué método permite obtener todas las constantes de un enum como un array?

Respuesta correcta: A) NombreEnum.values()

El método static implícito values() devuelve un array con todas las constantes del enum en el orden en que fueron declaradas.

5. ¿Puede un enum implementar un cuerpo de clase distinto para cada constante (constant-specific class body)?

Respuesta correcta: A) Sí, cada constante puede sobrescribir un método abstracto declarado en el enum con su propia implementación

Un enum puede declarar un método abstracto y cada constante puede proveer su propio cuerpo de implementación entre llaves, permitiendo comportamiento distinto por constante.

6. ¿Qué excepción se lanza si se llama a valueOf() de un enum con un nombre que no corresponde a ninguna constante?

Respuesta correcta: A) IllegalArgumentException

NombreEnum.valueOf("texto") lanza IllegalArgumentException si el texto no coincide exactamente con el nombre de ninguna constante declarada.

7. En un ejemplo donde una clase sellada 'SilClass' se declara con 'permits Armadillo', ¿qué ocurre si se intenta crear otra clase 'Perro' que también intente extender SilClass?

Respuesta correcta: A) Error de compilación, porque solo la clase o clases explícitamente listadas en la cláusula 'permits' (en este caso, únicamente Armadillo) están autorizadas a extender la clase sellada

Una clase sellada (sealed) restringe explícitamente, mediante la cláusula 'permits', qué clases pueden extenderla; cualquier intento de una clase no listada (como Perro en este ejemplo) de extender SilClass produce un error de compilación.

8. ¿Es correcto decir que un enum en Java 'funciona como una clase' con instancias fijas predefinidas?

Respuesta correcta: A) Sí, un enum es efectivamente un tipo especial de clase donde las instancias (constantes) están predefinidas y son fijas, pudiendo tener constructores, atributos y métodos igual que una clase normal

Un enum es, internamente, una clase especial cuyas instancias (las constantes declaradas) están fijadas de antemano; a pesar de esta restricción, puede declarar constructores (implícitamente privados), campos y métodos, comportándose en muchos aspectos como cualquier otra clase.

9. ¿Cuándo se vuelve obligatorio el punto y coma (;) después de la lista de constantes en la declaración de un enum?

Respuesta correcta: A) Cuando el enum incluye elementos adicionales después de las constantes, como constructores, atributos o métodos; en ese caso el punto y coma separa la lista de constantes del resto del cuerpo de la clase

Si un enum contiene únicamente la lista de constantes, el punto y coma final es opcional; pero en cuanto se añaden elementos adicionales (constructores, campos, métodos) después de las constantes, el punto y coma se vuelve obligatorio para separar la lista de constantes del resto de miembros del enum.

10. ¿Qué papel juega la encapsulación (campos private con accesores controlados) al diseñar un enum con atributos, como por ejemplo un enum Planeta con un atributo 'masa'?

Respuesta correcta: A) Permite proteger el estado interno de cada constante del enum (como la masa de cada planeta) exponiendo solo los métodos necesarios para leerlo, evitando modificaciones no controladas del atributo una vez inicializado

Igual que en cualquier clase, aplicar encapsulación en un enum (campos private, inicializados en el constructor y expuestos mediante métodos de solo lectura) protege la integridad del estado asociado a cada constante, evitando que código externo modifique valores que deberían permanecer fijos, como la masa de cada planeta en un enum astronómico.

11. ¿Puede el constructor de un enum declararse con acceso 'public' o 'protected'?

Respuesta correcta: A) No, el constructor de un enum solo puede ser private (o package-private/default, que en la práctica tiene un efecto equivalente al estar restringido al propio enum); nunca puede ser public ni protected

Como las instancias de un enum están limitadas exclusivamente a sus constantes predefinidas, el lenguaje prohíbe que su constructor sea public o protected (lo que permitiría crear instancias adicionales desde fuera); solo se permite private o el acceso por defecto (package), ambos con el efecto práctico de restringir la creación de instancias al propio enum.

12. ¿Es correcto decir que un enum siempre tiene, como mínimo, un constructor privado implícito si no se declara ninguno explícitamente?

Respuesta correcta: A) Sí, si no se declara ningún constructor explícito, el compilador genera uno implícito sin argumentos y con acceso private, igual que ocurre conceptualmente en una clase normal, pero respetando la restricción de que nunca puede ser public

Al igual que una clase normal, si un enum no declara explícitamente ningún constructor, el compilador provee uno implícito sin argumentos; en el caso de los enums, ese constructor implícito es siempre private, coherente con la restricción de que las instancias del enum están fijadas a sus constantes.

13. ¿Cuándo es estrictamente necesario colocar un punto y coma (;) después de la lista de constantes en un enum?

Respuesta correcta: A) Cuando después de las constantes se añade cualquier otro miembro (constructor, campos, métodos); en ese caso el punto y coma es obligatorio para delimitar dónde termina la lista de constantes

Si el enum contiene solo constantes (sin ningún otro miembro adicional), el punto y coma final es opcional; pero en cuanto se agregan constructores, campos o métodos después de la lista de constantes, ese punto y coma se vuelve obligatorio para el compilador.

14. ¿Qué elementos genera automáticamente el compilador para un record, evitando que el programador los escriba manualmente?

Respuesta correcta: A) El constructor canónico, los métodos de acceso (accessors) para cada componente, y las sobrescrituras de equals(), hashCode() y toString()

Un record genera automáticamente: el constructor canónico (que asigna cada componente a su campo correspondiente), los métodos accessor (con el mismo nombre del componente, sin prefijo 'get'), y las implementaciones de equals(), hashCode() y toString() basadas en todos los componentes.

15. ¿Puede un record declarar campos static adicionales, distintos de sus componentes?

Respuesta correcta: A) Sí, un record puede declarar campos static (compartidos por la clase, no por instancia), aunque no puede declarar campos de instancia adicionales fuera de sus componentes

Los records permiten declarar campos static adicionales (compartidos a nivel de clase), pero no admiten campos de instancia adicionales más allá de los componentes definidos en su cabecera, para preservar la transparencia de datos del record.

16. ¿Cuál es un error de compilación común al intentar instanciar directamente una clase abstracta con 'new', por ejemplo 'new Animal();' donde Animal es abstract?

Respuesta correcta: A) El compilador rechaza la instanciación directa, ya que una clase abstracta no puede instanciarse por sí misma; solo se pueden crear instancias de sus subclases concretas (no abstractas)

Intentar instanciar directamente una clase abstracta con 'new' siempre produce un error de compilación, independientemente de si declara o no métodos abstractos; una clase abstracta solo puede usarse como superclase de otras clases concretas que sí pueden instanciarse.

17. ¿Qué error de compilación ocurre si una clase concreta (no abstracta) extiende una clase abstracta pero no implementa todos los métodos abstractos heredados?

Respuesta correcta: A) Error de compilación: una clase concreta (no declarada como abstract) está obligada a proveer implementación para todos los métodos abstractos heredados; si no lo hace, debe declararse ella misma como abstract

Si una subclase no proporciona implementación de todos los métodos abstractos heredados y no se declara ella misma como abstract, el compilador genera un error, ya que de otro modo sería posible instanciar una clase con métodos sin implementación real.

18. ¿Qué error de compilación surge si una clase intenta implementar una interfaz sellada (sealed) sin estar listada en su cláusula 'permits'?

Respuesta correcta: A) Error de compilación, ya que solo las clases explícitamente autorizadas en la cláusula permits de la interfaz sellada pueden implementarla

Al igual que con las clases selladas, una interfaz sellada limita explícitamente, mediante 'permits', qué clases pueden implementarla; cualquier clase no autorizada que intente implementarla provoca un error de compilación.

11. Records

1. ¿Son mutables los componentes de un record?

Respuesta correcta: A) No, todos los campos de un record son implícitamente final e inmutables

Los records son inmutables por diseño: sus campos son implícitamente private final, sin setters generados.

2. ¿Qué es un 'constructor compacto' en un record?

Respuesta correcta: A) Una forma de escribir el constructor canónico sin repetir la lista de parámetros, útil para validar o normalizar valores antes de la asignación implícita

El constructor compacto se escribe como 'public NombreRecord { ... validaciones ... }' sin paréntesis de parámetros; el compilador asigna los campos automáticamente al final.

3. ¿Puede un record implementar interfaces?

Respuesta correcta: A) Sí, un record puede implementar una o varias interfaces, igual que una clase normal

Un record puede implementar cualquier interfaz (implements), aunque no puede extender otra clase, ya que implícitamente ya extiende java.lang.Record.

4. ¿Puede un record declarar campos de instancia adicionales fuera de los componentes definidos en su cabecera?

Respuesta correcta: A) No, un record solo puede tener los campos correspondientes a sus componentes declarados; sí puede tener campos static adicionales

Los records solo permiten campos de instancia correspondientes a los componentes de su cabecera (para mantener la transparencia de datos), aunque sí pueden declarar campos static adicionales.

5. ¿Qué genera el compilador si defines explícitamente el accessor de un componente de un record, por ejemplo `public int x() { return x * -1; }`?

Respuesta correcta: A) Usa la implementación personalizada en lugar del accessor generado automáticamente, permitiendo modificar qué se devuelve

Se puede sobrescribir explícitamente el método accessor de un componente; el compilador usará esa implementación personalizada en lugar de generar la trivial por defecto.

12. Sealed Classes y Pattern Matching Avanzado

1. ¿Para qué sirve la palabra clave 'sealed' en una clase o interfaz (Java 17)?

Respuesta correcta: A) Para restringir explícitamente qué clases pueden extenderla o implementarla, usando la cláusula 'permits'

'sealed' junto con 'permits' limita el conjunto de subtipos autorizados, permitiendo un control exhaustivo (útil junto a pattern matching en switch).

2. Una subclase de una clase sealed, ¿qué modificador debe declarar obligatoriamente?

Respuesta correcta: A) final, sealed o non-sealed

Cada subclase permitida de un tipo sealed debe declararse explícitamente como final, sealed (para seguir restringiendo) o non-sealed (para abrir la jerarquía de nuevo).

3. En Java 21, ¿qué aporta el pattern matching para switch sobre records (record patterns)?

Respuesta correcta: A) Permite desestructurar los componentes de un record directamente en el 'case' (ej. `case Punto(int x, int y) -> ...`)

Los record patterns (Java 21) permiten deconstruir un record en el propio patrón del case, extrayendo sus componentes como variables locales.

4. ¿Pueden las clases permitidas ('permits') de una clase sealed estar en un paquete diferente?

Respuesta correcta: A) Sí, pero solo si pertenecen al mismo módulo, o si están en el mismo paquete cuando no se usan módulos

Las subclases permitidas de un tipo sealed deben pertenecer al mismo módulo (en un proyecto modularizado) o, si no hay módulos, al mismo paquete que la clase sealed.

5. ¿Qué ventaja aporta un switch exhaustivo sobre una jerarquía sealed junto con pattern matching?

Respuesta correcta: A) El compilador puede verificar que se cubren todos los subtipos posibles, sin necesitar una rama 'default', mejorando la seguridad en compilación

Como el conjunto de subtipos de una clase/interfaz sealed es cerrado y conocido, el compilador puede verificar exhaustividad en un switch pattern matching sin exigir una rama default.

6. ¿Qué patrón usarías en un case para descomponer un record `Punto(int x, int y)` y aplicar una guarda adicional 'cuando x sea positivo' (Java 21, guarded patterns)?

Respuesta correcta: A) `case Punto(int x, int y) when x > 0 -> ...`

Java 21 introduce las guarded patterns con la palabra clave 'when' para añadir una condición booleana adicional al patrón deconstruido dentro de un case.

13. Generics, Wildcards y Type Erasure

1. ¿Qué problema resuelven los Generics en Java?

Respuesta correcta: A) Proveen seguridad de tipos en tiempo de compilación para colecciones y clases, evitando `ClassCastException` en tiempo de ejecución y castings manuales

Los generics permiten parametrizar tipos (`List<String>`, por ejemplo), detectando errores de tipo en compilación en vez de en runtime.

2. ¿Qué significa el comodín '?' extends Number' en una declaración de generics?

Respuesta correcta: A) Acepta el tipo Number o cualquiera de sus subtipos (útil para lectura/producción de datos, 'Producer Extends')

'? extends Number' es un wildcard covariante: acepta Number o subtipos. Sigue el principio PECS (Producer Extends, Consumer Super); con extends solo puedes leer, no añadir (excepto null).

3. ¿Por qué List<String> NO es un subtipo de List<Object> en Java (a diferencia de los arrays)?

Respuesta correcta: A) Porque los generics son invariantes: aunque String sea subtipo de Object, List<String> y List<Object> son tipos distintos sin relación de subtipo entre sí

A diferencia de los arrays (covariantes, lo que puede causar ArrayStoreException en runtime), los generics son invariantes por diseño para mantener la seguridad de tipos en compilación.

4. ¿Qué es 'type erasure' (borrado de tipos) en Java?

Respuesta correcta: A) El proceso por el cual la información de tipo genérico se elimina en tiempo de compilación, quedando solo el tipo raw (u Object) en el bytecode

Debido a type erasure, la JVM no conserva los parámetros de tipo en tiempo de ejecución; List<String> y List<Integer> comparten la misma clase List en bytecode.

5. ¿Qué significa el comodín '? super Integer' en una declaración de generics?

Respuesta correcta: A) Acepta Integer o cualquiera de sus supertipos (útil para escribir/consumir datos, 'Consumer Super')

'? super Integer' es un wildcard contravariante: acepta Integer o cualquier supertipo suyo (como Number u Object), permitiendo añadir elementos Integer de forma segura (principio Consumer Super de PECS).

6. ¿Se puede crear un array de un tipo genérico parametrizado, como new List<String>[10]?

Respuesta correcta: A) No, Java no permite crear directamente arrays de tipos genéricos parametrizados debido al conflicto entre la covarianza de arrays y el borrado de tipos

Crear arrays genéricos parametrizados (new List<String>[10]) no es válido porque, combinado con la covarianza de arrays y el type erasure, podría romper la seguridad de tipos en tiempo de ejecución.

7. ¿Qué es un método genérico y cómo se declara su parámetro de tipo?

Respuesta correcta: A) Un método que declara su propio parámetro de tipo antes del tipo de retorno, por ejemplo: public <T> T primero(List<T> lista)

Un método genérico declara su propio parámetro de tipo entre '<>' justo antes del tipo de retorno, independientemente de si la clase contenedora es o no genérica.

8. ¿Los métodos heredados de la clase Object (como equals(), hashCode() o toString()) cuentan para el límite de 'un único método abstracto' que debe tener una interfaz funcional?

Respuesta correcta: A) No, si una interfaz declara explícitamente un método abstracto con la misma firma que uno de Object (por ejemplo, un 'boolean equals(Object o)'), ese método NO cuenta para el límite de método abstracto único, ya que toda clase ya provee una implementación heredada de Object

Una regla especial permite que una interfaz funcional declare (re-declare) métodos abstractos que coincidan con métodos públicos de Object (como equals, hashCode, toString); estos no se cuentan hacia el límite de un único método abstracto, ya que cualquier clase que implemente la interfaz ya hereda una implementación de esos métodos desde Object.

9. ¿Qué ocurre si una interfaz funcional (con un único método abstracto propio) es extendida por otra interfaz que declara un segundo método abstracto adicional?

Respuesta correcta: A) La nueva interfaz (la que extiende) ya NO es una interfaz funcional, porque ahora tiene dos métodos abstractos en total (el heredado más el nuevo); dejaría de poder implementarse mediante una única lambda

Si una subinterfaz agrega un nuevo método abstracto a una interfaz funcional heredada, el total de métodos abstractos (heredado + nuevo) supera uno, por lo que la subinterfaz deja de cumplir la condición de interfaz funcional y no podría implementarse con una simple lambda.

10. ¿Qué función cumple la anotación @FunctionalInterface colocada sobre una interfaz?

Respuesta correcta: A) Indica la intención de que esa interfaz sea funcional, y hace que el compilador verifique y reporte un error si la interfaz NO cumple con la condición de tener exactamente un método abstracto; sin embargo, la anotación en sí no es obligatoria para que una interfaz sea tratada como funcional

@FunctionalInterface es una anotación opcional pero recomendada que documenta la intención de diseño y permite que el compilador valide en tiempo de compilación que la interfaz efectivamente tiene un único método abstracto; una interfaz sin esta anotación puede seguir siendo funcional si cumple el requisito de todos modos.

11. ¿Qué diferencia de contrato existe entre BiConsumer<T,U> y un UnaryOperator<T> en cuanto a la cantidad de argumentos y el retorno?

Respuesta correcta: A) BiConsumer<T,U> recibe dos argumentos (de tipos T y U) y no devuelve nada (accept); UnaryOperator<T> recibe un único argumento de tipo T y devuelve un valor del mismo tipo T (apply)

BiConsumer<T,U> consume dos valores de entrada sin producir retorno; UnaryOperator<T> transforma un único valor de entrada en un valor de salida del mismo tipo, siendo contratos claramente distintos en cantidad de parámetros y en presencia (o ausencia) de valor de retorno.

12. ¿Qué ventaja aporta migrar código que usaba clases tradicionales con herencia (por ejemplo, para representar distintos comportamientos) hacia el uso de interfaces funcionales con lambdas?

Respuesta correcta: A) Reduce significativamente el código repetitivo (boilerplate) al eliminar la necesidad de declarar clases completas por cada comportamiento distinto, permitiendo expresar la lógica de forma concisa directamente donde se necesita

Al reemplazar jerarquías de clases (o clases anónimas) usadas únicamente para representar comportamientos intercambiables por interfaces funcionales con lambdas, se reduce drásticamente el código repetitivo necesario, logrando una sintaxis mucho más concisa y directa para expresar esa lógica de comportamiento.

13. ¿Qué problema de diseño resolvían los generics (introducidos en Java 5) respecto a tener que crear una clase contenedora específica para cada tipo de dato (por ejemplo, una clase ContenedorDeString y otra ContenedorDeInteger por separado)?

Respuesta correcta: A) Los generics permiten escribir una única clase contenedora parametrizada por un tipo T, reutilizable para cualquier tipo de dato, eliminando la duplicación de código que antes requería una clase distinta por cada tipo específico

Antes de los generics, representar un contenedor reutilizable para distintos tipos requería duplicar código (una clase por tipo) o usar Object (perdiendo seguridad de tipos); los generics permiten declarar una única clase parametrizada (por ejemplo, Contenedor<T>) reutilizable con seguridad de tipos en compilación para cualquier tipo concreto.

14. ¿Qué significa declarar un método genérico a nivel de método, independientemente de si la clase que lo contiene es o no genérica, usando por ejemplo '<E> void metodo(E elemento)'?

Respuesta correcta: A) El método declara su propio parámetro de tipo E, que se determina de forma independiente cada vez que se invoca el método, sin necesidad de que la clase contenedora sea genérica ni de que E coincida con algún parámetro de tipo de la clase

Un método genérico declara su propio parámetro de tipo (como <E> antes del tipo de retorno) independientemente de si la clase es genérica; esto permite, por ejemplo, que una clase genérica Contenedor<T> tenga además un método genérico que use un parámetro de tipo E completamente distinto, determinado en cada invocación del método.

15. ¿Cuál es la convención de nomenclatura habitual para los nombres de parámetros de tipo genérico de una sola letra en Java, como T, E, K, V?

Respuesta correcta: A) T para 'Type' (tipo genérico general), E para 'Element' (elemento de una colección), K para 'Key' (clave) y V para 'Value' (valor), como en Map<K,V>

Aunque cualquier identificador válido podría usarse técnicamente, la convención ampliamente adoptada en la biblioteca estándar y la comunidad Java usa T (Type), E (Element, común en colecciones), K (Key) y V (Value, comunes en estructuras tipo Map<K,V>), entre otras letras similares para casos específicos.

16. ¿Qué representa un wildcard acotado superiormente, como 'List<? extends Number>', en cuanto a las operaciones permitidas sobre esa lista?

Respuesta correcta: A) Permite leer elementos de la lista tratándolos como Number (o un supertipo), pero no permite añadir elementos (salvo null), ya que el compilador no puede garantizar el tipo exacto de los elementos que realmente contiene la lista

Un wildcard '? extends Number' permite leer los elementos como Number de forma segura (son garantizadamente Number o subtipos), pero no permite añadir elementos (excepto null), porque el compilador no puede determinar el subtipo exacto que la lista realmente contiene en tiempo de ejecución (principio 'Producer Extends' de PECS).

17. ¿Qué representa un wildcard acotado inferiormente, como 'List<? super Integer>', en cuanto a las operaciones permitidas sobre esa lista?

Respuesta correcta: A) Permite añadir elementos de tipo Integer (o subtipos) de forma segura, ya que la lista garantiza ser de tipo Integer o algún supertipo de Integer; pero al leer, solo se puede garantizar que los elementos son al menos de tipo Object

Un wildcard '? super Integer' garantiza que la lista acepta Integer (o subtipos) como elementos añadidos de forma segura ('Consumer Super' de PECS), pero al leer un elemento solo se puede asumir con certeza que es de tipo Object, ya que el tipo real podría ser cualquier supertipo de Integer.

18. Antes de la introducción de generics en Java 5, ¿qué era necesario hacer al recuperar un elemento de una colección que internamente almacenaba objetos de tipo Object (como las collections 'raw' de aquella época)?

Respuesta correcta: A) Era necesario hacer un casting explícito manual del Object recuperado hacia el tipo real esperado (por ejemplo, '(String) lista.get(0)'), con el riesgo de un ClassCastException en tiempo de ejecución si el tipo real no coincidía

Antes de los generics, las colecciones almacenaban internamente referencias de tipo Object; recuperar un elemento requería un cast explícito manual hacia el tipo esperado, y si ese cast era incorrecto (el objeto real era de otro tipo), se producía ClassCastException solo detectable en tiempo de ejecución, no en compilación.

19. En una clase genérica como 'class Box<T> { private List<T> elementos; ... }', ¿qué gana el código en seguridad de tipos respecto a la versión sin generics equivalente (usando Object)?

Respuesta correcta: A) El compilador verifica en tiempo de compilación que solo se añadan y recuperen elementos del tipo T declarado, evitando errores de tipo que antes solo se detectaban en tiempo de ejecución mediante ClassCastException

Al parametrizar Box<T>, el compilador puede verificar en tiempo de compilación que solo se inserten y extraigan elementos del tipo T correspondiente, desplazando la detección de errores de tipo desde el tiempo de ejecución (con ClassCastException) hacia el tiempo de compilación, una mejora fundamental de seguridad de tipos.

20. ¿Es obligatorio, a nivel de compilador, usar los nombres de convención (T, E, K, V) para los parámetros de tipo genérico, o se trata solo de una recomendación de estilo?

Respuesta correcta: A) Es solo una convención de estilo ampliamente adoptada; el compilador acepta cualquier identificador válido como nombre de parámetro de tipo genérico, aunque usar nombres no convencionales puede dificultar la lectura del código para otros desarrolladores

Los nombres como T, E, K, V son puramente una convención de la comunidad Java para mejorar la legibilidad; el compilador acepta técnicamente cualquier identificador válido como nombre de parámetro de tipo genérico (por ejemplo, <MiTipo> sería sintácticamente válido, aunque poco convencional).

21. ¿Puede un método static de una clase genérica Box<T> usar directamente el parámetro de tipo T de la clase (por ejemplo, 'static T obtenerDefault() { ... }')?

Respuesta correcta: A) No, un método static no puede referirse al parámetro de tipo T de la clase, ya que T está asociado a cada instancia particular y los métodos static no dependen de ninguna instancia; si el método static necesita ser genérico, debe declarar su propio parámetro de tipo independiente

El parámetro de tipo T de una clase genérica se resuelve por cada instancia concreta (por ejemplo, Box<String> vs Box<Integer>); como los métodos static no están asociados a ninguna instancia particular, no pueden usar directamente el T de la clase, y si necesitan comportamiento genérico deben declarar su propio parámetro de tipo a nivel de método, como '<E> static E metodo(E valor)'.

22. ¿Cómo se puede usar Stream de forma segura sobre una colección genérica List<T> dentro de un método de una clase genérica, para procesar sus elementos?

Respuesta correcta: A) Invocando lista.stream() normalmente; como el compilador ya conoce el tipo T asociado a esa instancia particular, el Stream resultante es un Stream<T> con toda la seguridad de tipos habitual, permitiendo aplicar operaciones como filter/map/forEach sobre elementos de tipo T

Como el compilador conoce el tipo T concreto asociado a cada instancia de la clase genérica, invocar stream() sobre una List<T> produce normalmente un Stream<T> con toda la seguridad de tipos, permitiendo aplicar operaciones de la Stream API con el mismo nivel de garantías que sobre cualquier otro tipo concreto.

23. ¿Cuál es la sintaxis correcta para declarar un parámetro de tipo genérico T que debe ser tanto un subtipo de Number como implementar Comparable<T>, combinando ambas restricciones (bounded type con múltiples límites)?

Respuesta correcta: A) <T extends Number & Comparable<T>>

Para combinar múltiples límites (bounds) en un parámetro de tipo genérico, se usa 'extends' seguido de los límites separados por el símbolo '&' (no comas ni 'implements'); esto permite exigir simultáneamente que T sea subtipo de una clase y que implemente una o más interfaces.

24. Al declarar múltiples límites (bounds) para un parámetro de tipo genérico, como en '<T extends Number & Comparable<T>>', ¿qué regla de orden debe respetarse si uno de los límites es una clase?

Respuesta correcta: A) La clase debe declararse primero, antes de cualquier interfaz, en la lista de límites separados por '&'; solo se permite una clase como límite, pero pueden añadirse varias interfaces después de ella

Cuando se combinan varios límites en un parámetro de tipo genérico, si uno de ellos es una clase (como Number), esta debe aparecer primero en la lista antes que cualquier interfaz (como Comparable); intentar poner la interfaz antes que la clase produce un error de compilación.

25. ¿Cómo se relaciona la covarianza de tipos de retorno (al sobrescribir un método) con los generics, por ejemplo si una clase Contenedor<T> declara un método 'T obtener()' y una subclase concreta ContenedorDeTexto extiende Contenedor<String>?

Respuesta correcta: A) En ContenedorDeTexto, el método sobrescrito puede (y de hecho, efectivamente) devuelve String en lugar de T genérico, porque el compilador sustituye T por String en esa instancia concreta de la jerarquía, siendo coherente con las reglas de covarianza de tipos de retorno

Cuando una subclase concreta fija el parámetro de tipo genérico (por ejemplo, extendiendo Contenedor<String>), el compilador sustituye T por String en los métodos heredados; esto es coherente con la covarianza de tipos de retorno, ya que String (el tipo concreto) es efectivamente el tipo de retorno esperado en esa subclase específica.

26. Si se declara una clase genérica como 'class Contenedor<T extends CharSequence>', ¿por qué se puede usar directamente 'String' como argumento de tipo, por ejemplo 'Contenedor<String>'?

Respuesta correcta: A) Porque String implementa la interfaz CharSequence, por lo que cumple con la restricción del bounded type declarado (T extends CharSequence), permitiendo que String sea un argumento de tipo válido para esa clase genérica

CharSequence es una interfaz que String implementa (junto con StringBuilder y StringBuffer, entre otras); como la restricción declarada es 'T extends CharSequence' (donde 'extends' en este contexto de generics también cubre 'implementa una interfaz'), cualquier tipo que implemente CharSequence, incluido String, es un argumento de tipo válido.

27. ¿Por qué una lista declarada con un wildcard no acotado, como 'List<?> lista', solo permite operaciones de lectura muy limitadas (tratando los elementos como Object) y no permite añadir elementos (salvo null)?

Respuesta correcta: A) Porque el wildcard '?' representa 'algún tipo desconocido', y el compilador no puede garantizar qué tipo específico es realmente, por lo que solo permite operaciones seguras para cualquier tipo posible (como leer como Object), pero no añadir elementos de un tipo concreto que podría no coincidir con el tipo real desconocido

Un wildcard no acotado '?' representa un tipo completamente desconocido en tiempo de compilación; como el compilador no puede verificar qué tipo específico contiene realmente la lista, solo permite operaciones seguras para cualquier tipo (leer elementos tratándolos como Object) y prohíbe añadir elementos de un tipo concreto, ya que podría no coincidir con el tipo real oculto tras el wildcard.

28. ¿Qué es el 'diamond operator' (<>) introducido para simplificar la sintaxis de generics, y desde qué versión de Java está disponible?

Respuesta correcta: A) Es la sintaxis '<>' que permite omitir repetir el tipo genérico en el lado derecho de una instanciación, por ejemplo 'List<String> lista = new ArrayList<>();' en lugar de repetir '<String>' en ambos lados; está disponible desde Java 7

El diamond operator (<>) se introdujo en Java 7 (JEP relacionado con simplificación de generics) para evitar la redundancia de repetir el tipo parametrizado en ambos lados de una declaración con inicialización, ya que el compilador puede inferirlo del lado izquierdo.

29. ¿Pueden los records usar parámetros de tipo genérico, por ejemplo 'record Par<T, U>(T primero, U segundo) {}'?

Respuesta correcta: A) Sí, los records pueden ser genéricos igual que las clases normales, declarando sus propios parámetros de tipo que se usan en los tipos de sus componentes

Los records admiten parámetros de tipo genérico exactamente igual que las clases, permitiendo declarar records reutilizables para distintos tipos de datos, como un record Par<T,U> genérico que combina dos valores de tipos potencialmente distintos.

30. En un ejemplo con una jerarquía Helado (superclase) y HeladoDeChocolate (subclase), ¿por qué NO se puede asignar directamente un 'Contenedor<HeladoDeChocolate>' a una variable declarada como 'Contenedor<Helado>', aunque HeladoDeChocolate sea subtipo de Helado?

Respuesta correcta: A) Porque los generics son invariantes: a pesar de que HeladoDeChocolate es subtipo de Helado, Contenedor<HeladoDeChocolate> y Contenedor<Helado> son tipos completamente distintos sin relación de subtipo entre sí, por lo que la asignación directa no es válida sin wildcards

A diferencia de los arrays (covariantes), los generics son invariantes por diseño: aunque exista una relación de herencia entre los tipos parametrizados (HeladoDeChocolate y Helado), Contenedor<HeladoDeChocolate> y Contenedor<Helado> no tienen ninguna relación de subtipo entre sí; para lograr flexibilidad en estos casos se necesitan wildcards como Contenedor<? extends Helado>.

31. ¿Cómo permite un wildcard acotado, como 'Contenedor<? extends Helado>', aceptar de forma flexible tanto un Contenedor<Helado> como un Contenedor<HeladoDeChocolate>?

Respuesta correcta: A) El wildcard '? extends Helado' representa 'algún subtipo desconocido de Helado', por lo que una variable de ese tipo puede referenciar contenedores parametrizados con Helado o con cualquiera de sus subtipos, resolviendo la limitación de invariancia de los generics para casos de solo lectura

Un wildcard acotado superiormente, como '? extends Helado', permite que la referencia acepte contenedores parametrizados con Helado o cualquier subtipo suyo, proporcionando la flexibilidad de lectura que la invariancia estricta de los generics no permitiría con el tipo exacto Contenedor<Helado>.

32. ¿Qué relación tiene 'Comparator.naturalOrder()' con la interfaz Comparable, al usarse para ordenar una colección?

Respuesta correcta: A) Comparator.naturalOrder() devuelve un Comparator que delega en el propio compareTo() de los elementos (que deben implementar Comparable), aplicando el 'orden natural' definido por esa clase, en lugar de definir una lógica de comparación externa distinta

Comparator.naturalOrder() proporciona un Comparator que simplemente usa el orden natural ya definido por compareTo() en los elementos (que deben implementar Comparable), útil cuando se necesita pasar explícitamente un Comparator a un método pero se quiere conservar el orden natural existente.

33. ¿Por qué NO es posible crear directamente un array de un tipo genérico dentro de una clase genérica, como 'T[] array = new T[10];' dentro de una clase Contenedor<T>?

Respuesta correcta: A) Debido al type erasure, la JVM no conserva información del tipo T en tiempo de ejecución; permitir 'new T[10]' sin saber el tipo real produciría problemas de seguridad de tipos en tiempo de ejecución, por lo que el compilador lo prohíbe directamente

Como el compilador borra la información del parámetro de tipo T en tiempo de ejecución (type erasure), no hay forma de saber qué tipo de array crear realmente en runtime; por eso Java prohíbe la creación directa de arrays de un tipo genérico parametrizado dentro de una clase genérica.

34. ¿Puede un parámetro de tipo genérico T usarse directamente como un tipo primitivo, por ejemplo declarando una clase 'class Caja<T>' e intentando instanciarla como 'Caja<int>'?

Respuesta correcta: A) No, los generics en Java solo funcionan con tipos referencia (clases, no primitivos); para usar un tipo similar a int se debe usar su wrapper correspondiente, como Caja<Integer>

Debido al type erasure y a cómo están implementados los generics en Java, los parámetros de tipo solo pueden ser tipos referencia; para 'usar' un tipo primitivo con generics es necesario emplear su clase wrapper correspondiente (Integer, Double, Boolean, etc.), aprovechando el autoboxing/unboxing.

35. ¿Por qué una variable static de una clase genérica no puede declararse usando el parámetro de tipo T de esa clase, por ejemplo 'static T valorCompartido;' dentro de 'class Caja<T>'?

Respuesta correcta: A) Porque T está asociado a cada instancia particular de la clase genérica (Caja<String>, Caja<Integer>, etc.), mientras que una variable static es compartida por todas las instancias sin distinción de su tipo parametrizado específico, generando una contradicción de diseño

Como el parámetro de tipo T se resuelve de forma independiente para cada instancia concreta de la clase genérica, y una variable static es única y compartida por todas las instancias (sin importar su tipo parametrizado específico), no tiene sentido asociar esa variable static al tipo T, por lo que el compilador lo prohíbe.

36. Debido al type erasure, ¿por qué no se pueden declarar en la misma clase dos métodos sobrecargados cuyas firmas solo difieran en sus parámetros de tipo genérico, por ejemplo 'void metodo(List<String> lista)' y 'void metodo(List<Integer> lista)'?

Respuesta correcta: A) Porque tras el borrado de tipos, ambas firmas se convierten en 'void metodo(List lista)' (usando el tipo raw List), siendo exactamente la misma firma de método, lo que el compilador rechaza por firma duplicada

Debido al type erasure, en tiempo de ejecución (y para efectos de la firma del método usada por el compilador para detectar sobrecargas), List<String> y List<Integer> se reducen ambos al tipo raw List; esto hace que ambas declaraciones tengan la misma firma efectiva, produciendo un error de compilación por métodos duplicados.

37. ¿Qué comportamiento de solo lectura tiene una lista declarada con wildcard no acotado, como 'List<?> lista = obtenerListaDesconocida();', respecto a añadir elementos?

Respuesta correcta: A) No se puede invocar add() con ningún tipo de elemento (excepto null), ya que el compilador no puede garantizar qué tipo específico acepta realmente la lista detrás del wildcard '?'; sin embargo, sí se pueden leer elementos tratándolos genéricamente como Object

Con un wildcard no acotado, el compilador no tiene información sobre qué tipo específico contiene realmente la lista, por lo que prohíbe añadir cualquier elemento (salvo null, que es válido para cualquier tipo referencia), permitiendo solo operaciones de lectura seguras tratando los elementos como Object.

38. ¿Por qué un método como 'static double sumar(List<? extends Number> lista)' en una clase utilitaria CollectionUtils puede aceptar tanto List<Integer> como List<Double>, mientras que un parámetro 'List<Number> lista' sin wildcard NO aceptaría directamente List<Integer>?

Respuesta correcta: A) El wildcard '? extends Number' acepta una lista de Number o de cualquier subtipo suyo (Integer, Double, etc.), aprovechando la flexibilidad de lectura que dan los wildcards acotados; en cambio, 'List<Number>' exige exactamente ese tipo exacto, ya que los generics son invariantes y List<Integer> no es subtipo de List<Number>

Debido a la invariancia de los generics, un parámetro declarado como List<Number> solo acepta exactamente List<Number>, no List<Integer> ni List<Double>; usar el wildcard '? extends Number' resuelve esto, permitiendo que el método acepte listas de Number o de cualquier subtipo suyo, ideal para métodos que solo leen (suman) los valores sin necesidad de añadir elementos.

39. En un método utilitario que solo necesita LEER y sumar los valores numéricos de una colección (sin añadir elementos), ¿por qué es preferible usar 'List<? extends Number>' en lugar de un tipo genérico como '<T extends Number> List<T>'?

Respuesta correcta: A) Ambos enfoques son técnicamente válidos, pero el wildcard '? extends Number' comunica más claramente que el método solo lee los datos sin necesitar un parámetro de tipo nombrado, siendo la opción preferida cuando no se necesita referenciar ese tipo en ningún otro lugar de la firma o cuerpo del método

Cuando el tipo genérico no necesita reutilizarse en otra parte de la firma del método (por ejemplo, como tipo de retorno o en otro parámetro relacionado), el wildcard acotado es la convención preferida por ser más simple y expresar directamente la intención de solo lectura; un parámetro de tipo nombrado (T) suele reservarse para cuando se necesita 'recordar' ese tipo específico en otro lugar del método.

40. Dada la jerarquía A (superclase), B extends A, y C extends B, ¿qué tipo de wildcard permitiría a un método aceptar una lista de B, o de cualquier subtipo de B (incluyendo C), pero no de A?

Respuesta correcta: A) List<? extends B>

El wildcard '? extends B' acepta B o cualquier subtipo de B (en este caso, C también calificaría, ya que C extends B), pero no aceptaría A, ya que A es supertipo de B, no subtipo.

41. ¿Qué advertencia o conflicto puede surgir al declarar un record genérico que además implementa Comparable, como 'record Par<T extends Comparable<T>>(T valor) implements Comparable<Par<T>>'?

Respuesta correcta: A) Es necesario asegurar consistencia entre el bounded type del parámetro T (que ya exige ser Comparable<T>) y la implementación de Comparable<Par<T>> del propio record, para que compareTo() pueda delegar correctamente en el compareTo() de T sin conflictos de tipos ni advertencias del compilador por uso de tipos raw o incompatibles

Al combinar un bounded type (T extends Comparable<T>) con una implementación propia de Comparable<Par<T>>, es importante mantener consistencia en las relaciones de tipos para que el compareTo() del record pueda delegar correctamente en el compareTo() de T, evitando advertencias o errores de tipos incompatibles.

42. ¿Por qué Deque es frecuentemente preferido sobre LinkedList 'pura' cuando se necesita explícitamente un comportamiento de pila (LIFO) o cola (FIFO) en el código?

Respuesta correcta: A) Porque Deque expone una interfaz con métodos específicamente pensados para operaciones en ambos extremos (push/pop para LIFO, offer/poll para FIFO), comunicando más claramente la intención del código, aunque LinkedList sea frecuentemente la implementación subyacente utilizada

Deque es una interfaz que expone métodos claramente orientados a operaciones en ambos extremos de la estructura (push/pop, offerFirst/offerLast, etc.), comunicando mejor la intención de uso como pila o cola; LinkedList es una de las implementaciones concretas más comunes de Deque, pero programar contra la interfaz Deque es preferible por claridad y flexibilidad.

14. Comparable y Comparator

1. ¿Cuál es la diferencia principal entre Comparable y Comparator?

Respuesta correcta: A) Comparable se implementa en la propia clase para definir su orden natural (compareTo); Comparator se define externamente y permite múltiples estrategias de ordenación (compare)

Comparable define el orden natural de un objeto mediante compareTo(); Comparator es una interfaz separada para definir órdenes alternativos mediante compare(a,b), útil para múltiples criterios.

2. ¿Qué valor devuelve compareTo() si el objeto actual es 'mayor' que el que se compara?

Respuesta correcta: A) Un valor positivo

Por convención: negativo si el objeto es menor, 0 si es igual, positivo si es mayor. No se garantiza un valor exacto, solo el signo.

3. ¿Cómo se encadenan múltiples criterios de comparación usando Comparator (por ejemplo, por apellido y luego por nombre)?

Respuesta correcta: A) Comparator.comparing(Persona::getApellido).thenComparing(Persona::getNombre)

El método thenComparing() permite encadenar criterios de desempate adicionales tras el comparador principal.

4. ¿Qué ocurre si dos objetos son 'iguales' según equals() pero compareTo() no devuelve 0 para ellos?

Respuesta correcta: A) Se dice que la clase tiene un orden 'inconsistente con equals', lo cual puede causar comportamientos inesperados en colecciones ordenadas como TreeSet o TreeMap

Se recomienda que compareTo() sea consistente con equals(); si no lo es, colecciones basadas en orden como TreeSet/TreeMap pueden comportarse de forma sorprendente (por ejemplo, tratar como iguales elementos que equals() distingue).

5. ¿Cómo se invierte el orden producido por un Comparator existente, por ejemplo para ordenar de mayor a menor?

Respuesta correcta: A) comparator.reversed()

El método por defecto reversed() de Comparator devuelve un nuevo Comparator con el orden invertido respecto al original.

6. ¿Qué método estático de Comparator crea un comparador a partir de una función que extrae una clave comparable de cada elemento?

Respuesta correcta: A) Comparator.comparing(funcionExtractor)

Comparator.comparing(Function keyExtractor) crea un Comparator que ordena según el valor Comparable devuelto por esa función aplicada a cada elemento.

7. ¿Por qué se dice que Comparable 'se queda corto' cuando se necesita ordenar objetos por múltiples criterios distintos?

Respuesta correcta: A) Porque Comparable define un único método compareTo() por clase, lo que fija un único 'orden natural'; para ordenar por un criterio diferente (o varios) sería necesario modificar esa única implementación cada vez, en lugar de tener múltiples estrategias de orden simultáneas

Como Comparable se implementa directamente en la clase y define un solo método compareTo(), solo puede representar un único 'orden natural' fijo; para necesitar varios criterios de ordenamiento distintos (por ejemplo, por edad y también por nombre) se requiere Comparator, que permite crear múltiples estrategias de comparación externas e independientes de la clase.

8. ¿Puede usarse Comparator para ordenar objetos de una clase que NO implementa Comparable?

Respuesta correcta: A) Sí, Comparator es completamente independiente de si la clase implementa Comparable o no, ya que la lógica de comparación se define externamente en el propio Comparator

Una de las principales ventajas de Comparator es que no depende de que la clase de los objetos a comparar implemente ninguna interfaz especial; la lógica de comparación reside completamente en el propio objeto Comparator, externo a la clase que se está ordenando.

9. De las tres formas de implementar un Comparator mostradas (clase separada, clase anónima, expresión lambda), ¿qué tienen en común en cuanto al método que finalmente se debe proveer?

Respuesta correcta: A) Las tres formas terminan proveyendo una implementación del método compare(T o1, T o2), ya sea mediante una clase completa, una clase anónima al vuelo, o el cuerpo conciso de una lambda

Comparator es una interfaz funcional cuyo único método abstracto es compare(T o1, T o2); ya sea mediante una clase Comparator completa, una clase anónima, o una lambda, en el fondo las tres formas proveen una implementación de ese mismo método compare().

10. ¿Por qué la versión con expresión lambda de un Comparator suele considerarse más concisa que la versión con clase anónima equivalente?

Respuesta correcta: A) Porque la lambda elimina la sintaxis repetitiva de declarar la clase anónima, el nombre del método y los tipos de los parámetros (que el compilador puede inferir), dejando solo la lógica esencial de comparación

Una lambda elimina el boilerplate de declarar 'new Comparator<Tipo>() { public int compare(...) {...} }', dejando solo los parámetros y la lógica del cuerpo, ya que el compilador infiere el tipo de la interfaz funcional (Comparator) y de los parámetros a partir del contexto.

11. ¿Qué método estático de Comparator es común usar como punto de partida al construir una lambda o method reference para ordenar por un atributo específico, por ejemplo la edad de una lista de empleados?

Respuesta correcta: A) Comparator.comparing(Empleado::getEdad)

Comparator.comparing(keyExtractor) es el método de fábrica más común para construir un Comparator a partir de una función (lambda o method reference) que extrae la clave de comparación de cada objeto, como la edad de un empleado.

12. ¿Por qué es relevante recordar el 'cache de Integer' (valores entre -128 y 127) al analizar el comportamiento de un HashSet<Integer> en ejercicios de examen?

Respuesta correcta: A) Porque dentro de ese rango, los objetos Integer autoboxeados con el mismo valor son literalmente el mismo objeto compartido (gracias al cache), lo cual puede afectar el análisis de '==' entre elementos, aunque el comportamiento de duplicados de un HashSet siempre depende de equals()/hashCode(), no de identidad de referencia

El cache de Integer (-128 a 127) hace que los valores autoboxeados dentro de ese rango compartan el mismo objeto en memoria; esto es relevante para preguntas de examen sobre '==' entre Integers, aunque para la lógica de duplicados de un HashSet lo que realmente importa es equals()/hashCode() de Integer, que sí compara correctamente el valor numérico sin importar el cache.

13. ¿Qué ocurre en tiempo de ejecución si se intenta añadir elementos a un TreeSet cuyos elementos NO implementan Comparable, sin proporcionar tampoco un Comparator explícito al crear el TreeSet?

Respuesta correcta: A) Se lanza ClassCastException al intentar insertar el primer par de elementos que deban compararse entre sí, ya que el TreeSet no tiene ninguna forma de determinar su orden

Un TreeSet necesita determinar el orden de sus elementos, ya sea mediante el compareTo() de Comparable o mediante un Comparator explícito; si ninguno de los dos está disponible, se lanza ClassCastException en tiempo de ejecución al intentar comparar los elementos durante la inserción.

14. ¿Qué diferencia hay entre los métodos add()/remove() y offer()/poll() de la interfaz Deque en cuanto al manejo de casos límite (como una estructura llena o vacía)?

Respuesta correcta: A) add() y remove() lanzan una excepción si la operación no puede completarse (por ejemplo, remove() en una estructura vacía); offer() y poll() manejan esos mismos casos de forma más segura, devolviendo false o null respectivamente en lugar de lanzar una excepción

Deque ofrece dos familias de métodos: una que lanza excepciones ante condiciones límite (add, remove, element), y otra más 'segura' que devuelve un valor especial (false o null) en lugar de lanzar excepción (offer, poll, peek), útil cuando se prefiere manejar el caso límite sin try-catch.

15. ¿Qué devuelven los métodos `keySet()` y `entrySet()` de la interfaz `Map`?

Respuesta correcta: A) `keySet()` devuelve una vista de tipo `Set` con todas las claves del mapa; `entrySet()` devuelve una vista de tipo `Set` con todos los pares clave-valor (`Map.Entry`) del mapa, permitiendo iterar sobre ambos a la vez

`keySet()` proporciona una vista de las claves del mapa como un `Set` (sin duplicados, ya que las claves de un `Map` son únicas); `entrySet()` proporciona una vista de todos los pares clave-valor como objetos `Map.Entry`, siendo la forma más eficiente de iterar sobre ambos simultáneamente.

16. ¿Cómo se relaciona la interfaz `BiConsumer<K,V>` con el método `forEach()` de un `Map`, como en `'map.forEach((clave, valor) -> System.out.println(clave + "=" + valor));'`?

Respuesta correcta: A) El método `forEach()` de `Map` espera precisamente un `BiConsumer<K,V>`, ya que necesita procesar dos valores por cada entrada (la clave y el valor) sin producir ningún resultado de retorno, lo cual coincide exactamente con el contrato `accept(K,V)` de `BiConsumer`

`Map.forEach(BiConsumer<? super K, ? super V> action)` recorre cada entrada del mapa invocando el `BiConsumer` proporcionado con la clave y el valor de esa entrada como los dos argumentos, sin esperar ningún retorno, coincidiendo exactamente con el contrato `accept(K,V)` de `BiConsumer`.

17. ¿Qué hace el método `getOrDefault(clave, valorPorDefecto)` de la interfaz `Map`?

Respuesta correcta: A) Devuelve el valor asociado a la clave si existe en el mapa; si la clave no está presente, devuelve el `valorPorDefecto` proporcionado sin lanzar ninguna excepción ni modificar el mapa

`getOrDefault()` es una forma segura de consultar un mapa: devuelve el valor real si la clave existe, o el valor por defecto indicado si no existe, evitando tener que comprobar manualmente `containsKey()` antes de acceder con `get()`.

18. ¿Qué hace el método `merge(clave, valor, funcionRemapeo)` de la interfaz `Map` cuando la clave YA existe en el mapa?

Respuesta correcta: A) Aplica la función de remapeo (`BiFunction`) al valor existente y al nuevo valor proporcionado, y almacena el resultado como el nuevo valor asociado a esa clave (o elimina la entrada si la función devuelve `null`)

`merge()` combina el valor existente y el nuevo valor usando la función de remapeo proporcionada (`BiFunction`), y almacena el resultado; es especialmente útil para acumular valores, como sumar contadores asociados a una clave que ya existe.

19. ¿Por qué es importante validar contra `null` dentro de la implementación de `compareTo()` al implementar `Comparable` en una clase propia?

Respuesta correcta: A) Porque si `compareTo()` recibe un argumento `null` y no lo maneja explícitamente, se lanzaría `NullPointerException` al intentar comparar sus atributos internos contra los de un objeto inexistente; el contrato de `Comparable` espera que se lance `NullPointerException` de forma consistente, no un comportamiento indefinido

El contrato de `compareTo()` establece que debe lanzar `NullPointerException` si se compara contra `null`; sin una validación (o manejo) explícita, el código puede fallar de forma menos clara al intentar acceder a atributos de un objeto `null`, en lugar de fallar de manera consistente y esperada según el contrato de la interfaz.

20. ¿Cómo se encadenan varios criterios de ordenamiento con `Comparator` para ordenar records de `Empleado`, primero por departamento y luego por salario, en orden descendente para el salario?

Respuesta correcta: A) `Comparator.comparing(Empleado::departamento).thenComparing(Comparator.comparing(Empleado::salario).reversed())`

`thenComparing()` permite encadenar un criterio de desempate adicional; combinándolo con `reversed()` sobre el segundo comparador (el de salario), se logra ordenar primero por departamento (ascendente) y luego por salario en orden descendente como criterio de desempate.

21. ¿Por qué Arrays.binarySearch() (o Collections.binarySearch()) requiere que la colección o array esté ordenado en orden ascendente antes de aplicarse?

Respuesta correcta: A) Porque el algoritmo de búsqueda binaria divide repetidamente el rango de búsqueda a la mitad, descartando la mitad donde el elemento buscado no puede estar, según comparaciones que solo son válidas si los elementos están ordenados de forma ascendente y consistente

La búsqueda binaria depende fundamentalmente de que los elementos estén ordenados (de forma ascendente según el orden natural o el Comparator usado), ya que en cada paso descarta la mitad del rango basándose en la comparación con el elemento del medio; sobre datos desordenados, el resultado es indefinido/incorrecto aunque el método no lo detecte.

15. Lambdas e Interfaces Funcionales

1. ¿Cuál es la sintaxis correcta de una expresión lambda que recibe dos enteros y devuelve su suma?

Respuesta correcta: A) (a, b) -> a + b

La sintaxis de lambdas en Java es (parámetros) -> expresión_o_bloque. No se usa '=>' como en JavaScript.

2. ¿Qué interfaz funcional del paquete java.util.function representa una operación que recibe un argumento y devuelve un boolean?

Respuesta correcta: A) Predicate<T>

Predicate<T> tiene el método test(T t) que devuelve boolean, típicamente usado para filtrar (Stream.filter).

3. ¿Qué interfaz funcional representa una operación que NO recibe argumentos pero produce un resultado?

Respuesta correcta: A) Supplier<T>

Supplier<T> tiene el método get() sin parámetros que produce/devuelve un valor de tipo T, útil para provisión perezosa de valores.

4. ¿Puede una expresión lambda acceder y modificar una variable local externa capturada?

Respuesta correcta: A) No, solo puede leer variables locales externas que sean 'effectively final' (efectivamente finales); no puede reasignarlas

Las lambdas solo pueden capturar variables locales 'effectively final' (que no cambian tras su inicialización); intentar reasignarlas produce error de compilación.

5. ¿Qué interfaz funcional recibe un argumento y no devuelve ningún resultado (void)?

Respuesta correcta: A) Consumer<T>

Consumer<T> define el método accept(T t) que recibe un argumento y no retorna nada, útil para operaciones como forEach.

6. ¿Qué interfaz funcional recibe un argumento de tipo T y produce un resultado de tipo R distinto?

Respuesta correcta: A) Function<T,R>

Function<T,R> define apply(T t) devolviendo R, permitiendo transformar un tipo en otro. UnaryOperator<T> es un caso especial donde entrada y salida son del mismo tipo T.

7. ¿Puede una expresión lambda lanzar una excepción checked si la interfaz funcional que implementa no la declara en su método abstracto?

Respuesta correcta: A) No, si el método abstracto de la interfaz funcional no declara esa excepción checked, la lambda no puede lanzarla (no compila)

El cuerpo de una lambda debe respetar la firma (incluidas las excepciones checked declaradas) del método abstracto de la interfaz funcional que implementa; si el método no declara throws, la lambda no puede lanzar una checked exception.

8. ¿Qué requisito estructural debe cumplir una interfaz para poder ser implementada mediante una expresión lambda?

Respuesta correcta: A) Debe ser una interfaz funcional: declarar exactamente un único método abstracto (además de posibles métodos default o static)

Solo las interfaces funcionales (con exactamente un método abstracto) pueden implementarse mediante una expresión lambda, ya que el compilador necesita saber sin ambigüedad qué método está siendo implementado por el cuerpo de la lambda.

9. ¿Qué hace la interfaz funcional Predicate<T> y cuál es su método abstracto principal?

Respuesta correcta: A) Representa una condición evaluada sobre un objeto de tipo T, mediante el método test(T t), que devuelve true o false

Predicate<T> define el método test(T t) que evalúa una condición sobre un objeto de tipo T y devuelve un boolean, siendo ideal para operaciones de filtrado, como filtrar empleados por edad, salario o longitud del nombre.

10. Al filtrar un array de empleados usando distintos Predicate (por edad, por salario, por longitud de nombre), ¿qué ventaja aporta usar Predicate en lugar de escribir un bucle con if distinto para cada criterio?

Respuesta correcta: A) Permite encapsular cada criterio de filtrado como un objeto independiente y reutilizable, que puede combinarse o intercambiarse fácilmente sin duplicar la lógica del bucle de recorrido

Usar Predicate permite separar la lógica de 'qué se está evaluando' (el criterio) de 'cómo se recorre la colección', encapsulando cada condición como un objeto reutilizable e intercambiable, en lugar de escribir un bucle con lógica de filtrado duplicada para cada criterio distinto.

11. ¿Cómo se declara una interfaz funcional personalizada que use generics, por ejemplo una que reciba un tipo T y devuelva un tipo R?

Respuesta correcta: A) Declarando una interfaz con la anotación opcional @FunctionalInterface, un único método abstracto que use los parámetros de tipo genérico declarados en la propia interfaz (por ejemplo, interface Transformador<T, R> { R transformar(T entrada); })

Una interfaz funcional personalizada puede declarar sus propios parámetros de tipo genérico, como interface Transformador<T, R> { R transformar(T entrada); }, permitiendo crear implementaciones (mediante lambdas) reutilizables para distintos tipos concretos, similar a como funcionan Function<T,R> o Predicate<T> del paquete estándar.

12. ¿Qué paquete de la biblioteca estándar de Java provee un conjunto de interfaces funcionales predefinidas y listas para usar, como Predicate, Function, Supplier y Consumer?

Respuesta correcta: A) java.util.function

El paquete java.util.function contiene las interfaces funcionales genéricas más comunes de la biblioteca estándar (Predicate, Function, Supplier, Consumer, BiFunction, UnaryOperator, entre otras), evitando tener que declarar interfaces funcionales personalizadas para casos de uso habituales.

13. ¿El hecho de que una interfaz funcional contenga métodos static y default rompe la regla de que debe tener 'un único método abstracto'?

Respuesta correcta: A) No, la regla de 'un único método abstracto' solo aplica a los métodos SIN cuerpo (abstractos); los métodos static y default sí tienen implementación propia y no cuentan para ese límite, por lo que una interfaz funcional puede tener varios de ellos además de su único método abstracto

La restricción de 'interfaz funcional' se refiere exclusivamente a la cantidad de métodos abstractos (sin cuerpo): debe haber exactamente uno. Los métodos static y default tienen implementación propia y pueden ser tantos como se necesite, sin afectar la naturaleza funcional de la interfaz.

14. ¿Cómo se invoca un método static declarado dentro de una interfaz funcional, por ejemplo un método 'static Predicate<T> siempreVerdadero()' dentro de Predicate?

Respuesta correcta: A) Se invoca directamente sobre el nombre de la interfaz, como 'Predicate.siempreVerdadero()', sin necesidad de tener una instancia de la interfaz

Los métodos static de una interfaz pertenecen a la interfaz misma (no a las instancias), por lo que se invocan calificando con el nombre de la interfaz, igual que un método static de una clase normal.

15. ¿Por qué un método default de una interfaz funcional Sí requiere una instancia (una lambda o implementación concreta) para poder invocarse, a diferencia de un método static?

Respuesta correcta: A) Porque un método default forma parte del contrato de instancia de la interfaz, y su implementación suele operar sobre 'this' (la instancia actual), por lo que necesita un objeto concreto sobre el cual ejecutarse

Un método default es, en esencia, un método de instancia con implementación por defecto; puede referirse implícitamente a 'this' (por ejemplo, para combinarse con el propio predicado en `negate()`, `and()` u `or()`), por lo que necesita invocarse sobre una instancia concreta de la interfaz (como una lambda ya asignada a una variable).

16. ¿Qué hace el método default `negate()` de la interfaz `Predicate<T>`?

Respuesta correcta: A) Devuelve un nuevo `Predicate<T>` que representa la negación lógica del predicado original: donde el original devolvía true, el nuevo devuelve false, y viceversa

`negate()` es un método default de `Predicate` que, sin modificar el predicado original, devuelve un nuevo `Predicate` cuyo resultado es el opuesto lógico (la negación) del predicado sobre el que se invoca.

17. ¿Qué hacen los métodos default `and()` y `or()` de `Predicate<T>`, por ejemplo '`predicado1.and(predicado2)`'?

Respuesta correcta: A) Combinan dos predicados en uno nuevo: `and()` devuelve true solo si ambos predicados originales son true (equivalente lógico a `&&`); `or()` devuelve true si al menos uno de los dos es true (equivalente lógico a `||`)

`and()` y `or()` son métodos default que combinan lógicamente dos predicados en uno nuevo, replicando el comportamiento de los operadores `&&` y `||` respectivamente, sin modificar los predicados originales.

18. ¿Qué se entiende por 'función de orden superior' (higher-order function) en el contexto de un método que devuelve una expresión lambda?

Respuesta correcta: A) Es un método que trata las funciones (lambdas o referencias a método) como valores de primera clase: puede recibirlas como parámetros y/o devolverlas como resultado, en lugar de solo trabajar con datos simples

Una función de orden superior es aquella que puede recibir funciones como argumentos o devolver una función (lambda) como resultado; en Java esto se logra mediante interfaces funcionales, permitiendo que un método devuelva, por ejemplo, un nuevo `Predicate` o `Function` construido dinámicamente.

19. ¿Cuál es el contrato (entrada y salida) de la interfaz funcional `Supplier<T>`?

Respuesta correcta: A) No recibe ningún argumento y devuelve un valor de tipo T, mediante el método `get()`

`Supplier<T>` 'suministra' o produce un valor sin necesitar ningún argumento de entrada, a través del método `get()`; es útil para proveer valores de forma perezosa (lazy) o para fábricas de objetos.

20. ¿Cuál es la diferencia de contrato entre `Consumer<T>` y `BiConsumer<T,U>`?

Respuesta correcta: A) `Consumer<T>` recibe un único argumento de tipo T y no devuelve nada (`accept(T t)`); `BiConsumer<T,U>` recibe dos argumentos de tipos T y U distintos y tampoco devuelve nada (`accept(T t, U u)`)

`Consumer<T>` opera sobre un solo argumento sin retorno; `BiConsumer<T,U>` extiende ese concepto para operar sobre dos argumentos de tipos potencialmente distintos, también sin retorno, útil por ejemplo para iterar sobre pares clave-valor de un `Map`.

21. ¿Qué relación de contrato tiene `UnaryOperator<T>` respecto a `Function<T,R>`?

Respuesta correcta: A) `UnaryOperator<T>` es un caso especial de `Function<T,R>` donde el tipo de entrada y el tipo de salida son exactamente el mismo (T), es decir, `Function<T,T>`

`UnaryOperator<T>` extiende conceptualmente `Function<T,R>` fijando R igual a T; es decir, representa una operación que transforma un valor de tipo T en otro valor del mismo tipo T, como duplicar un número o convertir un String a mayúsculas.

22. ¿Qué relación de contrato tiene BinaryOperator<T> respecto a BiFunction<T,U,R>?

Respuesta correcta: A) BinaryOperator<T> es un caso especial de BiFunction donde ambos argumentos de entrada y el resultado son del mismo tipo T (equivalente a BiFunction<T,T,T>)

BinaryOperator<T> es un caso especial de BiFunction<T,U,R> donde los dos argumentos de entrada y el resultado son todos del mismo tipo T, útil por ejemplo para operaciones como sumar dos números del mismo tipo o combinar dos Strings.

23. ¿Por qué existen versiones 'primitivas' especializadas de las interfaces funcionales, como IntPredicate, IntFunction<R> o ToIntFunction<T>, en lugar de usar siempre las versiones genéricas con Integer?

Respuesta correcta: A) Para evitar el costo de autoboxing/unboxing entre int e Integer en operaciones frecuentes, mejorando el rendimiento al trabajar directamente con el tipo primitivo en lugar de su wrapper

Las interfaces funcionales especializadas para tipos primitivos (como IntPredicate, IntFunction, ToIntFunction, entre otras) evitan el costo adicional del autoboxing/unboxing que ocurriría al usar la versión genérica con el wrapper (como Predicate<Integer>), mejorando el rendimiento en operaciones frecuentes sobre datos primitivos.

24. ¿Qué método de la interfaz List permite eliminar todos los elementos que cumplan una condición dada, usando directamente un Predicate?

Respuesta correcta: A) removeIf(Predicate<? super E> filtro)

El método removeIf(Predicate) de la interfaz Collection (heredado por List) elimina de la colección todos los elementos para los cuales el Predicate dado devuelve true, sin necesidad de un bucle explícito con Iterator.

25. ¿Qué método de la interfaz List permite reemplazar cada elemento aplicando una función de transformación (UnaryOperator), modificando la lista en el mismo lugar?

Respuesta correcta: A) replaceAll(UnaryOperator<E> operador)

replaceAll(UnaryOperator<E>) recorre la lista y reemplaza cada elemento por el resultado de aplicarle la función dada, modificando la lista existente in situ en lugar de crear una nueva colección.

26. ¿Qué contrato define la interfaz funcional BooleanSupplier, y en qué se diferencia de Supplier<Boolean>?

Respuesta correcta: A) BooleanSupplier define el métodogetAsBoolean(), que no recibe argumentos y devuelve un boolean primitivo, evitando el autoboxing/unboxing que ocurriría al usar Supplier<Boolean> (que devuelve un objeto Boolean)

BooleanSupplier es la versión especializada para primitivos de Supplier, con el métodogetAsBoolean() que devuelve directamente un boolean primitivo, evitando el costo de autoboxing que tendría Supplier<Boolean> al trabajar con el objeto wrapper Boolean.

27. ¿Qué significa que una variable local sea 'effectively final' (efectivamente final) para poder ser capturada dentro de una lambda?

Respuesta correcta: A) Significa que la variable, aunque no se haya declarado explícitamente con la palabra 'final', nunca es reasignada después de su inicialización a lo largo de todo su alcance; si se reasignara en algún punto, dejaría de ser effectively final y la lambda no podría capturarla

Una variable 'effectively final' es aquella que, aunque no lleve la palabra clave final explícita, se comporta como si lo fuera porque nunca se reasigna tras su inicialización; esta es la condición que Java exige para que una variable local externa pueda ser capturada (leída) dentro de una lambda.

28. ¿Por qué una lambda puede leer libremente un campo de instancia de la clase contenedora, sin la restricción de 'effectively final' que sí aplica a las variables locales?

Respuesta correcta: A) Porque los campos de instancia no se 'capturan por valor' como las variables locales; la lambda accede a ellos a través de una referencia implícita a 'this' (el objeto contenedor), por lo que ve siempre el valor actual del campo, incluso si cambia después

Las variables locales se capturan 'por valor' (una copia fija en el momento de creación de la lambda), por eso deben ser effectively final; los campos de instancia, en cambio, se acceden a través de una referencia implícita al objeto contenedor ('this'), por lo que la lambda ve siempre su valor actualizado, sin restricción de effectively final.

29. ¿Qué conflicto de nombres puede surgir si un parámetro de una lambda tiene el mismo nombre que una variable local ya existente en el método contenedor, por ejemplo '(x) -> x + 1' cuando ya existe una variable local llamada 'x' en ese método?

Respuesta correcta: A) Error de compilación por conflicto de nombres ('variable already defined'), ya que el parámetro de la lambda no puede reutilizar el nombre de una variable local ya visible en ese mismo ámbito

A diferencia de otros lenguajes que permiten 'sombra' (shadowing) de variables, Java prohíbe que el parámetro de una lambda tenga el mismo nombre que una variable local ya visible en el ámbito que la contiene, generando un error de compilación por redefinición de variable.

30. ¿En qué se diferencia el alcance (scope) de una variable local declarada dentro del cuerpo de una lambda, respecto a una variable de instancia de la clase contenedora?

Respuesta correcta: A) La variable local dentro de la lambda solo es visible dentro del cuerpo de esa lambda específica; la variable de instancia es visible en toda la clase (según su modificador de acceso) y persiste mientras exista el objeto, no solo durante la ejecución de la lambda

Una variable declarada dentro del cuerpo de una lambda tiene un alcance limitado a esa lambda (similar a cualquier bloque de código), mientras que un campo de instancia pertenece al objeto y es accesible según su modificador de acceso durante toda la vida del objeto, no solo durante la ejecución puntual de la lambda.

31. ¿Qué requisito de tipos debe cumplirse para poder combinar dos Predicate con el método default or(), como en 'predicado1.or(predicado2)'?

Respuesta correcta: A) Ambos predicados deben operar sobre el mismo tipo genérico T (o tipos compatibles), ya que or() combina ambos en un nuevo Predicate<T> que evalúa la condición sobre el mismo tipo de argumento

Como or() es un método default de Predicate<T> que retorna otro Predicate<T> del mismo tipo genérico, ambos predicados combinados deben operar sobre el mismo tipo T (o uno compatible), de lo contrario el compilador no podrá unificar los tipos genéricos y producirá un error de compilación.

32. ¿Cuál es la diferencia de orden de ejecución entre los métodos default compose() y andThen() de la interfaz Function?

Respuesta correcta: A) f.compose(g) ejecuta primero g y luego aplica f al resultado (equivalente a f(g(x))); f.andThen(g) ejecuta primero f y luego aplica g al resultado (equivalente a g(f(x)))

f.compose(g) primero aplica la función g al argumento de entrada, y luego aplica f al resultado de g (es decir, f(g(x))); en cambio, f.andThen(g) aplica primero f y luego g al resultado de f (g(f(x))), invirtiendo el orden de ejecución respecto a compose().

33. Dadas dos funciones UnaryOperator<Integer> duplicar = x -> x * 2; y sumarUno = x -> x + 1;, ¿qué resultado produce 'duplicar.andThen(sumarUno).apply(3)'?

Respuesta correcta: A) 7 (primero se duplica 3 obteniendo 6, y luego se le suma 1, obteniendo 7)

andThen() ejecuta primero la función sobre la que se invoca (duplicar), y luego aplica la función pasada como argumento (sumarUno) al resultado: duplicar(3)=6, y luego sumarUno(6)=7.

34. ¿Puede usarse 'var' como nombre de un parámetro de una lambda, por ejemplo '(var x, var y) -> x + y'?

Respuesta correcta: A) Sí, se puede usar la palabra 'var' para inferir el tipo de los parámetros de una lambda desde Java 11, pero si se usa 'var' para un parámetro, debe usarse consistentemente para TODOS los parámetros de esa lambda, no se puede mezclar 'var' con tipos explícitos en la misma lista de parámetros

Desde Java 11, se permite usar 'var' para los parámetros de una lambda (por ejemplo, para poder añadir anotaciones), pero existe la regla de consistencia: si se usa 'var' para un parámetro, todos los demás parámetros de esa misma lambda deben usar también 'var', no pudiendo mezclarse con tipos explícitos o con la sintaxis sin tipo.

35. ¿Qué ocurre si una lambda declara un parámetro llamado 'var' usado como identificador simple, por ejemplo intentando escribir '(var) -> var + 1' donde 'var' es literalmente el nombre del parámetro?

Respuesta correcta: A) No es válido: 'var' no puede usarse como nombre de identificador para un parámetro de lambda, ya que 'var' es una palabra reservada contextual con un significado especial en ese contexto, no un identificador libre

Aunque 'var' no es técnicamente una palabra reservada en todos los contextos (se le llama 'reserved type name'), en el contexto de declaración de parámetros de lambda no puede usarse como el nombre de un parámetro, ya que ahí Java lo interpreta como el uso especial para inferencia de tipo, generando un conflicto/error de compilación si se intenta usar como simple identificador.

16. Referencias a Métodos (Method References)

1. ¿Cuál de estas es la sintaxis de una referencia a método estático?

Respuesta correcta: A) Clase::metodoEstatico

Referencia a método estático: Clase::metodo. Ejemplo: Integer.parseInt equivale a s -> Integer.parseInt(s).

2. ¿Qué representa la referencia a método String::toUpperCase usada como Function<String,String>?

Respuesta correcta: A) Una referencia a un método de instancia sobre un tipo arbitrario, equivalente a s -> s.toUpperCase()

Cuando se usa Clase::metodoDeInstancia sin especificar objeto, el primer parámetro de la interfaz funcional se convierte en el receptor de la llamada: equivale a s -> s.toUpperCase().

3. ¿Cómo se escribe una referencia a constructor para crear instancias de ArrayList mediante un Supplier?

Respuesta correcta: A) ArrayList::new

Clase::new es la sintaxis de referencia a constructor, usable como Supplier<ArrayList> u otra interfaz funcional compatible con la firma del constructor.

4. ¿Cuál es la sintaxis de una referencia a un método de instancia sobre un objeto particular ya existente, por ejemplo 'obj::metodo'?

Respuesta correcta: A) objetoExistente::metodoDeInstancia — el objeto ya está fijado como receptor de la llamada

Cuando se usa una referencia sobre un objeto ya creado (objeto::metodo), ese objeto queda fijado como receptor, y los parámetros de la interfaz funcional corresponden únicamente a los argumentos del método.

5. ¿A qué expresión lambda equivale la referencia de método Math::max usada como BinaryOperator<Integer>?

Respuesta correcta: A) (a, b) -> Math.max(a, b)

Math::max sobre una interfaz que recibe dos argumentos y devuelve un resultado equivale a (a, b) -> Math.max(a, b), ya que max es un método estático de dos parámetros.

6. ¿Cuáles son los cuatro tipos principales de referencias a métodos en Java?

Respuesta correcta: A) Referencia a método estático, referencia a método de instancia sobre un objeto particular ya existente, referencia a método de instancia sobre un objeto arbitrario de una clase, y referencia a constructor

Los cuatro tipos clásicos de referencias a métodos son: (1) Clase::metodoEstatico, (2) objeto::metodoDeInstancia (sobre un objeto ya existente), (3) Clase::metodoDeInstancia (sobre un objeto arbitrario que se pasa como primer parámetro), y (4) Clase::new (referencia a constructor).

7. ¿Por qué se dice que 'el contexto' (la interfaz funcional de destino) es fundamental para interpretar correctamente una referencia a método?

Respuesta correcta: A) Porque una misma sintaxis de referencia a método (por ejemplo, Clase::metodo) puede interpretarse de formas distintas según los parámetros y el tipo de retorno que exija la interfaz funcional a la que se está asignando, especialmente al distinguir entre una referencia a método estático y una referencia sobre un objeto arbitrario

El compilador interpreta una referencia a método comparando su firma contra el método abstracto de la interfaz funcional de destino (el 'target type'); esto es especialmente relevante cuando existe ambigüedad entre una referencia a método estático y una sobre un 'objeto arbitrario de una clase', ya que la sintaxis puede lucir similar pero el significado depende del contexto funcional esperado.

8. Dado 'Function<String, Integer> f = String::length;', ¿a qué tipo de referencia a método corresponde esta expresión?

Respuesta correcta: A) Referencia a un método de instancia sobre un objeto arbitrario de una clase: el primer parámetro de la interfaz funcional (el String de entrada) se convierte en el objeto sobre el cual se invoca el método length()

Como length() no es estático y no se especifica un objeto concreto antes de '::', esta es una referencia sobre un 'objeto arbitrario de una clase': el compilador toma el primer parámetro que recibe la interfaz funcional (un String) y lo usa como el objeto receptor de la llamada a length().

9. ¿Cómo afecta la sobrecarga de métodos (overloading) a la resolución de una referencia a método, por ejemplo 'Clase::metodo' cuando existen varias versiones sobrecargadas de 'metodo'?

Respuesta correcta: A) El compilador determina cuál versión sobrecargada usar comparando la firma de cada una contra el método abstracto de la interfaz funcional de destino, eligiendo (si existe) la que coincida en cantidad y tipos de parámetros y en el tipo de retorno

Cuando un método referenciado tiene varias sobrecargas, el compilador usa el contexto (la interfaz funcional de destino) para determinar cuál sobrecarga específica es compatible en número y tipo de parámetros, de forma análoga a como resuelve la sobrecarga en una llamada directa a método.

10. ¿Qué firma debe tener la interfaz funcional de destino para poder usarse con una referencia a constructor como 'ArrayList::new'?

Respuesta correcta: A) Debe tener un método abstracto cuyos parámetros (en tipo y cantidad) coincidan con alguno de los constructores disponibles de la clase, y cuyo tipo de retorno sea compatible con el tipo de la clase (o un supertipo de ella)

Una referencia a constructor debe ser compatible con el método abstracto de la interfaz funcional de destino: los parámetros deben coincidir con alguno de los constructores disponibles de la clase, y el tipo de retorno del método abstracto debe ser el tipo de la clase (o un supertipo), similar a cómo Supplier<ArrayList<String>> encaja con ArrayList::new si el constructor no recibe argumentos.

11. ¿Cómo se usaría una referencia a método de instancia sobre un objeto particular ya existente junto con la interfaz Consumer<T>, por ejemplo para imprimir cada elemento de una lista con System.out::println?

Respuesta correcta: A) 'System.out' es el objeto particular ya existente, y println es su método de instancia; la referencia System.out::println es compatible con Consumer<T> porque recibe un argumento (el objeto a imprimir) y no devuelve nada, igual que el método accept(T t) de Consumer

System.out es un objeto PrintStream ya existente; println(Object) es uno de sus métodos de instancia sobrecargados. Como println recibe un argumento y no retorna nada, encaja perfectamente con el contrato accept(T t) de Consumer<T>, permitiendo usar System.out::println directamente como implementación de un Consumer.

12. ¿Cómo encajaría una referencia a método estático de dos parámetros, como 'Integer::sum', con la interfaz BiFunction<Integer, Integer, Integer>?

Respuesta correcta: A) Integer.sum(int a, int b) es un método estático que recibe dos enteros y devuelve su suma; esto coincide exactamente con el contrato de BiFunction<Integer, Integer, Integer>, que recibe dos argumentos de tipo Integer y devuelve un Integer mediante apply(T t, U u)

Integer.sum(int, int) es un método estático con dos parámetros que retorna su suma; al coincidir en cantidad de parámetros y tipo de retorno con el método abstracto apply(T,U) de BiFunction<Integer,Integer,Integer>, la referencia Integer::sum puede usarse directamente para implementar esa interfaz funcional.

13. ¿Cómo se usaría una referencia a método de instancia sobre un objeto arbitrario de una clase junto con la interfaz BiConsumer<T,U>, por ejemplo con 'List::add'?

Respuesta correcta: A) El primer parámetro de BiConsumer (de tipo T, la lista) se convierte en el objeto receptor sobre el cual se invoca add(), y el segundo parámetro (de tipo U) se pasa como argumento del método add(elemento); es decir, List::add equivale a (lista, elemento) -> lista.add(elemento)

Cuando se usa Clase::metodoDeInstancia con una interfaz que recibe dos parámetros (como BiConsumer<T,U>), el primer parámetro se interpreta como el objeto receptor (sobre el que se invoca el método) y el segundo como el argumento de ese método; List::add encaja así con BiConsumer<List<E>, E>, equivalente a (lista, elemento) -> lista.add(elemento).

14. ¿Qué ventaja práctica ofrecen las referencias a métodos frente a escribir la lambda equivalente de forma explícita, cuando el cuerpo de la lambda simplemente invoca un método ya existente?

Respuesta correcta: A) Hacen el código más legible y conciso al eliminar la sintaxis redundante de los parámetros y el cuerpo de la lambda, dejando claro de un vistazo qué método existente se está reutilizando como implementación de la interfaz funcional

Cuando el cuerpo de una lambda simplemente delega en un método ya existente (como s -> s.toUpperCase() o (a,b) -> Integer.sum(a,b)), la referencia a método equivalente (String::toUpperCase, Integer::sum) es más concisa y comunica más claramente la intención de reutilizar ese método concreto, mejorando la legibilidad.

17. Streams API

1. ¿Qué caracteriza a los streams en Java respecto a las colecciones que los originan?

Respuesta correcta: A) Un stream no almacena datos propios; procesa datos de una fuente de forma perezosa (lazy) y puede consumirse una sola vez

Los streams no almacenan elementos; describen operaciones sobre una fuente de datos, se evalúan de forma perezosa y son de un solo uso.

2. ¿Cuál de las siguientes es una operación TERMINAL de Stream (no intermedia)?

Respuesta correcta: A) collect()

collect(), forEach(), count(), reduce() son operaciones terminales que disparan el procesamiento. filter, map, sorted, distinct son intermedias y perezosas.

3. ¿Qué hace el siguiente código?

```
List<String> nombres = List.of("Ana","Luis","Eva");
nombres.stream().filter(n -> n.length()>3).forEach(System.out::println);
```

Respuesta correcta: A) Imprime solo "Luis" porque es el único nombre con más de 3 caracteres

filter conserva solo elementos cuyo length() sea mayor a 3. 'Ana'(3) y 'Eva'(3) no cumplen; solo 'Luis'(4) pasa el filtro.

4. ¿Qué clase se usa típicamente para acumular resultados de un Stream en una colección mediante collect(), como Collectors.toList()?

Respuesta correcta: A) java.util.stream.Collectors

La clase utilitaria Collectors provee fábricas de Collector como toList(), toSet(), toMap(), joining(), groupingBy(), etc.

5. ¿Qué hace `Collectors.groupingBy()`?

Respuesta correcta: A) Agrupa los elementos de un stream en un Map según una función clasificadora, devolviendo un `Map<clave, List<elementos>>`

`groupingBy(funcionClasificadora)` agrupa los elementos según el resultado de esa función, generando un Map donde cada clave mapea a una lista (u otro collector downstream) de elementos.

6. ¿Qué devuelve la operación terminal `reduce()` en un Stream?

Respuesta correcta: A) Combina los elementos del stream en un único resultado acumulado, aplicando repetidamente una función de combinación binaria

`reduce()` combina todos los elementos usando una función asociativa (por ejemplo, sumar), produciendo un único valor acumulado, normalmente envuelto en `Optional` si no se especifica un valor identidad.

7. ¿Cuál es la diferencia entre un stream secuencial y uno paralelo (`parallelStream()`)?

Respuesta correcta: A) El stream paralelo divide el procesamiento entre múltiples hilos usando el `ForkJoinPool` común, mientras que el secuencial procesa los elementos uno a uno en un solo hilo

`parallelStream()` (o `stream().parallel()`) reparte el trabajo entre varios hilos del `ForkJoinPool` común, lo que puede acelerar operaciones costosas en datasets grandes, pero introduce consideraciones de orden y efectos secundarios que no aplican al stream secuencial.

8. ¿Qué método de Stream permite limitar la cantidad de elementos procesados, por ejemplo a los primeros 5?

Respuesta correcta: A) `limit(5)`

`limit(n)` es una operación intermedia que trunca el stream para que contenga como máximo n elementos.

9. ¿Qué método permite obtener un `IntStream` de números consecutivos, por ejemplo del 1 al 10?

Respuesta correcta: A) `IntStream.rangeClosed(1, 10)`

`IntStream.rangeClosed(1, 10)` genera un stream de enteros del 1 al 10 inclusive; `IntStream.range(1, 10)` generaría del 1 al 9 (excluyendo el límite superior).

10. ¿Por qué se dice que un Stream 'envuelve' una fuente de datos (como una colección) en lugar de ser una estructura de datos en sí misma?

Respuesta correcta: A) Porque un Stream no almacena elementos propios; describe una secuencia de operaciones (filter, map, etc.) a aplicar sobre los datos de la fuente original, calculando resultados de forma perezosa solo cuando se invoca una operación terminal

Un Stream no es una estructura de almacenamiento; es una abstracción que describe un pipeline de operaciones sobre una fuente de datos existente, ejecutándose de forma perezosa (lazy) y sin duplicar ni modificar necesariamente los datos originales.

11. ¿Por qué se dice que un Stream es de 'uso único', y qué ocurre si se intenta reutilizar un Stream después de haber ejecutado una operación terminal sobre él?

Respuesta correcta: A) Una vez ejecutada una operación terminal (como `collect()`, `forEach()` o `count()`), el Stream queda 'consumido' y no puede volver a usarse; intentar invocar otra operación sobre el mismo Stream lanza `IllegalStateException`

Tras ejecutar una operación terminal, el Stream se considera 'consumido' y cualquier intento de reutilizarlo (por ejemplo, aplicando otra operación intermedia o terminal sobre la misma referencia) lanza `IllegalStateException`; se debe crear un nuevo Stream desde la fuente original si se necesita procesar los datos nuevamente.

12. ¿Qué precaución especial se debe tomar al trabajar con un stream infinito, como el generado por `Stream.iterate()` o `Stream.generate()` sin límite explícito?

Respuesta correcta: A) Es imprescindible aplicar una operación intermedia que acote el stream, como `limit(n)`, antes de una operación terminal; de lo contrario, el programa intentaría procesar infinitos elementos y nunca terminaría (o consumiría toda la memoria/CPU disponible)

Los streams infinitos (generados con `iterate()` o `generate()` sin criterio de parada) requieren obligatoriamente una operación como `limit(n)` antes de aplicar una operación terminal; omitir este límite provoca que el programa intente procesar una secuencia sin fin, resultando en un bucle infinito o agotamiento de recursos.

13. ¿Qué diferencia hay entre las operaciones terminales `findFirst()` y `findAny()` de un `Stream`?

Respuesta correcta: A) `findFirst()` garantiza devolver el primer elemento que cumple la condición según el orden del stream (relevante en streams secuenciales u ordenados); `findAny()` puede devolver cualquier elemento que cumpla la condición, sin garantía de orden, lo cual permite mayor eficiencia en streams paralelos

`findFirst()` respeta el orden del encuentro del stream, garantizando devolver el primer elemento según ese orden; `findAny()` no ofrece esa garantía de orden, lo que permite a la JVM optimizar mejor su ejecución en streams paralelos, devolviendo cualquier elemento coincidente disponible.

14. ¿Por qué operaciones como `Stream.max(Comparator)` y `Stream.min(Comparator)` devuelven un `Optional` en lugar del valor directamente?

Respuesta correcta: A) Porque el stream podría estar vacío (sin ningún elemento), en cuyo caso no existiría ningún máximo o mínimo que devolver; `Optional` comunica explícitamente esa posible ausencia de resultado, en lugar de devolver `null` o lanzar una excepción inesperada

Al no poder garantizar que el stream tenga al menos un elemento, `max()` y `min()` devuelven `Optional<T>` para representar de forma explícita y segura el caso de un stream vacío (`Optional.empty()`), evitando el riesgo de `null` o de una excepción no controlada.

15. ¿Cuáles son las tres variantes (sobrecargas) del método `reduce()` de `Stream`, y qué las diferencia principalmente?

Respuesta correcta: A) `reduce(BinaryOperator)` que devuelve `Optional<T>` (sin valor identidad); `reduce(T identidad, BinaryOperator)` que devuelve `T` directamente (con un valor inicial/identidad); y `reduce(T identidad, BiFunction acumulador, BinaryOperator combinador)` para streams paralelos, donde el combinador une resultados parciales de distintos hilos

La primera variante no tiene valor identidad y devuelve `Optional` (por si el stream está vacío); la segunda recibe un valor identidad y devuelve directamente `T`; la tercera añade un 'combinador' adicional, necesario en streams paralelos para combinar los resultados parciales calculados independientemente en distintos hilos/particiones.

16. ¿Por qué el 'valor de identidad' (`identity`) usado en `reduce()` debe ser un elemento neutro respecto a la operación de combinación, por ejemplo 0 para una suma o 1 para una multiplicación?

Respuesta correcta: A) Porque combinar el valor de identidad con cualquier otro elemento mediante la operación no debe alterar el resultado (por ejemplo, sumar 0 no cambia nada, multiplicar por 1 tampoco), garantizando que el resultado final sea correcto independientemente de cómo se distribuya el cálculo (por ejemplo, en un stream paralelo)

El valor de identidad debe ser 'neutro' respecto a la operación (0 para suma, 1 para multiplicación, cadena vacía para concatenación), de modo que combinarlo con cualquier valor no altere el resultado; esta propiedad es esencial para que `reduce()` funcione correctamente también en streams paralelos, donde el valor de identidad puede combinarse en distintos puntos del cálculo distribuido.

17. ¿Cuál es la diferencia esencial entre las operaciones intermedias `filter()` y `map()` de un `Stream`?

Respuesta correcta: A) `filter()` selecciona un subconjunto de los elementos originales según una condición (`Predicate`), sin transformarlos; `map()` transforma cada elemento en otro valor (posiblemente de un tipo distinto), manteniendo la misma cantidad de elementos en el stream resultante

`filter()` reduce potencialmente la cantidad de elementos (descartando los que no cumplen el `Predicate`) sin cambiar su tipo o valor; `map()` transforma cada elemento en un nuevo valor (posiblemente de tipo distinto) mediante una `Function`, manteniendo la cantidad original de elementos en el stream resultante.

18. Al procesar un Stream<String> de números representados como texto (por ejemplo "3", "7", "2") para sumarlos como enteros, ¿qué combinación de operaciones sería típica usando map() y reduce()?

Respuesta correcta: A) stream.map(Integer::parseInt).reduce(0, Integer::sum); primero se transforma cada String a int con map() usando la referencia a método Integer::parseInt, y luego se suman todos los valores con reduce(), usando 0 como identidad e Integer::sum como operador binario

El patrón típico es usar map(Integer::parseInt) para transformar cada String en un int (realmente un Stream<Integer> por el autoboxing en un Stream genérico), y luego reduce(0, Integer::sum) para sumar todos esos valores, usando 0 como identidad y la referencia a método Integer::sum como operador binario de combinación.

19. ¿Qué hace Collectors.toCollection(TreeSet::new) al usarse en collect(), en comparación con Collectors.toList()?

Respuesta correcta: A) Permite recolectar los elementos del stream en un tipo de colección específico (en este caso, un TreeSet, que elimina duplicados y mantiene orden), en lugar de la List genérica que produce toList()

Collectors.toCollection(supplier) permite especificar exactamente qué tipo de colección mutable se desea como resultado (pasando una referencia a su constructor, como TreeSet::new), en lugar de conformarse con la List no especificada que produce Collectors.toList(); en el caso de TreeSet, además se obtiene automáticamente eliminación de duplicados y orden.

20. ¿Qué hace flatMap() en un Stream cuando cada elemento original produce, a su vez, otro Stream (por ejemplo, un Stream<List<Integer>>)?

Respuesta correcta: A) Aplica la función dada a cada elemento (que produce un Stream individual) y luego 'aplana' todos esos streams internos en un único Stream continuo, en lugar de dejar un Stream de streams anidados

flatMap() se usa cuando cada elemento del stream original se transforma en su propio Stream (por ejemplo, una lista de listas); en lugar de producir un Stream<Stream<T>> anidado, flatMap() aplanar todos esos streams internos en un único Stream<T> continuo.

21. ¿Cuál es la diferencia principal entre map() y flatMap() en cuanto a la relación entre elementos de entrada y de salida?

Respuesta correcta: A) map() transforma cada elemento en exactamente OTRO elemento (relación uno a uno); flatMap() transforma cada elemento en un Stream de posiblemente varios elementos, que luego se aplanan (relación uno a muchos)

map() mantiene una relación uno a uno (cada elemento se transforma en otro elemento); flatMap() maneja una relación uno a muchos (cada elemento puede expandirse en varios elementos mediante un Stream interno), aplanando el resultado final en un único stream continuo.

22. ¿Cuál es la diferencia clave entre peek() y forEach() en el contexto de un pipeline de Stream?

Respuesta correcta: A) peek() es una operación INTERMEDIA (perezosa, pensada principalmente para depuración, que no dispara la ejecución por sí sola); forEach() es una operación TERMINAL que sí dispara el procesamiento completo del stream

peek() es una operación intermedia que permite 'espiar' los elementos a medida que pasan por el pipeline (principalmente para depuración), pero no ejecuta nada por sí sola sin una operación terminal posterior; forEach() sí es una operación terminal que efectivamente dispara el procesamiento completo del stream.

23. ¿Por qué modificar estructuralmente (añadir o eliminar elementos de) la colección fuente mientras se procesa un Stream derivado de ella puede causar problemas o comportamiento indefinido?

Respuesta correcta: A) Porque el Stream no crea una copia independiente de los datos; opera sobre la fuente original de forma perezosa, por lo que modificar esa fuente durante el procesamiento puede causar ConcurrentModificationException o resultados inconsistentes/indefinidos

Como el Stream se evalúa de forma perezosa directamente sobre la fuente de datos (sin necesariamente copiar todo de antemano), modificar estructuralmente esa fuente (añadir/eliminar elementos) mientras el stream aún se está procesando puede provocar ConcurrentModificationException o comportamientos inconsistentes, similar al problema clásico de modificar una colección durante una iteración con for-each.

24. ¿Qué ventaja de rendimiento ofrecen los streams primitivos especializados (IntStream, LongStream, DoubleStream) frente a un Stream<Integer>, Stream<Long> o Stream<Double> genérico?

Respuesta correcta: A) Evitan el costo de autoboxing/unboxing al operar directamente sobre valores primitivos internamente, en lugar de envolver cada valor en su clase wrapper correspondiente (Integer, Long, Double), mejorando el rendimiento en operaciones numéricas intensivas

IntStream, LongStream y DoubleStream operan internamente con los valores primitivos correspondientes, evitando el overhead de autoboxing/unboxing que ocurriría con un Stream<Integer> genérico, lo que resulta en mejor rendimiento para cálculos numéricos intensivos, además de ofrecer métodos estadísticos convenientes como sum(), average(), max(), min().

25. ¿Cómo se puede convertir una List<Integer> en un DoubleStream, aplanando y transformando en un solo paso mediante flatMap y mapToDouble?

Respuesta correcta: A) listaDeListas.stream().flatMap(List::stream).mapToDouble(Integer::doubleValue); primero se aplanan la estructura anidada con flatMap(List::stream), y luego se convierte cada Integer a double primitivo con mapToDouble(Integer::doubleValue)

Cuando se tiene una estructura anidada (como una lista de listas de Integer), primero se usa flatMap(List::stream) para aplanarla en un Stream<Integer> continuo, y luego mapToDouble(Integer::doubleValue) convierte cada Integer al primitivo double correspondiente, produciendo finalmente un DoubleStream.

26. ¿Qué método se usa para extraer el valor primitivo double contenido en un OptionalDouble, análogo a get() en un Optional<T> genérico?

Respuesta correcta: A) getAsDouble()

Las versiones especializadas de Optional para primitivos (OptionalInt, OptionalLong, OptionalDouble) usan métodos con nombre específico como getAsDouble(), getAsInt() o getAsLong() en lugar del genérico get(), reflejando que devuelven directamente el primitivo correspondiente.

27. ¿Qué operaciones estadísticas convenientes ofrecen directamente los streams primitivos (IntStream, DoubleStream, etc.) que no están disponibles de la misma forma en un Stream<T> genérico?

Respuesta correcta: A) Métodos como sum(), average() (que devuelve OptionalDouble), max(), min() y summaryStatistics(), todos calculando directamente estadísticas numéricas sobre los valores primitivos del stream

Los streams primitivos exponen directamente métodos convenientes para estadísticas numéricas comunes: sum() (suma total), average() (promedio, como OptionalDouble por si el stream está vacío), max()/min() (como Optional del tipo primitivo correspondiente), y summaryStatistics() para obtener varias métricas combinadas en un solo objeto.

28. ¿En qué se diferencia el comportamiento de map() y filter() cuando se aplican sobre un Optional<T>, en comparación con su comportamiento sobre un Stream<T>?

Respuesta correcta: A) Sobre un Optional, map()/filter() operan sobre 'como máximo un valor' (aplicando la transformación o condición solo si el Optional tiene valor presente, devolviendo Optional vacío en caso contrario); sobre un Stream, operan sobre potencialmente muchos elementos de la secuencia

Conceptualmente, Optional puede verse como un 'contenedor de cero o un elemento'; map() y filter() aplicados sobre él operan sobre ese único valor posible (si está presente), devolviendo Optional vacío si no lo está o si no cumple la condición, mientras que en un Stream estas mismas operaciones procesan una secuencia potencialmente de muchos elementos.

29. ¿Por qué es recomendable evitar tener un 'Optional dentro de otro Optional' (por ejemplo, Optional<Optional<String>>), y cómo ayuda flatMap() en este contexto?

Respuesta correcta: A) Un Optional anidado es confuso e innecesariamente complejo de manejar (hay que 'desempaquetar' dos veces); usar flatMap() en lugar de map() cuando la función de transformación ya devuelve un Optional evita ese anidamiento, aplanando el resultado en un único Optional en lugar de uno doblemente envuelto

Si la función pasada a map() sobre un Optional ya devuelve otro Optional, el resultado sería un Optional<Optional<T>> anidado y confuso; usar flatMap() en su lugar 'aplanan' ese resultado, produciendo directamente un único Optional<T>, de forma análoga a cómo flatMap() aplanan streams anidados.

30. ¿Por qué una expresión lambda usada dentro de una operación de Stream (como map()) NO puede lanzar directamente una excepción checked, si la interfaz funcional correspondiente (como Function) no la declara?

Respuesta correcta: A) Porque el cuerpo de la lambda debe respetar la firma del método abstracto de la interfaz funcional que implementa; si ese método (como apply() de Function) no declara throws para esa excepción checked, el compilador rechaza que la lambda la lance directamente

Las interfaces funcionales estándar como Function, Predicate o Consumer no declaran excepciones checked en sus métodos abstractos; por lo tanto, una lambda que las implemente no puede lanzar una excepción checked directamente, siendo necesario envolverla en una unchecked (como RuntimeException) o manejarla dentro del propio cuerpo de la lambda.

31. ¿Cuál es una estrategia común para 'envolver' una excepción checked lanzada dentro de una lambda usada en un Stream, de forma que el código compile?

Respuesta correcta: A) Capturar la excepción checked dentro de un bloque try-catch en el cuerpo de la lambda, y relanzarla envuelta en una excepción unchecked (como una RuntimeException personalizada), que sí es compatible con la firma de la interfaz funcional sin necesidad de declarar throws

Una técnica común es capturar la excepción checked dentro de la lambda con try-catch y relanzarla envuelta en una excepción unchecked (por ejemplo, una RuntimeException personalizada), ya que las excepciones unchecked no requieren ser declaradas en la firma del método abstracto de la interfaz funcional, permitiendo que el código compile.

32. ¿Qué propósito cumple la interfaz Splitter en el procesamiento de streams?

Respuesta correcta: A) Permite dividir ('split') una fuente de datos en partes más pequeñas que pueden procesarse de forma independiente (potencialmente en paralelo), además de recorrer secuencialmente los elementos, siendo la base sobre la que se implementa el procesamiento paralelo eficiente de streams

Splitter (split + iterator) permite tanto recorrer elementos secuencialmente como dividir la fuente de datos en partes menores para su procesamiento independiente, siendo el mecanismo subyacente que la Stream API utiliza para implementar eficientemente el procesamiento paralelo de grandes colecciones.

33. ¿Qué hace el Collector Collectors.joining(delimitador) al aplicarse sobre un Stream<String>?

Respuesta correcta: A) Concatena todos los elementos String del stream en un único String resultante, insertando el delimitador especificado entre cada par de elementos consecutivos

Collectors.joining(delimitador) concatena todos los elementos de un Stream<String> en un único String, separando cada elemento con el delimitador indicado (y opcionalmente añadiendo prefijo y sufijo si se usa la sobrecarga de tres argumentos).

34. ¿Qué problema resuelve la función de fusión (merge function) que se puede proporcionar como tercer argumento de Collectors.toMap(keyMapper, valueMapper, mergeFunction)?

Respuesta correcta: A) Determina cómo combinar los valores cuando dos elementos del stream producen la MISMA clave al aplicar keyMapper, evitando la excepción IllegalStateException que ocurriría por defecto ante claves duplicadas

Sin una función de fusión, Collectors.toMap() lanza IllegalStateException si dos elementos del stream generan la misma clave; proporcionar una mergeFunction (BinaryOperator) permite especificar cómo combinar ambos valores en conflicto (por ejemplo, sumarlos, quedarse con el primero, o con el último), evitando esa excepción.

35. ¿Qué garantiza siempre el resultado de Collectors.partitioningBy(predicado), a diferencia de groupingBy()??

Respuesta correcta: A) partitioningBy() siempre produce un Map<Boolean, List<T>> con exactamente dos claves (true y false), incluso si alguna de las dos particiones queda vacía; groupingBy() puede producir un número variable de claves según los valores distintos generados por la función clasificadora

A diferencia de groupingBy() (que puede generar un número variable de grupos según los valores distintos que produzca la función clasificadora), partitioningBy() siempre produce exactamente dos particiones basadas en un Predicate: una para los elementos que cumplen la condición (true) y otra para los que no (false), incluso si una de las dos particiones resulta vacía.

36. ¿Qué hace la combinación 'Collectors.groupingBy(funcionClasificadora, Collectors.counting())'?

Respuesta correcta: A) Agrupa los elementos según la función clasificadora, y para cada grupo, en lugar de una lista de elementos, almacena el CONTEO (Long) de cuántos elementos cayeron en ese grupo

groupingBy() con un 'downstream collector' adicional (como counting()) no produce una lista de elementos por grupo, sino que aplica ese collector adicional sobre cada grupo; con counting(), el resultado es un Map<K, Long> donde cada valor es la cantidad de elementos que cayeron en esa clave/grupo, en lugar de la lista completa de elementos.

37. ¿Qué recomendación práctica se sugiere al construir un pipeline de Stream complejo con múltiples operaciones intermedias encadenadas?

Respuesta correcta: A) Comenzar construyendo y verificando el pipeline con operaciones simples primero, dividiendo la lógica en partes más pequeñas y manejables para facilitar la depuración, en lugar de escribir de una sola vez una cadena larga y compleja de operaciones

Ante pipelines complejos, se recomienda construir y probar el código incrementalmente con operaciones simples primero, dividiendo la lógica en partes más pequeñas y verificables, lo que facilita identificar errores y entender el comportamiento de cada etapa antes de combinar todo en un pipeline más largo y complejo.

38. ¿Qué ventaja ofrece calcular múltiples estadísticas (como suma, promedio, mínimo y máximo) en una sola pasada usando un Collector como summarizingInt/summarizingDouble, en lugar de procesar el stream varias veces (una por cada estadística)?

Respuesta correcta: A) Permite obtener todas esas métricas procesando la fuente de datos una única vez, evitando el costo de recorrer (o consumir) el stream múltiples veces, lo cual además sería imposible directamente ya que un stream es de uso único

Collectors como summarizingInt()/summarizingDouble() calculan de una sola pasada un conjunto completo de estadísticas (count, sum, min, max, average) en un único objeto de resumen, siendo mucho más eficiente que intentar recorrer el stream varias veces para cada estadística por separado (lo cual, de hecho, sería imposible sin recrear el stream, ya que es de uso único).

18. Optional

1. ¿Cuál es el propósito principal de la clase Optional<T>?

Respuesta correcta: A) Representar explícitamente la posible ausencia de un valor, evitando NullPointerException y forzando un manejo explícito del caso 'vacío'

Optional<T> es un contenedor que puede o no tener un valor, comunicando explícitamente en la firma del método que el resultado puede estar ausente.

2. ¿Qué hace Optional.ofNullable(valor) si 'valor' es null?

Respuesta correcta: A) Devuelve un Optional vacío (Optional.empty()) sin lanzar excepción

A diferencia de Optional.of() (que lanza NPE si el argumento es null), ofNullable() acepta null y produce un Optional vacío en ese caso.

3. ¿Cuál es una forma recomendada de obtener un valor por defecto de un Optional si está vacío?

Respuesta correcta: A) optional.orElse(valorPorDefecto)

orElse(valorDefault) devuelve el valor contenido si existe, o el valor por defecto indicado si el Optional está vacío. También existe orElseGet() y orElseThrow().

4. ¿Qué hace Optional.map(funcion) si el Optional está vacío?

Respuesta correcta: A) Devuelve directamente un Optional vacío, sin aplicar la función

Si el Optional está vacío, map() simplemente devuelve otro Optional vacío sin invocar la función de transformación, evitando NullPointerException.

5. ¿Qué excepción lanza optional.get() si el Optional está vacío?

Respuesta correcta: A) NoSuchElementException

Llamar a get() sobre un Optional vacío lanza NoSuchElementException; por eso se recomienda usar isPresent()/ifPresent(), orElse(), o orElseThrow() en su lugar.

6. ¿Cuál es una buena práctica respecto al uso de Optional como tipo de un campo de clase o parámetro de método?

Respuesta correcta: A) Se recomienda evitarlo: Optional está pensado principalmente como tipo de retorno de métodos, no para campos ni parámetros

La convención de la comunidad Java (y de los propios diseñadores del API) es usar Optional principalmente como tipo de retorno para indicar ausencia posible de valor, evitando su uso en campos de clase, parámetros o colecciones.

19. Collections Framework (List, Set, Map, Deque)

1. ¿Cuál es la diferencia principal entre List, Set y Map en el Collections Framework?

Respuesta correcta: A) List permite duplicados y mantiene orden de inserción; Set no permite duplicados; Map almacena pares clave-valor

List: colección ordenada indexada con duplicados permitidos. Set: no permite elementos duplicados. Map: asocia claves únicas a valores (no extiende Collection).

2. ¿Qué implementación de List garantiza acceso rápido $O(1)$ por índice, pero inserciones/eliminaciones costosas en el medio?

Respuesta correcta: A) ArrayList

ArrayList se respalda en un array, por lo que el acceso por índice es $O(1)$, pero insertar/eliminar en medio requiere desplazar elementos. LinkedList es más eficiente en inserciones/eliminaciones.

3. ¿Qué característica distingue a un TreeMap de un HashMap?

Respuesta correcta: A) TreeMap mantiene sus claves ordenadas (orden natural o Comparator); HashMap no garantiza ningún orden

TreeMap implementa NavigableMap y mantiene las claves ordenadas; HashMap ofrece acceso $O(1)$ promedio pero sin orden garantizado.

4. ¿Qué devuelve List.of(1,2,3) en cuanto a mutabilidad?

Respuesta correcta: A) Una lista inmutable; intentar añadir o eliminar elementos lanza UnsupportedOperationException

Los métodos de fábrica List.of(), Set.of(), Map.of() (Java 9+) devuelven colecciones inmutables; cualquier intento de modificarlas lanza UnsupportedOperationException.

5. ¿Qué debe sobrescribir correctamente una clase para usarse como clave en un HashMap de forma consistente?

Respuesta correcta: A) equals() y hashCode() (contrato de Object)

HashMap usa hashCode() para ubicar el bucket y equals() para verificar igualdad de claves; si no se sobrescriben correctamente y de forma consistente, el Map puede comportarse mal.

6. ¿Qué estructura de datos implementa típicamente una Queue de tipo FIFO (primero en entrar, primero en salir)?

Respuesta correcta: A) LinkedList (implementando Queue)

LinkedList implementa la interfaz Queue, permitiendo comportamiento FIFO mediante métodos como offer(), poll() y peek().

7. ¿Qué diferencia hay entre Iterator y ListIterator?

Respuesta correcta: A) ListIterator extiende Iterator añadiendo la capacidad de recorrer en ambas direcciones y modificar elementos durante la iteración (add, set)

ListIterator añade métodos como hasPrevious(), previous(), set() y add(), permitiendo recorrer una List en ambos sentidos y modificarla durante la iteración, algo que Iterator (más genérico) no ofrece.

8. ¿Qué excepción se lanza al modificar una colección (por ejemplo, con `remove()`) directamente mientras se itera sobre ella con un `for-each`, sin usar el propio `Iterator`?

Respuesta correcta: A) `ConcurrentModificationException`

Modificar estructuralmente una colección mientras se itera con `for-each` (que usa un `Iterator` internamente) sin pasar por los métodos del propio `Iterator` provoca `ConcurrentModificationException` en la mayoría de implementaciones.

9. ¿Cuáles son las tres interfaces principales del `Collections Framework` de Java que representan las estructuras de datos fundamentales?

Respuesta correcta: A) `List`, `Set` y `Map`

`List` (colección ordenada con duplicados permitidos), `Set` (sin elementos duplicados) y `Map` (pares clave-valor) son las tres interfaces fundamentales sobre las que se construye el resto del `Collections Framework` de Java.

10. ¿Qué estructura permite usar una misma implementación tanto como `Queue` (FIFO, primero en entrar-primero en salir) como `Stack` (LIFO, último en entrar-primero en salir)?

Respuesta correcta: A) `Deque` (double-ended queue), que permite inserciones y eliminaciones eficientes en ambos extremos, sirviendo tanto para comportamiento FIFO como LIFO según los métodos que se usen

`Deque` (interfaz que extiende `Queue`) permite añadir y quitar elementos tanto por el frente como por el final de forma eficiente, lo que la hace apta tanto para simular una `Queue` (FIFO, usando `offer/poll` de un extremo) como una `Stack` (LIFO, usando `push/pop` del mismo extremo).

11. ¿Qué criterio de ordenamiento usa un `TreeSet` por defecto para mantener sus elementos ordenados, si no se le proporciona un `Comparator` explícito?

Respuesta correcta: A) El orden natural definido por el método `compareTo()` de la interfaz `Comparable`, que los elementos deben implementar en ese caso

Si no se proporciona un `Comparator` explícito al crear un `TreeSet`, este usa el orden natural de los elementos definido por su implementación de `Comparable` (`compareTo()`); si los elementos no implementan `Comparable` y no se provee un `Comparator`, se produce una `ClassCastException` en tiempo de ejecución al intentar ordenar.

12. En un ejemplo con una clase `Cliente` añadida a un `HashSet<Cliente>`, si `Cliente` no sobrescribe `equals()` ni `hashCode()`, ¿qué ocurre al intentar añadir un 'Cliente' con los mismos datos (pero como un objeto diferente) para verificar si ya existe?

Respuesta correcta: A) El `HashSet` lo trata como un elemento distinto y lo añade igualmente, ya que sin sobrescribir `equals()/hashCode()` se usa la implementación heredada de `Object` (comparación por identidad de referencia), y dos objetos distintos con el mismo contenido no se consideran iguales

Si no sobrescribe `equals()` y `hashCode()`, `Object` usa comparación por identidad de referencia; un `HashSet`, que depende de estos métodos para detectar duplicados lógicos, tratará dos objetos `Cliente` con el mismo contenido pero distintas referencias como elementos diferentes, permitiendo insertar ambos aunque representen 'la misma' información de negocio.

13. ¿Por qué es importante que `hashCode()` sea consistente con `equals()` (es decir, que objetos considerados iguales por `equals()` produzcan el mismo `hashCode()`)?

Respuesta correcta: A) Porque las colecciones basadas en tablas hash (como `HashSet` o `HashMap`) usan `hashCode()` para determinar en qué 'bucket' buscar un elemento antes de usar `equals()` para confirmar la igualdad exacta; si dos objetos iguales según `equals()` tuvieran distinto `hashCode()`, la colección podría no encontrarlos correctamente como duplicados

Estructuras basadas en hash como `HashSet` y `HashMap` primero usan `hashCode()` para ubicar el bucket candidato donde buscar un elemento, y luego usan `equals()` para confirmar la coincidencia exacta dentro de ese bucket; si dos objetos son 'iguales' según `equals()` pero tienen `hashCode()` distintos, podrían terminar en buckets diferentes y la colección fallaría en detectarlos como el mismo elemento lógico.

14. ¿Qué tipo de lista devuelve Arrays.asList(array), y qué restricción particular tiene respecto a su tamaño?

Respuesta correcta: A) Devuelve una lista de tamaño fijo respaldada directamente por el array original; se pueden modificar los elementos existentes (set), pero no se pueden añadir ni eliminar elementos (add/remove lanzan UnsupportedOperationException)

Arrays.asList() crea una List de tamaño fijo respaldada directamente por el array proporcionado; se permite modificar elementos existentes con set() (lo cual afecta también al array original, ya que comparten el mismo almacenamiento), pero add() y remove() lanzan UnsupportedOperationException porque cambiarían el tamaño.

15. ¿Qué ocurre si se modifica un elemento del array original después de haber creado una lista con Arrays.asList(array)?

Respuesta correcta: A) El cambio se refleja también en la lista, ya que ambos (array y lista) comparten el mismo almacenamiento subyacente; no son estructuras independientes

Como Arrays.asList() no crea una copia independiente sino que la lista está respaldada directamente por el array proporcionado, cualquier modificación posterior a los elementos del array (o de la lista) se refleja automáticamente en el otro, ya que comparten el mismo almacenamiento subyacente.

16. ¿Qué diferencia hay entre List.copyOf(coleccionOriginal) y simplemente reutilizar la colección original para crear una lista 'inmutable'?

Respuesta correcta: A) List.copyOf() crea una nueva colección independiente, completamente inmutable, con una copia de los elementos actuales; los cambios posteriores en la colección original ya NO afectan a esta copia inmutable, a diferencia de Arrays.asList()

A diferencia de Arrays.asList() (que comparte almacenamiento con el array), List.copyOf() genera una copia independiente e inmutable de los elementos en el momento de la llamada; cambios posteriores en la colección de origen no afectan a esta copia inmutable resultante.

17. ¿Por qué insertar un elemento en una posición intermedia de una ArrayList es generalmente más costoso que hacerlo en una LinkedList?

Respuesta correcta: A) Porque ArrayList se respalda en un array interno, y al insertar en el medio es necesario desplazar todos los elementos posteriores una posición, lo cual es una operación O(n); LinkedList solo necesita reajustar las referencias de los nodos vecinos, una operación O(1) una vez ubicada la posición

Al estar respaldada por un array, ArrayList requiere desplazar físicamente los elementos posteriores a la posición de inserción, un costo O(n); LinkedList, al usar nodos enlazados, solo necesita ajustar un par de referencias, siendo O(1) una vez localizada la posición correcta (aunque encontrar esa posición sí puede ser O(n) si se accede secuencialmente).

18. ¿Qué hace el método removeIf(Predicate) al aplicarse sobre una colección mutable, por ejemplo 'lista.removeIf(n -> n < 0);' para eliminar todos los números negativos?

Respuesta correcta: A) Recorre la colección y elimina todos los elementos para los cuales el Predicate dado devuelve true, modificando la colección original directamente sin necesidad de un bucle explícito ni de gestionar manualmente un Iterator

removeIf(Predicate) modifica la colección original in situ, eliminando todos los elementos que cumplan la condición del Predicate proporcionado, evitando el patrón tradicional (y propenso a errores) de iterar manualmente con un Iterator para eliminar elementos durante el recorrido.

19. En una LinkedList<Integer>, ¿qué diferencia hay entre invocar remove(1) y remove(Integer.valueOf(1))?

Respuesta correcta: A) remove(1) interpreta el argumento como un índice (int) y elimina el elemento en la posición 1; remove(Integer.valueOf(1)) interpreta el argumento como un objeto Integer y elimina la PRIMERA OCURRENCIA del valor 1 en la lista, sin importar su posición

List tiene dos métodos sobrecargados remove(): remove(int index) que opera por posición, y remove(Object o) que opera por valor buscando la primera coincidencia; al pasar un int literal como 1, Java lo interpreta como índice (remove(int)), mientras que forzar un Integer explícito invoca la versión remove(Object), una trampa clásica de examen con listas de tipo Integer.

20. ¿Por qué esta ambigüedad entre `remove(int)` y `remove(Object)` es especialmente relevante en listas de tipo `Integer`, pero no representa un problema en listas de tipo `String`, por ejemplo?

Respuesta correcta: A) Porque `Integer` es un tipo que también puede interpretarse como un índice numérico (`int`), generando la ambigüedad de cuál sobrecarga de `remove()` aplica; `String` no tiene esa ambigüedad porque nunca podría confundirse con un índice `int`

La ambigüedad surge específicamente con listas de tipo `Integer` (o `Long` compatible con `int` mediante ciertas conversiones) porque un literal numérico puede interpretarse tanto como índice (`int`) como candidato a autoboxing hacia el tipo `Integer` almacenado; con `String`, al no existir esa doble interpretación posible, `remove(objetoString)` siempre se resuelve sin ambigüedad hacia `remove(Object)`.

21. ¿Qué combinación de pasos permite convertir el conjunto de pares clave-valor de un `Map` en una `List<Map.Entry<K,V>>` para, por ejemplo, poder ordenarlos con `Collections.sort()`?

Respuesta correcta: A) Invocar `mapa.entrySet()` para obtener un `Set<Map.Entry<K,V>>`, y luego pasar ese `Set` al constructor de una `ArrayList` (o usar `new ArrayList<>(mapa.entrySet())`) para obtener una lista mutable y ordenable

`entrySet()` devuelve una vista de tipo `Set` (no ordenable directamente con `Collections.sort`, que opera sobre `List`); para poder ordenarla, se debe envolver ese `Set` en una nueva `ArrayList` (por ejemplo, `'new ArrayList<>(mapa.entrySet())'`), obteniendo así una lista mutable de `Map.Entry` que sí puede ordenarse.

22. ¿Por qué `Collections.binarySearch()` sobre una `LinkedList` suele ser menos eficiente que sobre una `ArrayList`, aunque ambas puedan usarse con el método?

Respuesta correcta: A) Porque `LinkedList` no permite acceso directo (aleatorio) $O(1)$ a una posición intermedia como sí lo hace `ArrayList`; para acceder al 'elemento del medio' en cada paso del algoritmo de búsqueda binaria, `LinkedList` debe recorrer secuencialmente los nodos desde un extremo, degradando el rendimiento del algoritmo

El algoritmo de búsqueda binaria depende de acceder eficientemente a posiciones intermedias en cada paso; como `ArrayList` permite acceso indexado $O(1)$, esto es eficiente, pero `LinkedList` requiere recorrer secuencialmente los nodos desde un extremo para llegar a una posición intermedia ($O(n)$ por cada acceso), haciendo que `binarySearch()` sobre una `LinkedList` pierda gran parte de su ventaja teórica de $O(\log n)$.

23. ¿Qué papel juegan `Comparable` y `Comparator` al momento de decidir cómo se ordena un `TreeSet` de objetos personalizados antes de aplicar cualquier búsqueda o iteración ordenada?

Respuesta correcta: A) Si no se proporciona un `Comparator` al crear el `TreeSet`, este usará el orden natural definido por `compareTo()` (`Comparable`) de los elementos; si se proporciona un `Comparator` explícito al constructor del `TreeSet`, este tiene prioridad sobre el orden natural, sin importar si los elementos también implementan `Comparable`

Un `TreeSet` usa el `Comparator` proporcionado explícitamente en su constructor si existe, ignorando en ese caso el orden natural de `Comparable`; solo si no se proporciona ningún `Comparator`, recurre al orden natural definido por `compareTo()` de los elementos, que en ese caso deben implementar `Comparable`.

24. ¿Qué hace el método `setValue(nuevoValor)` de la interfaz `Map.Entry`, y qué efecto tiene sobre el `Map` original del cual proviene esa entrada?

Respuesta correcta: A) Modifica el valor asociado a esa entrada directamente en el `Map` subyacente del cual se obtuvo (siempre que ese `Map` sea mutable), reflejando el cambio inmediatamente en el mapa original, y devuelve el valor anterior que tenía esa entrada

`Map.Entry.setValue(V value)` actualiza el valor de esa entrada directamente en el mapa subyacente (si es mutable), y retorna el valor previo que tenía la entrada antes del cambio; esto permite modificar valores del mapa mientras se itera sobre su `entrySet()`, sin necesidad de volver a acceder al mapa con `put()`.

25. ¿Qué ocurre si se invoca setValue() sobre una entrada obtenida del entrySet() de un mapa inmutable, como uno creado con Map.of()?

Respuesta correcta: A) Se lanza UnsupportedOperationException, ya que las estructuras (y sus entradas) provenientes de colecciones inmutables como las creadas con Map.of() no permiten ningún tipo de modificación, incluido cambiar el valor de una entrada individual

Los mapas inmutables (como los creados con Map.of()) y sus vistas (entrySet(), las propias entradas) rechazan cualquier operación de modificación, incluyendo setValue() sobre una entrada individual, lanzando UnsupportedOperationException, consistente con la garantía general de inmutabilidad de esas estructuras.

26. Si se necesita modificar los valores de una lista o mapa obtenido de una fuente inmutable (por ejemplo, Map.of() o List.of()), ¿cuál es una estrategia común para lograrlo?

Respuesta correcta: A) Crear una copia mutable independiente a partir de la estructura inmutable (por ejemplo, con 'new ArrayList<>(listaInmutable)' o 'new HashMap<>(mapaInmutable)'), y aplicar las modificaciones sobre esa nueva copia mutable, dejando la estructura original inmutable intacta

La estrategia estándar es construir una nueva colección mutable a partir de los datos de la inmutable (usando el constructor de copia de ArrayList, HashMap, etc.), y realizar las modificaciones necesarias sobre esa copia mutable, sin alterar ni depender de que la estructura original inmutable permita cambios.

27. ¿En qué posición exacta de la firma de un método static debe declararse su propio parámetro de tipo genérico, por ejemplo en 'public static <T extends Comparable<T>> T maximo(List<T> lista)'?

Respuesta correcta: A) Inmediatamente antes del tipo de retorno del método (en este caso, antes de la segunda 'T'), no antes de la palabra 'static' ni en ningún otro lugar de la firma

La sintaxis correcta para un método genérico (static o no) requiere declarar el parámetro de tipo entre '<>' inmediatamente después de los modificadores (como static) y ANTES del tipo de retorno del método; en el ejemplo, '<T extends Comparable<T>>' aparece justo antes del tipo de retorno T, siendo la posición sintácticamente válida y obligatoria.

28. ¿Por qué un método genérico static que declara su propio parámetro de tipo T (como en '<T> void metodo(T valor)') Sí es válido, mientras que intentar usar directamente el T de la clase contenedora en un método static NO lo es?

Respuesta correcta: A) Porque en el primer caso, el método declara y resuelve su propio parámetro de tipo T de forma independiente en cada invocación, sin depender de ninguna instancia; en el segundo caso, el T de la clase está asociado a una instancia específica (Clase<String>, Clase<Integer>, etc.), lo cual es incompatible con un contexto static que no tiene instancia asociada

Un método genérico declara su propio parámetro de tipo, resuelto de forma independiente en cada llamada, sin ninguna dependencia de instancia, por lo que es perfectamente compatible con un contexto static; en cambio, el parámetro de tipo T de la propia clase genérica está ligado a una instancia concreta parametrizada, lo cual no tiene sentido en un método static que no opera sobre ninguna instancia.

20. Concurrencia y Virtual Threads

1. ¿Qué interfaz representa una tarea que puede devolver un resultado y lanzar excepciones checked, a diferencia de Runnable?

Respuesta correcta: A) Callable<V>

Callable<V> define call() que devuelve un valor V y puede lanzar excepciones checked; Runnable define run() sin retorno y sin poder lanzar checked exceptions.

2. ¿Qué provee un ExecutorService respecto a crear Thread manualmente?

Respuesta correcta: A) Un framework de alto nivel para administrar un pool de hilos reutilizables, enviar tareas (submit/execute) y gestionar su ciclo de vida

ExecutorService (java.util.concurrent) administra un conjunto de hilos, permitiendo enviar tareas sin gestionar manualmente la creación y destrucción de cada Thread.

3. ¿Qué son los 'virtual threads' introducidos en Java 21 (JEP 444)?

Respuesta correcta: A) Hilos ligeros gestionados por la JVM (no por el sistema operativo) que permiten crear millones de hilos concurrentes con bajo costo de memoria, ideales para aplicaciones con mucha I/O

Los virtual threads (Project Loom, estándar desde Java 21) son gestionados por la JVM en lugar de mapearse 1:1 a hilos del SO, permitiendo alta concurrencia con muy bajo overhead.

4. ¿Qué palabra clave permite garantizar que solo un hilo a la vez ejecute un bloque de código crítico?

Respuesta correcta: A) synchronized

'synchronized' aplicado a un método o bloque asegura exclusión mutua: solo un hilo puede ejecutar esa sección crítica sobre el mismo monitor a la vez.

5. ¿Cómo se crea rápidamente un ExecutorService basado en virtual threads en Java 21?

Respuesta correcta: A) Executors.newVirtualThreadPerTaskExecutor()

Executors.newVirtualThreadPerTaskExecutor() crea un ExecutorService que lanza una nueva virtual thread por cada tarea enviada.

6. ¿Qué método de la clase Thread permite pausar la ejecución del hilo actual durante un tiempo determinado?

Respuesta correcta: A) Thread.sleep(milisegundos)

Thread.sleep(ms) pausa la ejecución del hilo actual durante el tiempo indicado, sin liberar ningún monitor que tenga adquirido.

7. ¿Qué es una 'race condition' (condición de carrera) en programación concurrente?

Respuesta correcta: A) Una situación en la que el resultado del programa depende del orden impredecible de ejecución de múltiples hilos accediendo a datos compartidos sin la sincronización adecuada

Una race condition ocurre cuando el resultado depende del orden en que se intercalan operaciones de distintos hilos sobre recursos compartidos, produciendo resultados inconsistentes si no hay sincronización adecuada.

8. ¿Qué aporta la clase java.util.concurrent.atomic.AtomicInteger frente a un int normal en un entorno multihilo?

Respuesta correcta: A) Provee operaciones atómicas (como incrementAndGet()) que se ejecutan de forma indivisible sin necesidad de bloques synchronized, evitando condiciones de carrera

AtomicInteger (y clases similares) ofrece operaciones atómicas sin bloqueo explícito, útiles para contadores y acumuladores compartidos entre hilos, evitando problemas de concurrencia sin la sobrecarga de synchronized.

21. Manejo de Excepciones

1. ¿Cuál es la diferencia entre una excepción 'checked' y una 'unchecked'?

Respuesta correcta: A) Las checked deben declararse (throws) o manejarse obligatoriamente en compilación (extienden Exception); las unchecked no obligan a manejo explícito (extienden RuntimeException)

Las excepciones checked (como IOException) obligan al compilador a exigir manejo o declaración con throws; las unchecked (RuntimeException y subclases) no tienen esa obligación.

2. ¿Qué garantiza el bloque 'finally' en un try-catch-finally?

Respuesta correcta: A) Se ejecuta siempre, haya o no excepción, incluso si hay un return dentro del try o catch (salvo System.exit() o error grave de JVM)

El bloque finally se ejecuta siempre tras try/catch, independientemente de si hubo excepción o un return, salvo casos extremos como System.exit().

3. ¿Qué ventaja ofrece 'try-with-resources' introducido en Java 7?

Respuesta correcta: A) Cierra automáticamente los recursos que implementan AutoCloseable al finalizar el bloque try, sin necesidad de un finally explícito

try-with-resources cierra automáticamente cada recurso declarado (AutoCloseable/Closeable) al salir del bloque, en orden inverso a su declaración, incluso si ocurre una excepción.

4. ¿Qué clase es la superclase común de Exception y Error?

Respuesta correcta: A) Throwable

java.lang.Throwable es la raíz de la jerarquía; de ella derivan Exception (condiciones recuperables) y Error (problemas graves, normalmente no manejables).

5. ¿Se puede tener múltiples bloques catch para un mismo try, capturando distintos tipos de excepción?

Respuesta correcta: A) Sí, se pueden encadenar varios catch; el orden importa: las excepciones más específicas deben ir antes que las más generales

Java permite múltiples bloques catch; deben ordenarse de más específico a más general, ya que colocar una excepción general (como Exception) antes que una más específica provoca un error de compilación por código inalcanzable.

6. ¿Qué permite el 'multi-catch' introducido en Java 7 (catch (IOException | SQLException e))?

Respuesta correcta: A) Capturar varios tipos de excepción no relacionados jerárquicamente en un único bloque catch, evitando duplicar código de manejo

El multi-catch permite manejar varios tipos de excepción con el mismo bloque de código usando '|', siempre que no exista relación de herencia directa entre ellas (ya que en ese caso bastaría capturar la más general).

7. ¿Qué es una excepción personalizada (custom exception) y cómo se crea típicamente?

Respuesta correcta: A) Una clase que extiende Exception (checked) o RuntimeException (unchecked) para representar un error específico del dominio de la aplicación

Se crean excepciones personalizadas extendiendo Exception (si debe ser checked) o RuntimeException (si debe ser unchecked), típicamente añadiendo constructores que reciban un mensaje y/o causa.

8. ¿Qué es una 'suppressed exception' (excepción suprimida) en el contexto de try-with-resources?

Respuesta correcta: A) Es una excepción adicional lanzada durante el cierre automático de un recurso (en el método close()), que ocurre DESPUÉS de que ya se había lanzado una excepción principal en el bloque try; en lugar de reemplazar la excepción principal, se adjunta como 'suprimida' y puede recuperarse con getSuppressed()

Cuando ocurre una excepción en el bloque try y, adicionalmente, otra excepción ocurre al cerrar automáticamente un recurso (try-with-resources), la excepción original del try se propaga como la principal, y la excepción del close() se adjunta como 'suprimida', recuperable mediante el método getSuppressed() sobre la excepción principal, en lugar de perderse o reemplazar a la original.

9. ¿Es obligatorio incluir un bloque catch o finally al usar try-with-resources, como en 'try (Recurso r = new Recurso()) { ... }'?

Respuesta correcta: A) No, un try-with-resources puede existir por sí solo, sin bloque catch ni finally, ya que el propio mecanismo de cierre automático de recursos es la característica principal; catch y finally siguen siendo opcionales adicionales si se necesitan

A diferencia de un try tradicional (que requiere al menos un catch o un finally), un try-with-resources es sintácticamente válido por sí mismo, sin necesidad de catch ni finally, ya que el cierre automático de los recursos declarados es su propósito principal; catch y finally pueden añadirse opcionalmente si se necesita manejo adicional.

10. Si un try-with-resources declara múltiples recursos, como 'try (Recurso1 r1 = ...; Recurso2 r2 = ...) {...}', ¿en qué orden se cierran esos recursos al finalizar el bloque?

Respuesta correcta: A) En orden INVERSO al que fueron declarados: se cierra primero r2 y después r1, ya que el cierre sigue una disciplina similar a una pila (el último recurso abierto es el primero en cerrarse)

Los recursos declarados en un try-with-resources se cierran en orden inverso a su declaración (el último declarado es el primero en cerrarse), siguiendo una disciplina similar a una pila (LIFO), lo cual es relevante si existe alguna dependencia entre los recursos.

11. ¿Qué ocurre si un bloque 'finally' contiene una sentencia 'return' (o lanza una nueva excepción), y el bloque try (o catch) también había lanzado una excepción previamente?

Respuesta correcta: A) El return (o la nueva excepción) del bloque finally tiene prioridad y efectivamente 'descarta' o reemplaza la excepción original que se había lanzado en try/catch, la cual se pierde silenciosamente sin propagarse

Una trampa importante: si finally contiene un return o lanza una nueva excepción, esta 'gana' y efectivamente descarta/reemplaza cualquier excepción que estuviera propagándose desde el try o catch, la cual se pierde silenciosamente sin ningún rastro (a diferencia de las suppressed exceptions de try-with-resources, que sí se preservan).

12. ¿En qué orden deben cerrarse los recursos si una clase personalizada como Turkey implementa AutoCloseable y se anida dentro de otro recurso Duck que también lo implementa, ambos en el mismo try-with-resources?

Respuesta correcta: A) El recurso declarado en último lugar en la lista del try-with-resources se cierra primero (orden inverso de declaración), independientemente de si están conceptualmente 'anidados' o no en la lógica del programa

Independientemente de la relación lógica entre los recursos, el mecanismo de try-with-resources siempre cierra los recursos en el orden inverso exacto al que fueron declarados en la cabecera del try, garantizando un comportamiento predecible tipo pila (LIFO).

22. BigDecimal y Precisión Numérica

1. ¿Por qué se recomienda usar BigDecimal en lugar de double para cálculos monetarios?

Respuesta correcta: A) Porque double usa representación de punto flotante binario que introduce errores de redondeo; BigDecimal representa números decimales exactos con precisión arbitraria

double/float usan representación binaria IEEE 754 que no puede representar exactamente muchos decimales, causando errores acumulativos; BigDecimal evita esto mediante representación decimal exacta.

2. ¿Cuál es la forma recomendada de crear un BigDecimal a partir de un valor decimal exacto como 19.99?

Respuesta correcta: A) new BigDecimal("19.99") usando un String, para evitar imprecisión del double

Construir BigDecimal a partir de un double literal puede arrastrar la imprecisión binaria de ese double; usar el constructor con String es más seguro.

3. ¿Por qué comparar dos BigDecimal con equals() puede dar resultados inesperados, por ejemplo entre new BigDecimal("2.0") y new BigDecimal("2.00")?

Respuesta correcta: A) Porque equals() considera también la escala (número de decimales); para comparar solo el valor numérico se debe usar compareTo()

equals() en BigDecimal compara valor Y escala; '2.0' y '2.00' son matemáticamente iguales pero tienen distinta escala, por lo que equals() devuelve false. compareTo() sí compara solo el valor numérico.

4. ¿Qué método de BigDecimal se usa para redondear un valor a una cantidad específica de decimales con un modo de redondeo determinado?

Respuesta correcta: A) setScale(decimales, RoundingMode.HALF_UP) (u otro RoundingMode)

setScale(int newScale, RoundingMode mode) ajusta la escala (decimales) de un BigDecimal aplicando el modo de redondeo indicado, como HALF_UP, HALF_EVEN, etc.

5. ¿Por qué BigDecimal es inmutable, igual que String?

Respuesta correcta: A) Porque cada operación aritmética (add, subtract, multiply...) devuelve un nuevo objeto BigDecimal en lugar de modificar el original, garantizando consistencia y seguridad en entornos concurrentes

Al ser inmutable, cada operación como add() o multiply() retorna un nuevo BigDecimal; esto facilita el razonamiento sobre el código y la seguridad en concurrencia, similar al diseño de String.

24. Internacionalización (Locale, NumberFormat, ResourceBundle)

1. ¿Qué clase representa una región geográfica, política o cultural específica para adaptar formatos y textos (i18n)?

Respuesta correcta: A) `java.util.Locale`

Locale encapsula idioma y país/región, usado por clases como `NumberFormat`, `DateFormat` y `ResourceBundle` para adaptar la salida.

2. ¿Cómo se cargan mensajes traducidos según el idioma usando `ResourceBundle`?

Respuesta correcta: A) `ResourceBundle.getBundle("nombreBase", locale)` busca automáticamente el archivo `.properties` más específico que coincida con el `Locale` dado, con fallback al bundle por defecto

`ResourceBundle.getBundle` busca archivos como `mensajes_es_MX.properties`, `mensajes_es.properties`, hasta `mensajes.properties` (default), siguiendo el algoritmo de resolución de `Locale`.

3. ¿Qué clase se usa para formatear números o monedas de acuerdo a un `Locale` específico?

Respuesta correcta: A) `java.text.NumberFormat`

`NumberFormat.getInstance(locale)` o `NumberFormat.getCurrencyInstance(locale)` formatean números respetando las convenciones del `Locale` indicado.

4. ¿Cómo se crea un `Locale` específico para español de México?

Respuesta correcta: A) `Locale.of("es", "MX")` (o el constructor equivalente `new Locale("es", "MX")` en versiones previas)

Se construye combinando el código de idioma ISO (`es`) y el código de país ISO (`MX`); en versiones recientes se recomienda el método de fábrica `Locale.of(idioma, pais)` en lugar del constructor directo.

5. ¿Qué convención de nombres de archivo sigue un `ResourceBundle` basado en propiedades para el `Locale` francés (`fr_FR`), con nombre base 'mensajes'?

Respuesta correcta: A) `mensajes_fr_FR.properties`

La convención es `nombreBase_idioma_PAIS.properties`, por ejemplo `mensajes_fr_FR.properties`, con guiones bajos separando idioma y país.

6. ¿Qué diferencia hay entre usar el carácter '#' y el carácter '0' en un patrón de `DecimalFormat`, por ejemplo en `"#0.00"`?

Respuesta correcta: A) '#' representa un dígito OPCIONAL (que no se muestra si no hay valor en esa posición, evitando ceros innecesarios), mientras que '0' representa un dígito OBLIGATORIO (que siempre se muestra, rellenando con cero si no hay valor real en esa posición)

En los patrones de `DecimalFormat`, '#' representa un dígito que se muestra solo si existe (opcional, sin rellenar con ceros), mientras que '0' fuerza que siempre se muestre un dígito en esa posición, rellenando con '0' si no hay ningún valor real ahí, útil para asegurar un número mínimo de decimales o dígitos.

7. ¿Qué diferencia de resultado esperarías entre `NumberFormat.getCurrencyInstance(Locale.US)` y `NumberFormat.getCurrencyInstance(Locale.GERMANY)` al formatear el mismo valor numérico?

Respuesta correcta: A) Cada uno formatea el número según las convenciones monetarias de su región: `Locale.US` usaría el símbolo de dólar (\$) y el formato típico estadounidense (con punto decimal), mientras que `Locale.GERMANY` usaría el símbolo de euro (€) y las convenciones alemanas (como la coma como separador decimal)

`NumberFormat.getCurrencyInstance(locale)` adapta tanto el símbolo de moneda como las convenciones de formato (separador decimal, agrupación de miles) según las reglas culturales/regionales del `Locale` proporcionado, produciendo resultados visualmente distintos para distintos países aunque el valor numérico subyacente sea el mismo.

8. ¿Qué representa el 'compact number format' (formato de número compacto) introducido para representar valores grandes de forma abreviada, como "7K" o "7M"?

Respuesta correcta: A) Es una forma de NumberFormat que abrevia números grandes usando sufijos como K (miles), M (millones), etc., adaptando la abreviatura según el Locale y la magnitud del número, útil para interfaces de usuario donde se prioriza la brevedad sobre la precisión exacta

El formato de número compacto (disponible desde Java 12 a través de `NumberFormat.getCompactNumberInstance()`) representa valores numéricos grandes de forma abreviada usando sufijos como K, M, B según la magnitud, adaptándose además a las convenciones del Locale proporcionado.

9. ¿Qué excepción se lanza típicamente al intentar parsear (convertir de texto a número) un String que no tiene un formato numérico válido según el patrón esperado, usando `NumberFormat.parse()`?

Respuesta correcta: A) `ParseException`, una excepción checked que debe manejarse o declararse explícitamente

El método `parse()` de `NumberFormat` (y sus subclases como `DecimalFormat`) lanza `ParseException`, una excepción checked, si el texto proporcionado no puede interpretarse según el patrón numérico esperado; esto obliga a manejarla con try-catch o declararla con throws, a diferencia de `NumberFormatException` (unchecked) usada en otros contextos de parsing como `Integer.parseInt()`.

10. ¿Cuál es el orden exacto del algoritmo de búsqueda (fallback) que sigue `ResourceBundle.getBundle()` al buscar el archivo de propiedades más adecuado para un Locale específico, por ejemplo español de México (es_MX)?

Respuesta correcta: A) Primero busca el archivo más específico (`nombreBase_es_MX.properties`); si no lo encuentra, busca uno solo con el idioma (`nombreBase_es.properties`); si tampoco existe, recurre al archivo base sin sufijo de idioma/país (`nombreBase.properties`); si ninguno existe, lanza `MissingResourceException`

`ResourceBundle.getBundle()` sigue un algoritmo de fallback jerárquico: primero intenta el archivo más específico (idioma_país), luego solo idioma, y finalmente el archivo base sin sufijo; si ninguno de estos archivos existe, se lanza `MissingResourceException`.

11. ¿Qué excepción se lanza si se solicita una clave específica que no existe en NINGUNO de los archivos de propiedades encontrados por el algoritmo de fallback de `ResourceBundle`?

Respuesta correcta: A) `MissingResourceException`

`MissingResourceException` se lanza tanto si no se encuentra ningún archivo de bundle adecuado para el Locale, como si el archivo sí se encuentra pero la clave solicitada específicamente (mediante `getString()`) no existe dentro de ese archivo.

12. ¿Cómo se combina `MessageFormat` con `ResourceBundle` para producir mensajes dinámicos y localizados, por ejemplo "Hola {0}, bienvenido a {1}"?

Respuesta correcta: A) Se obtiene la plantilla del mensaje (con marcadores como {0}, {1}) desde el `ResourceBundle` mediante `getString()`, y luego se usa `MessageFormat.format(plantilla, parametro1, parametro2)` para sustituir esos marcadores con los valores reales proporcionados en tiempo de ejecución

El patrón típico es almacenar la plantilla del mensaje (con marcadores posicionales {0}, {1}, etc.) como el valor de una clave en el archivo .properties, obtenerla mediante `ResourceBundle.getString()`, y luego usar `MessageFormat.format(plantilla, valores...)` para sustituir esos marcadores con los valores reales, logrando mensajes localizados y parametrizados a la vez.

13. ¿Cuál es una diferencia clave entre usar la clase Properties directamente y usar ResourceBundle para gestionar configuración o mensajes de una aplicación?

Respuesta correcta: A) Properties se centra en la carga y gestión manual de pares clave-valor desde cualquier fuente (útil para configuración general, incluso modificable en tiempo de ejecución con setProperty()); ResourceBundle está específicamente diseñado para internacionalización, con su propio algoritmo de fallback automático basado en Locale

Properties es una clase de propósito más general para cargar/gestionar pares clave-valor (por ejemplo, configuración de una aplicación), permitiendo incluso modificar valores en tiempo de ejecución con setProperty(); ResourceBundle está específicamente orientado a la internacionalización, con su propio mecanismo de selección automática de archivo según el Locale, algo que Properties no ofrece por sí sola.

14. ¿Qué hace el método getProperty(clave, valorPorDefecto) de la clase Properties si la clave solicitada no existe en el objeto Properties?

Respuesta correcta: A) Devuelve el valorPorDefecto proporcionado como segundo argumento, sin lanzar ninguna excepción

getProperty(clave, valorPorDefecto) es una forma segura de consultar un objeto Properties: si la clave no existe, simplemente devuelve el valor por defecto proporcionado como segundo argumento, sin lanzar ninguna excepción, a diferencia de ResourceBundle.getString() que sí lanzaría MissingResourceException en un caso análogo.

15. ¿Por qué usar un patrón como "HH:mm" con un objeto LocalDate (en lugar de LocalDateTime o LocalTime) provoca un error en tiempo de ejecución al formatear?

Respuesta correcta: A) Porque LocalDate no contiene ninguna información de hora (HH, mm); el patrón de formato debe ser compatible con los campos que realmente contiene el objeto temporal, y usar patrones de hora sobre un LocalDate lanza una excepción en tiempo de ejecución (UnsupportedTemporalTypeException)

DateTimeFormatter exige que el patrón solicitado sea compatible con los campos que el objeto temporal realmente posee; como LocalDate solo tiene año/mes/día (sin hora), usar patrones de hora como HH:mm lanza UnsupportedTemporalTypeException en tiempo de ejecución, aunque el código compile sin problema.

16. ¿Es correcto asumir que ResourceBundle, tras encontrar un archivo específico para un Locale (por ejemplo, zoo_es.properties), seguirá buscando también en el archivo default por cada clave individual que falte?

Respuesta correcta: A) No, es un error común: una vez que ResourceBundle determina cuál archivo de bundle usar según el algoritmo de fallback de Locale, usa exclusivamente ESE archivo (no vuelve a buscar en otros Locales) para las claves; si una clave específica no está en ese archivo bundle elegido, se lanza MissingResourceException directamente

Una vez que el algoritmo de fallback de Locale determina el archivo de bundle a utilizar (el más específico disponible), ResourceBundle NO continúa buscando en otros archivos de fallback para claves individuales faltantes; si la clave no existe en ese bundle ya seleccionado, se lanza MissingResourceException, un error común de comprensión sobre el mecanismo de fallback.

17. ¿Por qué el patrón de DateTimeFormatter "HH o'clock" (sin escapar la comilla simple) produce un error, y cómo se corrige correctamente?

Respuesta correcta: A) La comilla simple (') tiene un significado especial en los patrones de DateTimeFormatter (delimita texto literal); para incluir una comilla simple literal dentro del texto, debe escaparse duplicándola, como en "HH 'o'clock"

En los patrones de DateTimeFormatter, la comilla simple (') se usa para delimitar secciones de texto literal (que no deben interpretarse como símbolos de formato); para incluir una comilla simple literal dentro de ese texto, debe escaparse escribiéndola dos veces (") dentro de la sección delimitada, como en "HH 'o'clock".

18. ¿Cuál es la sintaxis correcta para construir un objeto Locale para inglés de Estados Unidos, evitando el error común de escribir 'new Locale("en_US")' como un único String?

Respuesta correcta: A) new Locale("en", "US") (idioma en minúscula y país en mayúscula como argumentos separados), o alternativamente usando Locale.Builder para mayor claridad

El constructor tradicional de Locale espera el código de idioma (en minúscula) y el código de país (en mayúscula) como argumentos SEPARADOS, no como un único String combinado con guion bajo; escribir 'new Locale("en_US")' como un solo argumento es un error común, ya que Java interpretaría todo el string como el código de idioma.

19. Según las reglas repasadas en los ejercicios, ¿qué caracteres de un patrón DecimalFormat determinan si un dígito es obligatorio o solo opcional en el resultado formateado?

Respuesta correcta: A) '0' indica un dígito obligatorio (se muestra siempre, incluso rellenando con cero); '#' indica un dígito opcional (no se muestra si no hay valor real en esa posición)

Confirmando la regla revisada en los ejercicios: '0' obliga a mostrar un dígito en esa posición del patrón (rellenando con cero si no hay valor), mientras que '#' solo lo muestra si existe un dígito real ahí, evitando ceros innecesarios en el resultado final.

27. Patrones de Diseño (Strategy)

1. ¿Qué problema resuelve el patrón de diseño Strategy?

Respuesta correcta: A) Permite definir una familia de algoritmos intercambiables, encapsulando cada uno en una implementación separada, de modo que el algoritmo pueda variar independientemente del código que lo usa

Strategy define una interfaz común para varios algoritmos intercambiables; el contexto delega en la estrategia concreta, permitiendo cambiarla en tiempo de ejecución sin modificar el cliente.

2. En Java moderno, ¿qué característica del lenguaje permite implementar el patrón Strategy de forma muy concisa sin crear clases separadas para cada algoritmo?

Respuesta correcta: A) Expresiones lambda que implementan una interfaz funcional que representa la estrategia

Si la interfaz Strategy es funcional, se puede pasar una lambda o method reference como estrategia en lugar de crear una clase concreta por cada algoritmo.

3. ¿Cuál es una ventaja de usar Strategy frente a múltiples bloques if-else o switch para seleccionar un algoritmo?

Respuesta correcta: A) Facilita añadir nuevas estrategias sin modificar el código existente que las usa (principio abierto/cerrado)

Strategy favorece el principio Open/Closed: se pueden añadir nuevas estrategias creando nuevas implementaciones, sin modificar el código cliente existente.

4. En el patrón Strategy, ¿qué rol cumple la clase 'Contexto'?

Respuesta correcta: A) Mantiene una referencia a una estrategia (interfaz común) y delega en ella la ejecución del algoritmo, sin conocer los detalles de la implementación concreta

El Contexto mantiene una referencia a la interfaz Strategy y delega en ella la ejecución concreta, pudiendo cambiar de estrategia en tiempo de ejecución sin modificar su propio código.

09. Clases Anidadas, Internas, Locales y Anónimas

1. ¿Qué es una clase interna (inner class) no estática en Java?

Respuesta correcta: A) Una clase definida dentro de otra clase que mantiene una referencia implícita a una instancia de la clase contenedora, permitiendo acceder a sus miembros de instancia

Una clase interna no estática está ligada a una instancia de la clase externa (se crea como externa.new Interna()), y puede acceder directamente a los miembros de instancia de esa clase externa.

2. ¿Qué es una clase local (local class) en Java?

Respuesta correcta: A) Una clase definida dentro del cuerpo de un método, visible solo dentro de ese método

Una clase local se declara dentro de un bloque de método y solo es visible/utilizable dentro de ese bloque, a diferencia de las clases anónimas que no tienen nombre propio.

3. ¿Puede una clase interna no estática acceder directamente a variables private de la clase externa que la contiene?

Respuesta correcta: A) Sí, las clases internas tienen acceso completo a todos los miembros (incluidos private) de la clase externa que las contiene

Una de las principales ventajas de las clases internas es su acceso irrestricto a todos los miembros, incluidos los private, de la clase que las contiene.

4. ¿Puede una interfaz declararse como 'sealed' en Java, restringiendo qué clases o interfaces pueden implementarla o extenderla?

Respuesta correcta: A) Sí, al igual que las clases, una interfaz puede declararse sealed con una cláusula permits, restringiendo qué clases pueden implementarla y qué interfaces pueden extenderla

El modificador sealed puede aplicarse tanto a clases como a interfaces; una interfaz sellada usa también la cláusula 'permits' para limitar qué clases implementadoras o qué subinterfaces están autorizadas.

5. ¿Cuál es la diferencia principal entre una clase interna (inner class) no estática y una clase anidada estática (static nested class)?

Respuesta correcta: A) La clase interna no estática requiere una instancia de la clase externa para existir y tiene acceso a sus miembros de instancia; la clase anidada estática no requiere esa instancia y no tiene acceso implícito a los miembros de instancia de la clase externa

Una clase interna no estática está ligada a una instancia específica de la clase externa (se crea con externa.new Interna()) y accede directamente a sus miembros de instancia; una clase anidada estática se comporta más como una clase de nivel superior, sin necesitar ni tener acceso implícito a una instancia externa.

6. ¿Qué es una 'clase local' (local class) y dónde se declara?

Respuesta correcta: A) Es una clase declarada dentro del cuerpo de un método (o bloque), visible únicamente dentro de ese método/bloque donde fue definida

Una clase local se define dentro del cuerpo de un método o bloque de código, y su visibilidad/uso queda restringido a ese ámbito local; a diferencia de una clase anónima, sí tiene un nombre propio y puede tener múltiples constructores.

7. ¿Qué error común se debe evitar al trabajar con clases anidadas y el acceso a variables locales del método que las contiene?

Respuesta correcta: A) Intentar que una clase local o anónima acceda a una variable local externa que no sea 'effectively final' (que se reasigna después de declararse), lo que produce un error de compilación

Una trampa común es que las clases locales y anónimas solo pueden capturar variables locales del método contenedor si estas son 'effectively final' (no se reasignan después de su inicialización); intentar acceder a una variable que sí cambia produce un error de compilación.

8. ¿Cómo se relaciona la encapsulación con el diseño de clases anidadas privadas dentro de una clase externa?

Respuesta correcta: A) Una clase anidada declarada como private solo es accesible dentro de la clase externa que la contiene, lo cual permite ocultar detalles de implementación interna que no deberían exponerse fuera de esa clase

Declarar una clase anidada como private es una forma de encapsulación a nivel de diseño: esa clase (y sus detalles de implementación) queda completamente oculta y solo es utilizable desde dentro de la clase externa que la declara, sin exponerse a ningún código externo.

23. API de Fecha y Hora (java.time)

1. ¿Qué clase del paquete `java.time` representa una fecha sin hora, como '2026-07-07'?

Respuesta correcta: A) `LocalDate`

`LocalDate` representa solo una fecha (año, mes, día) sin componente de hora ni zona horaria.

2. ¿Qué clase representa una fecha y hora sin información de zona horaria?

Respuesta correcta: A) `LocalDateTime`

`LocalDateTime` combina fecha y hora (`LocalDate` + `LocalTime`) sin asociar ninguna zona horaria específica.

`ZonedDateTime` y `OffsetDateTime` sí incluyen esa información.

3. ¿Son mutables los objetos de las clases del paquete `java.time` como `LocalDate` y `LocalDateTime`?

Respuesta correcta: A) No, son inmutables; métodos como `plusDays()` devuelven una nueva instancia en lugar de modificar el objeto original

Al igual que `String` y `BigDecimal`, las clases del API `java.time` (introducido en Java 8 para reemplazar la antigua y problemática clase `Date/Calendar`) son inmutables; sus métodos de modificación devuelven siempre un nuevo objeto.

4. ¿Qué clase se usa para representar una cantidad de tiempo basada en fechas (años, meses, días), como la diferencia entre dos `LocalDate`?

Respuesta correcta: A) `Period`

`Period` representa una cantidad de tiempo en términos de años/meses/días (basado en fechas del calendario); `Duration` representa cantidades de tiempo basadas en segundos/nanosegundos, útil entre instantes temporales.

5. ¿Qué método se usa para calcular la diferencia entre dos `LocalDate` en días?

Respuesta correcta: A) `ChronoUnit.DAYS.between(fecha1, fecha2)`

`ChronoUnit.DAYS.between(inicio, fin)` calcula la cantidad de días entre dos objetos temporales; también puede usarse `Period.between()` para obtener años/meses/días por separado.

26. Módulos Java (JPMS)

1. ¿Qué archivo declara un módulo en el sistema de módulos de Java (JPMS, introducido en Java 9)?

Respuesta correcta: A) `module-info.java`

`module-info.java`, ubicado en la raíz del código fuente del módulo, contiene la declaración del módulo: su nombre, qué paquetes exporta y qué módulos requiere.

2. ¿Qué directiva dentro de `module-info.java` indica que un módulo depende de otro módulo?

Respuesta correcta: A) `requires nombreModulo;`

'requires' declara una dependencia de compilación y ejecución hacia otro módulo. 'uses' se refiere a servicios consumidos, no a dependencias de módulo directas.

3. ¿Qué directiva permite que otros módulos accedan a las clases públicas de un paquete específico?

Respuesta correcta: A) `exports nombrePaquete;`

'exports' hace visible un paquete (sus tipos públicos) a otros módulos que lo requieran; sin exportar el paquete, sus clases permanecen encapsuladas dentro del módulo, incluso si son públicas.

4. ¿Cuál es uno de los principales beneficios del sistema de módulos (JPMS) respecto al classpath tradicional?

Respuesta correcta: A) Permite encapsulamiento fuerte a nivel de módulo: un paquete no exportado no es accesible desde fuera, incluso si sus clases son públicas

JPMS introduce encapsulamiento fuerte: solo los paquetes explícitamente exportados con 'exports' son visibles fuera del módulo, mejorando la seguridad y el diseño frente al classpath tradicional donde todo lo público era accesible.

5. ¿Qué directiva se usa en `module-info.java` para declarar que un módulo abre un paquete a la reflexión (por ejemplo para frameworks como Hibernate)?

Respuesta correcta: A) `opens nombrePaquete;`

'opens' permite el acceso reflexivo (incluso a miembros privados vía `reflection`) a un paquete desde otros módulos, útil para frameworks que instancian objetos dinámicamente.

00. Información del examen de certificación

1. ¿Cuál es la duración del examen de certificación Oracle Java 21 (1Z0-830), en comparación con el examen Java 17 (1Z0-829)?

Respuesta correcta: A) El examen Java 21 dura 2 horas, más tiempo que las 90 minutos del examen Java 17

Según lo señalado en el curso, el examen de certificación Java 21 otorga 2 horas de duración, un incremento notable respecto a los 90 minutos del examen Java 17, lo cual da más margen para resolver las preguntas.

2. ¿Qué tema importante del examen Java 17 fue eliminado en el examen de certificación Java 21?

Respuesta correcta: A) JDBC (Java Database Connectivity)

Uno de los cambios señalados entre ambos exámenes es que los temas relacionados con JDBC fueron eliminados del temario del examen de certificación Java 21, a diferencia del examen Java 17 que sí los incluía.

3. ¿Cuál es el porcentaje mínimo de aciertos necesario para aprobar tanto el examen Java 17 como el Java 21, y cuántas preguntas tiene cada examen?

Respuesta correcta: A) 68% de aciertos mínimo, con 50 preguntas en ambos exámenes

Ambos exámenes (Java 17 y Java 21) mantienen el mismo formato: 50 preguntas y un puntaje mínimo de aprobación del 68%, según lo mencionado en la comparación del curso.

4. Aproximadamente, ¿qué porcentaje del temario del examen Java 17 sigue siendo aplicable y relevante para el examen Java 21?

Respuesta correcta: A) Entre el 85% y el 90%

Se menciona que aproximadamente el 85-90% de los temas cubiertos en el examen Java 17 continúan siendo relevantes y aplicables al examen Java 21, por lo que estudiar para uno prepara en gran medida para el otro.

5. ¿Qué ventaja tiene el vale (voucher) de examen mencionado en el curso respecto a qué exámenes de certificación Java puede usarse?

Respuesta correcta: A) Es válido para varios exámenes de certificación Oracle Java, incluyendo Java 8, 11, 17 y 21, no solo para uno específico

El vale de examen mencionado (con un costo aproximado de \$245 más impuestos y una ventana de 6 meses para programarlo) es válido para varios exámenes de certificación Oracle Java, entre ellos Java 8, 11, 17 y 21.

25. Entrada/Salida (I/O, File, Path, Scanner)

1. ¿Cuál es la diferencia principal entre la clase heredada `java.io.File` y la interfaz `java.nio.file.Path` introducida en NIO.2?

Respuesta correcta: A) File representa de forma más limitada una ruta de archivo o directorio con un conjunto reducido de operaciones; Path (parte de la API NIO.2, más moderna) ofrece una representación más rica y flexible del sistema de archivos, integrándose con la clase utilitaria Files para operaciones más completas

`java.io.File` es la clase original (desde las primeras versiones de Java) para representar rutas de archivos/directorios, con capacidades limitadas; `java.nio.file.Path` (parte de NIO.2, introducido en Java 7) ofrece una API más moderna y completa, normalmente usada junto a la clase utilitaria `Files` para realizar operaciones sobre el sistema de archivos.

2. ¿Qué método de la clase utilitaria `Files` permite obtener un objeto `Path` a partir de un `String` que representa una ruta, como una alternativa moderna a crear un `File` directamente?

Respuesta correcta: A) `Path.of("ruta/al/archivo.txt")` (o el método equivalente `Paths.get()`, usado en versiones previas a esta simplificación)

`Path.of(String...)` es el método de fábrica moderno (Java 11+) para crear un objeto `Path` a partir de una o más cadenas que representan la ruta; en versiones anteriores se usaba comúnmente `Paths.get(String...)` con el mismo propósito.

3. ¿Qué método de la clase Files permite verificar si un archivo o directorio existe en el sistema de archivos, dado un objeto Path?

Respuesta correcta: A) Files.exists(path)

Files.exists(Path) es el método utilitario que verifica si el archivo o directorio referenciado por el Path existe realmente en el sistema de archivos, devolviendo un boolean.

4. ¿Cuál es la diferencia conceptual entre un 'stream de bytes' (como InputStream/OutputStream) y un 'stream de caracteres' (como Reader/Writer) en la API de I/O de Java?

Respuesta correcta: A) Los streams de bytes trabajan con datos binarios sin interpretación de codificación de texto (útiles para imágenes, archivos binarios, etc.); los streams de caracteres están diseñados específicamente para texto, manejando la codificación de caracteres (charset) de forma apropiada

InputStream/OutputStream (y sus subclases) operan sobre datos binarios crudos (bytes), apropiados para cualquier tipo de dato incluyendo archivos no textuales; Reader/Writer están diseñados específicamente para trabajar con texto, gestionando automáticamente la conversión entre bytes y caracteres según una codificación (charset) determinada.

5. ¿Por qué se recomienda usar try-with-resources al trabajar con streams de I/O como FileInputStream o BufferedReader?

Respuesta correcta: A) Porque estos streams implementan AutoCloseable (o Closeable) y mantienen recursos del sistema operativo (como descriptores de archivo) que deben liberarse explícitamente; try-with-resources garantiza su cierre automático incluso si ocurre una excepción durante su uso

Los streams de I/O gestionan recursos limitados del sistema operativo (como descriptores de archivo) que deben cerrarse explícitamente para evitar fugas de recursos; try-with-resources garantiza ese cierre automático de forma segura, incluso si ocurre una excepción durante el procesamiento, siendo la práctica recomendada sobre el manejo manual con finally.

6. ¿Qué método de la clase Files permite leer todo el contenido de un archivo de texto pequeño en una única List<String>, con cada línea como un elemento?

Respuesta correcta: A) Files.readAllLines(path)

Files.readAllLines(Path) lee todo el contenido de un archivo de texto y lo devuelve como una List<String>, donde cada elemento corresponde a una línea del archivo; es conveniente para archivos pequeños que caben cómodamente en memoria.

7. ¿Por qué se prefiere usar BufferedReader (envolviendo un FileReader) en lugar de leer un archivo de texto carácter por carácter directamente con FileReader, en términos de rendimiento?

Respuesta correcta: A) BufferedReader introduce un búfer interno que reduce drásticamente la cantidad de operaciones de lectura físicas al sistema de archivos, leyendo bloques más grandes de una vez y sirviendo los datos desde memoria, en lugar de acceder al disco en cada lectura individual de carácter

Acceder al disco para cada lectura individual es costoso; BufferedReader mantiene un búfer interno que lee bloques más grandes de datos de una vez, sirviendo las lecturas posteriores (carácter por carácter o línea por línea) desde ese búfer en memoria, mejorando significativamente el rendimiento frente a leer directamente con FileReader sin búfer.

8. ¿Qué método de la clase Files permite escribir una colección de líneas de texto directamente a un archivo, como alternativa conveniente a usar un Writer manualmente?

Respuesta correcta: A) Files.write(path, listaDeLineas)

Files.write(Path, Iterable<? extends CharSequence>) permite escribir directamente una colección de líneas de texto a un archivo, sin necesidad de crear y gestionar manualmente un objeto Writer, simplificando la escritura de archivos de texto sencillos.

9. ¿Qué clase se usa comúnmente para leer datos ingresados por el usuario desde la entrada estándar (System.in) de forma sencilla, incluyendo métodos como nextInt() o nextLine()?

Respuesta correcta: A) Scanner (java.util.Scanner)

Scanner es la clase más comúnmente usada para leer y parsear la entrada del usuario desde System.in de forma conveniente, ofreciendo métodos como nextInt(), nextDouble(), nextLine(), next(), entre otros, para leer distintos tipos de datos.

10. ¿Qué diferencia hay entre los métodos `next()` y `nextLine()` de `Scanner` al leer texto ingresado por el usuario?

Respuesta correcta: A) `next()` lee solo hasta el siguiente espacio en blanco (un único 'token'), mientras que `nextLine()` lee toda la línea completa hasta el salto de línea, incluyendo espacios intermedios

`next()` se detiene en el primer espacio en blanco, devolviendo solo esa palabra/token; `nextLine()` lee la línea completa (incluyendo espacios) hasta el próximo salto de línea, siendo la opción adecuada para leer frases completas ingresadas por el usuario.

11. ¿Qué ventaja ofrece la clase `System.console()` sobre `Scanner` al interactuar con el usuario en una aplicación de consola, particularmente para leer contraseñas?

Respuesta correcta: A) `Console` ofrece el método `readPassword()`, que lee la entrada del usuario SIN mostrarla en pantalla (oculta los caracteres tecleados), una funcionalidad de seguridad que `Scanner` no provee directamente

Una ventaja clave de `Console` sobre `Scanner` es el método `readPassword()`, que permite leer información sensible (como contraseñas) sin mostrarla en pantalla mientras el usuario la escribe; sin embargo, `System.console()` puede devolver `null` en algunos entornos (como muchos IDEs), donde no hay una consola interactiva real disponible.

12. ¿Qué puede ocurrir si se llama a un método de `Scanner` como `nextInt()` cuando la entrada del usuario no corresponde a un número entero válido?

Respuesta correcta: A) Se lanza `InputMismatchException`, una excepción `unchecked` que indica que el token leído no coincide con el tipo esperado

Si la entrada disponible no puede interpretarse como el tipo esperado (por ejemplo, texto no numérico al llamar `nextInt()`), `Scanner` lanza `InputMismatchException`, una excepción `unchecked` (`RuntimeException`), sin necesidad de declararla explícitamente aunque sí conviene manejarla para evitar que el programa termine abruptamente.

13. ¿Qué método de `Scanner` permite verificar si existe más entrada disponible para leer como un entero, antes de invocar `nextInt()` y arriesgarse a una excepción?

Respuesta correcta: A) `hasNextInt()`

`hasNextInt()` (y sus variantes como `hasNextDouble()`, `hasNext()`, `hasNextLine()`) permite verificar de forma segura si el siguiente token disponible puede interpretarse como el tipo esperado, antes de invocar el método `next`-correspondiente y arriesgarse a una excepción si no lo es.

14. ¿Por qué es importante cerrar un `Scanner` que envuelve `System.in` una vez que ya no se necesita, y qué precaución especial existe al hacerlo en aplicaciones que puedan necesitar leer más entrada después?

Respuesta correcta: A) Cerrar el `Scanner` libera recursos asociados, pero cerrar un `Scanner` que envuelve `System.in` también cierra el propio flujo de entrada estándar subyacente; si se intenta leer más entrada después de cerrarlo (incluso con un nuevo `Scanner`), se producirá un error, por lo que conviene cerrar el `Scanner` de `System.in` solo al finalizar definitivamente la interacción con el usuario

Como `Scanner` implementa `Closeable`, cerrarlo (`close()`) libera sus recursos; sin embargo, al envolver `System.in`, cerrar el `Scanner` también cierra el stream subyacente de entrada estándar del programa, por lo que cualquier intento posterior de leer más entrada (incluso con un `Scanner` completamente nuevo) fallaría, siendo una consideración importante en programas que necesitan múltiples ciclos de lectura de usuario.